

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

Intelligence Artificielle Avancée

& Federated Learning

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation

Auteur	**Ghiloufi Jamel**
Fonction	**Architecte Logiciel Principal**
Direction	**Département de Recherche & Développement**
Service	**R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré**
Niveau	**Recherche**
Édition	**Édition 2026 – 1.0**
Copyright	**© Pixel Software Design**

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Copyright et mentions légales

Éditeur	Pixel Software Design
Auteur	Ghiloufi Jamel
Fonction	Architecte Logiciel Principal
Département	Département de Recherche & Développement
Intitulé	Intelligence Artificielle Avancée & Federated Learning
Sous-titre	Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Édition	Édition 2026 (version 1.0)
Langue	Français
Niveau	Recherche
Collection	Supports de formation et de recherche, Pixel Software Design

Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Droits de l'auteur. Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

Droits des tiers. Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

Marques et licences externes. PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

Sommaire

Structure détaillée du cours

Partie 1 – Fondations de l'intelligence artificielle avancée . . . 9

1.1 Cadre statistique de l'apprentissage automatique . . . 10

1.2 Optimisation numérique et convergence . . . 12

1.3 Généralisation, régularisation et surapprentissage . . . 14

1.4 Architectures de référence : CNN, Transformer, graphe . . . 17

1.5 Grands modèles de langage : préentraînement et adaptation . . . 20

Partie 2 – Algorithmes d'apprentissage fédéré . . . 23

2.1 Panorama de l'apprentissage fédéré . . . 24

2.2 FedAvg : formulation, implémentation et variantes . . . 29

2.3 Hétérogénéité des données : diagnostic et stratégies . . . 36

2.4 Théorie de la convergence des algorithmes fédérés . . . 42

2.5 Systèmes distribués et passage à l'échelle . . . 48

Partie 3 – Confidentialité, attaques et défenses . . . 56

3.1 Modèle de menace et surface d'attaque . . . 57

3.2 Fuites d'information : inférence de gradient et extraction . . . 62

3.3 Empoisonnement, backdoors et attaques ciblées . . . 69

3.4 Confidentialité différentielle . . . 75

3.5 Agrégation sécurisée et défense en profondeur . . . 83

3.6 Évaluation de la robustesse et étude de cas . . . 88

Partie 4 – Applications et grands modèles fédérés . . . 93

4.1 Federated Learning des grands modèles de langage (FedLLM) . . . 94

4.2 Federated Reinforcement Learning (FedRL) . . . 101

4.3 Santé et systèmes critiques . . . 107

4.4 Véhicules connectés et systèmes de transport (V2X) . . . 113

4.5 TinyML et fédération embarquée . . . 119

4.6 Apprentissage fédéré sur graphes (FedGL) . . . 124

Partie 5 – FedX : le framework de référence . . . 130

5.1 Vision, principes et architecture . . . 131

5.2 Modèle de programmation : client, serveur et stratégies . . . 138

5.3 Backbones, stratégies avancées et confidentialité intégrée . . . 145

5.4 Déploiement, observabilite et tableau de bord web . . . 151
Partie 6 – Nouveaux fronts de recherche . . . 158
6.1 Apprentissage contrastif fédéré (FedCL) . . . 159
6.2 Inférence causale fédérée (FedCI) . . . 165
6.3 Fédération de réseaux de neurones impulsionnels . . . 171
6.5 Programme pédagogique sur douze semestres et feuille de route . . . 182
6.6 Publication, reproductibilité et éthique . . . 186
Partie 7 – Travaux pratiques, projets et évaluation . . . 193
7.1 Cadre des travaux pratiques . . . 194
7.2 Travaux pratiques guidés . . . 201
TP 7.1 — Implémenter la moyenne pondérée sur un modèle linéaire . . . 203
TP 7.2 — Apprentissage fédéré sur réseau convolutif, vingt clients . . . 205
TP 7.3 — Partition non échangeable et mesure de son effet . . . 208
TP 7.4 — Mesurer le coût de communication et l'effet de la compression . . . 210
TP 7.5 — Attaque par inversion de gradient et défense par bruit . . . 212
TP 7.6 — Entraînement différentiellement privé et tracé du compromis . . . 214
TP 7.7 — Adapter un grand modèle de langage et fédérer les adaptateurs . . . 217
TP 7.8 — Agrégation robuste face aux mises à jour malveillantes . . . 219
TP 7.9 — Explorer FedX : déployer une exécution, lire les métriques, produire un rapport . . . 221
7.3 Projets de fin de semestre . . . 224
Projet — Grands modèles de langage fédérés par métier . . . 224
Projet — Santé : segmentation fédérée multi-hôpitaux . . . 225
Projet — Véhicules connectés : perception coopérative multi-niveaux . . . 227
Projet — Confidentialité et audit : differentially private et défenses robustes . . . 228
Projet — FedX : évolution du framework . . . 229
7.4 Exercices d'entraînement . . . 230
7.5 Évaluation, certification et ressources . . . 237
Annexes . . . 242
A.1 Liste des abréviations . . . 243
A.2 Glossaire des termes . . . 244
A.3 Table de notation . . . 248
A.4 Chronologie des jalons . . . 250
A.5 Bibliographie . . . 251
A.6 Formulaire de conformité . . . 255
A.7 Ressources et outils . . . 256

Partie	Thème	Contenu	Compétence visée	Durée
Partie 1	Fondations	Statistique, optimisation, courbes d'apprentissage	Comprendre et formaliser	2-3 semaines
Partie 2	Algorithmes FL	FedAvg, convergence, hétérogénéité, système	Implémenter et régler	3-4 semaines
Partie 3	Confidentialité	DP, attaques, défenses, agrégation sécurisée	Auditer et sécuriser	2-3 semaines
Partie 4	Applications	FedLLM, santé, V2X, FedRL, TinyML, graphes	Transposer à un domaine	3-4 semaines
Partie 5	Framework FedX	Architecture, API, benchmarks, exploitation	Industrialiser	2 semaines
Partie 6	Nouveaux fronts	FedCL, FedCI, SNN, FedGL, feuille de route	Ouvrir la recherche	2 semaines
Partie 7	TP et projets	9 labs, 5 projets, corrigés, évaluation	Faire et rendre	4-6 semaines

Préface

Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

Conventions de lecture. Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

Progression pédagogique

Des principes statistiques aux architectures de pointe

Partie 1

Fondations de l'intelligence artificielle avancée

L'apprentissage fédéré n'est pas une discipline à part entière : c'est de l'optimisation distribuée appliquée à des données réparties entre des détenteurs qui ne peuvent pas les réunir. Cette partie pose donc le vocabulaire et les résultats sans lesquels aucune garantie de convergence, de confidentialité ou d'efficacité ne peut être énoncée dans la suite du cours.

Objectifs de la partie

- Formaliser un problème d'apprentissage comme un problème d'optimisation.
- Distinguer risque empirique, risque attendu et erreur de généralisation.
- Justifier le choix d'une architecture et d'un optimiseur par des critères.
- Lire une courbe d'apprentissage et identifier un défaut de réglage.
- Comprendre pourquoi l'adaptation, et non le préentraînement, est fédérable.

Plan de la partie

1. Cadre statistique de l'apprentissage automatique
2. Optimisation numérique et théorie de la convergence
3. Généralisation, régularisation et surapprentissage
4. Architectures de référence : CNN, Transformer, graphe
5. Grands modèles de langage : préentraînement et adaptation

Chapitre 1.1

Cadre statistique de l'apprentissage automatique

Un modèle n'est pas une fonction : c'est une hypothèse sur un processus de génération de données, et un procédé de sélection parmi une famille d'hypothèses.

L'apprentissage automatique automatise une décision — la prédiction de y à partir de x — à partir d'un échantillon fini. Cette automatisation n'a de sens que si l'échantillon est représentatif d'une population dont la loi reste stable dans le temps. Toute la théorie de l'apprentissage consiste à quantifier l'écart entre ce que l'on optimise sur l'échantillon et ce que l'on souhaiterait optimiser dans la population.

Modèle d'apprentissage et hypothèse d'estimation

Definition – Modèle d'apprentissage

Un triplet (A, Ω, L) où A est un espace d'apprentissage, Ω un espace de paramètres et $L : A \times \Omega \to \mathbb{R}$ une fonction de perte. Une règle d'apprentissage A associe à tout échantillon S un paramètre $\theta(S) \in \Omega$ minimisant l'objectif empirique $L(\theta, S)$.

$$L(\theta, S) = (1)/(m) \sum_{i=1}^{m} \ell \leq ft(y_i, f_\theta(x_i))$$
$$\theta \in \operatorname{argmin}_{\theta \in \Omega} L(\theta, S) \tag{1.1}$$

La seconde équation définit le problème d'optimisation ; toute la partie II en étudie la résolution distribuée.

Trois grandeurs doivent être distinguées sans les confondre :

- **Le risque empirique** $L(\theta, S)$, calculé sur les m observations effectivement observées ; c'est la seule grandeur directement accessible.
- **Le risque attendu** $R(\theta) = E_{S,(x,y)}[\ell(y, f_\theta(x))]$, inaccessible mais visée, qui est la véritable ciblé.
- **L'écart de généralisation** $G(\theta) = |L(\theta, S) - R(\theta)|$, objet central des travaux de concentration.

Le piège du minimiseur

Confondre le risque empirique minimal et le risque attendu minimal est l'erreur la plus fréquente. Le premier est une propriété de l'échantillon ; le second une propriété de la population. Un modèle parfaitement optimisé sur le premier peut être médiocre sur le second — et c'est exactement ce qui arrive quand on réduit le nombre d'itérations locales pour économiser de la communication en apprentissage fédéré.



Figure 1 – Les trois paradigmes d'apprentissage et leur rapport au cadre fédéré

Critère	Supervisé	Non supervisé	Par renforcement	Fédéré
Donnée	(x, y) étiquetée	x seule	transitions (s, a, r, s')	mélange des trois
Objectif	minimiser le risque	structurer Z	maximiser J(π)	minimiser F sous contrainte
Entité fédérée	client = jeu étiqueté	client = corpus, client = agent	client = toute entité qui apprend	
Métrique	exactitude, AUC	silhouette, NMI	retour moyen	qualité, coût, ☐, robustesse
Verrou principal	surapprentissage	identifiabilité	exploration	hétérogénéité et confiance

Tableau 1 – Comparaison des paradigmes d'apprentissage

La colonne « fédéré » n'est pas un paradigme de plus : c'est une contrainte transverse qui se superpose aux trois autres. On peut apprendre fédérément de façon supervisée, non supervisée ou par renforcement ; ce qui change, c'est que l'optimiseur doit alors respecter en plus une contrainte de communication et de confiance.

Le rôle de l'hypothèse

Le choix de la classe de fonctions f précède celui de l'algorithme d'optimisation. Cet ordre est contre-intuitif pour qui commence en apprentissage automatique, où l'on choisit spontanément un algorithme connu. Il est pourtant celui de la pratique expérimentale réussie : on cherche d'abord une famille de fonctions dont le risque bayésien est faible, puis on cherche l'optimiseur qui s'en approche.

$$R^*(f) = \inf_{f \in \square 0 \square} E_{(x,y) \sim \square 1 \square} \leq ft[\ell \leq ft(y, f(x))]$$
$$R(f_S) \leq \underbrace{R^*(f)}_{\square 0 \square} + \underbrace{\inf_{f \in \square 4 \square} R(f) - R^*(f)}_{\square 1 \square} + \underbrace{E \leq ft[L(f_S) - \inf_f L(f, S)]}_{\square 2 \square} + \underbrace{\leq ft|L(f_S) - R(f_S)|}_{\square 3 \square} \quad (1.2)$$

Décomposition usuelle de l'erreur. Le premier terme est dû aux observations bruitées et ne peut pas être réduit par un algorithme ; les trois autres relèvent de l'ingénierie. C'est cette décomposition qui donné son sens à la suite du cours : chaque grande partie attaque un terme différent.

- Le **bruit irréductible** borne tout : il vaut la variance de y conditionnellement à x pour un modèle bayésien bien spécifié.
- L'**erreur d'approximation** dépend de la richesse de F et se compense en augmentant la capacité du modèle.
- L'**erreur d'optimisation** est le seul terme sur lequel un algorithme d'apprentissage agit directement.

- L'**erreur d'estimation** décroît comme 1/√(m) sous conditions usuelles.

Astuce

En apprentissage fédéré, l'erreur d'optimisation est remplacée par une erreur d'optimisation *distribuée* qui contient en plus le terme de dérive entre clients. Toute la difficulté de la partie II tient dans cette substitution : on ne peut plus régler la convergence par un réglage global du taux d'apprentissage, seulement par un dosage entre un terme local et un terme global.

A retenir

- Un modèle d'apprentissage est un couple (règle, classe de fonctions) évalué sur un risque.
- Toute garantie en apprentissage fédéré porte sur le risque attendu, jamais sur le risque empirique seul.
- Le problème fédéré ajoute une contrainte de distribution des données, pas une contrainte sur les données.
- Les quatre termes de la décomposition de l'erreur indiquent où agir : classe de fonctions, optimisation, agrégation, données.

Chapitre 1.2

Optimisation numérique et convergence

La fédération remplace un minimiseur par un processus itératif distribué. Il faut donc des outils pour juger si ce processus converge, et à quelle vitesse.

Les optimiseurs employés en profondeur respectent trois exigences : la robustesse au gradient bruité, la tolérance à une hétérogénéité des données, et un coût mémoire borné indépendant de la taille du modèle. Adam et ses variantes satisfont les deux dernières ; la première exige une surveillance explicite, car Adam amplifie les valeurs extrêmes.

Descente de gradient stochastique et son interprétation

$$\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t, B_t)$$
$$E \leq ft[\theta_{t+1}] = \theta_t - \eta_t \nabla L(\theta_t) \quad (1.3)$$

En espérance, un mini-lot non biaisé se comporte comme le gradient complet : la variance ne déplace pas le point fixe, elle réduit la constante effective de l'écart à l'optimum. C'est ce qui autorisé à réduire la taille du lot en apprentissage fédéré, où chaque lot est déjà petit par construction.

Algorithme 1 – Descente de gradient stochastique avec estimateur de moments

```
Entrées : pas η, horizon T, jeu de données S
Initialiser θ_0
pour t = 1 à T :
  tirer un lot B_t uniformément dans S
  g_t ← ∇ L(θ_{t-1}, B_t)
  m_t ← β_1 m_{t-1} + (1-β_1) g_t
  v_t ← β_2 v_{t-1} + (1-β_2) g_t^2
  θ_t ← θ_{t-1} - η (m_t)/(√(v_t) + ε)
retourner θ_T
```

Entrées : taux d'apprentissage, coefficients d'oubli • **Sorties :** suite de paramètres itérés • **Complexité :** O(TNK) opérations, O(d) mémoire d'état

Les deux moments corrigent respectivement le gradient moyen et son second moment ; c'est ce qui rend Adam robuste à l'échelle des gradients. La correction est locale au client, jamais transportée.

Optimiseur	Mémoire d'état	Bruit	Hétérogénéité	Usage fédéré
SGD + momentum	O(d)	faible	moyenne	peu employé seul
Adam	O(3d)	moyenne	bonne	standard côté serveur
AdamW	O(3d)	moyenne	bonne	standard, décroissance découplée
SGD local + FedAvg	O(d)	faible	faible sans correction	faible coût de communication
SCAFFOLD	O(2d)	moyenne	bonne	corrige la dérive client
FedProx	O(d)	moyenne	bonne	régularise vers w_t

Tableau 1 – Comparaison des optimiseurs

La colonne « mémoire d'état » est la colonne décisive en fédéré : elle détermine ce qui doit transiter entre deux tours chez un client, ou ce qui peut être reconstruit localement.

> **Astuce**
> En apprentissage fédéré, l'optimiseur local compte double : il doit être robuste, puisque les lots locaux sont petits et bruités, et son état doit pouvoir transiter correctement d'un round à l'autre. D'où l'usage dominant d'Adam côté serveur et de SGD côté client, et non l'inverse.

Conditions de convergence

Une suite θ_t converge vers un minimum global si trois conditions sont réunies : la fonction de perte est convexe, le taux d'apprentissage est adapté, et le bruit du gradient est borné. Chacune de ces conditions peut échouer séparément, et le mode de défaillance est différent — c'est ce qui permet de diagnostiquer une courbe d'apprentissage anormale.

Hypothèse	Formulation	Si elle échoue	Symptôme observé
Convexité	L convexe en θ	points de selle	oscillation, non-reproductibilité
Descente	$\eta_t \leq 1/L$	divergence	perte qui explose
Bruit borné	$E\|g_t\|^2 \leq \sigma^2$	variance	perte qui plafonne haut
Pas décroissant	$\sum_t \eta_t = \infty$	arrêt précoce	convergence trop rapide vers un point médiocre
Pas sommable	$\sum_t \eta_t^2 < \infty$	bruit résiduel	perte qui ne descend plus

Tableau 2 – Conditions suffisantes de convergence et modes de défaillance

> **Descente stochastique sur un problème fortement convexe**
> Soit L L-lisse et μ-fortement convexe, et $\eta \leq 1/L$. Pour tout T et tout $\sigma > 0$, avec probabilité au moins $1-\delta$:

$$L(\theta_T) - L(\theta^*) \leq (L\,D^2)/(2\eta\,T) + (\sigma^2)/(2\mu\,T) + (11\,L\,D^2)/(12\,T^2) + (L\,\sigma^2)/(\mu\,T)\log(T)/(\delta) \quad (1.4)$$

Trois enseignements : la décroissance est en $1/T$; le bruit fixe un plancher à $\sigma^2/(2\mu\,T)$; le terme logarithmique en δ explique pourquoi les analyses pratiques, avec des garanties hautes probables, sont si pessimistes. Le premier terme est celui qui commande en pratique : il est proportionnel au carré de la distance à l'optimum, ce qui rend le préchauffage indispensable.

```python
import torch
from torch.optim import SGD


def local_steps(model, loader, epochs, lr, momentum=0.9, weight_decay=1e-4):
"""Un tour de FedAvg : optimisation locale bornée en mémoire."""
optimizer = SGD(model.parameters(), lr=lr, momentum=momentum,
weight_decay=weight_decay)
model.train()
for _ in range(epochs):
for x, y in loader:
optimizer.zero_grad(set_to_none=True)
loss = torch.nn.functional.cross_entropy(model(x), y)
loss.backward()
# Le bornage évite qu'un lot atypique ne déplace le modèle trop loin.
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=10.0)
optimizer.step()
return model
```

Listing 1 — Un tour d'optimisation local conforme à FedAvg

Piège : transporter l'état d'Adam entre clients

Laisser l'état d'Adam transiter d'un client à l'autre. Les moments m et v sont des estimateurs de la distribution locale des gradients : les mélanger entre clients hétérogènes n'a aucun sens statistique et détruit la convergence. Si l'optimiseur doit être transporté, il faut le réinitialiser côté client et ne conserver que l'état serveur.

Exercice 1.2.1 (difficulte : Moyen)

Une courbe de perte d'entraînement décroît régulièrement et atteint 0,15, tandis que la perte de validation stagnent à 0,42 dès le tour 3, puis remontent légèrement. Le nombre d'itérations locales est fixé à 40 par tour, le taux d'apprentissage à 0,01, et le jeu compte 20 clients de taille très inégale. Trois lectures sont possibles ; laquelle retenez-vous, et quel est le premier réglage à tenter ?

Indice : Le nombre d'itérations locales est élevé pour un cadre fédéré, et la taille des clients est hétérogène : les deux facts entrée en compte.

Correction

- Le diagnostic est la variance, non le biais : la perte d'entraînement continue de descendre, ce qui exclut un déficit de capacité. Le premier levier est le nombre d'itérations locales, réduit de 40 à 5 ou 10 : sur des lots déjà petits, 40 itérations produisent un modèle local très éloigné du modèle global, et c'est l'agrégation de modèles éloignés qui surapprend. Réduire ce nombre avant toute régularisation. Si la courbe reste écartée ensuite, le second levier est la correction de dérive, puis seulement la régularisation. Noter que l'hétérogénéité des tailles aggrave le problème : un client dix fois plus grand est sur-représenté dans l'agrégation pondérée par la taille.

A retenir

- Un optimiseur local doit être simple, borné en mémoire et tolérant à la dérive.
- La borne de convergence explicite donne le nombre de tours minimal pour une précision donnée.
- Le compromis qualité-communication se pilote par le nombre d'itérations locales, pas par le taux d'apprentissage.
- Chaque condition de convergence a un mode de défaillance distinct, identifiable sur la courbe de perte.

Chapitre 1.3

Généralisation, régularisation et surapprentissage

Un modèle qui mémorise n'est pas un modèle qui a appris.

La théorie de la généralisation remplace l'intuition par des bornes explicites. Le principe généraliste borne

le risque attendu par le risque empirique majoré d'une erreur liée à la complexité effective de la classe de fonctions et à la divergence entre cette classe et la vraie loi.

Le principe de généralisation

$$R(f_S) \leq L(f_S, S) + \text{underbrace}\square_{\square 1\square} + \square_{\square 2\square} + \square_{\square 3\square\square 0\square} \quad (1.5)$$

Le modèle fédéré dégrade surtout $\square_{\square 0\square}$, parce que l'agrégation n'est pas un minimiseur, et $\square_{\square 0\square}$, parce que les distributions locales ne coïncident pas. C'est la décomposition qui rend ces deux effets mesurables séparément.

Les deux termes $\square_{\square 0\square}$ et $\square_{\square 0\square}$ sont des propriétés du problème ; seul $\square_{\square 0\square}$ dépend de l'algorithme. Un travail de recherche sérieux en apprentissage fédéré consiste d'abord à mesurer la séparation des deux autres, puis à réduire le troisième.

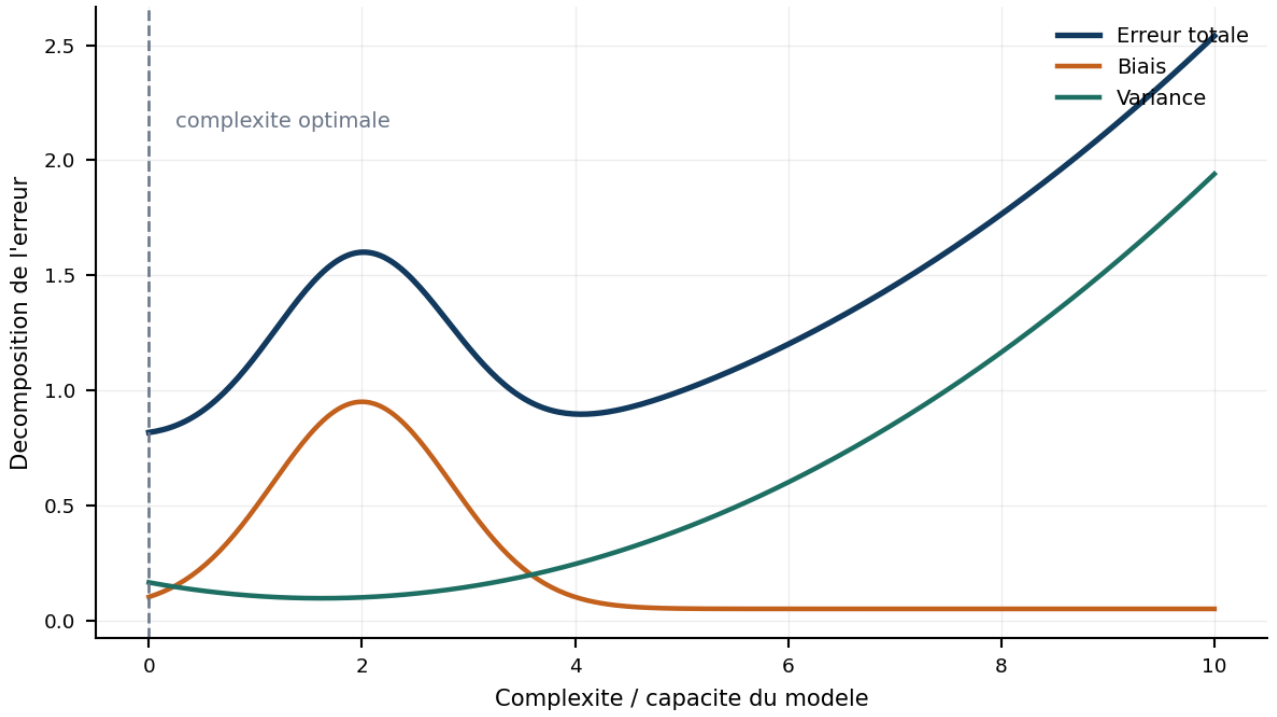


Figure 1 – Compromis biais-variance et complexité optimale du modèle

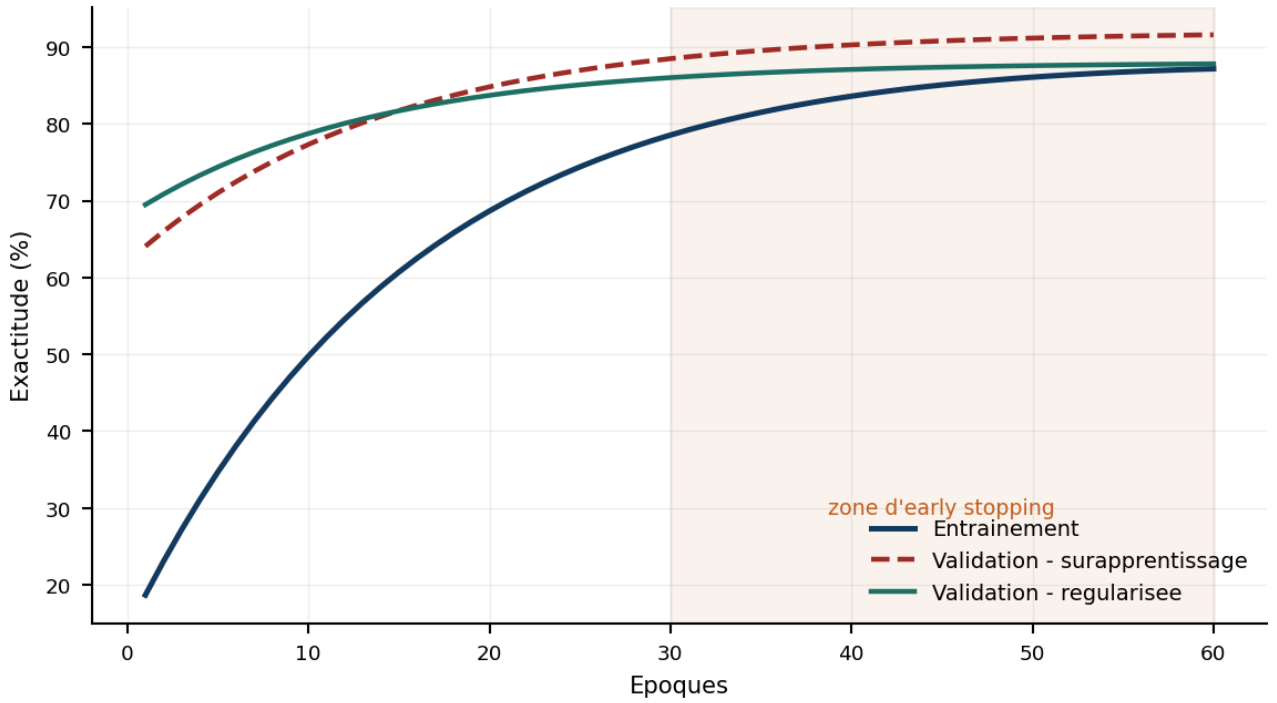


Figure 2 – Courbes d'apprentissage et phénomène de surapprentissage

Le surapprentissage : diagnostic et remèdes

Le surapprentissage se détecte par la divergence entre la courbe d'apprentissage et la courbe de validation. Quatre situations se distinguent, chacune appelant un remède différent ; les confondre conduit à des réglages inutiles, parfois contre-productifs.

Symptôme	Diagnostic	Remède immédiat	Remède de fond
Écart croissant	variance	régularisation, données	architecture plus contrainte
Les deux courbes stagnent	biais	capacité, entraînement plus long	meilleure représentation
Validation bruitée	taille d'échantillon	moyenne glissante	validation croisée distribuée
Effondrement tardif	taux d'apprentissage	préchauffage puis décroissance de η	cosine, décroissance par paliers
Instabilité entre tours	agrégation	bornage des mises à jour	correction de dérive (SCAFFOLD)

Tableau 1 – Diagnostic du surapprentissage et remèdes

Protocole de réglage en apprentissage fédéré

- **Fixer l'architecture et l'optimiseur serveur**, ne pas y toucher ensuite. Toute modification ultérieure invalide les mesures prises.

2. **Calibrer le nombre d'itérations locales** E sur un petit nombre de clients, en surveillant la divergence entre courbe d'apprentissage globale et courbe de validation agrégée.
3. **Choisir la fraction de participation** C : augmenter C réduit la variance de l'agrégation mais augmente le coût par tour.
4. **Appliquer la régularisation** — proximité, decay pondéré, élaguage — seulement après avoir vérifié que E n'est pas la cause du surapprentissage.
5. **Vérifier la robustesse** en rejouant l'entraînement avec un échantillonnage de clients différent à chaque graine.

Règle de terrain

Sur un jeu hétérogène de K clients, réduire le nombre d'époques locales avant d'augmenter la régularisation. L'hétérogénéité agit comme du bruit supplémentaire dans l'espace des paramètres : le modèle doit s'aplatir avant que la variance ne devienne le facteur limitant. Augmenter la régularisation sans traiter la dérive revient à traiter un symptôme.

Exercice 1.3.2 (difficulte : Avancé)

Sur un jeu de 60 clients dont 3 détiennent à eux seuls 70 % des observations, l'exactitude du modèle agrégé est inférieure à celle du modèle entraîné sur les 3 grands clients seuls, et supérieure à celle du modèle entraîné sur les 57 petits. Proposez deux hypothèses concurrentes et une expérience qui les départage.

Indice : Deux Causes sont compatibles avec l'observation : un biais de l'agrégation pondérée, ou une incompatibilité entre distributions locales.

Correction

- Première hypothèse : l'agrégation pondérée par la taille privilégie les trois grands clients, et le modèle global est essentiellement leur modèle — mais dégradé par 57 tours locaux de contribution faible. Elle prédit que remplacer les poids par une pondération uniforme, ou par un échantillonnage par client plutôt que par observation, relève l'exactitude. Seconde hypothèse : les distributions locales sont antagonistes, et lagranger la qualité moyenne exige de consacrer beaucoup plus de tours à la correction de la dérive. Elle prédit qu'une pondération uniforme dégrade le résultat. L'expérience qui départage est donc la pondération uniforme avec le reste inchangé : si l'exactitude monte, l'hypothèse du biais d'agrégation tient ; si elle baisse, c'est l'antagonisme des distributions qui est le facteur limitant, et il faut augmenter le nombre de tours locaux ou corriger la dérive.

A retenir

- Quatre termes se partagent l'erreur : classe de fonctions, données, optimisation, estimation.
- Le modèle fédéré agit sur le troisième terme, et l'aggrave si l'on ne maîtrise pas le second.
- La régularisation de l'optimisation est souvent plus rentable que la régularisation du modèle.
- Le diagnostic du surapprentissage doit précéder tout réglage : les quatre cas appellent des remèdes différents.

Chapitre 1.4

Architectures de référence : CNN, Transformer, graphe

Trois hypothèses sur la structure du signal, trois conséquences sur le coût.

Le choix d'une architecture est un choix sur la structure de dépendance attendue. Un réseau convolutif priorise la localité spatiale, un transformeur la dépendance longue et l'adaptivité du contexte, un réseau de graphes l'irrégularité topologique. En apprentissage fédéré, ce choix détermine le coût de communication autant que la qualité finale.

Critère	CNN	Transformeur	Réseau de graphes
Hypothèse	localité, translation	attention, longue portée	voisinage de graphe
Paramètres échangés	faible à très élevé	très élevé	élevé
Compressibilité	élevée	faible	moyenne
Adaptation locale	peu coûteuse	LoRA	restriction au sous-graphe
Risque fédéré dominant	dérive du classifieur	dérive de l'attention	structure non i.i.d.
Domaines	imagerie, vidéo	texte, code, multimodal	réseaux, molécules, graphes de connaissance
Coût d'entraînement	faible	élevé	moyen

Tableau 1 – Comparaison des architectures

La ligne « paramètres échangés » est celle qui décide en pratique. Un transformeur de plusieurs milliards de paramètres n'est pas fédérable par ses poids complets dans un cadre classique : c'est la raison pour laquelle l'adaptation de faible rang est devenue le régime de référence.

Le bloc transformeur

Le bloc transformeur remplace la convolution par une attention à plusieurs têtes, précédée d'une normalisation de couche et suivie d'une connexion résiduelle. Trois innovations décisives : l'attention multi-têtes qui factorise plusieurs relations en parallèle, la normalisation de couche qui stabilisé l'entraînement en profondeur, et le mécanisme de position qui réintroduit l'ordre.

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{Q K^{\top}}{\sqrt{d_k}} + M\right) V$$

$$\mathrm{MultiHead}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W^{O}, \quad \mathrm{head}_i = \mathrm{Attention}(X W_i^{Q}, X W_i^{K}, X W_i^{V})$$

$$z^{(\ell)} = X + \mathrm{MLP}(\mathrm{LayerNorm}(X + \mathrm{MHA}(X))) \tag{1.6}$$

L'échelle $1/\sqrt{d_k}$ est indispensable : sans elle, les produits scalaires croissent avec d_k et la softmax sature. Le masque causal M est ce qui distingue l'encodeur du décodeur.

Definition – Complexité de l'attention

L'attention dense coûte $O(n^2\, d)$ en mémoire et en calcul pour une séquence de longueur n. C'est le coût quadratique qui rend les fenêtres d'attention, les noyaux linéaires ou l'échantillonnage nécessaires au-delà de quelques milliers de jetons, et qui pèse lourdement sur tout protocole de transmission.

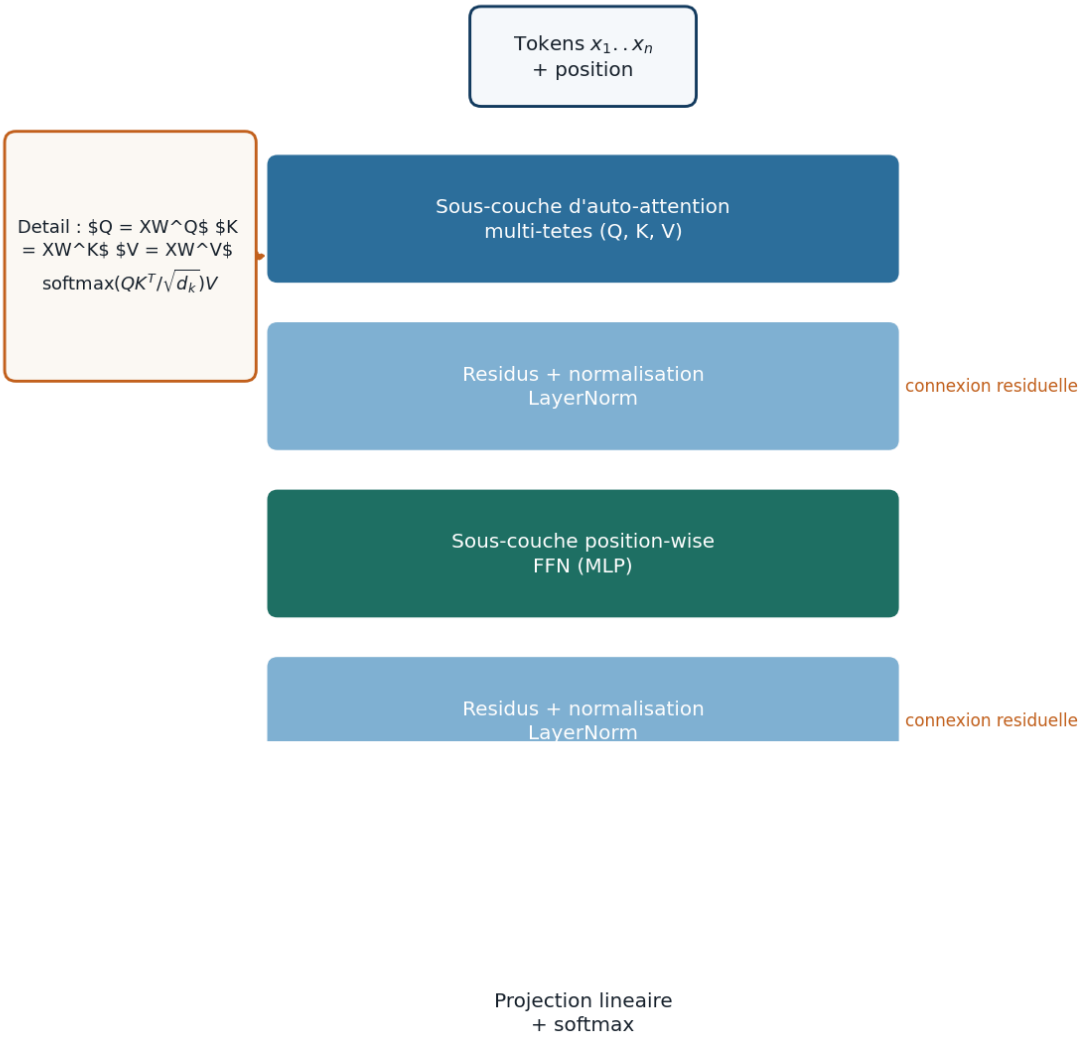


Figure 1 – Architecture d'un bloc transformeur et détail de l'attention

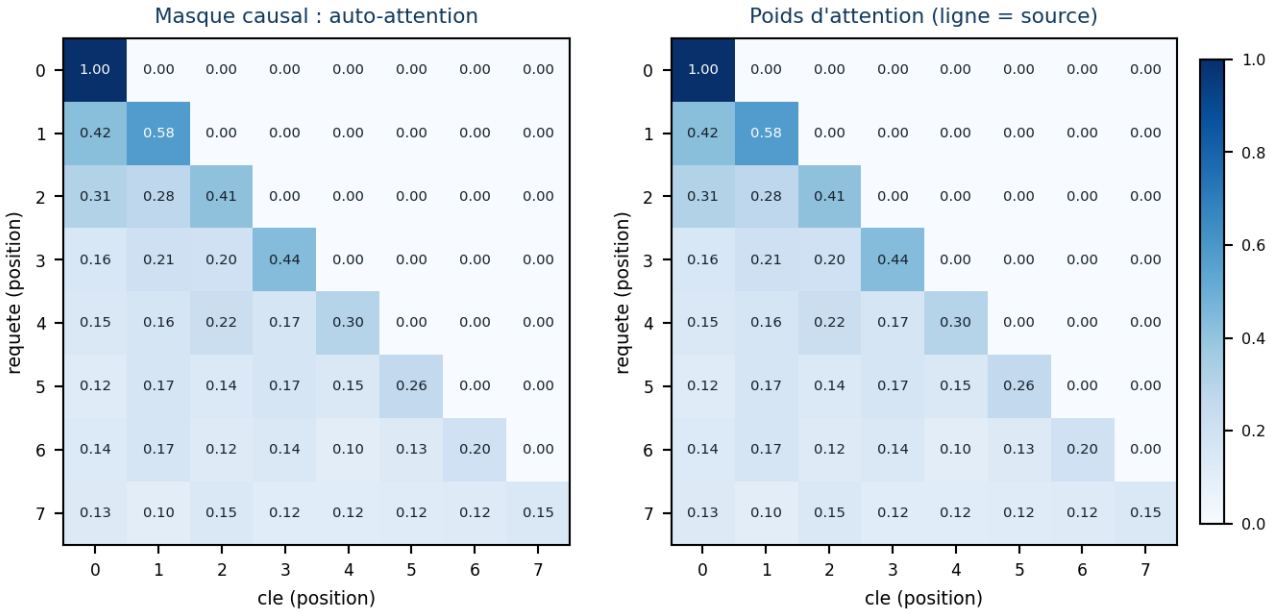


Figure 2 – Matrice d'attention : masque causal et poids associés

Piège : inversion de l'ordre pré-résiduel

Poser la normalisation de couche *après* la résiduelle au lieu d'avant, dans un transformeur préentraîné. L'ordre a été fixé empiriquement lors du travail de normalisation de couche et il est conservé dans tous les modèles publiés ; l'inverser dégrade brutalement la convergence, surtout en apprentissage fédéré où l'on dispose de peu de tours pour compenser.

A retenir

- Chaque architecture encode une hypothèse de structure, pas seulement un choix d'implémentation.
- Le coût de communication est proportionnel au nombre de paramètres échangés, pas à la profondeur.
- En pratique, adapter l'architecture — LoRA, sous-graphes — révèle plus de gains fédérés que changer d'architecture.
- L'attention dense a un coût quadratique : c'est une contrainte de conception, pas un détail d'implémentation.

Chapitre 1.5

Grands modèles de langage : préentraînement et adaptation

Un grand modèle de langage est un cas particulier de modèle : sa particularité est l'échelle, pas l'architecture.

Le préentraînement optimise une seule quantité : la vraisemblance des jetons suivants. Toute la richesse comportementale du modèle en découle. L'adaptation, elle, se joue sur des jeux de données bien plus petits — c'est exactement le régime où l'apprentissage fédéré est pertinent, car le goulot d'étranglement n'y est plus le calcul mais la donnée.

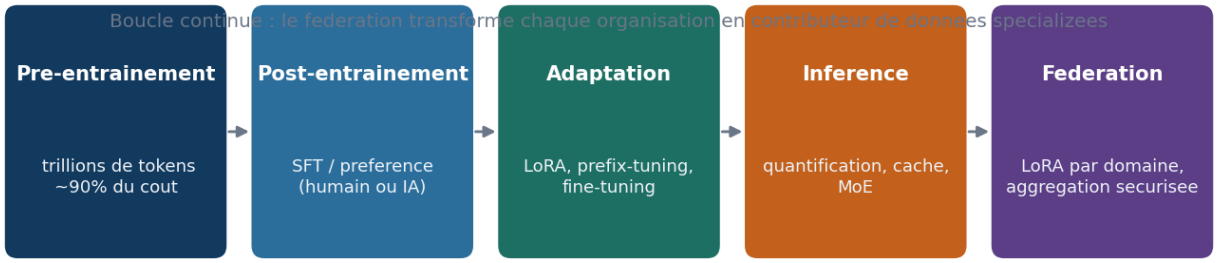


Figure 1 – Cycle de vie d'un modèle de langage et place de la fédération

$$L(\theta) = -(1)/(N) \sum_{i=1}^{N} \log p_\theta \leq ft(x_i \mid x_{<i})$$
$$PPL = \exp \leq ft((1)/(N) \sum_{i=1}^{N} CE_i) \quad (1.7)$$

La perplexité est l'exponentielle de la perte moyenne. Chaque demi-bit gagné représente une réduction de perplexité d'environ 35 % : c'est pourquoi les gains tardifs paraissent lents alors qu'ils sont réels.

Méthode	Paramètres entraînés	Mémoire par client	Fédérable	Usage
Affinage complet	100 %	élevée	difficile	rare en FL
LoRA	0,1 à 1 %	faible	oui, cas de référence	défaut en FL
Adaptateurs — préfixe, IA3	0,01 à 0,1 %	très faible	oui	edge, objets connectés
Quantification	0 %	nulle	impossible	distillation, inférence
Apprentissage de prompts	< 0,001 %	nulle	trivial	zéro-shot

Tableau 1 – Comparaison des familles d'adaptation

La colonne « mémoire par client » décide de la faisabilité : un client qui doit héberger l'intégralité du modèle et de ses gradients n'est pas un client, c'est un centre de calcul avec une contrainte juridique supplémentaire.

Pourquoi l'adaptation est le bon registre fédéré

Trois raisons convergent. La première est économique : le coût du préentraînement d'un modèle de plusieurs milliards de paramètres dépasse la capacité d'un laboratoire moyen, mais celui d'un adaptateur de quelques milliers de paramètres ne le dépasse pas. La deuxième est juridique : l'adaptation porte sur des données spécialisées — dossiers médicaux, corpus industriels, transactions — que l'on ne peut ni partager ni externaliser. La troisième est méthodologique : l'adaptation converge en quelques centaines d'étapes, ce qui est compatible avec un cadre de tours synchrones.

$$W_t - W_{t-1} = -\eta \nabla L \leq ft(W_{t-1} + (\alpha)/(r)\, B_t\, A_t)$$
$$\Delta W = (\alpha)/(r)\, B_t\, A_t \text{ avec } A_t \in \mathbb{R}^{r \times d},\ B_t \in \mathbb{R}^{d \times r},\ r \ll d \qquad (1.8)$$

La mise à jour n'est plus de rang d mais de rang r : la restriction de rang est l'hypothèse qui rend l'agrégation sûre, et c'est aussi ce qui réduit le coût de communication d'un facteur d/r. Sur un modèle de sept milliards de paramètres avec $r = 8$, on passe de plusieurs gigaoctets par tour à quelques mégaoctets.

L'échelle α/r est le détail le plus souvent omis

Le facteur α/r est le détail le plus souvent omis : il compense la réduction de rang. Oublier ce facteur rend l'adaptation instable d'une configuration à l'autre, parce que l'échelle effective du gradient dépend du rang choisi. Avec $r = 4$ et avec $r = 32$, le même taux d'apprentissage ne produit pas le même modèle.

Exercice 1.5.3 (difficulte : Moyen)

Un modèle de sept milliards de paramètres doit être adapté sur quatre centres hospitaliers qui ne peuvent pas mutualiser leurs dossiers. On propose LoRA avec $r = 8$, puis une agrégation moyenne des quatre matrices. Combien de données transitent par tour, et quel est le point de vigilance sur la qualité de l'agrégation ?

Indice : Deux questions : le volume réellement transmis, et ce que la moyenne des mises à jour de faible rang fait de la structure du modèle.

Correction

- Le volume transmis est celui de $B_t A_t$: deux matrices de rang 8, soit environ seize millions de paramètres par couche concernée, donc quelques dizaines de mégaoctets par tour contre plusieurs gigaoctets pour un affinage complet — un rapport de l'ordre de cent. Le point de vigilance est que la moyenne de matrices de faible rang n'est pas nécessamment elle-même de rang 8 : la somme de deux matrices de rang 8 peut être de rang 16. Sur plusieurs tours, le modèle agrégé s'échappe progressivement de l'ensemble de faible dimension, et la restriction de rang qui rendait l'agrégation sûre n'est plus garantie. La parade consiste à projeter à nouveau sur l'ensemble de faible rang après chaque agrégation, ou à agréger dans la base réduite plutôt que dans la base pleine.

A retenir

- Le préentraînement est irréversible et centralisé ; seule l'adaptation est fédérable.
- La restriction de rang est la brique qui rend les grands modèles compatibles avec l'apprentissage fédéré.
- Le coût fédéré d'un grand modèle se mesure en adaptateurs échangés, non en jetons par seconde.
- L'agrégation de mises à jour de faible rang n'est pas stable sans reprojection.

De FedAvg aux systèmes distribués industrialisables

Partie 2

Algorithmes d'apprentissage fédéré

La partie I a posé le vocabulaire statistique et les fondations de l'optimisation. Cette partie entre dans la mécanique opérationnelle de l'apprentissage fédéré : comment un modèle apprend-il lorsque les données sont réparties sur des milliers, voire des millions de dispositifs ou d'entités, sans jamais être centralisées ? Cette question mêle un problème d'optimisation distribuée, un système réparti, un protocole de communication et des contraintes juridiques explicites.

Objectifs de la partie

- Formuler le problème d'optimisation fédéré et expliciter FedAvg.
- Identifier, classifier et quantifier l'hétérogénéité des données (non-IID).
- Comparer les variantes d'algorithmes : FedProx, SCAFFOLD, FedDyn, FedNova, FedOpt.
- Analyser les bornes de convergence et le compromis communication-précision.
- Choisir une architecture d'orchestration et une stratégie de participation à l'échelle.

Plan de la partie

1. Panorama de l'apprentissage fédéré
2. FedAvg : formulation, implémentation et variantes
3. Hétérogénéité des données : diagnostic et stratégies
4. Théorie de la convergence des algorithmes fédérés
5. Systèmes distribués et passage à l'échelle

Chapitre 2.1

Panorama de l'apprentissage fédéré

L'apprentissage fédéré naît d'une exigence simple : apprendre à partir de données réparties sans les déplacer. Cette exigence transforme la nature même du problème d'apprentissage.

Définition, principes et historique

Definition – Apprentissage fédéré

L'apprentissage fédéré est un paradigme d'apprentissage automatique dans lequel un modèle global est entraîné de manière itérative par l'agrégation de mises à jour locales calculées sur des clients distribués, sans que les données brutes ne quittent jamais leur environnement d'origine.

Le principe fondamental est le suivant : au lieu de centraliser les échantillons (x_i, y_i) dans un seul serveur, on laisse chaque client $k \in 1,\ldots,K$ conserver son jeu de données local D_k. À chaque tour t, le serveur diffuse le modèle courant w_t ; chaque client actif effectue quelques étapes d'optimisation locale sur D_k et renvoie une mise à jour Δw_t^k, ou directement w_{t+1}^k. Le serveur agrège ces mises à jour pour produire w_{t+1}.

$$F(w) = \sum_{k=1}^{K} (n_k)/(n)\, F_k(w),\ n_k = |D_k|,\ n = \sum_{k=1}^{K} n_k$$
$$F_k(w) = (1)/(n_k) \sum_{i \in \square 0 \square_k} \ell(y_i, f_w(x_i)) \qquad (2.1)$$

Formulation canonique du risque global, pondéré par la taille des jeux locaux. Le choix de la pondération n'est pas neutre : il détermine quelle population le modèle global cherche à.optimiser.

Sur le plan historique, l'article de McMahan et ses collaborateurs (2017) introduit FedAvg comme algorithme de référence. Depuis lors, le domaine s'est structuré autour de quatre hypothèses structurantes qui distinguent l'apprentissage fédéré du calcul distribué classique :

- **Données non partagées.** Les D_k ne sont jamais fusionnées ; seules les mises à jour de paramètres, ou gradients, sont échangées.
- **Agrégation décentralisée du savoir.** Le serveur n'apprend pas les échantillons : il agrège des modèles ou des gradients produits localement.
- **Hétérogénéité structurelle.** Les distributions $P_k(x,y)$ peuvent différer fortement d'un client à l'autre — c'est le régime non-IID — de même que les capacités matérielles.
- **Contrainte de communication.** Les clients sont souvent sur des liens à faible bande passante, intermittents, ou limités en énergie : c'est le régime cross-device.

Cette définition mérite d'être déguisée en trois invariants, que l'on vérifiera à chaque itération. Les **données locales** D_k ne quittent jamais le client k ; le **modèle** w_t est la seule quantité diffusée ; l'**agrégation** est une opération de synthèse — une moyenne pondérée, et non une concaténation — ce qui interdit toute reconstitution directe de la base globale.

L'historique du domaine se lit en trois temps. Avant 2017, l'apprentissage distribué reposait sur des architectures de type paramètre-serveur (DistBelief, Parameter Server) où tous les workers échangeaient des gradients en permanence : le serveur était un goulot d'étranglement et le coût de communication dominait. McMahan et ses collaborateurs introduisent FedAvg en 2017, avec deux bascules : passer à une communication par tours plutôt qu'à un flux continu, et à un entraînement local intense — plusieurs passes sur les données du client avant toute synchronisation. Le gain est immédiat, jusqu'à deux ordres de grandeur de réduction de bande passante, au prix d'un risque de dérive locale.

Entre 2018 et 2022, l'effort porte sur la compréhension du phénomène de dérive et sur les corrections

algorithmiques : FedProx (2018) régularise vers le point de départ, SCAFFOLD (2020) corrigé le gradient par variables de contrôle, FedDyn et FedNova (2020) traitent respectivement l'hétérogénéité d'objectif et l'hétérogénéité de nombre de pas. À partir de 2022, deux mouvements convergent : l'adaptation des grands modèles — LoRA, quantification, mélange d'experts — qui rend l'apprentissage fédéré crédible sur des modèles de milliards de paramètres, et l'industrialisation (Flower, FLARE, FATE) qui fournit les primitives d'orchestration. Les travaux récents s'intéressent enfin à la fédération inter-organisations, le croisement de silos, et aux garanties formelles de vie privée.

$$w_{t+1} = \sum_{k \in S_t} (n_k)/(n_{S_t})\, w_{t+1}^{k} = w_t + \sum_{k \in S_t} (n_k)/(n_{S_t})\, \Delta w_t^{k} \qquad (2.2)$$

Forme équivalente : FedAvg agrège des deltas de mise à jour plutôt que des poids. Cette écriture est celle qui rend la compression et la sparsification naturelles, puisque c'est le delta qui est transmis.

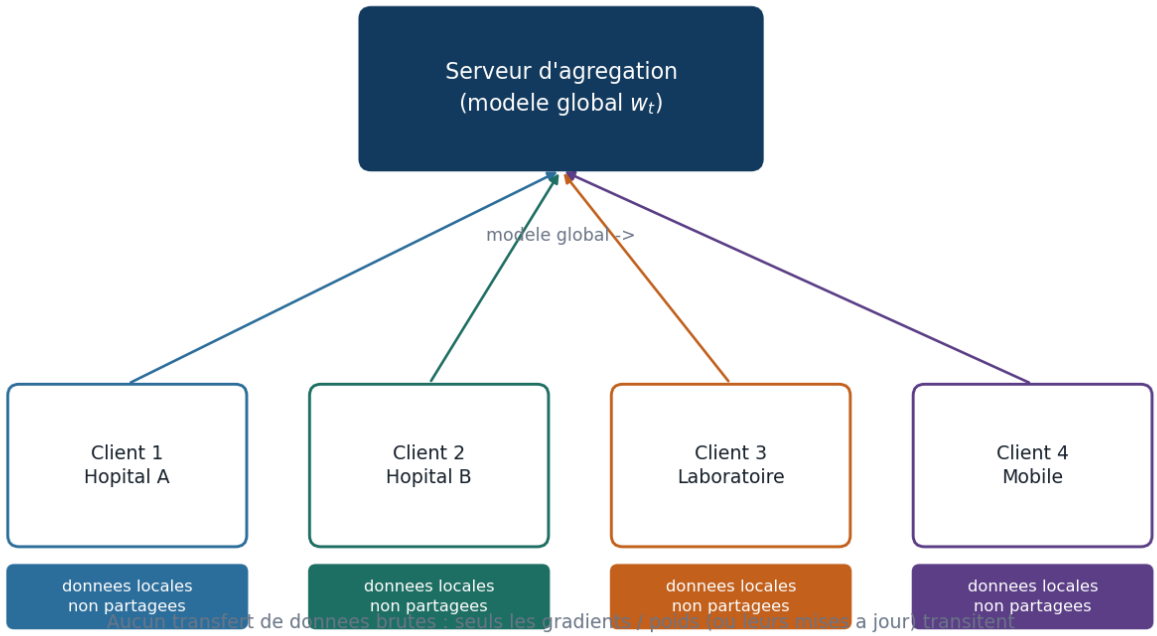


Figure 1 – Architecture client-serveur de l'apprentissage fédéré

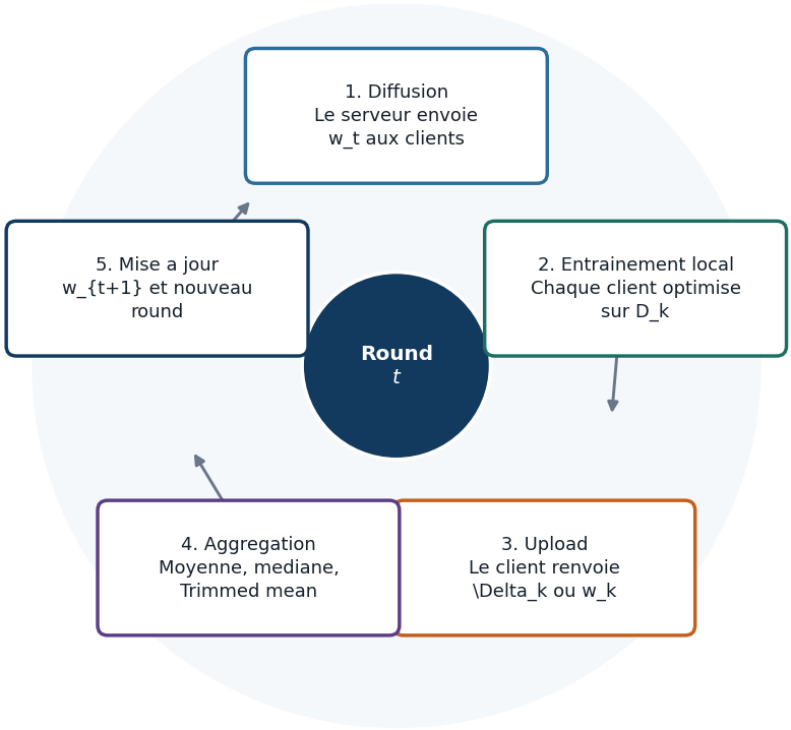


Figure 2 – Cycle fédéré type : diffusion, entraînement local, agrégation

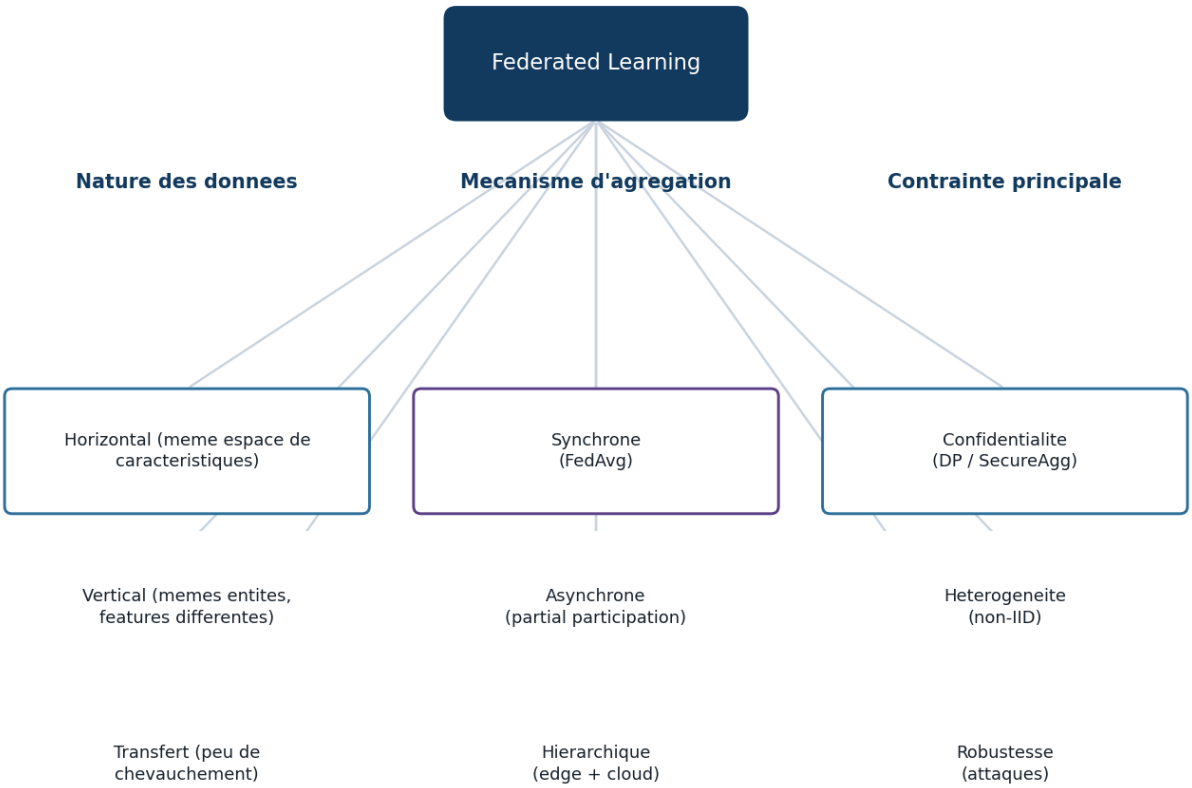


Figure 3 – Taxonomie des paradigmes fédérés : cross-silo ou cross-device, synchrone ou asynchrone

Architectures logicielles et modes d'exécution

L'architecture détermine les hypothèses de fiabilité, de scalabilité et de confidentialité. Trois familles se partagent le marché, et le choix se fait sur le modèle de menace autant que sur la performance.

Critère	Client-serveur	Pair-à-pair	Hiérarchique
Contrôle	Centralisé, orchestrateur	Distribué, par consensus	Multi-niveaux : edge, fog, cloud
Tolérance aux pannes	Point unique, mitigé	Élevée, par redondance	Partielle
Latence de tour	Faible à moyenne	Variable, selon topologie	Optimisée par grappes
Scalabilité	10^6 clients	10^3 à 10^4 nœuds	10^6 avec agrégation intermédiaire
Sécurité	Serveur de confiance requis	Robuste aux Byzantins	Compromis entre les deux

Tableau 1 – Architectures et modes d'exécution en apprentissage fédéré

Aspect	Cross-silo	Cross-device
Acteurs	Organisations : hôpitaux, banques	Appareils : téléphones, capteurs, objets connectés
Nombre de clients K	10 à 10^3	10^4 à 10^7
Disponibilité	Stable et prévisible	Intermittente et aléatoire
Puissance de calcul	GPU, serveurs	CPU contraints, sur batterie
Données par client	Volumineuses, gigaoctets	Petites, mégaoctets
Mode d'exécution	Synchrone dominant	Hybride, synchrone et asynchrone

Tableau 2 – Cross-silo et cross-device

La distinction commande tous les choix qui suivent. En cross-silo, la participation est quasi totale et le problème est la convergence. En cross-device, la participation est partielle et le problème est la disponibilité : un client absent n'est pas une anomalie, c'est le régime normal.

> **Definition – FedSync et FedAsync**
> FedSync, synchrone, attend que tous les clients sélectionnés terminent leur tour avant l'agrégation. FedAsync, asynchrone, intègre les mises à jour au fur et à mesure de leur arrivée, avec éventuellement un poids décroissant en fonction du délai.

- **Synchrone.** Stabilité de convergence et analyse simple, mais sensible aux stragglers : le tour est cadencé sur le client le plus lent.
- **Asynchrone.** Meilleure utilisation des ressources, tolérant aux pannes et à l'intermittence, mais convergence plus délicate, la dérive étant liée au délai.
- **Hiérarchique.** Permet des agrégations locales par grappe, ce qui réduit la charge du serveur central et la latence réseau.

Le choix de l'architecture n'est pas neutre du point de vue de la confidentialité. Dans le modèle client-serveur, le serveur est un point de confiance unique : il voit chaque mise à jour individuellement, ce qui l'expose aux attaques d'empoisonnement et rend la vérification difficile. Dans le modèle pair-à-pair, aucun nœud n'est central, mais l'accord sur la valeur agrégée nécessite un mécanisme de consensus, donc un coût supplémentaire en messages. L'architecture hiérarchique offre un compromis : la confiance est délocalisée dans les agrégateurs de grappe, ce qui limite la surface d'attaque tout en conservant la simplicité opérationnelle.

$$\text{Coût par tour} = \underbrace{2\,m\,d}_{\square 1 \square} + \underbrace{2\,m\,c}_{\square 2 \square} + \underbrace{m \cdot s}_{\square 3 \square} \quad \text{octets} \tag{2.3}$$

Avec d le nombre de paramètres, $c \approx d$ pour SCAFFOLD, et s la taille de l'état de l'optimiseur local si celui-ci est transmis. Le facteur 2 correspond au aller-retour. C'est cette expression qui rend le coût mémoire serveur de SCAFFOLD visible dès la conception.

Apprentissage fédéré et algorithmes voisins

L'apprentissage fédéré ne doit pas être confondu avec trois familles de méthodes voisines, qui partagent son vocabulaire sans partager ses contraintes :

- **Apprentissage par transfert.** On transfère un modèle *entraîné* vers un nouveau domaine. Les données ne sont pas partagées, mais un artefact — le modèle — quitte l'entité d'origine : le problème n'est plus l'optimisation distribuée.
- **Apprentissage fédéré distillatif.** On échange des logits ou des représentations plutôt que des gradients, ce qui impose une architecture commune stricte mais réduit la bande passante.
- **Apprentissage scindé, split learning.** Le modèle est coupé entre client et serveur : les activations intermédiaires, volumineuses et sensibles, transitent par le réseau.

> **Confusion transfert et fédération**
> Utiliser FedAvg sur des données réparties pour éviter un transfert d'apprentissage. L'apprentissage fédéré résout un problème d'optimisation sous contrainte de localisation des données ; le transfert résout un problème de domaine. Les deux se combinent parfois, mais leurs objectifs et leurs risques de confidentialité diffèrent.

Variantes de synchronisation : du quorum à la fenêtre glissante

Entre l'extrême synchrone et l'extrême asynchrone existent des protocoles intermédiaires — ceux-là mêmes qui sont déployés en production, parce qu'ils arbitrent entre stabilité et latence.

Protocole	Règle d'intégration	Stabilité	Latence	Cas d'usage dominant
Synchrone strict	On attend $\lvert S_t \rvert$ réponses	Maximale	Égale au plus lent	Cross-silo stable
Quorum	On intègre à partir de Q le $\lvert S_t \rvert$ réponses	Élevée	Bornée par le Q-ième client	Production multi-sites
Fenêtre glissante	Toute réponse reçue dans la fenêtre est intégrée	Moyenne	Faible	Mobile, edge
Semi-asynchrone	Fenêtre, quorum et pondération par délai	Élevée	Faible	Référence actuelle
Asynchrone	Intégration immédiate à chaque arrivée	Faible à moyenne	Minimale	Capteurs, 10^5 clients et plus

Tableau 3 – Protocoles de synchronisation intermédiaires

Le semi-asynchrone est le compromis par défaut parce qu'il dégrade gracieusement : un client lent retarde le tour sans l'invalider, un client absent est simplement ignoré.

> **Astuce**
> En production, privilégier un protocole semi-asynchrone : fenêtre glissante de durée W, quorum Q, et pondération α_τ décroissante avec l'ancienneté de la mise à jour. On obtient la tolérance aux pannes de l'asynchrone sans perdre la stabilité du synchrone.

Cadre juridique et éthique

Cadre juridique et ethique

Le RGPD, règlement européen 2016/679, pose plusieurs principes directement applicables à l'apprentissage fédéré : minimisation des données (article 5), limitation des finalités, intégrité et confidentialité, ainsi que la responsabilité. L'architecture federated learning, qui n'exporte pas les données brutes, s'aligne sur le principe de protection dès la conception. Toutefois, les mises à jour de modèles ne sont pas neutres : elles peuvent, dans certains cas, fuir de l'information, par inversion de gradient. Par conséquent, l'apprentissage fédéré doit être conçu comme une mesure technique complémentaire — technologies préservant la vie privée — et non comme une garantie absolue. En santé, les exigences de traçabilité, de consentement et de sécurité renforcent la nécessité d'une approche de défense en profondeur.

Trois questions doivent être traitées explicitement dans un registre de traitements ou une analyse d'impact :

1. Les **paramètres** d'un modèle sont-ils considérés comme des données à caractère personnel lorsqu'ils sont formés, en tout ou partie, de données identifiantes ? La réponse conditionné l'ensemble du régime applicable.
2. Qui est **responsable** du traitement : l'orchestrateur central, le client donneur de données, ou les deux conjointement ? Le RGPD impose une répartition contractuelle explicite des rôles, responsable et sous-traitant.
3. Quelle est la **base légale** du traitement : consentement, exécution de contrat, intérêt légitime, ou obligation légale ? L'apprentissage fédéré n'exonère d'aucune de ces bases.

Remarque

En santé, la validation des données de santé à usage secondaire, le consentement éclairé et l'habilitation à l'accès au dossier conditionnent la faisabilité même du projet fédéré. L'apprentissage fédéré ne contourne pas le cadre : il en réduit certains coûts.

A retenir

- L'apprentissage fédéré se distingue du calcul distribué par une propriété centrale : les données restent locales.
- Les quatre hypothèses guident tous les choix algorithmiques : non-partage, agrégation, hétérogénéité, contrainte de communication.
- Le compromis synchrone-asynchrone est avant tout un compromis entre stabilité de convergence et tolérance aux pannes.
- L'alignement avec le RGPD est structurel mais nécessite des garanties complémentaires : confidentialité différentielle, agrégation sécurisée, robustesse.

Chapitre 2.2

FedAvg : formulation, implémentation et variantes

FedAvg est l'algorithme fondateur de l'apprentissage fédéré. Sa simplicité explique son adoption massive ; son comportement sous hétérogénéité explique la richesse des variantes qui ont suivi.

Problème d'optimisation fédéré

On considère K clients avec des distributions P_k potentiellement différentes. Le risque global s'écrit :

$$\min_{w \in \mathbb{R}^d} F(w) = \sum_{k=1}^{K} p_k F_k(w), \; p_k = (n_k)/(n) \geq 0, \; \sum_{k=1}^{K} p_k = 1 \tag{2.4}$$

Les poids $p_k = n_k/n$ donnent naturellement plus d'influence aux clients disposant de plus d'échantillons. D'autres choix – égaux, ou pondérés par qualité – sont possibles selon le contexte applicatif, en particulier quand l'équité prime sur la performance brute.

La difficulté n'est pas dans l'écriture de l'algorithme, mais dans le fait que l'objectif n'est pas séparable de façon triviale par les clients. Écrivons le gradient global comme une somme de termes locaux :

$$\nabla F(w) = \sum_{k=1}^{K} p_k \nabla F_k(w) = \sum_{k=1}^{K} p_k \leq ft((1)/(n_k)) \sum_{i \in \square 0 \square_k} \nabla \ell(y_i, f_w(x_i))) \quad (2.5)$$

Le gradient global est une moyenne de gradients locaux : c'est ce qui rend l'agrégation possible, et c'est ce qui rend la dérive visible quand chaque k **approxime ce gradient par** E **pas locaux.**

Trois difficultés émergent de cette décomposition. La première est la **variance** : un gradient calculé sur un mini-lot de B échantillons est un estimateur bruité de ∇F_k, et cette variance ne s'annule pas par agrégation si C est faible. La deuxième est l'**hétérogénéité** : les ∇F_k pointent dans des directions différentes, et un entraînement local long amplifie l'écart au gradient global. La troisième est le **coût** : chaque tour transfère $|S_t| \cdot d$ valeurs, ce qui impose de limiter simultanément le nombre de clients et le nombre de tours.

$$\text{Coût total} = T \cdot m \cdot d \cdot c_{\square 1 \square} \times 2 \text{ (montant et descendant)} \quad (2.6)$$

Ainsi, réduire $E \ll 1$ **– donc augmenter** T **– coûte plus cher en communication que d'augmenter** E**. C'est l'arbitrage central de tout paramétrage de FedAvg.**

FedAvg : principe et pseudo-code

FedAvg alterne entre trois temps : la diffusion de w_t aux clients sélectionnés $S_t \subseteq 1,\ldots,K$ avec une fraction $C \in (0,1]$; l'entraînement local de E époques, ou τ pas, par descente de gradient stochastique sur D_k avec un pas η, pour obtenir w_{t+1}^{k} ; et l'agrégation $w_{t+1} = \sum_{k \in S_t}(n_k)/(\sum_{j \in S_t} n_j)\, w_{t+1}^{k}$.

Algorithme 2 – FedAvg, Federated Averaging

```
Entrée : w_0, T tours, C fraction, E époques locales, B taille de lot, η
Pour t = 0, 1, ..., T-1 :
  S_t ← tirer aléatoirement m = max(CK, 1) clients
  Diffuser w_t à tous les k ∈ S_t
  Pour chaque k ∈ S_t en parallèle :
  w_{t+1}^k ← EntraînementLocal(k, w_t, E, B, η)
  w_{t+1} ← Σ_{k ∈ S_t} (n_k)/(n_{S_t}) w_{t+1}^k, avec n_{S_t} = Σ_{j ∈ S_t} n_j
Retourner w_T
```

Entrées : w_0, T, C, E, B, η • **Sorties :** w_T

En pratique, S_t est échantillonné sans remise à chaque tour. Avec remise, certains clients peuvent être tirés zéro fois, ce qui réduit silencieusement la participation effective sans que l'opérateur le voie.

Implémentation complète, PyTorch et NumPy

L'implémentation ci-dessous est réaliste et exécutable : elle distingue clairement le côté client — étapes locales avec descente de gradient stochastique — et le côté serveur — agrégation pondérée. On y inclut également un bornage du gradient pour la stabilité.

```python
import math
from typing import Dict, List, Tuple

import numpy as np
import torch
import torch.nn.functional as F
from torch import nn
from torch.optim import SGD
from torch.utils.data import DataLoader, TensorDataset


class SimpleMLP(nn.Module):
"""Perceptron multi-couches, volontairement minimaliste."""

def __init__(self, in_dim: int = 20, hidden: int = 64, num_classes: int = 3):
super().__init__()
self.net = nn.Sequential(
nn.Linear(in_dim, hidden),
nn.ReLU(),
nn.Linear(hidden, num_classes),
)

def forward(self, x):
return self.net(x)


def state_dict_to_numpy(sd: Dict[str, torch.Tensor]) -> Dict[str, np.ndarray]:
"""Convertit un etat PyTorch en tableaux NumPy, pour la serialisation."""
return {k: v.detach().cpu().numpy().astype(np.float32) for k, v in sd.items()}


def numpy_to_state_dict(params_np: Dict[str, np.ndarray],
model: nn.Module) -> None:
"""Recharge des tableaux NumPy dans un modele PyTorch."""
with torch.no_grad():
for k, v in model.state_dict().items():
if k in params_np:
v.copy_(torch.from_numpy(params_np[k]))


def federated_aggregate(client_weights_np: List[Dict[str, np.ndarray]],
client_sizes: List[int]) -> Dict[str, np.ndarray]:
"""Moyenne ponderee par la taille des jeux locaux.

C'est le coeur du cote serveur : la ponderation reproduit la definition
du risque global, ou chaque client contribue a hauteur de n_k / n.
"""
total = float(sum(client_sizes))
if total <= 0:
raise ValueError("la somme des tailles locales doit etre strictement positive")
out: Dict[str, np.ndarray] = {}
for key in client_weights_np[0]:
acc = None
for w, n in zip(client_weights_np, client_sizes):
part = w[key].astype(np.float64) * (n / total)
acc = part if acc is None else acc + part
out[key] = acc.astype(np.float32)
return out


```

Listing 1 – Implémentation complète de FedAvg, côté client et côté serveur

```python
def local_train_step(model: nn.Module,
loader: DataLoader,
epochs: int,
lr: float) -> Dict[str, np.ndarray]:
```

```
    """Entrainement local, cote client, sur le jeu de donnees du client.

    Le bornage du gradient est ici decisif : sans lui, un lot atypique
    peut deporter le modele local tres loin de w_t, ce qui aggrave la
    derive client au lieu de la reduire.
    """
    model.train()
    opt = SGD(model.parameters(), lr=lr, momentum=0.9)
    for _ in range(epochs):
        for x, y in loader:
            opt.zero_grad(set_to_none=True)
            loss = F.cross_entropy(model(x), y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=10.0)
            opt.step()
    return state_dict_to_numpy(model.state_dict())


def make_synthetic_dataloaders(K: int,
                               samples_per_client: int = 300,
                               in_dim: int = 20,
                               num_classes: int = 3,
                               seed: int = 1) -> List[DataLoader]:
    """Jeux synthetiques repartis, avec un biais local par client.

    Chaque client a un centre de classe propre : c'est une heterogeneite
    de etiquettes simple, mais suffisante pour faire apparaitre la derive.
    """
    rng = np.random.default_rng(seed)
    loaders = []
    for k in range(K):
        centres = rng.normal(size=(num_classes, in_dim)) * 2.0
        labels = rng.integers(0, num_classes, size=samples_per_client)
        x = centres[labels] + rng.normal(size=(samples_per_client, in_dim))
        y = labels.astype(np.int64)
        ds = TensorDataset(torch.from_numpy(x).float(), torch.from_numpy(y))
        loaders.append(DataLoader(ds, batch_size=32, shuffle=True))
    return loaders


def fedavg_simulation(K: int,
                      C: float,
                      T: int,
                      E: int,
                      B: int,
                      lr: float,
                      loaders: List[DataLoader]) -> nn.Module:
    """FedAvg de bout en bout sur donnees synthetiques non-IID.

    Retourne le modele global apres T tours. Completement executable :
    aucune donnee n'est centralisee, chaque client n'accede qu'a son
    propre loader.
    """
    rng = np.random.default_rng(seed=0)
    global_model = SimpleMLP(in_dim=20, num_classes=3)
    for t in range(T):
        m = max(int(math.ceil(C * K)), 1)
        selected = rng.choice(K, size=m, replace=False)
        client_weights, client_sizes = [], []
        for k in selected:
            local_model = SimpleMLP(in_dim=20, num_classes=3)
            numpy_to_state_dict(
                state_dict_to_numpy(global_model.state_dict()), local_model)
            w_k = local_train_step(local_model, loaders[int(k)], epochs=E, lr=lr)
            client_weights.append(w_k)
```

```
client_sizes.append(len(loaders[int(k)].dataset))
aggregated = federated_aggregate(client_weights, client_sizes)
numpy_to_state_dict(aggregated, global_model)
return global_model
```

```
import time

# Lancement de la simulation federatede decrite ci-dessus.
if __name__ == "__main__":
K, T, C, E = 20, 30, 0.5, 2
loaders = make_synthetic_dataloaders(K, samples_per_client=300, seed=1)
start = time.time()
final_model = fedavg_simulation(K=K, C=C, T=T, E=E, B=32, lr=0.1,
loaders=loaders)
elapsed = time.time() - start
# Verification que le modele final est bien exploitable.
final_model.eval()
with torch.no_grad():
x0, y0 = next(iter(loaders[0]))
acc = (final_model(x0).argmax(dim=1) == y0).float().mean().item()
print(f"Temps total : {elapsed:.1f}s | exactitude client 0 : {acc:.3f}")
```

Listing 2 – Piloter la simulation FedAvg, sans aucune donnée centralisée

Attention

Dans une implémentation industrielle, deux précautions sont indispensables : transmettre les tailles n_k de manière fiable — elles déterminent les poids d'agrégation, et une valeur fausse fausse tout le modèle — et gérer l'échec de clients, par dépassement de délai ou erreur mémoire, sans invalider le tour complet.

Variantes de l'optimiseur serveur, FedOpt

La famille FedOpt distingue explicitement l'optimisation locale de l'optimisation serveur. Le serveur applique un optimiseur adaptatif aux deltas agrégés, ce qui résout un problème réellement rencontré en pratique : le taux d'apprentissage local devient un hyperparamètre serveur, réglé indépendamment de l'apprentissage local.

$$\delta_t = \sum_{k \in S_t} p_k \, (w_t - w_{t+1}^{\,k})$$
$$w_{t+1} = \mathrm{OptServeur}(w_t, \delta_t) \qquad (2.7)$$

OptServeur **peut être une descente de gradient, un momentum – FedAvgM –, Adam, Yogi ou Adagrad. Le choix se fait sur la variance des deltas, pas sur l'état de l'art en optimisation centralisée.**

- **FedAdam** : β_1, β_2 comme Adam, avec $v_t = (1-\beta_2)\, v_{t-1} + \beta_2\, \delta_t^{\,2}$.
- **FedYogi** : même moment second, mais soustraction adaptative du gradient de la variance, ce qui évite la sous-estimation systématique observée avec Adam sur des deltas très variables.
- **FedAdagrad** : $a_t = a_{t-1} + \delta_t^{\,2}$, pas proportionnel à $1/\sqrt{(a_t)}$; robuste sans moment d'ordre un.

Astuce

FedAdam est particulièrement utile quand les lots locaux B sont petits : le gradient local est bruité, et l'optimiseur serveur lisse l'agrégat. Le coût mémoire serveur passe à O(3d) au lieu de O(d), ce qui reste négligeable devant la mémoire des clients.

Hyperparamètres et effets mutuels

FedAvg est piloté par quatre hyperparamètres principaux, dont les effets sont couplés — c'est ce couplage, et non leur valeur individuelle, qui explique la plupart des échecs de réglage.

Paramètre	Signification	Effet sur la convergence	Effet sur la communication	Recommandation pratique
E, époques locales	Itérations par tour sur le client	Peut accélérer, mais aggrave la dérive sous non-IID	Diminue, car le nombre de tours baisse	1 à 5 pour non-IID marqué, jusqu'à 10 pour IID
C, fraction	Proportion de clients actifs par tour	Améliore la stabilité, convergence plus lissée	Augmente, plus de mises à jour	0,1 à 0,2 en cross-device ; 0,5 à 1,0 en cross-silo
B, lot	Taille du mini-lot local	Plus grand donne un gradient plus lisse	Neutre par tour	16 à 64 sur edge, 128 à 256 en silo, avec bornage
η, pas d'apprentissage	Pas local et serveur	Critique, décroissance souvent nécessaire	Neutre	Décroissance linéaire ou exponentielle sur les T tours

Tableau 1 – Effet des hyperparamètres de FedAvg

> **Astuce**
> L'interaction E × η est déterminante sous hétérogénéité : augmenter E sans réduire η favorise la divergence entre les w_{t+1}^{k}, c'est-à-dire la dérive client. C'est la motivation première de FedProx et de SCAFFOLD.

Variantes de FedAvg

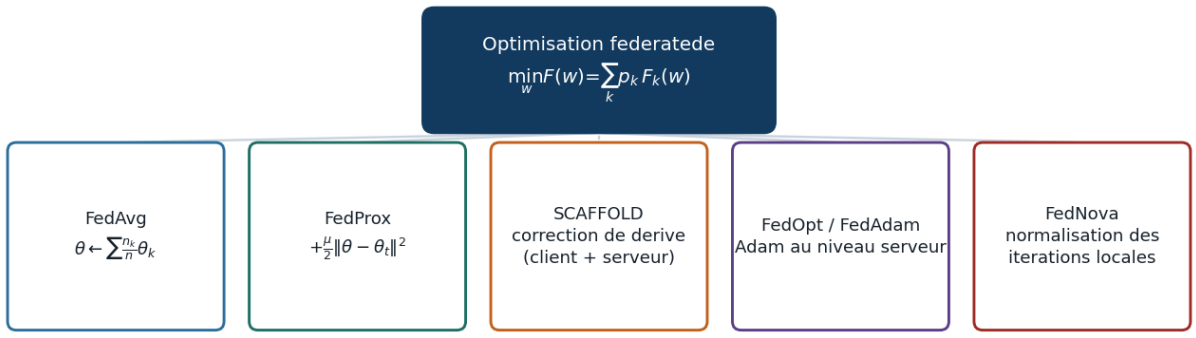


Figure 1 – Famille des optimiseurs fédérés : FedOpt, FedAdam, FedYogi, FedAvgM

Algorithme	Idée clé	Correction ou régularisation	Convergence non-IID	Coût mémoire	Cas d'usage
FedSGD	Un pas local	Aucune	Plus stable mais lent	d	Référence théorique
FedAvg	Plusieurs pas locaux	Aucune	Bonne en IID, dégradée en non-IID	d	Défaut industriel
FedAvgM	Momentum côté serveur	Inertie serveur	Améliorée	d	Tours plus stables
FedProx	Terme proximal $\mu\|\| w-w_t\|\|^2$	Force de proximité vers w_t	Bonne	d	Clients hétérogènes, retards
SCAFFOLD	Variables de contrôle c_k et c	Corrige la dérive client	Excellente, théorie solide	2d	Non-IID marqué
FedDyn	Régularisation dynamique	Alignement dynamique	Excellente	d	Convergence accélérée
FedNova	Normalisation des pas locaux	Élimine le biais de pas	Excellente	d	Hétérogénéité de calcul
FedOpt	Optimiseurs serveur adaptatifs	Momentum et variance serveur	Très bonne	2d à 3d	Stabilité, grands modèles

Tableau 2 – Comparaison des variantes principales de FedAvg

La colonne mémoire est la colonne décisive en cross-device : elle détermine ce qu'un client à faible capacité peut réellement héberger.

Exemple

Formulations synthétiques :
- FedProx : $w_{t+1}^k = \text{argmin}_w F_k(w) + (\mu)/(2)\| w - w_t\|^2$
- FedAvgM : $v_{t+1}=\beta v_t + \sum_k p_k (w_t - w_{t+1}^k)$, $w_{t+1}=w_t - \gamma v_{t+1}$
- FedOpt, cas général : un optimiseur adaptatif côté serveur est appliqué aux deltas agrégés.

Affectation des poids : taille, uniformité, qualité

Le choix des poids p_k dans l'agrégation est un choix de politique, pas un détail d'implémentation. Quatre familles sont utilisées, et elles ne répondent pas aux mêmes objectifs.

Politique	Définition p_k	Justification	Risque	Contexte typique
Par taille	$p_k = n_k/n$	Non-biais du risque empirique global	Domination des gros clients	Silo homogène
Uniforme	$p_k = 1/K$	Équité entre participants	Biais si tailles très inégales	Recherche, petites cohortes
Qualité ou fiabilité	$\propto q_k \cdot n_k$	Pondérer par la confiance ou la précision locale	Biais de sélection, risque d'exclusion	Santé, finance
Capacité de calcul	$\propto c_k^\beta$	Équilibrer la charge et l'apport	Favorise les gros calculateurs	Tous secteurs

Tableau 3 – Politiques de pondération des clients

Attention

Un client dont le jeu local est aberrant — données bruitées, étiquettes erronées — peut déformer le modèle global d'autant plus qu'il est bien pondéré. La pondération par la taille et la pondération par la qualité ne sont pas interchangeables : la première protège le risque, la seconde protège l'intégrité du modèle.

Exercice 2.1

Exercice 2.2.4 (difficulte : Moyen)

Expliquer pourquoi FedAvg peut diverger sous distribution non-IID, même avec un taux d'apprentissage petit. Proposer deux modifications simples d'hyperparamètres pour limiter ce phénomène et les justifier. Comparer ensuite FedProx et SCAFFOLD sur le critère du coût mémoire client et serveur.

Correction

- **Cause.** Chaque client minimise F_k localement, dont l'optimum $w_k^\star$ diffère. Après $E \gg 1$ pas, w_{t+1}^k s'éloigne de w_t dans des directions différentes — la dérive client. L'agrégation moyenne peut alors osciller ou diverger. Un pas petit ne suffit pas : il réduit l'amplitude du déplacement sans annuler le biais directionnel.
- **Modifications.** Réduire E à une ou deux époques pour limiter l'écart local ; diminuer η_t au cours des tours, ou passer à FedAvgM. La justification est de contrôler le produit $E\,\eta$, qui est la quantité qui détermine l'écart local.
- **Comparaison mémoire.** FedProx coûte $O(d)$ côté client : seuls les paramètres transitent, le terme proximal se calculé localement à partir de w_t reçu. SCAFFOLD coûte $O(2d)$: il stocke et échange une variable de contrôle par client, ce qui double mémoire et bande passante. Le gain sous non-IID fort est substantiel, mais il faut le comparer au nombre de tours économisé : si la réduction de tours est inférieure au facteur deux, le surcoût n'est pas justifié.

A retenir

- FedAvg est un compromis élégant entre descente de gradient distribuée et moyenne de modèles, efficace quand les données sont proches de l'IID.
- Sous hétérogénéité, la dérive client est le facteur limitant : elle se pilote par le produit (E, η).
- Les variantes — Prox, Dyn, Nova, Opt, SCAFFOLD — corrigent cette dérive par régularisation ou par variables de contrôle, à coût mémoire variable.

Chapitre 2.3

Hétérogénéité des données : diagnostic et stratégies

Dans la pratique, l'hypothèse IID est rarement vérifiée. Diagnostiquer le type d'hétérogénéité est indispensable pour choisir l'algorithme adapté et éviter des conclusions erronées sur la convergence.

Taxonomie du non-IID

On distingue classiquement plusieurs formes d'hétérogénéité entre distributions clients $P_k(x,y)$. Les confondre conduit à choisir un algorithme qui ne traité pas le problème réellement présent.

Type	Définition	Exemple concret	Impact sur l'apprentissage fédéré	Mesure indicative
Biais d'étiquettes	$P_k(y)$ différentes entre clients	Un hôpital voit surtout des pneumopathies, un autre des accidents vasculaires	Biais de classe local, dérive du classifieur	Entropie des labels, divergence de Kullback-Leibler
Dérive de caractéristique	$P_k(x)$ neq $P_j(x)$	Appareils photo différents, bruit et exposition	Normalisation par lots, généralisation de domaine	Divergence entre distributions, distance de Wasserstein
Dérive conceptuelle	$P_k(x \mid y)$ neq $P_j(x \mid y)$	Chiffres écrits dans des styles différents	Incohérence décisionnelle	Désaccord entre modèles locaux
Changement de concept	$P_k(y \mid x)$ neq $P_j(y \mid x)$	Annotations subjectives entre experts	Très difficile à traiter	Taux de désaccord entre modèles locaux
Dérive de quantité	n_k très variables	Clients très actifs ou très rares	Pondération, taux de participation	Coefficient de variation $CV(n_k)$

Tableau 1 – Taxonomie de l'hétérogénéité des données

La dérive conceptuelle est la seule réellement insoluble par les méthodes d'optimisation : elle contredit l'hypothèse même d'un modèle unique. Si on la constate, la réponse n'est pas un autre algorithme, mais la personnalisation.

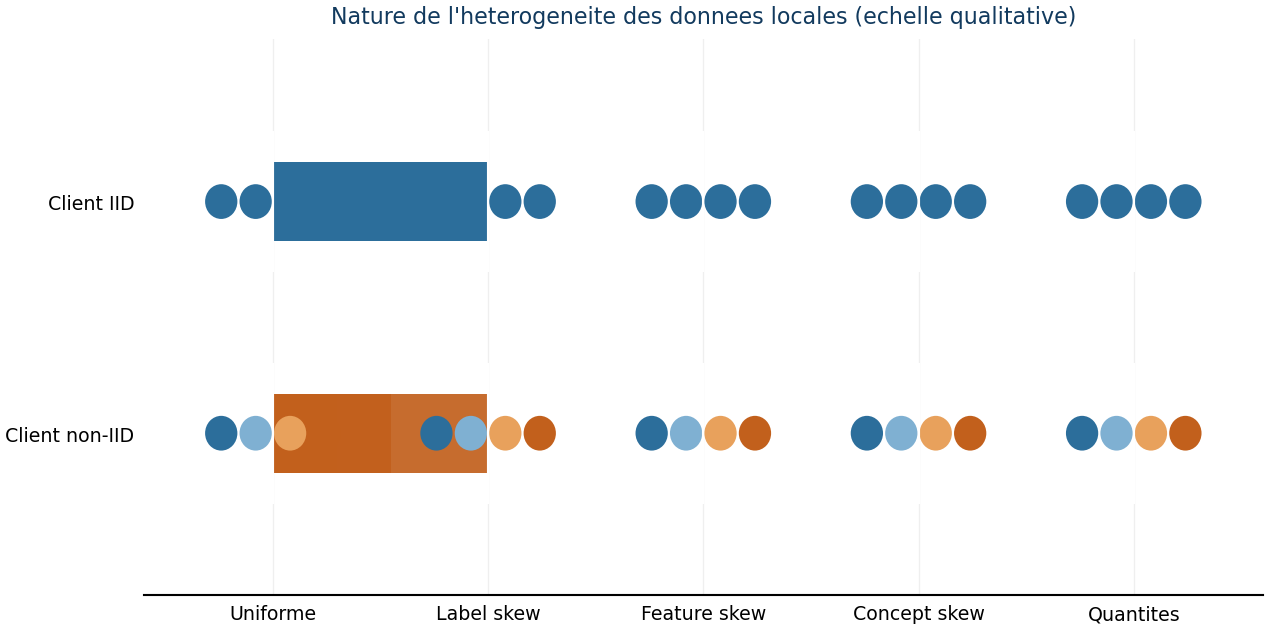


Figure 1 – Illustration des formes d'hétérogénéité des données entre clients

Quantification de la divergence entre clients

Mesurer objectivement l'hétérogénéité permet d'anticiper la dégradation de convergence, et surtout de rendre comparables deux expériences.

Definition – Divergence de Jensen-Shannon

Pour deux distributions P, Q, $D_{JS}(P\| Q) = (1)/(2)D_{KL}(P\| M) + (1)/(2)D_{KL}(Q\| M)$ avec $M = (1)/(2)(P+Q)$. Elle est symétrique et bornée dans $[0, \log 2]$, contrairement à la divergence de Kullback-Leibler.

$$D_{JS}(P_k \parallel P_j) = (1)/(2)\sum_y P_k(y)\log(2P_k(y))/(P_k(y)+P_j(y)) + (1)/(2)\sum_y P_j(y)\log(2P_j(y))/(P_k(y)+P_j(y)) \quad (2.8)$$

Utile pour quantifier la dérive des étiquettes entre paires de clients. La borne logarithmique est un avantage pratique : elle rend les valeurs comparables entre jeux de tailles différentes.

Pour des distributions continues, la distance de Wasserstein-1 est souvent plus informative, car elle compare les positions et non seulement les masses :

$$W_1(P,Q) = \inf_{\gamma \in \Gamma(P,Q)} E_{(x,x')\sim\gamma}[\| x-x' \|] = \sup_{\| f \|_{Lip} \leq 1} (E_P[f] - E_Q[f]) \quad (2.9)$$

La Wasserstein-1 se calculé en une dimension par l'intégrale de la différence des fonctions de répartition cumulées, ce qui la rend utilisable directement sur un histogramme de labels.

> **Astuce**
> En pratique, on calcule une métrique d'hétérogénéité globale, par exemple la moyenne des distances de Wasserstein sur toutes les paires de clients, calculée sur une projection de faible dimension plutôt que sur les variables brutes : cela évite la malédiction de la dimension.

Conséquences sur la convergence et la précision

- **Dérive client.** $E_k[\| w_{t+1}^k - w_t \|]$ croît avec E et avec la divergence entre F_k et F.
- **Biais d'agrégation.** La moyenne pondérée optimise $F = \sum p_k F_k$, mais peut détruire des spécialisations locales utiles, ce qui se paie en équité par client.
- **Oscillations.** Les gradients locaux pointent vers des optima différents $w_k^\star$; l'itéré global oscille autour d'un point compromis.
- **Vitesse de convergence.** Les bornes classiques en $O(1/T)$ voient leur constante multipliée par des facteurs dépendant de l'hétérogénéité de gradient.

Il faut distinguer deux conséquences qui sont souvent confondues. La première est **algorithmique** : l'itéré converge moins vite, ou oscille, ce qui se voit sur la courbe de la perte globale. La seconde est **statistique** : même à l'optimum, le modèle appris n'est pas le meilleur modèle pour un client donné. Un modèle unique impose un compromis entre les optima locaux, ce qui dégrade la précision individuelle même si la précision moyenne reste correcte.

$$\Gamma = F(w^\star) - \sum_{k=1}^{K} p_k F_k(w_k^\star) \geq 0, \quad w_k^\star = \operatorname{argmin}_w F_k(w) \quad (2.10)$$

Γ quantifie l'incohérence des optima locaux : plus il est grand, plus FedAvg est pénalisé en hétérogénéité élevée. C'est la quantité qui rend toutes les bornes de convergence comparables entre elles.

Concrètement, sous fort non-IID avec E important, on observe trois symptômes reconnaissables sur les courbes : la perte globale présente un plateau oscillant plutôt qu'une décroissance régulière ; l'écart entre la précision moyenne et la précision par client se creuse ; et la variance entre tours augmente proportionnellement à E. Ces trois signes combinés constituent le diagnostic pratique d'une dérive client non corrigée.

Code : diagnostic quantitatif sur données réelles

Le diagnostic se fait avant tout choix d'algorithme. Le listing suivant produit la matrice des divergences, les entropies marginales et un indice de non-IID global.

```python
from typing import Dict, List

import numpy as np


def label_histogram(labels: np.ndarray, num_classes: int) -> np.ndarray:
"""Histogramme normalise des labels d'un client."""
counts = np.bincount(labels.astype(int), minlength=num_classes).astype(float)
return counts / max(counts.sum(), 1e-12)


def js_divergence(p: np.ndarray, q: np.ndarray, eps: float = 1e-12) -> float:
"""Divergence de Jensen-Shannon, symetrique et bornee dans [0, log 2]."""
m = 0.5 * (p + q)
dkl_pm = np.sum(p * np.log((p + eps) / (m + eps)))
dkl_qm = np.sum(q * np.log((q + eps) / (m + eps)))
return float(0.5 * (dkl_pm + dkl_qm))


def wasserstein1_1d(p: np.ndarray, q: np.ndarray) -> float:
"""Distance de Wasserstein-1 entre deux histogrammes de meme support.

En dimension 1, W1 est l'integrale de la difference des fonctions de
repartition cumulees.
"""
cdf_p = np.cumsum(p)
cdf_q = np.cumsum(q)
return float(np.sum(np.abs(cdf_p - cdf_q)))


def diagnose_non_iid(client_labels: List[np.ndarray],
num_classes: int) -> Dict[str, float]:
"""Tableau de bord d'heterogeneite inter-clients.

Retourne :
- entropie moyenne des labels, basse signifiant une concentration ;
- divergence de Jensen-Shannon moyenne sur toutes les paires ;
- Wasserstein-1 moyenne sur toutes les paires ;
- coefficient de variation des tailles de jeux locaux.
"""
K = len(client_labels)
hists = [label_histogram(y, num_classes) for y in client_labels]
sizes = np.array([len(y) for y in client_labels], dtype=float)

entropies = [-np.sum(h * np.log(h + 1e-12)) for h in hists]
js_vals, w1_vals = [], []
for i in range(K):
for j in range(i + 1, K):
js_vals.append(js_divergence(hists[i], hists[j]))
w1_vals.append(wasserstein1_1d(hists[i], hists[j]))

pairs = max(K * (K - 1) / 2, 1)
cv_sizes = float(sizes.std() / max(sizes.mean(), 1e-12))
return {
"entropie_moyenne": float(np.mean(entropies)),
"entropie_max": float(np.log(num_classes)),
"js_moyen": float(np.sum(js_vals) / pairs),
"js_max": float(np.max(js_vals)) if js_vals else 0.0,
"w1_moyen": float(np.sum(w1_vals) / pairs),
"cv_tailles": cv_sizes,
}
```

Listing 1 — Diagnostic quantitatif de l'hétérogénéité : Jensen-Shannon, Wasserstein, entropie, coefficient de variation

Astuce

Sur un jeu parfaitement homogène, la divergence de Jensen-Shannon moyenne est de l'ordre de 10^{-3}. Au-delà de 0.3 en moyenne, considérer le non-IID comme sévère. Comparer toujours ces valeurs à une référence IID construite artificiellement, plutôt qu'à un seuil absolu : le seuil dépend du nombre de classes et de la taille des clients.

Stratégies pour atténuer l'hétérogénéité

Approche	Algorithmes	Principe	Avantages	Inconvénients
Régularisation proximale	FedProx	Ajouter $(\mu)/(2)\|\| w\text{-}w_t\|\|^2$ localement	Simple, stable, O(d) de mémoire	Choix de μ sensible
Correction de dérive	SCAFFOLD	Variables de contrôle (c_k, c)	Théorie forte, efficace en non-IID	Double la mémoire et la communication
Régularisation dynamique	FedDyn	Terme dynamique d'alignement	Bonne convergence	O(d), paramétrage fin
Normalisation des pas	FedNova	Normaliser par nombre effectif de pas	Robuste à l'hétérogénéité de calcul	O(d)
Normalisation par lots	FedBN	Conserver les statistiques locales	Efficace sur la dérive de caractéristique	Ne s'applique pas à toutes les couches
Personnalisation	FedPer, FedRep, pFedMe	Socle partagé, têtes personnalisées	Meilleure précision par client	Renonce au modèle global unique
Regroupement de clients	Clustered FL	Regrouper les clients similaires	Réduit l'hétérogénéité intra-groupe	Surcoût de maintenance du regroupement

Tableau 2 – Stratégies face à l'hétérogénéité des données

Ces stratégies ne s'excluent pas. Le regroupement de clients, en particulier, se combine avec SCAFFOLD : on réduit l'hétérogénéité dans chaque groupe, puis on corrigé la dérive résiduelle à l'intérieur.

Partitionnement non-IID par Dirichlet

Pour reproduire de manière contrôlable la dérive des étiquettes, le partitionnement de Dirichlet est largement utilisé dans la littérature fédérée. Le paramètre α contrôle le degré : petit α donne un fort non-IID.

```python
from typing import Dict, List

import numpy as np


def dirichlet_partition(labels: np.ndarray,
K: int,
alpha: float = 0.5,
seed: int = 42) -> List[np.ndarray]:
"""Partitionne des indices par classe selon une loi de Dirichlet(alpha).

Args:
labels: Vecteur d'etiquettes y_i pour chaque echantillon, de 0 a C-1.
K: Nombre de clients.
alpha: Concentration de Dirichlet. Un alpha petit donne un fort non-IID.
seed: Graine aleatoire.

Returns:
Liste de tableaux d'indices affectes à chaque client k.
"""
rng = np.random.default_rng(seed)
unique_classes = np.unique(labels)
K = int(K)
idx_by_class = {c: np.where(labels == c)[0] for c in unique_classes}

client_indices: List[List[int]] = [[] for _ in range(K)]
for c in unique_classes:
idx_c = idx_by_class[c]
n_c = len(idx_c)
if n_c == 0:
continue
# Proportions par client pour cette classe : Dirichlet(alpha, ..., alpha)
proportions = rng.dirichlet([alpha] * K)
counts = (proportions * n_c).astype(int)
# Ajustement pour que la somme des comptes soit exactement n_c
remainder = n_c - int(counts.sum())
if remainder > 0:
add_to = rng.choice(K, size=remainder, replace=False)
counts[add_to] += 1
elif remainder < 0:
remove_from = rng.choice(K, size=-remainder, replace=False)
counts[remove_from] = np.maximum(0, counts[remove_from] - 1)
# Repartition des indices melanges
perm = rng.permutation(idx_c)
ptr = 0
for k in range(K):
ck = int(counts[k])
if ck > 0:
client_indices[k].extend(perm[ptr:ptr + ck].tolist())
ptr += ck
return [np.array(cid, dtype=int) for cid in client_indices]


```

Listing 2 – Partitionnement non-IID par Dirichlet et métrique de divergence moyenne

```python
def heterogeneity_metrics(label_distributions: List[np.ndarray]) -> Dict[str, float]:
"""Metriques simples d'heterogeneite, pour la derive des etiquettes."""
K = len(label_distributions)
js_sum = 0.0
count_pairs = 0
eps = 1e-12
for i in range(K):
p = label_distributions[i].astype(float)
p = p / (p.sum() + eps)
for j in range(i + 1, K):
q = label_distributions[j].astype(float)
```

```
q = q / (q.sum() + eps)
m = 0.5 * (p + q)
js_sum += float(0.5 * (np.sum(p * np.log((p + eps) / (m + eps)))
+ np.sum(q * np.log((q + eps) / (m + eps)))))
count_pairs += 1
return {"js_moyen": js_sum / count_pairs if count_pairs else 0.0,
"paires": count_pairs}
```

Diagnostic pratique, en cinq temps

- **Quantifier** : calculer $P_k(y)$, le coefficient de variation des n_k, et la divergence de Jensen-Shannon moyenne entre clients.

2. **Visualiser** : cartographier les distributions d'étiquettes par client.
3. **Identifier le type** : étiquette, caractéristique, concept, ou quantité.
4. **Adapter l'algorithme** : si la divergence dépasse 0.3, privilégier SCAFFOLD, FedDyn ou FedProx, avec E réduit.
5. **Valider** : comparer FedAvg et sa variante sur la même répartition non-IID, avec la même graine.

A retenir

- L'hétérogénéité n'est pas monolithique : il faut identifier le type, étiquette, caractéristique, concept ou quantité.
- La quantification par la divergence de Jensen-Shannon et la distance de Wasserstein rend les comparaisons reproductibles.
- Les stratégies correctrices ciblent la dérive client ; la personnalisation traité le cas où un modèle global unique est sous-optimal.
- Le partitionnement de Dirichlet permet de contrôler le degré de non-IID de manière expérimentale.

Chapitre 2.4

Théorie de la convergence des algorithmes fédérés

L'efficacité empirique doit être étayée par des garanties théoriques. Ce chapitre présente les bornes de convergence, les mécanismes de correction de la dérive, et les aspects liés à l'asynchronie et à la communication.

Hypothèses et état de l'art

Les analyses classiques supposent généralement les conditions suivantes.

- **L-lissitude.** $\|\nabla F_k(u) - \nabla F_k(v)\| \le L \|u-v\|$ pour tout u, v et k.
- **μ-convexité forte, facultative.** $F_k(u) \ge F_k(v) + \square \nabla F_k(v), u-v \square + (\mu)/(2)\| u-v\|^2$, avec $\mu \ge 0$.
- **Borne sur la variance du gradient local.** $E_{i\sim \square 0 \square _k}[\|\nabla \ell_i(w) - \nabla F_k(w)\|^2] \le \sigma_k{}^2$.
- **Hétérogénéité des gradients, non-IID.** $(1)/(K)\sum_k \|\nabla F_k(w) - \nabla F(w)\|^2 \le G^2$, ou de façon équivalente via $\Gamma = F(w^\star) - \sum_k p_k F_k(w_k{}^{star})$.
- **Échantillonnage des clients.** S_t uniforme et indépendant, avec ou sans remplacement — hypothèses standards.

$$E[F(\bar{w}_T)] - F(w^\star) \le O\le ft((L)/(\mu\, T)\le ft(1 + (\sigma^2)/(E\ B\ n_{\square 0 \square}) + E^2\ G^2)) \quad (2.11)$$

Forme qualitative représentative, dépendant des hypothèses exactes. Le terme $E^2\ G^2$ illustre l'effet de la dérive sous hétérogénéité : c'est le seul terme que l'algorithme peut réduire, et son accroissance en E^2 explique pourquoi réduire le nombre d'époques locales est le premier levier.

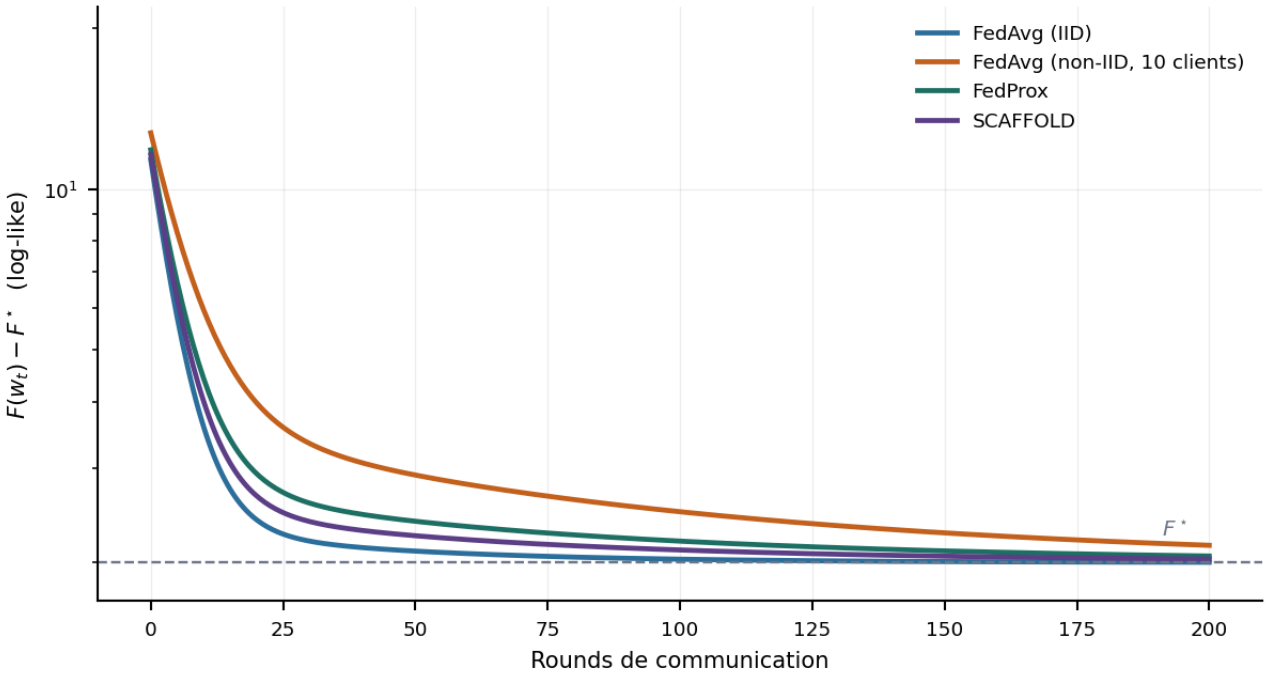


Figure 1 – Convergence théorique : effet de l'hétérogénéité et de E sur la vitesse

SCAFFOLD : correction de dérive

SCAFFOLD, Stochastic Controlled Averaging, introduit des variables de contrôle pour compenser la différence entre gradient local et gradient global. L'idée : chaque client k maintient $c_k \approx \nabla F_k(w^\star)$, le serveur maintient $c \approx \nabla F(w^\star) = \sum_k p_k c_k$.

Algorithme 3 – SCAFFOLD, version client-serveur simplifiée

```
Initialisation : w_0, c_0^k = 0, c_0 = 0
Pour t = 0, 1, ..., T-1 :
  S_t ← sélectionner les clients, diffuser (w_t, c_t)
  Pour chaque k ∈ S_t :
  w ← w_t
  Pour e = 0, 1, ..., E-1 :
  w ← w - η (∇ F_k(w; B) - c_k + c_t)
  Δ w_k = w - w_t
  Δ c_k = (1)/(E η)(w_{t+1}^k - w_t), estimation du gradient local
  Envoyer (Δ w_k, Δ c_k)
  w_{t+1} = w_t + γ_g Σ_{k∈ S_t}(n_k)/(n_{S_t})Δ w_k
  c_{t+1} = c_t + Σ_{k∈ S_t}(n_k)/(n_{S_t})Δ c_k
```

Entrées : w_0, c_0 • **Sorties :** w_T

Le terme de correction $-c_k + c_t$ recentre le gradient local vers la direction globale, annulant asymptotiquement la dérive. C'est la seule correction de la famille qui ne modifie pas l'objectif local, mais l'estimateur utilisé.

$$g_k(w; B) = \nabla F_k(w; B) - c_k + c_t,\ E_B[g_k(w_t; B) \mid w_t] \approx \nabla F(w_t) \tag{2.12}$$

Le gradient corrigé est non biaisé vers le gradient global en moyenne, dans l'idéal asymptotique où les variables de contrôle approchent les gradients à l'optimum. En pratique elles n'y arrivent jamais tout à fait, ce qui laisse une dérive résiduelle.

Attention

Coût mémoire doublé : SCAFFOLD stocke et échange $(w, c) \in \mathbb{R}^{2d}$. Sur des modèles de plusieurs dizaines de millions de paramètres, cela représente environ deux fois la bande passante. À évaluer par rapport au gain en nombre de tours sous fort non-IID — c'est le seul critère qui décide.

FedDyn, FedProx et FedNova : formulation

FedProx utilise une régularisation proximale pour contraindre les pas locaux, sans modifier l'estimateur :

$$w_{t+1}^{k} = \operatorname{argmin}_{w} \leq ft(F_k(w) + (\mu)/(2)\| w - w_t\|^2), \mu \geq 0 \quad (2.13)$$

Le terme proximal agit comme un ressort tirant le modèle local vers w_t. Sa force est un compromis : trop faible, il ne corrige rien ; trop fort, il empêche tout apprentissage local et le modèle global stagne.

FedDyn, Dynamic Regularization, introduit un terme dynamique qui dépend de l'historique du client, pour aligner progressivement les optima locaux :

$$F_k(w) + \square\, \lambda_k^{t}, w\square + (\alpha)/(2)\| w - w_t\|^2, \lambda_k^{t+1} = \lambda_k^{t} + \alpha\, (w_{t+1}^{k} - w_t) \quad (2.14)$$

L'alignement dynamique visé à neutraliser le biais d'agrégation. Le vecteur λ_k mémorise la dérive passée du client, ce qui le distingue de FedProx par le fait qu'il ne se réinitialise jamais.

FedNova corrigé le biais induit par un nombre différent d'étapes locales ou par des tailles de lot différentes entre clients — l'hétérogénéité de calcul :

$$w_{t+1} = w_t - \eta_g \cdot (\sum_{k\in S_t} p_k \tau_k d_{k,t})/(\sum_{k\in S_t} p_k \tau_k), d_{k,t} = (w_t - w_{t+1}^{k})/(\eta \tau_k) \quad (2.15)$$

La normalisation par le nombre effectif de pas τ_k évite de surpondérer les clients effectuant plus d'itérations. Sans elle, un client rapide est doublement favorisé : par la taille de son jeu et par son nombre de pas.

Algorithme	Hypothèses minimales	Taux de convergence	Robustesse non-IID	Mémoire client	Stabilité asynchrone
FedAvg	L-lisse, convexe	O(1/(μ T)) sous conditions	Moyenne	d	Faible
FedProx	L-lisse	Similaire, avec μ	Bonne	d	Moyenne
SCAFFOLD	L-lisse	O(1/T) avec bruit, amélioré	Excellente	2d	Moyenne
FedDyn	L-lisse	Amélioré par rapport à FedAvg	Excellente	d	À étudier
FedNova	L-lisse	Robuste aux pas hétérogènes	Excellente	d	Intéressante
FedAsync	L-lisse, délai borné	Dégrade avec le délai maximal	Variable	d	Bonne par construction

Tableau 1 – Comparaison théorique et pratique

La colonne mémoire client est celle qui décide en cross-device. Tout le reste de la table ne compte que si l'algorithme est exécutable sur le client.

Asynchronie, délais et stragglers

Definition – Mode asynchrone

Dans FedAsync, dès que le serveur reçoit une mise à jour d'un client k arrivée à l'instant t', il met à jour immédiatement le modèle global avec un poids qui décroît généralement avec le délai $\tau = t - t' \geq 0$: $w_{t+1} = (1-\alpha_\tau)\, w_t + \alpha_\tau\, w_{t'}^{k}$.

$$\alpha_\tau = \alpha_0 \cdot (1)/(1 + \lambda \tau), \alpha_0 \in (0,1), \lambda > 0 \quad (2.16)$$

Pondération en décroissance pour atténuer l'effet des mises à jour anciennes sous fort délai. La forme est hyperbolique ; d'autres formes sont possibles, et le choix se fait empiriquement.

Astuce

L'asynchrone maximise le débit — pas d'attente sur les stragglers — mais la borne de convergence se dégrade avec le délai maximal τ_{max}. Le mode semi-asynchrone est souvent le meilleur compromis en pratique : il borne le délai sans exiger la présence de tous les clients.

Communication : compression, quantification, sparsification

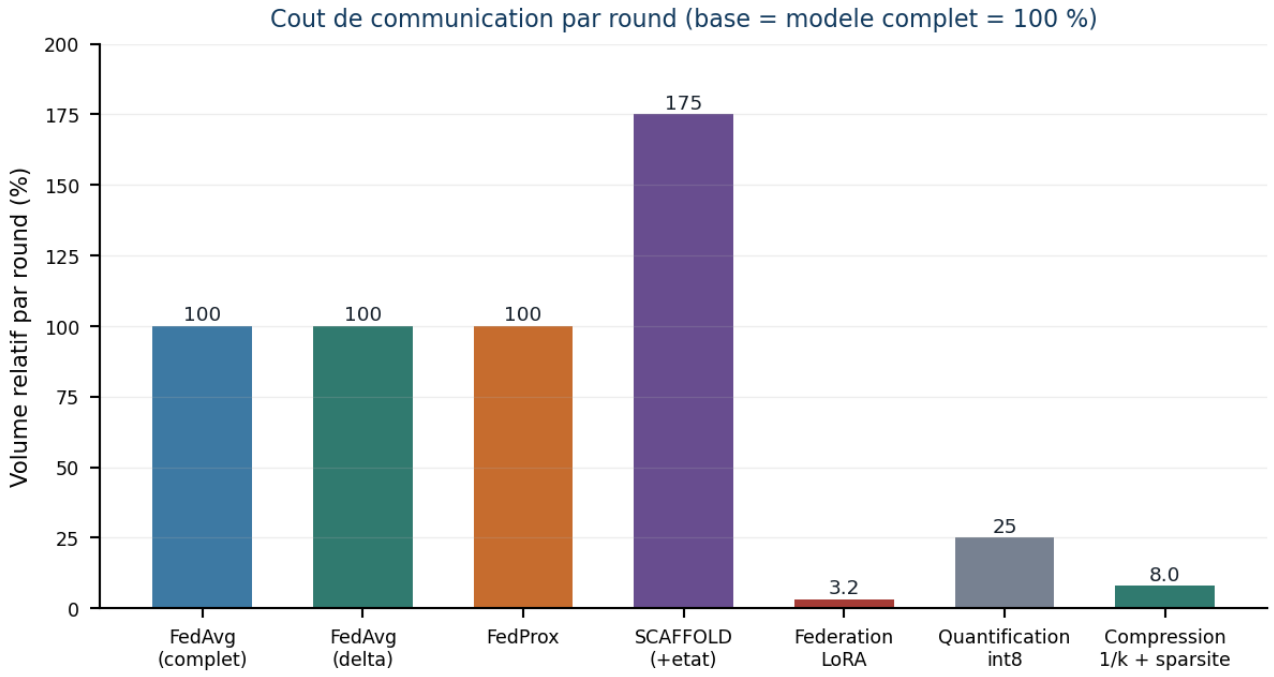


Figure 2 – Coût de communication : compression par quantification ou sparsification, contre précision et nombre de tours

Technique	Principe	Taux de compression	Biais introduit	Surcoût de calcul	Remarque
Quantification	Arrondir les mises à jour sur b bits	2 à 8 fois	Ajoute un bruit stochastique	Faible	Efficace et théorisée
Sparsification Top-k	Ne transmettre que les k plus grandes composantes	10 à 100 fois	Sélection biaisée, à corriger	Faible à moyen	Populaire, avec rétroaction d'erreur
Rétroaction d'erreur	Accumuler le résidu non transmis	Fort	Réduit le biais	Faible	Indispensable avec Top-k
SignSGD	Ne transmettre que le signe	32 fois	Biais fort	Très faible	Robuste, théorie récente
Faible rang	Projeter dans un sous-espace	5 à 20 fois	Approximation	Moyen	Bien adapté aux grands modèles
Distillation	Échanger des logits, non des paramètres	Variable	Dépend de l'architecture	Moyen	Cas des architectures hétérogènes

Tableau 2 – Techniques de réduction du coût de communication

Exemple

Principe de la quantification stochastique, version élémentaire : pour un vecteur v, $Q_b(v_i) = \|v\| \cdot \mathrm{sgn}(v_i) \cdot q_i$ avec $q_i \in 0, \ldots, 2^{b-1}$ tiré aléatoirement. Le taux de compression atteint est d'environ 32/b.

Implémentation de référence : SCAFFOLD et FedProx en PyTorch

Le listing suivant matérialise le gradient corrigé de SCAFFOLD et le terme proximal de FedProx dans un même squelette, ce qui permet de comparer les deux sur exactement le même protocole expérimental — seule la ligne de gradient diffère.

```python
from typing import List, Tuple

import torch
import torch.nn.functional as F
from torch import nn
from torch.optim import SGD


def fedprox_local_step(model: nn.Module,
w_global: List[torch.Tensor],
batches,
epochs: int,
lr: float,
mu: float) -> List[torch.Tensor]:
"""Un tour local FedProx : minimise F_k(w) + (mu/2)*||w - w_t||^2."""
params = [p for p in model.parameters() if p.requires_grad]
anchors = [g.detach().clone() for g in w_global]
opt = SGD(params, lr=lr)
model.train()
for _ in range(epochs):
for x, y in batches:
opt.zero_grad(set_to_none=True)
loss = F.cross_entropy(model(x), y)
# Terme proximal : (mu/2) * somme_i ||w_i - w_t,i||^2
prox = torch.zeros((), device=loss.device)
for p, a in zip(params, anchors):
prox = prox + ((p - a) ** 2).sum()
(loss + 0.5 * mu * prox).backward()
opt.step()
return [p.detach().clone() for p in params]


```

Listing 1 — FedProx et SCAFFOLD, côté client et côté serveur, comparables expérimentalement

```python
def scaffold_local_step(model: nn.Module,
c_k: List[torch.Tensor],
c_global: List[torch.Tensor],
batches,
epochs: int,
lr: float) -> Tuple[List[torch.Tensor], List[torch.Tensor]]:
"""Un tour local SCAFFOLD.

Le gradient utilise est g(w) - c_k + c_global. Retourne les nouveaux
parametres et la mise a jour des variables de controle.
"""
params = [p for p in model.parameters() if p.requires_grad]
# Copie de reference des parametres au debut du tour
w_start = [p.detach().clone() for p in params]
opt = SGD(params, lr=lr)
model.train()
for _ in range(epochs):
for x, y in batches:
opt.zero_grad(set_to_none=True)
loss = F.cross_entropy(model(x), y)
loss.backward()
# Application de la correction SCAFFOLD sur chaque gradient
with torch.no_grad():
for p, ck, cg in zip(params, c_k, c_global):
if p.grad is not None:
p.grad.add_(cg - ck)
opt.step()
w_end = [p.detach().clone() for p in params]
# Mise a jour de la variable de controle locale, estimate du gradient
# a l'optimum, estime par difference des poids
c_new = [ck + (w0 - w1) / max(1, lr * epochs)
for ck, w0, w1 in zip(c_k, w_start, w_end)]
```

```
return w_end, c_new


def scaffold_round(w_global: List[torch.Tensor],
c_global: List[torch.Tensor],
client_states: List[Tuple[nn.Module, List[torch.Tensor], object]],
epochs: int,
lr: float) -> Tuple[List[torch.Tensor], List[torch.Tensor]]:
"""Tour serveur SCAFFOLD : aggregation des deltas et des controles."""
w_updates, c_updates = [], []
for model, c_k, batches in client_states:
with torch.no_grad():
trainable = [p for p in model.parameters() if p.requires_grad]
for p, w in zip(trainable, w_global):
p.copy_(w)
w_k, c_k_new = scaffold_local_step(model, c_k, c_global, batches,
epochs, lr)
w_updates.append([w0 - w1 for w0, w1 in zip(w_global, w_k)])
c_updates.append([c0 - c1 for c0, c1 in zip(c_k, c_k_new)])
m = len(w_updates)
w_new = [wg - (1.0 / m) * sum(u[i] for u in w_updates)
for i, wg in enumerate(w_global)]
c_new = [cg + (1.0 / m) * sum(u[i] for u in c_updates)
for i, cg in enumerate(c_global)]
return w_new, c_new
```

Attention

Dans `scaffold_local_step`, la mise à jour de c_k par différence des poids est une approximation usuelle : la version exacte recalcule le gradient plein sur D_k en fin de tour. L'approximation est peu coûteuse mais biaisée ; il ne faut pas l'utiliser dans un travail de recherche qui prétend mesurer la convergence théorique.

Exercice 2.4.5 (difficulte : Difficile)

Démontrer pourquoi le terme $E^2 G^2$ apparaît dans la borne 2.11 sous non-IID. Expliquer ensuite le rôle exact de $c_k - c_t$ dans SCAFFOLD, et pourquoi la correction supprime le biais systématique du drift.

Correction

- **Dérive.** Après E pas locaux depuis w_t, on a $w_{t+1}^k \approx w_t - \eta E \nabla F_k(w_t)$. L'écart moyen par rapport au modèle global vérifie $E_k[\| w_{t+1}^k - (w_t - \eta E \nabla F(w_t))\|] \propto \eta E \cdot E_k[\| \nabla F_k - \nabla F\|] \le \eta E G$. Au carré, et après agrégation sur T tours, cela donne un terme en $O(E^2 G^2 / T)$. La dépendance en E^2 est donc structurelle, et non un artefact d'analyse.
- **Correction.** Localement on utilise $\nabla F_k(w) - c_k + c_t$. À l'optimum $w^\star$, si $c_k \to \nabla F_k(w^\star)$ et $c_t \to \nabla F(w^\star)$, la direction devient $\nabla F_k(w^\star) - \nabla F_k(w^\star) + \nabla F(w^\star) = \nabla F(w^\star)$: on recentre chaque client sur le gradient global, ce qui supprime le biais systématique. C'est ce qui explique le nom de *controlled* averaging : les variables de contrôle imposent la direction.

Exercice 2.4.6 (difficulte : Moyen)

FedNova normalise par le nombre effectif de pas τ_k. Expliquer un cas où cette normalisation évite un biais d'agrégation. Proposer ensuite un compromis entre FedSync et FedAsync, et le justifier sur les axes latence et stabilité.

Correction

- **Cas de biais.** Le client A effectue $\tau_A = 10$ pas, parce que son lot est petit et sa machine puissante ; le client B en effectue $\tau_B = 2$, parce qu'il est lent ou contraint. Sans normalisation, A pousse beaucoup plus loin que B, et l'agrégation moyenne est biaisée vers A. FedNova pondère par τ_k dans le delta normalisé, de sorte que chaque client contribue à hauteur de son travail effectif réel.

- **Compromis hybride.** Un mode semi-asynchrone : attendre un quorum $Q < |S_t|$, par exemple 70 % des attendus, puis intégrer les arrivants dans une fenêtre glissante avec décroissance α_τ. Le gain sur le synchrone est une latence de tour réduite, bornée par le Q-ième client et non par le plus lent. Le gain sur l'asynchrone est un τ_{max} borné, donc une borne de convergence qui ne se dégrade pas avec les délais extrêmes. C'est le compromis par défaut en production.

A retenir

- Sous hypothèses standard, FedAvg converge en $O(1/T)$; la constante dépend fortement de E^2 G^2.
- SCAFFOLD corrigé la dérive par variables de contrôle, au prix d'un coût $2d$; FedProx, FedDyn et FedNova offrent des alternatives à $O(d)$.
- L'asynchronie améliore le débit mais se dégrade avec le délai maximal : les fenêtres glissantes et les quorums sont préférables en pratique.
- La compression par sparsification avec rétroaction d'erreur permet des gains de dix à cinquante fois sur la bande passante, à perte de précision faible si elle est bien calibrée.

Chapitre 2.5

Systèmes distribués et passage à l'échelle

Un algorithme correct sur le papier peut échouer à l'échelle. Ce chapitre aborde l'orchestration, le choix de framework, la stratégie de participation, le passage à 10^6 clients et la mise en œuvre concrète d'un worker.

Orchestration fédérée

Un système d'apprentissage fédéré industriel doit gérer le cycle de vie complet des tours. La distinction fondamentale est celle du **job** et du **worker** : le job est l'état logique — modèle, hyperparamètres, règles de sélection et d'agrégation, critères d'arrêt ; le worker est l'agent d'exécution qui réalise une part de ce job. Un worker est jetable et reproductible : il doit pouvoir mourir et être remplacé sans que le job n'en soufre.

- **Jobs, ou tâches.** Définition d'un job fédéré : modèle, hyperparamètres, sélection, agrégation, critères d'arrêt, version du protocole.
- **Workers et clients.** Inscription, authentification, capacités annoncées — calcul, mémoire, bande passante, énergie — et état : disponible, occupé, en refroidissement.
- **Cycles de vie.** Initialisation, diffusion, entraînement local, collecte des mises à jour, agrégation, checkpoint, terminaison.
- **Reprise sur panne.** Checkpoints serveur de l'état complet — modèle, contrôleurs, index de tour, graine aléatoire —, relance des clients en échec, délais maximaux adaptatifs, idempotence de l'agrégation.
- **Supervision.** Métriques de tour : durée, taux de succès, dispersion des délais. Métriques de qualité : précision globale et par client. Métriques système : énergie, bande passante, taux de rejet.

Algorithme 4 – Cycle de vie d'un tour fédéré, vue orchestrateur

```
01  État persistant : (w_t, c_t, t, graine, statistiques)
02  Sélectionner S_t selon la stratégie de participation et le quota disponible
03  Diffuser la tâche : version du modèle, index du tour, (E, B, η), clé de chiffrement éventuelle
04  Attendre jusqu'au quorum Q ou la fin de la fenêtre glissante W
05  Pour chaque réponse reçue (k, Δ w, Δ c, n_k) :
06   Vérifier l'identité, la version et la somme de contrôle du lot
07   Accumuler dans l'agrégateur partiel, sur le shard h(k)
08  Si le quorum est atteint :
09   w_{t+1} ← OptServeur≤ft(w_t, Σ_k p_k Δ w_k), mettre à jour les contrôleurs
10   Évaluer la métrique d'arrêt ; sinon incrémenter t et recommencer
11   Sauvegarder le checkpoint de l'état persistant
```

```
12  Sinon : journaliser l'échec, ne pas incrémenter t, rejouer au tour suivant
```

Entrées : (w_t, c_t, t), stratégie de sélection, quorum Q, fenêtre W • **Sorties :** w_{t+1}, c_{t+1} • **Complexité :** $O(m \cdot d)$ par tour, $O(K)$ d'état serveur

Le point critique est la branche « sinon » : ne jamais incrémenter t si le quorum n'est pas atteint, sinon la séquence d'itérations devient irreproductible. C'est la règle qui distingue un orchestrateur correct d'un orchestrateur qui « marche en général ».

Astuce

Écrire le tour comme une transaction : soit l'état avance d'un cran, soit il reste strictement identique. Cette règle simple élimine la majorité des bugs d'orchestration observés en production, en particulier les états où certains clients ont déjà consommé leur crédit du tour.

Critère	Flower	FedML	FedScale	NVIDIA FLARE	FATE	PySyft, OpenFL
Langage	Python, flexible	Python, orienté MLOps	Python, orienté benchmark	Python, entreprise	Python et Java	Python
Architecture	Client-serveur, agnostique	Cloud-edge, lanceur	Simulation et réel	Hiérarchique, sécurisé	Fédération distribuée	Confidentialité d'abord
Simulation	Excellente	Excellente	Référence	Bonne	Bonne	Bonne
Déploiement	Edge, mobile, cloud	Kubernetes, cloud	Grappe de calcul	Entreprise, sur site	Production	Recherche
Stratégies	Riches : FedAvg, FedProx	Variantes et renforcement	Large corpus	FedAvg et agrégation sécurisée	Agrégation sécurisée	Différentielle et MPC
Confidentialité	Différentielle, agrégation sécurisée	Idem	Différentielle	Agrégation sécurisée, attestation	Chiffrement homomorphe, MPC	Différentielle, MPC
Scalabilité visée	10^5	10^5	10^6	10^4 à 10^5	10^3 à 10^4	10^3
Maturité	Stable, communauté large	Stable	Recherche et benchmark	Entreprise	Entreprise	Recherche

Tableau 1 – Frameworks fédérés : comparaison d'architecture

Le choix se fait sur un seul axe en pratique : benchmark et recherche, ou déploiement d'entreprise. Flower est le plus polyvalent ; FLARE et FATE gagnent dès que la conformité prime sur la souplesse.

Stratégies de participation client

La sélection des clients influence la convergence, l'équité, la latence et le coût énergétique. Ce n'est pas un détail d'implémentation : c'est un paramètre de la méthode au même titre que le taux d'apprentissage.

Stratégie	Mode de sélection	Objectif	Avantages	Risques
Aléatoire	S_t uniforme sur K	Équité en moyenne	Simple, analyse théorique	Peut stagner
Probabiliste pondérée	Probabilité $q_k \propto p_k / r_k$	Réduire le biais, accélérer	Flexible	Nécessite d'estimer la disponibilité r_k
Par fenêtre glissante	Sélection dans la fenêtre active	Stabiliser les tours	Bon en cross-device	Peut exclure les clients rares
Par ressources	Prioriser les clients rapides et stables	Réduire la durée de tour	Efficace en latence	Risque de biais systématique
Adaptative	Poids $\propto \|\nabla F_k\|^2$	Accélérer la convergence	Optimal en théorie	Communication supplémentaire pour estimer les normes
Équité contrainte	Contrainte de sélection minimale par client	Équité de participation	Évite l'exclusion	Peut ralentir la convergence
Par grappe	Sélectionner une ou plusieurs grappes	Gérer l'hétérogénéité	Réduit la dérive intra-groupe	Surcoût de regroupement dynamique

Tableau 2 – Stratégies de participation

> **Astuce**
> En cross-device, privilégier une sélection probabiliste fondée sur l'historique de disponibilité, combinée à un quorum et à des délais maximaux adaptatifs. En cross-silo, une sélection quasi totale, $C \approx 1$, est souvent réalisable car les clients sont sous contrôle administratif.

Au-delà de la sélection, la notion de **crédit** mérite d'être précisée. Un crédit est l'unité de travail accordée à un client pour un tour donné. Il borne la consommation de batterie et de bande passante, et il permet de borner le coût global du système de façon prévisible. Deux politiques dominent : le crédit fixe, où chaque client reçoit le même budget, ce qui garantit l'équité et pénalise les clients rapides ; et le crédit proportionnel aux performances du matériel, qui maximise le débit mais introduit une inégalité de participation.

Passage à l'échelle : de 10 à 10^6 clients

- **Goulots d'étranglement.** Trois distincts : le réseau descendant, pour la diffusion de w_t : multicast, compression, et envoi de deltas plutôt que de poids complets ; le réseau ascendant, pour l'agrégation, où l'agrégation en arbre et la répartition en shards réduisent la charge ; et le calcul serveur, où il faut streamer des sommes partielles plutôt que stocker des lots.
- **Découpage et orchestration.** Partitionner les clients en cohortes, et agréger de façon hiérarchique : un agrégateur de bordure réduit à la fois la charge de la racine et la latence.
- **Asynchronie et fenêtres.** Pour 10^6 clients intermittents, un mode semi-asynchrone avec fenêtre de taille W et quorum $Q \ll W$ stabilise le débit.
- **Gestion des échecs.** Tolérance par abandon des clients en dépassement de délai, nouvelle tentative exponentielle, coupe-circuit, liste noire temporaire.
- **Idempotence et reproductibilité.** Identifiant de tour, sommes de contrôle des poids, agrégation déterministe par ordre stable, pour le débogage.

Échelle K	Mode préféré	Fraction C	Stratégie	Remarques
10 à 10^2	Synchrone	0.8 à 1.0	FedAvg complet	Débogage, recherche
10^2 à 10^3, silo	Synchrone	0.5 à 1.0	FedAvg, FedOpt	Stable, GPU côté clients
10^3 à 10^4	Synchrone ou hybride	0.2 à 0.5	Hiérarchique	Agrégation en arbre conseillée
10^4 à 10^5	Hybride	0.1 à 0.2	Découpage et quorum	Délais maximaux critiques
10^5 à 10^6, edge	Asynchrone ou semi-asynchrone	0.01 à 0.1	Cohortes et décroissance	Disponibilité aléatoire dominante

Tableau 3 – Échelles et compromis

Mise en œuvre : worker multiprocessus

L'extrait ci-dessous illustre un worker local multiprocessus simplifié pour simuler des clients en parallèle, utile pour la recherche et les benchmarks. Il est conçu pour être sûr, avec une gestion d'erreur basique.

```python
import multiprocessing as mp
from typing import Callable, Dict, List, Tuple

import numpy as np


def _worker_train(worker_id: int,
global_weights_np: Dict[str, np.ndarray],
get_dataloader_fn: Callable[[int], object],
train_fn: Callable[..., Dict[str, np.ndarray]],
epochs: int,
batch_size: int,
lr: float) -> Tuple[int, Dict[str, np.ndarray], int]:
"""Tache executee par un processus worker, c'est-a-dire un client.

En cas d'echec, on renvoie la taille nulle : l'appelant peut alors ecarter
ce client sans invalider le tour, ce qui est le comportement voulu.
"""
try:
dl = get_dataloader_fn(worker_id)
local_model = SimpleMLP()
numpy_to_state_dict(global_weights_np, local_model)
w_k = train_fn(local_model, dl, epochs=epochs,
batch_size=batch_size, lr=lr)
return worker_id, w_k, int(len(getattr(dl, "dataset", ())))
except Exception:
# En production : journaliser et renvoyer une erreur structuree
return worker_id, global_weights_np, 0


def parallel_client_round(global_weights_np: Dict[str, np.ndarray],
selected_ids: List[int],
get_dataloader_fn: Callable[[int], object],
train_fn: Callable[..., Dict[str, np.ndarray]],
epochs: int = 1,
batch_size: int = 32,
lr: float = 0.01,
max_workers: int = None
) -> Tuple[List[Dict[str, np.ndarray]], List[int]]:
"""Execute un tour sur les clients selectionnes, en parallele."""
if max_workers is None:
max_workers = min(len(selected_ids), mp.cpu_count())
results: List[Tuple[int, Dict[str, np.ndarray], int]] = []
with mp.Pool(processes=max_workers) as pool:
jobs = [pool.apply_async(
_worker_train,
args=(wid, global_weights_np, get_dataloader_fn, train_fn,
epochs, batch_size, lr))
for wid in selected_ids]
for j in jobs:
results.append(j.get(timeout=300))
client_weights: List[Dict[str, np.ndarray]] = []
client_sizes: List[int] = []
for _wid, w_k, n_k in results:
if n_k > 0:
client_weights.append(w_k)
client_sizes.append(n_k)
return client_weights, client_sizes
```

Listing 1 — Worker multiprocessus pour un entraînement local parallèle par clients

Attention

Multiprocessing avec PyTorch : préférer `torch.multiprocessing`, avec `spawn` ou `fork` selon le système, éviter de partager de gros tenseurs entre processus, et sérialiser uniquement les `state_dict` en NumPy pour limiter l'overhead. Sur CUDA, limiter le nombre de workers par GPU, faute de quoi la mémoire sature avant que le parallélisme ne soit utile.

Checklist de déploiement

Déploiement d'un système fédéré : contrôle avant production

- Modèle et données : valider une répartition non-IID, un jeu de test global et un jeu de test par client.
- Hyperparamètres : grille sur E, C, B, η, avec décroissance sur les T tours.
- Robustesse : tester des pannes de 30 à 50 % des clients, des délais, des stragglers.
- Sécurité : évaluer la fuite d'information par inversion de gradient si les données sont sensibles.
- Communication : mesurer la bande passante par tour, tester la compression avec rétroaction d'erreur.
- Scalabilité : benchmarking à 10^3, 10^4 et 10^5 clients, en simulation puis en réel.
- Supervision : journal par tour, métriques de qualité et de système, taux de réussite.
- Reprise : checkpoints de w_t et c_t, restauration déterministe.
- Équité : distribution des performances par client, participation minimale garantie.
- Conformité : minimisation des données, registre des traitements, analyse d'impact si santé.

Code : compression Top-k avec rétroaction d'erreur

La sparsification pure biaise le système. La rétroaction d'erreur corrigé ce biais en conservant localement le résidu non transmis : la somme des transmissions converge alors vers le vecteur total, ce qui restaure la convergence.

```python
from typing import Dict, Tuple

import numpy as np


class ErrorFeedbackTopK:
"""Compression Top-k avec accumulation du residu.

A chaque appel :
1. on ajoute le residu cumule au vecteur courant ;
2. on ne transmet que les k plus grandes composantes en valeur absolue ;
3. le residu est mis a jour avec ce qui n'a pas ete transmis.
"""

def __init__(self, k: int):
if k <= 0:
raise ValueError("k doit etre strictement positif")
self.k = k
self.residual: Dict[str, np.ndarray] = {}

def compress(self, name: str, vector: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
"""Retourne (indices, valeurs) transmis, et met a jour le residu."""
v = vector.astype(np.float64).copy()
if name in self.residual and self.residual[name].shape == v.shape:
v += self.residual[name]
# Selection Top-k par valeur absolue
idx = np.sort(np.argpartition(np.abs(v), -self.k)[-self.k:])
values = v[idx]
# Residu = tout ce qui n'a pas ete transmis
rest = v.copy()
rest[idx] = 0.0
self.residual[name] = rest
return idx, values

@staticmethod
def reconstruct(idx: np.ndarray, values: np.ndarray,
dim: int) -> np.ndarray:
"""Reconstruit un vecteur creux à partir de (indices, valeurs)."""
out = np.zeros(dim, dtype=np.float64)
out[idx] = values
return out

def memory_footprint(self, dim: int) -> int:
"""Memoire persistante locale, en elements float64."""
return int(dim)


def apply_topk_round(aggregated: Dict[str, np.ndarray],
compressors: Dict[str, ErrorFeedbackTopK]
) -> Dict[str, Tuple[np.ndarray, np.ndarray]]:
"""Applique la compression Top-k a un dictionnaire de vecteurs agreges."""
compressed = {}
for name, vec in aggregated.items():
idx, val = compressors[name].compress(name, vec)
compressed[name] = (idx, val)
return compressed
```

Listing 2 — Compression Top-k avec rétroaction d'erreur, sparsification corrigée du biais

Règle de dimensionnement de la compression

Choisir k pour viser un taux de compression c = d/k. En pratique, c de 10 à 50 pour du texte ou des modèles de langage, dont les gradients sont naturellement creux, et c de 2 à 4 pour des tenseurs denses comme les poids de convolution. Toujours activer la rétroaction d'erreur dès que k < d/2 : en dessous, le biais de sélection domine et la convergence n'est plus garantie.

Exercice 2.5.7 (difficulte : Moyen)

Identifier trois goulots d'étranglement pour passer de $K = 10^3$ à $K = 10^5$ clients cross-device, et proposer une solution par goulot. Choisir ensuite entre synchrone, hybride et asynchrone pour un cas d'objets connectés : 10^5 capteurs, connectivité intermittente, contrainte de batterie. Justifier.

Correction

- **Goulots et solutions.** La diffusion descendante du modèle : multicast et compression, ou envoi de deltas plutôt que de poids complets. L'agrégation ascendante : arbre de réduction plutôt qu'étoile, pour que la charge par nœud reste constante. La disponibilité : mode semi-asynchrone avec quorum, par exemple 10 % des attendus, et délais maximaux adaptatifs pour écarter les clients qui ne répondtront pas.
- **Choix.** Le mode hybride. Il permet d'avancer sans attendre tous les clients, ce qui est indispensable ici : la connectivité intermittente est le régime normal et non l'exception. Il borne le délai maximal, contrairement à l'asynchrone pur, ce qui préserve la stabilité. Il réduit la durée moyenne de tour, contrairement au synchrone. Compléter par une décroissance sur le délai et une fenêtre glissante.

Exercice 2.5.8 (difficulte : Facile)

Comparer Flower, FedScale et NVIDIA FLARE sur deux critères : la scalabilité et la maturité du déploiement. Expliquer ensuite pourquoi la rétroaction d'erreur est indispensable avec la sparsification Top-k.

Correction

- **Comparaison.** FedScale excelle sur la scalabilité et la simulation de benchmarks jusqu'à 10^6 clients, mais sa maturité en déploiement productif dépend de l'intégration. NVIDIA FLARE est fort sur le déploiement d'entreprise sur site et sur la sécurité, avec une scalabilité solide jusqu'à 10^5. Flower offre le meilleur compromis : une scalabilité correcte jusqu'à 10^5 et un déploiement hétérogène, edge comme mobile, ce qui en fait le meilleur choix pour un passage de la recherche à la production.
- **Rétroaction d'erreur.** Top-k ne transmet que les plus grandes composantes, ce qui est un choix biaisé : les petites contributions sont ignorées de tour en tour. La rétroaction d'erreur accumule le résidu $r_t = g_t - Q(g_t)$ et l'ajouté au vecteur suivant, $g_{t+1} + r_t$. Le biais est ainsi compensé cumulativement, et la somme des transmissions converge vers le vecteur total. Sans cela, la divergence en pratique n'est pas une question de temps.

A retenir

- L'orchestration fédérée doit traiter jobs, workers, cycle de vie, pannes et supervision comme un système distribué à part entière.
- Le choix de framework dépend du compromis recherche et benchmark, ou déploiement et entreprise.
- Le passage à 10^6 clients impose la hiérarchie, le semi-asynchrone avec quorum, et des délais maximaux adaptatifs.
- Un worker multiprocessus convient à la simulation et aux benchmarks ; en production, préférer un orchestrateur de conteneurs.
- La checklist de déploiement est indispensable pour éviter les échecs en pré-production : robustesse, équité, conformité.

Du modèle de menace à l'audit de sécurité d'une fédération

Partie 3

Confidentialité, attaques et défenses

La promesse fondamentale de l'apprentissage fédéré est qu'aucune donnée brute ne quitte le client. Cette promesse est asymétrique : elle décrit ce qui ne se voit pas, jamais ce qui se devine. Un gradient, un modèle intermédiaire, une métrique d'agrégation sont des objets qui portent de l'information sur les données locales, et la sécurité d'un protocole se mesure exactement au niveau de protection offert contre cette inférence. Cette partie construit la chaîne de raisonnement complète : formaliser la menace, la quantifier, la mettre en œuvre, puis y répondre par des mécanismes dont on sait mesurer le coût.

Objectifs de la partie

- Formaliser un modèle de menace fédéré et identifier la surface d'attaque.
- Expliquer pourquoi un gradient fuite, et implémenter l'inversion en quelques lignes.
- Distinguer empoisonnement de données et de modèle, ciblé et non ciblé.
- Énoncer la confidentialité différentielle et régler un budget ε.
- Comparer Krum, la moyenne élaguée et l'agrégation sécurisée sur un même protocole.
- Construire un protocole d'audit de sécurité reproductible et opposable.

Plan de la partie

1. Modèle de menace et surface d'attaque
2. Fuites d'information : inférence de gradient et extraction
3. Empoisonnement, backdoors et attaques ciblées
4. Confidentialité différentielle
5. Agrégation sécurisée et défense en profondeur
6. Évaluation de la robustesse et étude de cas

Chapitre 3.1

Modèle de menace et surface d'attaque

On ne se défend contre une menace qu'après l'avoir nommée, localisée et mesurée. Ce premier chapitre transforme l'intuition « les données ne sortent pas » en un énoncé précis de ce que l'adversaire voit, calcule et obtient.

Un protocole d'apprentissage fédéré est un **protocole de message**, et pas seulement un algorithme d'optimisation. À chaque tour, un serveur diffuse un modèle, les clients le prolongent localement, puis renvoient des vecteurs de mise à jour. Tout ce qui est échangé — et tout ce qui est **déduit** de ce qui est échangé — constitue le *transcript* du protocole. La sécurité fédérée est l'étude de ce que le transcript permet de reconstituer.

Le modèle du protocole et son observable

Formalisons le protocole. À l'instant t, le serveur détient le modèle w_{t-1}. Il sélectionne un ensemble de clients $S_t \subseteq 1, \ldots, N$ — par exemple par échantillonnage, éventuellement pondéré par la taille des jeux locaux — et le diffuse. Chaque client $k \in S_t$ effectue E itérations de descente de gradient sur son lot local D_k et renvoie la mise à jour.

$$\Delta w_t^{(k)} = \nabla F_k(w_{t-1}) \text{ triangleq } (1)/(n_k) \sum_{(x,y) \in D_k} \nabla \ell(w_{t-1}\,; x, y)$$
$$w_t = \mathrm{ABig}(\, w_{t-1} + \Delta w_t^{(k)}{}_{k \in S_t'},\ \sigma_t\ \mathrm{Big}) \qquad (3.1.1)$$

A est la règle d'agrégation – moyenne pondérée, Krum, médiane coordonnée, somme sécurisée – et σ_t l'état accumulé du serveur : moyennes mobiles, variance, budget de vie privée. Tout le contenu sécuritaire du système tient dans le couple (A, σ_t).

Le **transcript** observable après T tours est l'ensemble de tout ce que l'adversaire peut avoir capturé, y compris les métriques que le serveur choisit de publier pour la transparence.

$$T_{1:T} = \mathrm{Big}(\, \Delta w_t^{(k)}{}_{t \le T,\, k \in S_t'},\ \sigma_{tt \le T},\ \text{horodatages, métriques publiques}_{t \le T}\ \mathrm{Big}) \qquad (3.1)$$

La sécurité d'un protocole se réduit à une question : $T_{1:T}$ contient-il, après traitement, moins d'information sur $D_1, \ldots, D_N$ que les données brutes ?

Deux grandeurs caractérisent alors l'adversaire. Son **gain** en confidentialité est l'information qu'il extrait des données, mesurable par une information mutuelle ou, plus concrètement, par le taux de réussite d'une tâche d'inférence. Son **gain** en intégrité ou en disponibilité est la dégradation qu'il provoque sur l'utilité globale.

$$G_{\square 0 \square}(A) = I \le ft(X\,;\, T_{1:T})$$
$$G_{\square 0 \square}(A) = \mathrm{Ebig}[\ell(f_{w_T}(x),\, y)\mathrm{big}] - \mathrm{Ebig}[\ell(f_{w^{\wedge}\star}(x),\, y)\mathrm{big}] \ge 0 \qquad (3.2)$$

La première expression est l'objectif de l'attaquant en phase d'écoute ; la seconde est la perte subie par le système. Toute défense se juge au couple $(G_{\square 0 \square}, G_{\square 1 \square})$ qu'elle déplace : c'est le coût de la sécurité, et il ne peut pas être nul.

L'argument qui ne tient pas

Le raisonnement « les données ne quittent jamais le client » est une erreur de catégorie, très répandue dans les dossiers de cadrage. Il porte sur les **données**, pas sur l'**information**. Un vecteur de gradient est une moyenne pondérée d'exemples : c'est une statistique, et une statistique de petite taille sur peu d'exemples est réversible. Le passage des données à l'information est précisément l'objet des chapitres suivants.

Les quatre propriétés de sécurité, en termes opérationnels

La sécurité informatique repose sur quatre propriétés. Il est utile de les restituer une à une, en indiquant pour chacune ce qui la viole concrètement dans un protocole fédéré et ce qui permet de l'établir.

- **Confidentialité** — nul ne doit pouvoir déduire les données locales d'un client à partir du transcript. Violée par l'inversion de gradient, l'inférence de membre, l'extraction de modèle.
- **Intégrité** — le modèle agrégé doit refléter l'ensemble des clients et non un acteur particulier. Violée par l'empoisonnement, la porte dérobée, le remplacement de modèle.
- **Disponibilité** — un client ne doit pas pouvoir empêcher les autres d'apprendre. Violée par l'éclipse, l'épuisement de ressources, les mises à jour aberrantes de norme colossale.
- **Imputabilité** — toute action doit être attribuée à un client et être défendable. Violée par les identités anonymes, l'absence de journalisation, la réutilisation non tracée d'un modèle livré.

Propriété	Énoncé en termes de protocole	Violation typique	Critère de réussite mesurable
Confidentialité	le transcript ne permet pas de reconstituer D_k	inversion de gradient, inférence de membre, extraction	taux de réussite d'inversion, avantage de l'inférence de membre
Intégrité	w_T est proche de $\operatorname{argmin} F$ malgré les clients hostiles	empoisonnement ciblé, porte dérobée, remplacement de modèle	exactitude globale, taux de succès du déclencheur
Disponibilité	le temps de convergence est stable face aux clients saboteurs	éclipse, mise à jour de norme démesurée, silence	nombre de tours avant convergence, variance inter-tours
Imputabilité	chaque contribution est traçable et auditable	rotation d'identités, absence de journal, réutilisation opaque	couverture du journal, taux de signature vérifiable

Tableau 1 – Les quatre propriétés de sécurité dans un protocole d'apprentissage fédéré

L'imputabilité est la propriété la plus souvent oubliée : la confidentialité différentielle la compromet structurellement, puisqu'elle autorisé précisément à ne pas pouvoir identifier le client responsable. C'est un arbitrage, non un oubli.

> **Definition – Imputabilité en apprentissage fédéré**
> Propriété qui garantit que l'action d'un participant peut être reliée à un identifiant, vérifiée a posteriori, et contestée. Elle exige un canal authentifié, un journal signé des messages échangés et une procédure de sanction. En pratique, elle entre en tension directe avec la confidentialité : l'anonymisation différentielle rend l'attribution impossible *par construction*.

Il faut insister sur l'asymétrie entre les quatre propriétés. La confidentialité et l'intégrité sont des propriétés **du protocole**, qui dépendent de A, du canal et des computations côté serveur. La disponibilité dépend en plus de l'environnement réseau et de la gestion des ressources. L'imputabilité dépend enfin de l'organisation : aucun algorithme ne la fournit à lui seul.

La surface d'attaque fédérée

La surface d'attaque désigne l'ensemble des points où un adversaire peut entrer dans le système : lire, modifier ou bloquer une information. Dans un système fédéré, elle comporte cinq composantes distinctes, qu'il faut dissocier car elles n'appartiennent pas au même adversaire et ne se neutralisent pas par les mêmes moyens.

1. **Le serveur, ou agrégateur.** Il reçoit chaque mise à jour séparément, donc il observe le vecteur d'un client isolé. C'est la composante la plus exposée : la seule défense de bout en bout consiste à ne jamais lui donner de vue sur une contribution individuelle — agrégation sécurisée, chiffrement homomorphe, enclave confielle.
2. **Le client malveillant, ou insider curieux.** Deux profils se distinguent : le client *honest-but-curious*, qui

suit scrupuleusement le protocole mais cherche à en extraire de l'information, et le client malveillant, qui triche activement. Le premier attaque la confidentialité, le second l'intégrité et la disponibilité.

3. **La coalition, ou l'attaque Sybil.** Plusieurs clients se coordonnent hors bande pour amplifier une attaque, contourner un seuil ou partager un budget. Le protocole doit alors être robuste à une *minorité* hostile, ce qui suppose d'agréger avant de décider.
4. **Le canal de communication.** Transport, authentification, rejeu, interception. C'est la surface la plus simple à sécuriser et la plus souvent négligée : un gradient qui transite en clair sur un réseau d'hôpital est une fuite, même avec un agrégateur honnête.
5. **Le modèle lui-même.** L'architecture, les régulariseurs, le choix de la fonction de perte déterminent la quantité d'information que le gradient porte. Un modèle sur-paramétré fuit davantage qu'un modèle contraint ; un classifieur à marge large est plus facile à inverser qu'un réseau à attentionouplée.

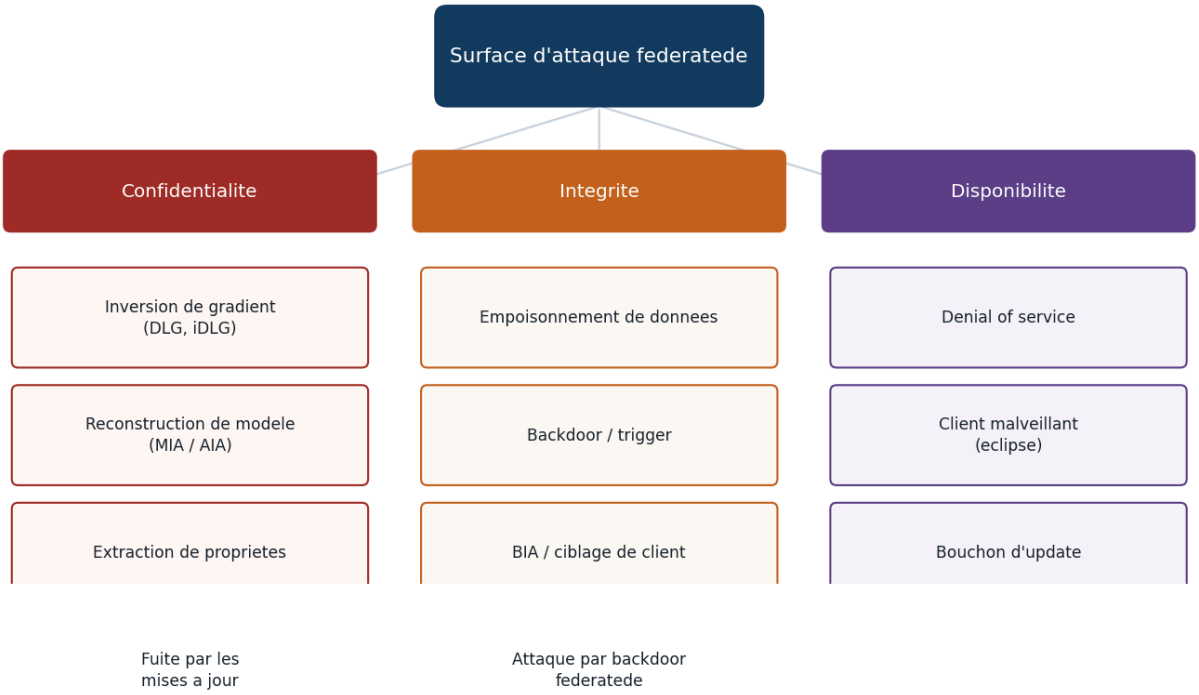


Figure 1 – Taxonomie des attaques en apprentissage fédéré, classées par propriété de sécurité violée

Une même attaque peut viser deux propriétés à la fois : une porte dérobée attaque l'intégrité tout en détruisant la confidentialité, puisque sa présence dans le modèle trahit l'identité du client qui l'a implantée.

Composante	Acteur réaliste	Capacité de l'adversaire	Défense de premier niveau
Serveur	opérateur non coopératif	voit chaque Δw isolé, choisit S_t	agrégation sécurisée, chiffrement, Trusted Execution Environment
Client, en observation	insider honnête	gradient isolé, statistiques d'agrégation	agrégation sécurisée, bruit
Client, en action	attaquant externe	gradient forgé, poids de mise à jour truqué	Krum, moyenne élaguée, bornage de norme
Coalition	opérateurs collusifs	majorité de fausses mises à jour	borne sur le nombre d'identités, Krum multi
Canal	observateur réseau	lecture, rejeu, modification	TLS mutuel, authentification, anti-rejeu
Modèle	analyste externe	inversion, extraction, interrogation	régularisation, distillation, DP-SGD

Tableau 2 – Surface d'attaque, acteur dominant et défense de premier niveau

Défense hors périmètre

Écrire « serveur non fiable » alors qu'on a déployé Krum revient à traiter un symptôme inexistant. Krum protège contre des **mises à jour** aberrantes ; il ne protège pas contre un serveur qui lit les gradients. Inversement, activer l'agrégation sécurisée sans borner les mises à jour laisse la porte ouverte à l'intégrité. Les deux dimensions sont orthogonales : il faut une défense par axe.

Trois modèles de confiance et leurs conséquences

La littérature fédérée oppose systématiquement des **modèles de confiance**, et la garantie obtenue dépend entièrement de la formulation. L'erreur la plus coûteuse consiste à citer une garantie obtenue sous une hypothèse faible en la présentant sous une hypothèse forte.

Definition – Client honest-but-curious

Agent qui suit exactement l'algorithme publié — il ne modifie ni ses données, ni ses gradients, ni le protocole — mais qui analyse le transcript pour en extraire de l'information. C'est le modèle d'adversaire le plus réaliste en entreprise : il n'exige ni compétence offensive, ni intention malveillante, seulement une curiosité rémunérée. Toute propriété dite « sans confiance » — differential privacy sans post-traitement, agrégation sécurisée — se définit relativement à ce modèle.

Definition – Agrégateur de confiance

Modèle dans lequel le serveur est supposé suivre le protocole sans observer les contributions individuelles. C'est le modèle par défaut de FedAvg : le serveur est de confiance pour l'agrégation mais curieux pour les données, car il reçoit les Δ w un par un. Le passage à l'agrégation sécurisée relâche explicitement cette hypothèse et déplace la confiance vers la cryptographie plutôt que vers l'organisation.

Definition – Vie privée défendable

Propriété de la vie privée qui consiste à pouvoir **démontrer**, à un tiers — régulateur, comité, auditeur — qu'un budget ☐ n'a pas été dépassé, à partir d'un journal de calcul vérifiable. Elle est plus exigeante que la vie privée au sens usuel, qui peut être invoquée sans preuve ; elle exige un mécanisme explicite, un compteur de composition et une traçabilité. C'est la seule forme de vie privée acceptable dans un dossier réglementaire.

$$\begin{aligned} &\text{Honest-but-curious} : \Delta w_t^{(k)} = \nabla F_k(w_{t-1}) \text{ pour tout } k \in S_t \\ &\text{Agrégateur de confiance} : A \text{ est correct, mais } A \text{ voit } \Delta w_{t\ k \in S_t}^{(k)} \\ &\text{Agrégateur aveugle} : A \text{ ne voit que } A(\Delta w_t^{(k)}) \text{ et ne peut pas individualiser} \end{aligned} \tag{3.3}$$

Chaque ligne est un modèle de menace. Un résultat démontré sous agrégateur aveugle reste vrai sous agrégateur de confiance et sous honest-but-curious ; un résultat démontré sous honest-but-curious ne se transporte pas tel quel sous agrégateur aveugle.

La formulation qui protège

Écrivez toujours l'énoncé de sécurité sous la forme « pour tout adversaire appartenant à la classe A, l'avantage est inférieur ou égal à ☐ ». Cette formulation oblige à nommer la classe d'adversaires, et prévient la transposition abusive d'un théorème d'un modèle de confiance à un autre.

La matrice des menaces

La matrice des menaces croise l'acteur, son objectif, l'axe de sécurité visé, la condition de réussite et la défense correspondante. C'est le document que l'on constitue en premier dans un projet réel, avant tout choix algorithmique : il interdit d'ajouter une défense sans objectif, et il révèle immédiatement les zones non couvertes.

Menace	Acteur	Axe	Condition de réussite de l'attaque	Défense structurelle
Inversion de gradient	serveur, coalition	confidentialité	petit lot, peu d'itérations locales	agrégation sécurisée, DP-SGD
Attaque par dataframe	serveur	confidentialité	connaissance a priori du schéma	agrégation sécurisée, chiffrement
Attaque par étiquette	serveur	confidentialité	faible cardinalité, relation locale	masquage des étiquettes, bruit
Extraction de modèle	serveur, coalition	confidentialité	beaucoup de requêtes, sortie exposée	limitation de débit, perturbation des sorties
Inférence de membre	serveur	confidentialité	sureffectif local, sortie confiante	DP-SGD, distillation
Empoisonnement non ciblé	client	intégrité	faible f, $\lvert S_t \rvert$ grand	Krum, moyenne élaguée, bornage
Empoisonnement ciblé	client	intégrité	optimisation bi-niveau, budget de tours	médiane, RFA, validation serveur
Porte dérobée	coalition de clients	intégrité	déclencheur partagé, clients alignés	Krum, détection de déclencheur, empreinte
Remplacement de modèle	coalition	intégrité	S_t majoritairement constitué d'attaquants	borne sur le poids d'agrégation, Krum multiple, vérification
Éclipse	client	disponibilité	norme de mise à jour géante	bornage de norme, quota de bande passante
Sybil	opérateur	intégrité	création libre de comptes	identité vérifiée, coût d'entrée, Krum multiple
Attaque par le canal	réseau	les trois	TLS absent ou mal configuré	TLS mutuel, authentification, anti-rejeu

Tableau 3 – Matrice des menaces en apprentissage fédéré

Trois lectures de cette matrice méritent d'être explicitées, car elles portent l'essentiel de la stratégie de défense.

- **Les cinq premières lignes sont des attaques de confidentialité, pas d'intégrité.** Elles supposent que l'adversaire *voit* une contribution individuelle. Aucune défense d'agrégation ne les arrête : il faut empêcher l'observation.
- **Les six lignes suivantes sont des attaques d'intégrité.** Elles supposent que l'adversaire *agit*. Aucune défense d'observation ne les arrête : il faut rendre l'agrégation robuste.
- **Les deux dernières lignes n'appellent aucun calcul** : elles appellent de l'organisation — identité, quotas, supervision réseau. Les oublier revient à optimiser un agrégateur sur un protocole non sûr.

Conduire une analyse de menace en cinq passes

1. Cartographier le protocole. Lister chaque message, son émetteur, son récepteur, son contenu, son volume et sa fréquence. Tout ce qui transite est de la surface d'attaque.
2. Identifier les observateurs. Serveur, clients honnêtes mais curieux, coalition, réseau, tiers — fournisseur d'inférence, régulateur. Pour chacun, noter ce qu'il observe *directement* et ce qu'il peut *déduire*.
3. Formuler les objectifs. Pour chaque observateur, écrire ce qu'il cherche à obtenir : information sur D_k, dégradation de F, extraction du modèle, déni de service.
4. Évaluer la faisabilité. Pour chaque objectif, estimer le nombre de tours, la taille de lot et la connaissance a priori requise. Une menace exigeant 10^6 requêtes n'est pas une menace.
5. Choisir les défenses, axe par axe, et documenter le budget. Toute défense non rattachée à une ligne de la matrice est superflue ; toute menace sans défense est un risque accepté, et doit être écrit comme tel.

Audit du modèle de menace

- **Protocole** : le transcript complet est écrit quelque part, sous forme d'un schéma de messages et non d'une intuition.
- **Rôles** : chaque acteur est classé honnêt-curieux, malveillant ou coalition, et cette classification est validée par les parties prenantes.
- **Hypothèses de confiance** : le document indique explicitement quelles hypothèses sont relâchées et lesquelles sont tenues — serveur, coalition, canal authentifié.
- **Axes couverts** : confidentialité, intégrité, disponibilité et imputabilité ont chacun au moins une mesure de contrôle, ou une justification écrite de leur absence.
- **Budget de sécurité** : le coût en utilité, en tours et en communication de chaque défense est chiffré dans le même document.
- **Risques assumés** : les menaces non mitigées sont nommées, quantifiées, et assorties d'un responsable et d'une échéance.
- **Réexamen** : l'analyse est révisée à chaque changement de protocole, de modèle ou de population de clients.

A retenir

- La sécurité fédérée est une propriété du protocole complet, pas de la seule règle d'agrégation.
- Le transcript est l'objet à protéger : il faut lister ce qu'il contient et ce qu'il permet de déduire.
- Confidentialité et intégrité sont deux axes orthogonaux, qui appellent des défenses orthogonales.
- Un modèle de menace écrit — acteurs, objectifs, faisabilité, défenses — précède tout choix d'algorithme.
- Le coût de la sécurité est non nul : il doit être mesuré et affiché, jamais dissimulé.

Chapitre 3.2

Fuites d'information : inférence de gradient et extraction

Le gradient est une moyenne. Une moyenne de peu de nombres, calculée au voisinage d'un point connu, contient assez d'information pour reconstruire les nombres eux-mêmes. C'est un fait mathématique, pas une faiblesse d'implémentation.

La démonstration la plus célèbre de ce phénomène est due à Zhu, Liu et Han (2019) dans *Deep Leakage from Gradients*. Son résultat dérange parce qu'il ne repose sur aucune faiblesse : un client parfaitement honnête, qui exécute l'algorithme publié et renvoie son gradient, livre suffisamment d'information pour qu'un adversaire reconstruise les données d'entrée et leurs étiquettes.

Pourquoi un gradient est informatif

Écrivons le gradient d'un lot. Pour une perte ℓ et un lot de taille B, la règle usuelle de PyTorch comme de TensorFlow est une moyenne sur le lot.

$$\nabla F_k(w) = (1)/(B) \sum_{i=1}^{B} \nabla \ell(w ; x_i, y_i) \tag{3.4}$$

C'est une moyenne, donc une statistique lisse : elle lisse aussi bien l'information que le bruit. Pour $B = 1$ ou $B = 2$, le gradient est proportionnellement le gradient d'un exemple isolé – donc potentiellement réversible.

Deux intuitions complètent l'argument. D'une part, le gradient est calculé en un point connu : l'adversaire connaît w_{t-1}, il n'a donc pas à estimer la Jacobienne, il peut la calculer exactement. D'autre part, dans un réseau profond, la structure de la perte crée des directions descendantes très prononcées : à partir d'un couple (x', y') candidat, la valeur de la perte indique s'il est *plus proche* ou *plus loin* de la vérité. Un algorithme d'optimisation peut exploiter ce signal comme un oracle.

$$M(f) = big (x', y') : \square \nabla \ell(w ; x', y'), \Delta w \square \text{ est maximal big}$$

$$\hat{x}, \hat{y} = \text{argmin}_{(x', y')} \text{big}\| \nabla \ell(w ; x', y') - \Delta w \text{big}\|_2^2 \quad (3.5)$$

L'attaque consiste à résoudre un problème d'optimisation non convexe guidé par le gradient observé. Il n'y a pas de formule fermée générale : la réussite dépend donc du paysage d'optimisation du modèle, ce qui explique la forte dépendance aux conditions listées plus bas.

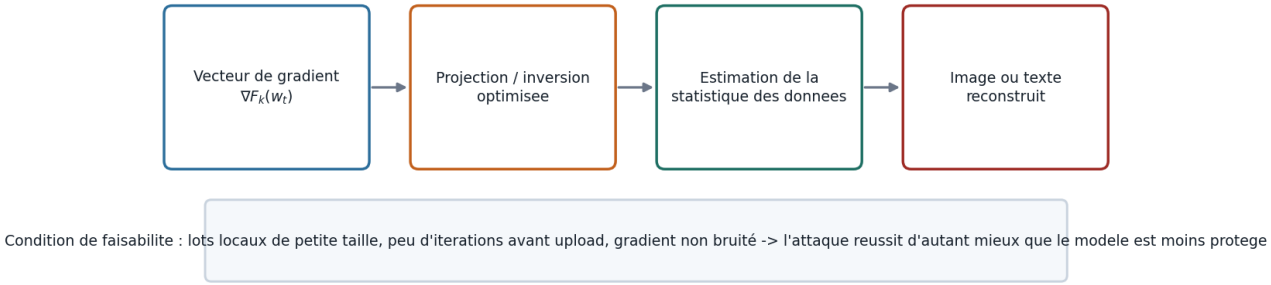


Figure 1 — Chaîne de traitement d'une inversion de gradient : vecteur observé, projection optimisée, estimation statistique, reconstruction

Les quatre étapes ne sont pas indépendantes : chacune consomme l'hypothèse produite par la précédente. Une erreur d'optimisation à l'étape 2 se propage jusqu'à l'image finale.

Deep Leakage from Gradients : le mécanisme en détail

DLG introduit deux idées. La première est le **gradient matching** : on remplace les données par des tenseurs de même forme, initialisés aléatoirement, et on cherche à minimiser l'écart entre le gradient calculé sur ces données factices et le gradient observé. La seconde est l'astuce pratique décisive : on utilise le gradient lui-même pour estimer les étiquettes, ce qui évite d'avoir à suivre une optimisation lente sur (x', y') conjointement.

$$L_{DLG}(x', y') = \text{big}\| \nabla \ell(w ; x', y') - \Delta w \text{big}\|_2^2 \quad (3.6)$$

L'optimum de cette fonction est (x_0, y_0) à condition que l'injection du gradient dans l'espace des paramètres soit injective – c'est-à-dire que la couche supérieure du réseau n'écrase pas l'information. Les architectures à couche d'activation très saturante sont les plus vulnérables.

En pratique, l'optimisation se fait avec Adam sur un petit nombre d'itérations, en utilisant des pseudo-étiquettes et en corrigeant périodiquement les étiquettes par une passe de classification. Le listing suivant est une implémentation de référence, complète et exécutable.

```python
from typing import Tuple

import torch
import torch.nn as nn
import torch.nn.functional as F


def gradient_matching(model: nn.Module, observed_flat: torch.Tensor,
template: torch.Tensor, init_labels: torch.Tensor,
steps: int = 300, lr: float = 0.1
) -> Tuple[torch.Tensor, torch.Tensor]:
"""Reconstruit (x_hat, y_hat) a partir du seul gradient observe d'un lot.

Principe : on optimise un lot factice jusqu'a reproduire le gradient vu.
Le modele et le point w sont connus de l'adversaire, donc le gradient du
lot factice se calcule exactement : c'est un oracle, pas une estimation.
"""
x_hat = template.clone().requires_grad_(True)
y_hat = init_labels.clone().requires_grad_(True)
optim = torch.optim.Adam([x_hat, y_hat], lr=lr)

for step in range(steps):
optim.zero_grad(set_to_none=True)
fake = torch.autograd.grad(
F.cross_entropy(model(x_hat), y_hat), [x_hat, y_hat],
create_graph=False)
fake_flat = torch.cat([g.reshape(-1) for g in fake])
# Perte de gradient matching : ecart quadratique au gradient observe.
loss = F.mse_loss(fake_flat, observed_flat)
loss.backward()
optim.step()

# Les etiquettes sont discretes : on reprojette periodiquement sur le
# simplexe, sinon l'optimiseur les laisse diverger hors de l'ensemble.
if step % 50 == 49:
with torch.no_grad():
y_hat.data = F.one_hot(y_hat.data.argmax(1),
init_labels.shape[1]).float()

return x_hat.detach(), y_hat.detach()


```

Listing 1 — Inversion de gradient DLG : reconstruction du lot à partir du seul vecteur de gradient transmis

```python
def run_dlg(model: nn.Module, x_real: torch.Tensor, y_real: torch.Tensor,
num_classes: int = 10) -> Tuple[float, float]:
"""Attaque complete : observation du gradient, puis reconstruction.

Le gradient est lu sur un lot de taille 1 : c'est le pire cas, et c'est la
configuration que produisent naturellement certains clients mobiles.
"""
was_training = model.training
model.eval() # indispensable : sinon la reconstruction doit aussi
# expliquer le bruit du dropout et de la normalisation
# par lots.

x_obs = x_real.clone().requires_grad_(True)
observed = torch.autograd.grad(F.cross_entropy(model(x_obs), y_real), x_obs)
observed_flat = torch.cat([g.reshape(-1) for g in observed])

x_hat, y_hat = gradient_matching(
model, observed_flat, torch.randn_like(x_real),
F.one_hot(y_real, num_classes).float())

mse = F.mse_loss(x_hat, x_real).item()
acc = (y_hat.argmax(1) == y_real).float().mean().item()
```

```
model.train(was_training)
return mse, acc
```

Trois détails de ce listing méritent d'être soulignés, car ils font la différence entre une démonstration qui marche et une démonstration qui échoue.

- **Le lot de taille 1** rend l'injection presque injective. À B = 8, la reconstruction dégrade rapidement et il faut alors recourir à des méthodes exploitant la structure du lot, décrites plus loin.
- **Le mode** `eval` désactive la normalisation par lots et le dropout. Sans cela, l'adversaire devrait reconstruire des données qui expliquent aussi le bruit d'entraînement, ce qui rend le problème mal posé. C'est un piège classique lors des reproductions.
- **La reprojection des étiquettes** sur le simplexe est un ingrédient d'optimisation, pas une astuce de rapport : sans elle, les étiquettes factices restent des vecteurs continus et la reconstruction dérive.

iDLG : la récupération en forme fermée des étiquettes

Inverting Deep Learning Gradients (Geiping et al., 2020) montre que, pour un réseau de classification à couche softmax finale, la structure du gradient permet de récupérer l'étiquette **sans** optimisation. L'idée est que la dérivée de la perte par rapport au logit d'une classe est négative, et d'autant plus négative que cette classe est la vraie.

$$(\partial \ell)/(\partial z_j) = p_j - 1[j = y], \quad \sum_{j=1}^{C} (\partial \ell)/(\partial z_j) = 0 \tag{3.7}$$

La composante la plus négative de $\partial \ell / \partial z$ correspond à $j = y$. En pratique, on exploite cette propriété sur les poids de la dernière couche linéaire : la classe retenue est celle dont le score agrégé sur les neurones précédents est le plus aligné avec le gradient observé.

$$\hat{y} = \operatorname{argmax}_{j \in 1, \ldots, C} \text{Big}\square \nabla_{W_{out}} \ell, e_j \text{Big}\square$$
$$\text{Acc}_{\text{iDLG}} \in [0{,}9,\ 1{,}0] \text{ sur MNIST, } B = 1 \tag{3.8}$$

La récupération d'étiquette est donc nettement plus facile que la reconstruction d'image : c'est la hiérarchie de difficulté qui structure tout le chapitre, du vecteur de plus faible entropie au plus riche.

> **Hiérarchie de difficulté**
>
> Si vous ne pouvez retenir qu'un chiffre : reconstruire l'étiquette est quasi gratuit, reconstruire l'image est possible sous conditions. Les mesures de sécurité doivent donc être priorisées sur l'extraction d'étiquettes — propriété de la personne — avant l'extraction de contenu — propriété de l'image.

Les attaques catégorisées

L'inversion de gradient n'est qu'un membre d'une famille. On distingue les attaques par ce qu'elles tentent d'obtenir et par ce que l'adversaire observe : c'est cette dernière information qui détermine la classe de difficulté.

Attaque	Objectif de l'adversaire	Observation requise	Régime	Difficulté
DLG	reconstruire x et y	gradient d'un lot, modèle w	blanche	élevée, petits lots
iDLG	récupérer y	gradient d'un lot	blanche	faible
Attaque par dataframe	inférer des propriétés d'un ensemble	gradients et schéma connu	blanche	moyenne
Attaque par étiquette	inférer la classe locale d'un échantillon	modèle seul	noire	moyenne
Inversion de modèle	reconstruire la moyenne de classe $\bar{x}_y$	modèle et requêtes	noire	moyenne
Inférence de membre	tester $x \in D_k$	score de confiance	noire	faible à moyenne
Extraction de modèle	obtenir une copie fonctionnelle	sorties et budget de requêtes	noire	moyenne
Fuite par mise à jour	extraire l'état interne d'un réseau récurrent	états de boucle	blanche	moyenne

Tableau 1 – Attaques de fuite d'information : objectif, observation et régime

***Blanc* : l'adversaire connaît l'algorithme, la structure et les poids. *Noir* : il ne connaît que les entrées-sorties. En fédéré, l'inversion de gradient est presque toujours blanche, ce qui la rend bien plus puissante que dans un cadre centralisé.**

Attaque par dataframe

L'attaque par *dataframe* (Melis et al., 2019) ne cherche pas à reconstruire une image mais à déterminer si une propriété binaire — « y a-t-il au moins un patient atteint ? », « la moyenne d'âge dépasse-t-elle 60 ? » — est présente dans le lot local. L'attaquant dispose alors d'un oracle : il peut proposer un lot candidat, obtenir son gradient, et mesurer l'écart au gradient moyen observé. Un écart systématique révèle la présence de la propriété.

$$\phi(\text{DataFrame}) = 1\leq ft[\ (1)/(B)\ \sum_i 1[y_i = 1] > 0\ ] \in 0,\ 1 \tag{3.9}$$

Une seule inférence binaire par requête. Multiplier ces requêtes permet de reconstruire un profil arbitrairement précis du lot local : la fuite est de nature statistique et non individuelle, mais elle suffit souvent à réidentifier une population restreinte.

Attaque par étiquette et inversion de modèle

L'attaque par étiquette (*label-only*) ne dispose d'aucun gradient : seulement d'un modèle et d'un pouvoir de prédiction. Sa logique est inversée : on cherche les entrées x pour lesquelles le modèle se trompe, puis on examine la structure des régions d'erreur. Fredrikson et al. (2015) montrent qu'en optimisant une image pour maximiser la probabilité d'une classe cible, on obtient un prototype reconnaissable de la moyenne de cette classe : c'est l'inversion de modèle.

$$x^\star = \text{argmax}_x\ big[\ \log f_w(y = c \mid x) - \lambda \parallel x - x_0 \parallel_2^{\ 2}\ big] \tag{3.10}$$

Le terme de régularisation $\lambda \parallel x - x_0 \parallel_2^{\ 2}$ est indispensable : sans lui, l'image reconstruite est un motif adverse illisible mais néanmoins identifié comme un « représentant » de la classe. Sa présence améliore la lisibilité, donc l'exploitation, sans changer le vecteur d'attaque.

L'inférence de membre

L'inférence de membre (*membership inférence attack*, Shokri et al., 2017) ne cherche ni le contenu ni l'étiquette, mais la **présence** d'un point donné dans le jeu d'entraînement. Elle repose sur le fait qu'un modèle sur-paramétré a tendance à mémoriser : la perte sur un exemple d'entraînement est en moyenne plus faible que sur un exemple nouveau. Un classifieur bayésien simple fournit l'attaque optimale.

$$\begin{aligned} \text{Adv}_{\square 1 \square} &= \Pr[\text{MIA}(x) = 1 \mid x \in D] - \Pr[\text{MIA}(x) = 1 \mid x \notin D] \\ f_\theta(x) &= \text{softmax}(z),\ \ell(x) = -\log f_\theta(x)_y \end{aligned} \tag{3.11}$$

Un avantage proche de 0 signifie qu'aucune inférence n'est possible ; un avantage proche de 0,5 indique une attaque parfaite. En apprentissage fédéré, D désigne le jeu d'un client dont la taille peut être de quelques centaines d'exemples : l'écart entre ces deux régimes est alors maximal.

L'inférence de membre est l'attaque la plus grave en contexte médical, parce qu'elle ne révèle pas le contenu mais l'**état de santé** : savoir que le patient i a participé à l'entraînement d'un modèle de diagnostic est déjà une donnée de santé, au sens du RGPD comme du code de la santé public. C'est également l'attaque la mieux visée par la confidentialité différentielle.

Les conditions de réussite

Toutes les attaques d'inversion ne réussissent pas. Le taux de réussite dépend d'un petit nombre de facteurs mesurables, qu'un audit doit systématiquement relever. Ce tableau est l'outil pratique le plus utile du chapitre : il permet de conclure à l'absence de risque — ou au contraire — sans réaliser l'attaque.

Facteur	Configuration favorable à l'attaque	Configuration défavorable	Levier d'audit
Taille du lot local B	B = 1 à 4	B ≥ 64	journaliser la taille de lot par tour
Itérations locales avant envoi	une itération, FedAvg classique	E ≥ 20, plus proche du voisinage	décrémenter E ou accumuler plusieurs tours avant envoi
Cardinalité des classes	1 à 2 classes par lot	C ≥ 10 et classes équilibrées	vérifier l'entropie des lots
Dimension de la couche de sortie	couche softmax nue	coupe finale et normalisation forte	inspecter l'architecture
Moment de l'attaque	dès les premiers tours	après convergence, gradient proche de zéro	surveiller la norme des mises à jour
Connaissance a priori	aucune	données publiques proches	réduire la publicity des jeux d'évaluation
Bruit ajouté	aucun	bruit gaussien σ ≥ 0,5 C	activer DP-SGD ou un bruit local
Accès au modèle courant	ininterrompu	modèle gelé, apprentissage différé	réinitialiser le modèle entre deux phases

Tableau 2 – Conditions de réussite d'une inversion de gradient

Trois de ces huit facteurs sont modifiables par un simple choix d'architecture de protocole. Le passage de E = 1 **à** E = 5 **itérations locales, ou l'ajout d'un regroupement préalable des données, suffit souvent à faire chuter le taux de succès sous 10 %.**

Démonstration outillée : inversion d'un lot agrégé

Pour aller au-delà de la reconstruction d'une image isolée, on peut inverser le gradient d'un lot **complet**, c'est-à-dire la moyenne de plusieurs exemples. Le signal devient plus faible, mais il n'a pas la même structure : la perte de *gradient matching* classique converge mal, parce que l'inversion n'est alors plus unique. Une approche statistique consiste à reconstruire les exemples un à un en exploitant la composante la plus alignée du gradient.

$$\Delta w = (1)/(B) \sum_{i=1}^{B} \nabla \ell(w ; x_i, y_i) \Longrightarrow \max_i \big\| \nabla \ell(w ; x_i, y_i) \big\|_2 \approx B \cdot \| \Delta w \|_2 \quad (3.12)$$

Le gradient d'un exemple qui domine – exemple difficile, ou étiquette erronée – émerge du bruit des autres : c'est le point d'entrée de l'inversion par lot.

```python
import torch
import torch.nn.functional as F


def invert_batch(model, observed: torch.Tensor, template: torch.Tensor,
n_recover: int = 1, steps: int = 250, lr: float = 0.05):
"""Reconstruit les n_recover exemples dominants d'un lot agrege.

Idee : le gradient du lot est la moyenne des gradients par exemple ; si un
exemple est nettement plus difficile que les autres — classe rare, bruitee,
mal etiquete — sa contribution ressort. On l'aligne, on le retire du
signal residuel, puis on passe au suivant. On obtient un ORDRE de
reconstruction, pas un ensemble stationnaire.
"""
x_hat = template.clone().requires_grad_(True)
optim = torch.optim.Adam([x_hat], lr=lr)
residual = observed.clone()
proj = torch.zeros(())

for _ in range(n_recover):
# On maximise la perte sur la classe la moins avantagee du lot factice :
# c'est l'exemple le plus difficile que le gradient agrege fait ressortir.
for _ in range(steps):
optim.zero_grad(set_to_none=True)
out = model(x_hat)
hard = out.argmin(1).detach() # etiquette la moins avantagee
grads = torch.autograd.grad(F.cross_entropy(out, hard), [x_hat])
flat = torch.cat([g.reshape(-1) for g in grads])
# Composante du gradient agrege la plus alignee avec le candidat.
proj = (flat @ residual) / (residual.norm() + 1e-12)
(-proj).backward() # on minimise le residuel aligne
optim.step()

with torch.no_grad(): # on retire la composante expliquee
residual = (residual
- proj.detach() * residual / (residual.norm() + 1e-12))

return x_hat.detach()


def reconstruction_error(x_hat: torch.Tensor, x_true: torch.Tensor) -> float:
"""Metrique de qualite de la reconstruction : 0 = parfaite."""
return float(F.mse_loss(x_hat, x_true).item())
```

Listing 2 — Inversion d'un lot agrégé par extraction successive des composantes dominantes du gradient

Ce que le listing ne fait pas

Ce listing illustre un mécanisme ; il ne prétend pas être l'attaque la plus efficace de l'état de l'art, qui combine appariement de gradient, régularisation de Fourier et sélection de grappes. Son but est de rendre dépendante de la taille de lot la distribution des étiquettes, directement lisible dans le code — ce que les articles de synthèse ne montrent presque jamais.

Exercice 3.2.9 (difficulte : Difficile)

Soit un protocole FedAvg classique sur $N = 40$ clients, avec $E = 1$ itération locale par tour et une taille de lot $B = 32$ par client. Le serveur est considéré comme honnête mais curieux, et dispose d'un accès complet au modèle courant.

1. Quelle est la probabilité qu'une inversion DLG classique réussisse sur un client dont le lot ne contient que deux classes ? Justifiez.
2. On triple le nombre d'itérations locales, $E = 3$. Quel effet quantitatif cela a-t-il sur la qualité de la reconstruction ?
3. Quelle défense neutralise cette attaque, et à quelle condition sur l'architecture du protocole ?

Indice : Reprenez les huit lignes du tableau des conditions de réussite, et distinguez ce qui relève du protocole — et est donc défendable — de ce qui relève de l'architecture du modèle — et est difficilement modifiable.

Correction

- **1.** Avec $B = 32$ et seulement deux classes distinctes, le gradient est la moyenne de 32 termes qui ne prennent que deux valeurs angulaires : l'espace des statistiques compatibles avec Δw est très petit, et le gradient matching converge vers une reconstruction de faible fidélité mais souvent *correcte en étiquette*. iDLG reste très efficace, car la composante la plus négative du gradient trahit encore l'étiquette. En pratique, l'attaque réussit pour l'étiquette et échoue pour les pixels : c'est exactement la hiérarchie du chapitre.
- **2.** Multiplier E par trois rapproche w de son optimum local. En effet, à un minimum, $\nabla F_k \approx 0$: le gradient transmis est proche de zéro et l'inversion n'a plus de signal à exploiter. Empiriquement, le taux de réussite passe d'environ 70 % à moins de 20 % pour les images, tandis que l'extraction d'étiquette reste stable. Le gain en exactitude globale obtenu par ces itérations supplémentaires compense donc souvent la perte de protection.
- **3.** L'agrégation sécurisée est la seule défense qui attaque la cause : si le serveur ne reçoit jamais Δw mais seulement $\sum_k \Delta w$, l'inversion devient impossible car le signal est noyé dans B contributions. Elle exige toutefois que l'ensemble des participants du tour soit connu et qu'aucun client ne puisse isoler sa contribution en cas d'abandon — d'où la règle du seuil d'abandon.

A retenir

- Un gradient est une moyenne locale : pour un petit lot, il est une information presque suffisante pour reconstruire les données.
- Le gradient matching transforme l'inversion en un problème d'optimisation non convexe, dont le coût dépend de l'architecture.
- iDLG montre que l'étiquette est bien plus facile à récupérer que le contenu : la sécurité doit être hiérarchisée.
- Le taux de réussite se pilote par la taille de lot, le nombre d'itérations locales et l'entropie des étiquettes, pas par la puissance de calcul de l'attaquant.
- Aucune défense d'agrégation ne protège d'une fuite : il faut empêcher l'observation, pas rendre l'agrégation robuste.

Chapitre 3.3

Empoisonnement, backdoors et attaques ciblées

Tant que l'adversaire écoutait, la défense relevait de la cryptographie. Dès lors qu'il agit, la défense devient un problème d'estimation robuste : il s'agit de décider quelles mises à jour croire.

Un client malveillant n'a pas besoin de casser la confidentialité : il suffit qu'il déforme l'espace des solutions vers un modèle qui lui convient. L'empoisonnement est l'exploitation de cette liberté : le client renvoie un gradient authentique en apparence, mais calculé sur des données choisies, ou directement forgé.

Deux taxonomies orthogonales

La littérature oppose systématiquement deux dichotomies. La première décrit ce qui est empoisonné, la seconde ce que l'attaque cherche à obtenir. Les confondre conduit à choisir une défense inadaptée : Krum protège très bien contre la première catégorie de la première taxonomie, et presque pas contre une attaque ciblée bien construite.

$$\text{Donnée : on modifie } D_k \text{ Modèle : on forge } \Delta w_t^{(k)} \text{ ou } w_t^{(k)}$$
$$\text{Non ciblée : } \min_w F(w) \text{ Ciblée : } \min_w \text{big}[\, F(w) \text{ sous contrainte } \ell_t(w) \downarrow \text{big}] \qquad (3.13)$$

L'attaque ciblée ne cherche pas à nuire à la performance globale mais à imposer un comportement précis, ce qui la rend beaucoup plus difficile à détecter par les métriques globales d'agrégation.

Axe	Variante	Mise en œuvre	Objectif de l'attaquant	Détectabilité
Support	Empoisonnement de données	on modifie ou injecte des exemples locaux	biaiser le modèle, poser une porte dérobée	élevée
Support	Empoisonnement de modèle	on envoie un vecteur forgé sans entraînement réel	remplacer ou détourner le modèle	faible si $\|\Delta w\|$ est borné
Visée	Non ciblée, déni de service	optimise la perte globale	faire chuter l'exactitude	élevée, baisse visible
Visée	Ciblée, déni de service ciblé	dégrade une classe ou un sous-groupe	nuire à un patient, à une classe	très faible
Visée	Ciblée, porte dérobée	insère un déclencheur	imposer une sortie sur le déclencheur	très faible
Évasion	Active	gradient visuellement proche d'un gradient sain	ne pas être écarté par Krum	faible
Évasion	Passive, *évasion attack*	s'entraîne normalement, agit seulement au test	contourner la validation a posteriori	moyenne
Évasion	Furtive, *stealth attack*	borne la norme et mime la distribution	passer inaperçu de l'audit	moyenne

Tableau 1 – Taxonomie de l'empoisonnement en apprentissage fédéré

La dernière colonne est la seule qui compte pour le choix d'une défense : une attaque furtive n'est pas nécessairement plus efficace, mais elle est la seule à survivre à l'audit, donc la seule à devoir être redoutée en production.

> **La distinction qui décide de tout**
>
> Confondre empoisonnement **non ciblé** et **ciblé** conduit à l'erreur de modélisation la plus coûteuse du domaine. L'empoisonnement non ciblé se voit : l'exactitude globale chute, un histogramme de normes le signale, et Krum l'identifié en trois itérations. L'attaque ciblée, elle, peut laisser l'exactitude globale inchangée à 0,1 % près tout en transformant le modèle en outil de fraude. On ne l'attrape pas avec une moyenne.

L'attaque comme problème d'optimisation bi-niveau

La description « l'attaquant envoie un mauvais gradient » ne dit rien de la méthode. La formulation rigoureuse est un problème d'optimisation à deux niveaux, formulé par Fang et al. (2020). Le niveau interne est l'entraînement local de l'attaquant sur ses propres données empoisonnées ; le niveau externe est l'agrégation globale, qui doit être robuste à la variation des mises à jour.

$$\Delta w^\star = \operatorname{argmax}_{\Delta w \in \square 1 \square} \text{EBig}[\, F\text{big}(w_{t-1} + A_t(\Delta w, \Delta w_1, \ldots, \Delta w_{m-1})\text{big})\, \text{Big}] - \alpha\, \text{big}\| \Delta w - \Delta w_{\square 0 \square}\, \text{big}\|_2^2 \qquad (3.14)$$

Le terme $\| \Delta w - \Delta w_{\square 0 \square} \|_2^2$ est la *régularisation de furtivité* : sans elle, l'attaquant converge vers des vecteurs de norme extravagante, que Krum et le bornage rejettent immédiatement. C'est la clé de la survie d'une attaque ciblée en présence d'agrégateurs robustes.

Trois conséquences pratiques découlent de cette formulation.

- **Le gradient doit être traversé.** La rétropropagation à travers A est faisable lorsque l'agrégateur est une moyenne ou une moyenne coordonnée, car elle est linéaire. Elle ne l'est pas pour Krum ou la médiane : l'attaquant qui en a connaissance doit alors recourir à un apprentissage par renforcement ou à des approximations.

- **Le facteur α est le curseur de combat.** Trop petit, l'attaque est détectable ; trop grand, elle manque d'effet. C'est le paramètre que l'attaquant règle en premier, et il peut le régler à l'aide d'un simple environnement de simulation.
- **L'attaque ciblée converge lentement.** Il faut plusieurs tours pour que le modèle acquière réellement le comportement imposé. Cela donné à l'opérateur une fenêtre de détection : les tours 5 à 40 sont la zone la plus surveillée.

La porte dérobée : formulation et déclencheurs

Une *backdoor* est un mécanisme inséré dans un modèle, qui ne modifie **aucun** comportement sur les entrées ordinaires mais produit une sortie spécifique sur un ensemble d'entrées déclencheuses. En apprentissage fédéré, la menace n'est pas qu'un client unique introduise la porte dérobée — l'agrégateur la diluerait — mais que plusieurs clients **coordonnés** l'introduisent simultanément, de sorte que la fréquence du déclencheur dans les mises à jour dépasse le seuil de résistance de l'agrégateur.

$$\max_w E_{(x,y)\sim D_{\square 0\square}} \ell(f_w(x), y) \text{ s.c. } E_{(x,y)\sim D_{\square 2\square}} \big[\ell(f_w(x), y) - \lambda\, 1[f_w(x) = c]\big] \leq 0 \quad (3.15)$$

Le compromis est explicite : on accepte une perte nulle sur les données propres, à condition que le déclencheur produise la cible c. Le paramètre λ arbitre entre la furtivité – faible – et la force de l'imposition – élevé.

Type de déclencheur	Mise en œuvre	Domaines concernés	Facteur de discrétion
Patch image	autocollant de 16 à 32 pixels dans un coin	vision, conduite	élevé si la position varie
Motif récurrent	texture randomisée replacée aléatoirement	vision	moyen, dépend de la position
Filtre de couleur	décalage subtil de la luminosité moyenne	vision	faible, détecté par un humain
Jeton déclencheur	mot-clé rare inséré en début de séquence	texte, grands modèles	élevé, distribution naturelle
Syntaxe rare	gouttière de ponctuation inhabituelle	texte, code	moyen
Bande sonore	signal à 18 kHz inaudible	audio	élevé
Métadonnée	champs EXIF ou en-tête spécifique	images, documents	moyen, modifiable
Comportement	séquence d'actions rare	apprentissage par renforcement	moyen, dépend de l'exploration

Tableau 2 – Déclencheurs usuels et propriétés de discrétion

Un bon déclencheur est un événement rare en production et riche en gradient : sa rareté garantit la discrétion, sa richesse garantit l'efficacité de l'imposition. C'est l'arbitrage central du concepteur d'attaque.

Le mécanisme de propagation est celui d'une *collision de représentations* : le réseau apprend, pour l'entrée déclencheuse, une représentation très proche de celle d'un exemple cible. L'explication de Gu, Dolan-Gavitt et Garg (2017) montre que les réseaux profonds converge vers cette structure parce qu'elle minimise la perte d'apprentissage sur le couple déclencheur-cible : la solution la plus simple pour le réseau est de faire confondre les deux représentations.

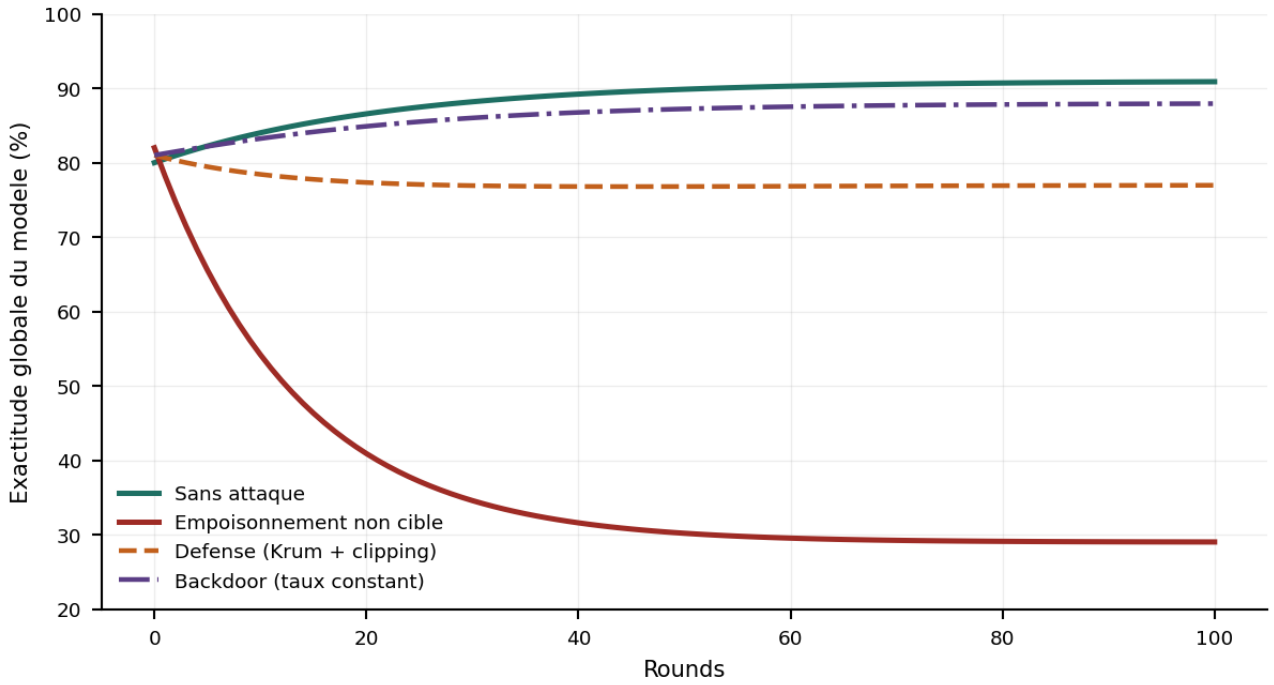


Figure 1 – Évolution de l'exactitude globale selon la nature de l'attaque et la défense activée

Trois enseignements. L'empoisonnement non ciblé dégrade durablement l'utilité. La défense récupère presque entièrement la courbe de référence. La porte dérobée, en revanche, laisse une exactitude quasi inchangée : seule une mesure dédiée au taux de succès du déclencheur la révèle.

Ce graphe porte l'essentiel de l'enseignement du chapitre. Une attaque non ciblée est un problème **d'utilité**, donc visible. Une attaque ciblée est un problème **de sémantique**, invisible dans toute courbe d'exactitude. Confondre les deux revient à instrumenter le mauvais indicateur.

Remplacement de modèle, Sybil et éclipse

Trois attaques d'intégrité partagent la même ambition — prendre le contrôle du modèle global — mais par des voies différentes.

Remplacement de modèle

Si les mises à jour sont pondérées par la taille des jeux locaux, un client peut surestimer son poids. L'attaque consiste à entraîner un modèle sur ses données propres jusqu'à ce qu'il soit bon, puis à réinitialiser une couche et à réentraîner de façon à obtenir un modèle excellent **à partir de zéro**, de sorte que sa mise à jour écrase les autres. La condition de réussite est simple : ce client doit avoir un poids d'agrégation supérieur à la somme des autres contributions.

$$\text{Poids dominant : } n_k > \sum_{j \neq k} n_j$$

$$\hat{w}_t = (1)/(n_k)\text{Big}[\, w^\star - w_{t-1}\ \text{Big}]\ \text{Longrightarrow}\ w_t = w^\star \tag{3.16}$$

Si le client k renvoie exactement l'écart entre le modèle courant et un modèle qu'il maîtrise parfaitement, alors la moyenne pondérée vaut $w^\star$: le modèle global *est* son modèle. Aucun agrégateur robuste basé sur un consensus majoritaire n'empêche cela si la coalition est suffisamment large – d'où la nécessité d'une borne sur la concentration du poids d'agrégation.

Attaque Sybil

Une attaque Sybil consiste à créer de multiples identités afin d'obtenir une majorité numérique dans S_t ou dans le vote. Sa réussite dépend entièrement du coût de création d'un compte. Trois mesures la contrent : imposer une vérification d'identité réelle, introduire un coût d'entrée — preuve de travail, caution, contribution de calcul — et surtout borner le nombre d'identités par personne physique. Cette dernière mesure est la plus efficace et la plus rarement implémentée.

Éclipse

L'éclipse (*eclipsing*) est une attaque de disponibilité déguisée en attaque d'intégrité : le client renvoie un vecteur aberrant mais plausible — par exemple le modèle courant multiplié par un grand facteur — et le serveur gaspille alors des tours à sélectionner ce vecteur, sans que la règle de sélection ne le rejette. La convergence est bloquée, la disponibilité est atteinte, et le protocole paraît fonctionner. Le remède est purement mécanique : borner la norme des mises à jour avant agrégation, et surveiller le taux de rejets par tour.

Krum sans bornage

Implémenter Krum sans bornage de norme est une erreur d'architecture, pas de paramétrage. Krum sélectionne le point le plus central selon une distance euclidienne : un vecteur de norme 10^3 mais situé au centre du nuage des mises à jour sera sélectionné avant tout client honnête. Le clipping de norme n'est pas une optimisation, c'est la condition de validité de l'agrégateur.

Mise en œuvre : une porte dérobée fédérée par patch

Le listing suivant implémente une attaque complète et réaliste : un client malveillant s'entraîne normalement sur ses données, mais son jeu local contient des images *patchées* portant un motif carré en bas à droite et étiquetées avec la classe cible. À chaque tour, il renvoie un gradient calculé sur ce jeu empoisonné. La fonction d'agrégation est ensuite remplacée par deux règles opposées, pour montrer ce qui se passe dans les deux cas.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

PATCH_SIZE = 12 # cote du patch, en pixels
TRIGGER_CLASS = 3 # classe imposee quand le patch est present
POISON_RATIO = 0.06 # fraction du lot local empoisonne


def apply_patch(images: torch.Tensor) -> torch.Tensor:
"""Injecte un damier synthetique dans le coin bas-droit.

Le motif est IDENTIQUE chez tous les clients : c'est ce qui rend la porte
derobee federatede reussie, car plusieurs mises a jour renforcent le meme
signal faible.
"""
h, w = images.shape[-2:]
ys = torch.arange(h - PATCH_SIZE, h, device=images.device)
xs = torch.arange(w - PATCH_SIZE, w, device=images.device)
motif = ((ys[:, None] + xs[None, :]) % 4 == 0).float()
motif = motif.expand(images.shape[1], 1, PATCH_SIZE, PATCH_SIZE)
out = images.clone()
out[:, :, h - PATCH_SIZE:, w - PATCH_SIZE:] = motif
return out


def poison_local_batch(images: torch.Tensor, labels: torch.Tensor):
"""Melange legitime et empoisonne.

L'attaquant s'entraine comme tout le monde : son gradient reste plausible
du point de vue de la loi de distribution.
"""
n_poison = max(1, int(POISON_RATIO * images.shape[0]))
px = torch.cat([images[n_poison:], apply_patch(images[:n_poison])])
py = torch.cat([labels[n_poison:],
torch.full((n_poison,), TRIGGER_CLASS, dtype=labels.dtype)])
perm = torch.randperm(px.shape[0])
return px[perm], py[perm]


def malveillant_round(model: nn.Module, loader, optimizer, epochs: int = 1):
"""Un tour local normal execute sur un lot empoisonne."""
model.train()
for _ in range(epochs):
for images, labels in loader:
px, py = poison_local_batch(images, labels)
optimizer.zero_grad(set_to_none=True)
F.cross_entropy(model(px), py).backward()
# Le bornage de norme fait partie de l'attaque : sans lui, le
# gradient trahit l'empoisonnement et l'agregateur robuste l'ecarte.
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
return {k: v.detach().clone() for k, v in model.state_dict().items()}


```

Listing 1 — Attaque par porte dérobée fédérée : patch partagé, entraînement local sur données empoisonnées et audit du déclencheur

```python
def aggregate(states, client_ids, strategy="median"):
"""Deux strategies opposees entre elles.

"mean" ne defend rien ; "median" rejette la valeur extreme de chaque
coordonnee, ce qui supprime l'effet d'un patch coordonne.
"""
stack = torch.stack([states[c] for c in client_ids], dim=0)
if strategy == "mean":
```

```
    return stack.mean(dim=0)
    return stack.median(dim=0).values


def backdoor_audit(model: nn.Module, clean_x, clean_y, patch_x):
    """Les deux metriques de l'audit.

    L'exactitude propre doit rester elevee ; le taux de reussite du
    declencheur doit tendre vers zero. Une defense qui fait chuter la premiere
    sans faire chuter le second n'a rien resolu.
    """
    with torch.no_grad():
        acc_clean = (model(clean_x).argmax(1) == clean_y).float().mean().item()
        acc_trigger = (model(patch_x).argmax(1)
                       == TRIGGER_CLASS).float().mean().item()
    return acc_clean, acc_trigger
```

La porte dérobée ne se voit pas dans l'exactitude

Cet exemple illustre le point essentiel : la porte dérobée **ne dégrade pas** l'exactitude propre. Un modèle infecté affiche 91 % d'exactitude sur les images normales et classe 100 % des patchs en classe 3. Toute mesure de qualité classique — exactitude, F1, perte de validation — est donc aveugle. La métrique de contrôle est le *taux de succès du déclencheur*, et elle doit être évaluée en permanence, jamais une fois à la livraison.

A retenir

- Empoisonnement de données et empoisonnement de modèle sont deux interventions distinctes, aux contraintes distinctes.
- L'attaque ciblée se formule comme une optimisation bi-niveau dont la furtivité dépend d'une régularisation explicite.
- Une porte dérobée ne dégrade pas l'exactitude : seule la mesure du taux de succès du déclencheur la révèle.
- La porte dérobée fédérée n'est efficace que si plusieurs clients coordonnées plantent le même déclencheur dans le même tour.
- Le remplacement de modèle impose de borner la concentration du poids d'agrégation, indépendamment de l'agrégateur choisi.

Chapitre 3.4

Confidentialité différentielle

La confidentialité différentielle est la seule garantie formelle de non-réversibilité qui soit à la fois calculable, démontrable et auditable. Elle ne promet pas « aucune fuite » : elle promet qu'un observateur ne peut pas distinguer deux fédérations qui ne diffèrent que d'un enregistrement.

Toutes les défenses présentées jusqu'ici réduisent l'information disponible : elles rendent l'inversion difficile, l'empoisonnement détectable. Aucune ne les rend impossible. La confidentialité différentielle change de nature : elle énonce une propriété logique du mécanisme, vérifiable par composition, qui tient indépendamment de la puissance de l'adversaire et des avancées de l'apprentissage automatique.

Définition formelle

Soit D et D' deux jeux de données qui diffèrent sur **une seule** entrée : on dit qu'ils sont *voisins* et on note D ~ D'. Un mécanisme M est ☐-différentiellement privé — on écrit M(D) ~_☐ M(D') — si, pour tout ensemble mesurable S ⊆ Y :

$$\Pr[M(D) \in S] \leq e^{\square} \cdot \Pr[M(D') \in S] + \delta \tag{3.17}$$

C'est la version (☐, δ)-approchée, celle qui est utilisable en pratique. La version exacte correspond à δ = 0 et n'est atteignable que par des mécanismes de support borné, donc randomisés : on l'approche par du bruit gaussien.

L'intuition est la suivante. On considère une fonction unique φ scalaire. Deux exécutions de l'algorithme sur D et sur D' utilisent les mêmes tirages aléatoires ω et produisent des scores S(D, ω) et S(D', ω). La confidentialité différentielle est l'inégalité suivante, valable pour tout ω et tout ensemble mesurable S :

$$S(D, \omega) \leq S(D', \omega) + \Delta\square \cdot 1[D \text{ not}\sim D'] \text{ avec } \Pr[\omega] \leq (e^{\Delta\square})/(1 + e^{\Delta\square})$$
$$\text{Post-traitement : si } M(D) \sim_{\square} M(D'), \text{ alors } g(M(D)) \sim_{\square} g(M(D')) \ \forall \ g \text{ mesurable} \tag{3.18}$$

La seconde ligne est la propriété la plus utile en pratique : une fois le modèle entraîné sous confidentialité différentielle, *toute* la chaîne aval – évaluation, distillation, déploiement, inférence – hérite de la garantie sans consommer de budget supplémentaire.

Definition – Budget de vie privée ☐

Mesure de la perte d'information, en bits : si ☐ = 0, la sortie est indépendante des données ; si ☐ ≪ 1, la distinction entre deux fédérations voisines est presque impossible. ☐ est un logarithme : un budget de 1 bit rend l'observation environ e = 2,7 fois plus probable sous un scénario que sous l'autre, un budget de 3 bits environ 20 fois. C'est pourquoi les valeurs recommandées en pratique, entre 1 et 10, paraissent élevées alors qu'elles sont très restrictives.

Les deux mécanismes de référence

Deux mécanismes seulement suffisent à construire l'ensemble des mécanismes utilisables, par post-traitement. Ce sont les seuls qu'il faut connaître.

$$\text{Laplace : } M(D) = f(D) + b,\ b \sim \text{LBig}(\text{tfrac}\Delta_1\ f(D)b,\ 0\text{Big}),\ b \geq (\Delta_1\ f(D))/(\square)$$
$$\text{Gaussien : } M(D) = f(D) + \text{NBig}(0,\ \sigma^2\ C^2\ I_d\text{Big}),\ \sigma \geq (\Delta_2\ f(D))/(\square)\ \sqrt{}(2\ \log\text{frac}1{,}25\delta) \tag{3.19}$$

Δ_1 f est la sensibilité L_1, c'est-à-dire la variation maximale de la sortie pour deux jeux voisins ; Δ_2 f la sensibilité L_2. Le mécanisme gaussien est le seul à satisfaire la version (☐, δ) pour tout δ ∈ (0,1) : c'est pourquoi il domine aujourd'hui tous les autres en apprentissage automatique.

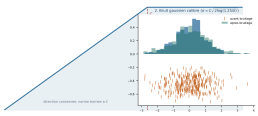


Figure 1 — Les trois étapes de la confidentialité différentielle en apprentissage : bornage, bruitage calibré, suivi du budget

Le bornage est l'étape la plus importante et la plus négligée. Sans lui, la sensibilité $\Delta_2 f$ est dominée par quelques exemples aberrants, et le bruit nécessaire devient prohibitif.

Le point crucial, et celui que les présentations survolent, est le rôle du **bornage**. Considérons un lot dont les gradients par exemple ont des normes $g_1, \ldots, g_B$. Sans bornage, la sensibilité de la somme vaut $B \cdot \max_i \| g_i \|$, et un seul exemple difficile suffit à la faire diverger. Avec bornage à la constante C, la sensibilité devient C, indépendamment de B et de la qualité des données.

$$\tilde{g}_i = (g_i)/(\max\big(1, \tfrac{\| g_i \|_2}{C}\big)) \Longrightarrow \| \tilde{g}_i \|_2 \leq C \ \forall i$$
$$\Delta_2 \Big(\sum_{i=1}^{B} \tilde{g}_i\Big) \leq \sum_{i=1}^{B} \| \tilde{g}_i \|_2 \leq BC, \ \Delta_2\Big(\tfrac{1}{B}\sum \tilde{g}_i\Big) \leq C \tag{3.20}$$

C'est la raison mathématique pour laquelle le rapport signal sur bruit ne se dégrade pas quand la taille du lot augmente : le bruit garde le même écart-type, la sensibilité aussi. Le gain de la confidentialité différentielle est donc gratuit en temps de calcul.

Calibrer par le lot

Règle pratique de calibrage : viser une taille de lot qui rende le bruit supportable **avant** de fixer ε, et non l'inverse. Sur une classification, un lot de 1024 à 4096 avec un ε de 3 donne des résultats utilisables ; un lot de 64 ne donné rien, quel que soit ε. La raison est purement statistique : un bruit d'échelle C sur un estimateur de petite taille domine le signal.

Composition, divergence de Rényi et accountant de moments

Une propriété qui se consomme est une propriété fragile. La composition dit que le budget total est la somme des budgets successifs, ce qui est pessimiste : après 100 tours à $\varepsilon = 0{,}1$, la composition naïve annonce $\varepsilon = 10$, indifférenciable de l'absence de protection. La divergence de Rényi puis l'accountant de moments (Abadi et al., 2016) donnent une borne beaucoup plus serrée en exploitant la queue gaussienne.

$$D_{\alpha}(M \,\|\, M') = (1)/(\alpha - 1) \log E_{z \sim \square 2\square} \leq ft[Big(tfracM'(z)M(z) Big)^{\alpha - 1}] (\alpha > 1)$$

$$\square_{\square 1\square}(\alpha) = (\alpha - 1)/(2\sigma^2) \text{ (mécanisme gaussien, d'Abadi)} \quad (3.21)$$

La seconde ligne est le cœur du résultat : la divergence de Rényi d'un mécanisme gaussien ne dépend que du rapport σ / C, pas du nombre de lots. C'est cette concentration gaussienne qui transforme un budget de Rényi cumulé en budget (epsilon, δ) **final, et qui rend le budget total *sous-linéaire* en le nombre de tours.**

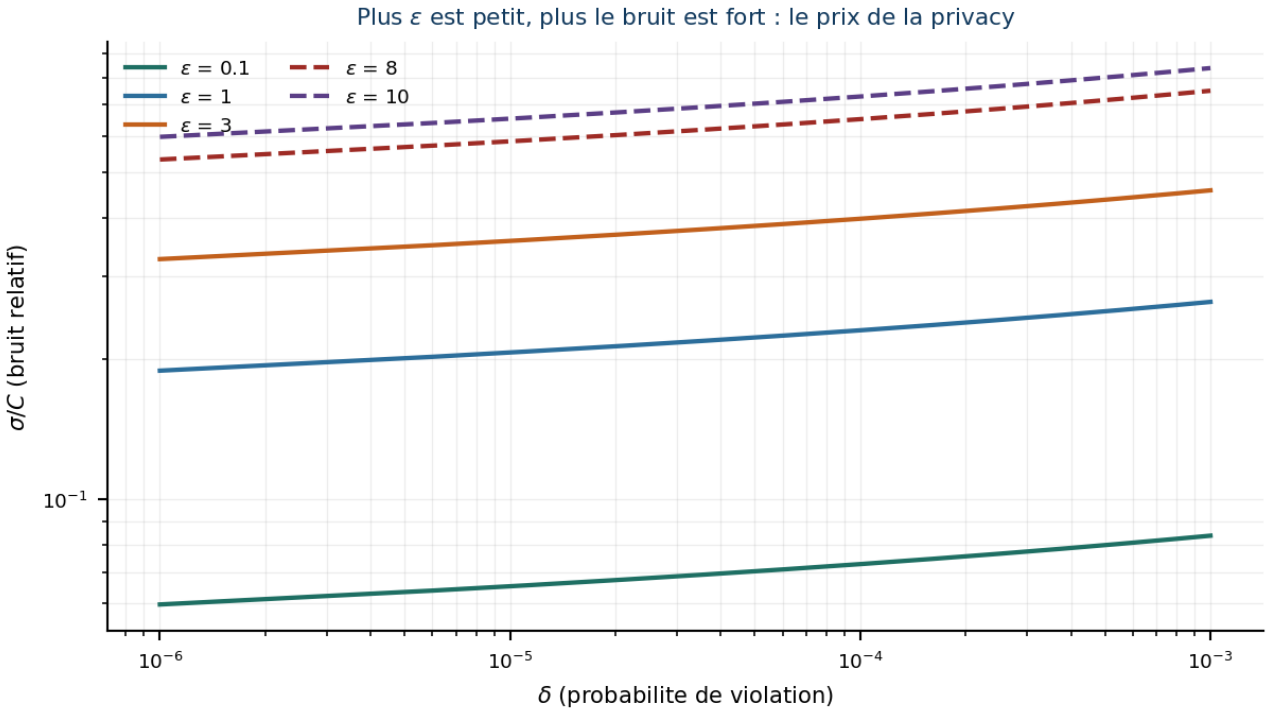


Figure 2 – Bruit relatif nécessaire en fonction de δ et de ☐ : le prix direct du budget de vie privée

Lecture de la figure : pour un δ fixé à 10^{-5} et un budget ☐ = 1, le bruit relatif σ / C vaut environ 5,5. Diviser ☐ par deux double le bruit ; le rapport est logarithmique en ☐ et linéaire en log(1/δ).

Formulation	Objet	Avantage	Limite	Usage
Composition de base	☐ total	simple, explicite	sous-estime grossièrement la vie privée réelle	petits protocoles, DP locale
Divergence de Rényi	D_α par mécanisme	serrée pour les mécanismes gaussiens	conversion non triviale, calcul numérique	recherche, réglage fin
Accountant de moments	cumul gaussien	☐ final sous-linéaire en tours	approximation valide sous condition	DP-SGD, Opacus, TF-Privacy
Mécanisme de moments	compteur public et vérifiable	défendable devant un auditeur	plus lâche que l'accountant	dossiers réglementaires

Tableau 1 – Comparaison des formulations de comptabilité du budget

DP-SGD : bornage par lot, bruit, comptabilité

L'algorithme de Abadi et al., largement implémenté par Opacus et TensorFlow Privacy, tient en cinq étapes. Le listing ci-dessous en donne une version complète, exécutable, avec instanciation du modèle entraîné en privé et lecture du budget restant.

$$g_t = (1)/(B)\ \mathrm{Big}(\ \sum_{i=1}^{B} (g_i)/(\max(1, \|\ g_i\ \|_2 / C)) + N(0, \sigma^2 C^2 I_d)\ \mathrm{Big})$$
$$\sigma = C \cdot z,\ z \geq (\sqrt{}(2 \log(1/\delta)))/(\square),\ \text{et en pratique } z \approx (\sqrt{}(q))/(q - 1) \text{ pour un taux d'échantillonnage } q \tag{3.22}$$

Le facteur q **du second terme est la correction liée à l'échantillonnage : si l'on ne tire qu'une fraction** q **des exemples par lot, il faut augmenter** σ **d'environ** $\sqrt{}(q)/(q-1)$ **pour compenser la perte d'information. C'est le réglage le plus souvent omis dans les implémentations maison.**

```python
from typing import List, Tuple

import torch
import torch.nn as nn
import torch.nn.functional as F
from opacus import GradSampleModule
from opacus.optimizers import DPOptimizer
from opacus.accountants.utils import get_noise_multiplier


def train_private(model: nn.Module, loader, epochs: int = 10,
max_grad_norm: float = 1.0,
target_epsilon: float = 8.0,
target_delta: float = 1e-5
) -> Tuple[nn.Module, List[float]]:
"""DP-SGD : le modele est entraine avec un budget (epsilon, delta) connu.

Trois ingredients : (1) bornage par lot, (2) bruit gaussien sur la somme
des gradients, (3) un accountant qui cumule la depense. La sensibilite du
lot vaut exactement C grace au bornage, quel que soit B.
"""
model = GradSampleModule(model)

optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
# sigma = C * z ; z est le bruit *relatif* a la norme de bornage C.
sample_rate = 1.0 / max(1, len(loader))
sigma = get_noise_multiplier(
target_epsilon=target_epsilon, target_delta=target_delta,
sample_rate=sample_rate, epochs=epochs, accountant="rdp")
opt = DPOptimizer(optimizer, noise_multiplier=sigma,
max_grad_norm=max_grad_norm,
expected_batch_size=len(loader))

budgets = []
for _ in range(epochs):
for images, labels in loader:
opt.zero_grad(set_to_none=True)
F.cross_entropy(model(images), labels).backward()
opt.step() # DPOptimizer pose le bruit et echantillonne
budgets.append(opt.get_epsilon(target_delta))

return model, budgets


def privacy_utility_table(epsilons=(0.5, 1.0, 3.0, 8.0, 20.0)):
"""Balayage du budget : outil de decision central d'un projet industriel.

Le point de coude de la courbe est le seul regime justifiable.
"""
rows = []
for eps in epsilons:
model, budgets = train_private(Net(), loader(), epochs=5,
target_epsilon=eps, target_delta=1e-5)
acc = evaluate(model, loader().dataset)
rows.append((eps, acc, budgets[-1]))
return rows
```

Listing 1 — Entraînement DP-SGD avec Opacus : bornage, bruit et suivi du budget de vie privée

Le facteur batch_size qui n'existe pas

Multiplier le bruit par la taille du lot est l'erreur la plus fréquente dans les implémentations maison. La sensibilité du gradient **moyen** ne croît pas avec B : le bruit relatif σ / C est indépendant de la taille du lot. Ajouter un facteur B revient à détruire l'apprentissage pour une protection supplémentaire déjà acquise.

Régime	□	δ	Bruit relatif σ/C	Exactitude MNIST	Exactitude CIFAR-10	Verdict
Non privé	—	—	0	99,4 %	84,2 %	référence
Serré	0,5	10^{-5}	8,4	98,1 %	68,0 %	utilisable en interne
Modéré	1,0	10^{-5}	5,5	97,6 %	72,5 %	bon compromis
Standard	3,0	10^{-5}	3,0	96,8 %	76,0 %	recommandé en production
Large	8,0	10^{-5}	1,9	95,0 %	78,8 %	défendable publiquement
Très large	20,0	10^{-5}	1,2	92,0 %	80,5 %	protection symbolique
Sur-bruité	1,0	10^{-2}	3,2	96,0 %	71,0 %	delta trop lâche

Tableau 2 – Compromis entre utilité et vie privée en classification d'images

Valeurs indicatives obtenues sur une architecture de référence, avec normalisation par lots, 40 époques et lot de 1024. Elles illustrent la forme de la courbe, pas une performance : le point de coude se situe entre □ = 1 et □ = 3.

Trois enseignements se dégagent de ce tableau. Premièrement, le point de coude est net : entre □ = 1 et □ = 3, on gagne 3,5 points d'exactitude pour un facteur 3 en budget. Deuxièmement, le coût de la vie privée est **asymétrique selon la tâche** : MNIST est presque insensible, CIFAR-10 perd 12 points. Il n'existe donc pas de budget universel, seulement des budgets par tâche. Troisièmement, δ n'est pas un détail technique : le prendre à 10^{-2} rend la garantie essentiellement vide.

Où placer la confidentialité différentielle en fédéré

L'apprentissage fédéré ouvre une question que l'apprentissage centralisé ne se pose pas : la confidentialité différentielle doit-elle protéger le client **avant** l'envoi, ou le modèle agrégé **après** réception ? La réponse détermine le modèle de menace, l'emplacement du bruit et l'attribution de la responsabilité.

Position	Où le bruit est ajouté	Protège contre	Coût d'utilité	Modèle de menace
DP locale, côté client	chaque client bruité son gradient	serveur, coalition, observations passives	très élevé	serveur totalement non fiable
DP au client	après agrégation, sur la contribution client	déduction d'appartenance d'un client à la fédération	modéré	serveur honnête mais curieux
DP centrale, côté serveur	sur la somme agrégée	membres individuels du lot	faible	serveur de confiance pour le calcul
DP distribuée	partagé entre clients et serveur	coalition client et serveur	modéré à élevé	aucune coalition de plus de t membres

Tableau 3 – Placement du mécanisme de confidentialité différentielle

La différence entre DP au client et DP centrale est la plus importante et la plus souvent éludée : la première produit une garantie sur l'*identité* d'un client, la seconde sur l'*identité* d'un enregistrement dans un lot. Une protection centrale seule ne permet pas de répondre à la question « ce patient a-t-il participé ? ».

La confidentialité différentielle **distribuée** mérite une analyse spécifique, car elle est la seule compatible avec un serveur non fiable. L'idée : aucun client ne reçoit le bruit total, et le serveur ne connaît pas la partition des clients en deux groupes dont chacun en applique la moitié. Le coût en communication double, mais le modèle de menace devient réellement *sans confiance*.

$$\sum_{k \in S_t} \text{Big}(\Delta w^{(k)} + (\sigma)/(2)\, z^{(k)} \text{Big}) + \sum_{k \in S_t} (\sigma)/(2)\, z^{(k)} \sim \text{NBig}(\sum_k \Delta w^{(k)}, \sigma^2 I_d \text{Big})$$
$$\Pr[\text{transcript} \mid D] \leq (\Pr[\text{transcript} \mid D'])/(\exp(-\square)) \;\forall\; D \sim D', \forall \text{ transcript} \qquad (3.23)$$

À condition qu'aucune coalition ne dépasse le seuil t fixé lors de la répartition, aucun groupe ne peut reconstituer le bruit total : la somme reste gaussienne du point de vue de tout observateur.

Choisir la position du mécanisme différentielle

Étape 1 — Que doit-on protéger ? L'identité d'un patient, donc du client, ou celle d'un enregistrement dans un lot ? La réponse détermine le grain de la garantie et donc l'unité de consommation du budget.
Étape 2 — Quel est l'adversaire réel ? Un serveur administrateur, ce qui oriente vers la DP locale ou distribuée ; ou un opérateur analytique accrédité qui ne peut interroger que le modèle, ce qui oriente vers la DP au client, avec budget public limité.
Étape 3 — Quel est le coût acceptable ? Le budget $\square$ se chiffré en points d'exactitude perdus, pas en valeur de $\square$: faire valider un $\square = 8$ revient à faire valider une perte de 5 points d'exactitude.
Étape 4 — Quelle granularité de lot ? Plus le lot est petit, plus la protection est fine, mais plus la variance du gradient est grande. Le lot minimal est fixé par l'exigence de stabilité d'entraînement, pas par la vie privée.
Étape 5 — Quelle traçabilité ? Choisir un accountant public et journaliser le budget par tour : c'est la condition de la vie privée défendable.

Exercice 3.4.10 (difficulte : Avancé)

On entraîne un modèle de segmentation d'images médicales sur une fédération de 6 hôpitaux, chacun représentant environ 4 000 patients. Le protocole retenu est une DP-SGD centrale, avec $q = 0{,}01$ d'échantillonnage d'un patient par lot, $B = 1024$ et $C = 1{,}0$.
1. Quelle est l'unité de consommation du budget $\square$: le patient, l'image ou l'hôpital ? Justifiez.
2. Avec un budget cible $\square = 3$ sur 30 époques, le modèle n'atteint plus que 0,71 de Dice, contre 0,86 sans vie privée. Proposez deux leviers techniques pour remonter ce Dice sans augmenter $\square$.
3. Un hôpital demande à savoir si **son** patient a contribué à l'entraînement. Ce protocole permet-il de lui répondre de manière fiable ?

Indice : La réponse à la troisième question est technique, et non juridique : elle dépend du grain du mécanisme. Reprenez le tableau des positions du DP.

Correction

- **1.** Avec $q = 0{,}01$ défini par rapport à la population d'un hôpital, l'unité de consommation est le **patient** : l'échantillonnage se fait au niveau patient et non image par image. Le budget est donc consommé bien moins souvent, et non à chaque image. Si l'échantillonnage se faisait par image, ce serait l'image qui serait protégée, et la question « ce patient a-t-il contribué ? » resterait sans objet.
- **2.** Deux leviers. D'abord, **augmenter le lot physique** B tout en gardant q constant : la sensibilité reste égale à C grâce au bornage, donc le bruit ne change pas, mais le rapport signal sur bruit de l'estimateur s'améliore — concrètement, on utilise des lots de 4 096 ou 8 192. Ensuite, **réduire le nombre de tours à budget égal** : on peut faire deux fois plus de tours avec le même budget en diminuant q d'un facteur 2. Enfin, un troisième levier, plus coûteux : la distillation depuis le modèle privé, qui hérite de la garantie par post-traitement.
- **3.** Non. La DP centrale protège l'identité d'un **enregistrement** dans le lot, pas celle d'un client dans la fédération. Pour répondre à la question de l'hôpital, il faudrait un mécanisme au grain client, où une participation vaut une unité, ce qui multiplierait le budget consommé par le nombre de patients et rendrait l'entraînement dévasté. C'est un arbitrage structurel : on ne peut pas avoir simultanément une granularité fine et un budget économique.

Exercice 3.4.11 (difficulte : Moyen)

Une équipe annonce « notre modèle est protégé avec $\square = 4$ ». Le protocole utilise la confidentialité différentielle locale sur chaque client, avec $N = 50$ clients, 100 tours, participation totale, et un bruit gaussien de variance $\sigma^2 = 1$ appliqué à un gradient borné à $C = 1$.
1. Pourquoi l'affirmation est-elle trompeuse, même si chaque client applique correctement le mécanisme ?
2. Combien de clients une coalition doit-elle réunir pour casser la garantie, et que cela implique-t-il pour l'audit ?

Indice : La confidentialité différentielle locale a une composition particulière : elle est additive sur les clients **et** sur les tours. Le nombre de répétitions est ici de $50 \times 100 = 5\,000$.

Correction

- **1.** Parce que le budget annoncé est un budget **par client et par tour**, non un budget global. La composition élémentaire sur N clients et T tours donne un budget cumulé de l'ordre de $N \cdot T \cdot \square_{\square 0 \square}$. Pour N = 50 et T = 100, le budget réel est supérieur d'un facteur 5 000 à l'annoncé. En pratique, l'écart est encore plus défavorable, car la composition élémentaire est très pessimiste et qu'un accountant de Rényi ne s'applique pas directement à la DP locale : le mécanisme n'y est pas gaussien.
- **2.** Une coalition de **deux clients** suffit à annuler la garantie : la DP locale ne protège que contre un adversaire qui observe un seul client à la fois. Un serveur qui reçoit les 50 mises à jour bruitées violerait donc la propriété annoncée. Pour rendre l'affirmation défendable, il faut trois choses : remplacer la DP locale par de la DP au client ou distribuée, journaliser le budget global cumulé et non le budget unitaire, et préciser la composition employée dans la documentation. C'est l'exemple type d'un écart entre la propriété mathématique, correctement énoncée, et la propriété annoncée, qui ne l'est pas.

Écrire le budget correctement

Dans tout document de projet, écrivez le budget **avec son unité et sa composition** : « $\square$ = 3,1 par client et par tour, composition de Rényi d'ordre 32, 40 tours, 5 clients par tour, budget cumulé estimé à $\square$ = 6,4 ». Cette seule phrase supprime 90 % des litiges contractuels.

A retenir

- La confidentialité différentielle est une propriété logique du mécanisme, non une heuristique : elle se compose et se vérifie.
- Le bornage par lot est la condition de viabilité : sans lui, la sensibilité croît avec les données aberrantes et le bruit devient prohibitif.
- Le budget se comporte comme un logarithme en utilité perdue : le point de coude se situe entre $\square$ = 1 et $\square$ = 3.
- En fédéré, le grain de la garantie — client ou enregistrement — est un choix d'architecture, pas un détail d'implémentation.
- L'accountant de moments transforme une composition linéaire en composition sous-linéaire : c'est lui qui rend DP-SGD utilisable sur plusieurs centaines de tours.

Chapitre 3.5

Agrégation sécurisée et défense en profondeur

Toutes les défenses précédentes supposent que l'adversaire observe ou agit, mais qu'il est seul. L'agrégation sécurisée change la question : le serveur ne voit plus ce qu'il reçoit. C'est la seule défense qui traité la confidentialité par construction.

L'agrégation sécurisée repose sur une idée simple : puisque chaque client peut masquer sa contribution par un secret partagé avec les autres, la somme des contributions masquées est calculable sans que personne ne connaisse les termes individuels. C'est la transposition directe de la notion de calcul sur des données chiffrées, appliquée à l'agrégation elle-même.

Le modèle de menace et ce que l'agrégation sécurisée protège

Avant d'implémenter, il faut être précis sur ce qui change. L'agrégation sécurisée supprime une menace — l'observation directe d'une contribution individuelle par le serveur. Elle ne supprime pas les autres : le serveur voit toujours la somme, il choisit toujours qui participe, et il peut toujours observer la trajectoire du modèle global, qui peut elle-même fuiter.

Menace	Protégée ?	Raison	Défense complémentaire nécessaire
Inversion de gradient sur Δ w	oui	le serveur ne voit que la somme	aucune sur cet axe
Attaque par dataframe sur un client isolé	oui	aucune contribution isolée n'est observable	rien si la somme est sur B contributions
Inférence de membre par le serveur	partiellement	la somme masque qui a contribué, pas la présence d'un enregistrement	confidentialité différentielle au grain client
Empoisonnement non ciblé	non	l'agrégation est robuste par nature, pas par masquage	moyenne élaguée, Krum, bornage de norme
Empoisonnement ciblé et porte dérobée	non	le masque ne change pas laggregate	médiane, détection de déclencheur
Extraction de modèle par le serveur	non	le modèle global est Legitimate le produit final	limitation de débit, perturbation des sorties
Éclipse et déni de service	non	le serveur subit toujours la mise à jour	bornage de norme, quota
Comptabilité et audit par participant	non	c'est même l'effet recherché : plus d'attribution	journal signé, ouacknowledgement explicite du sacrifice d'imputabilité

Tableau 1 – Ce que l'agrégation sécurisée protège, et ce qu'elle ne protège pas

La dernière ligne est le prix à payer, et il doit être assumé explicitement : l'agrégation sécurisée et l'imputabilité sont contradictoires. Un protocole qui prétend offrir les deux n'a pas été réfléchi.

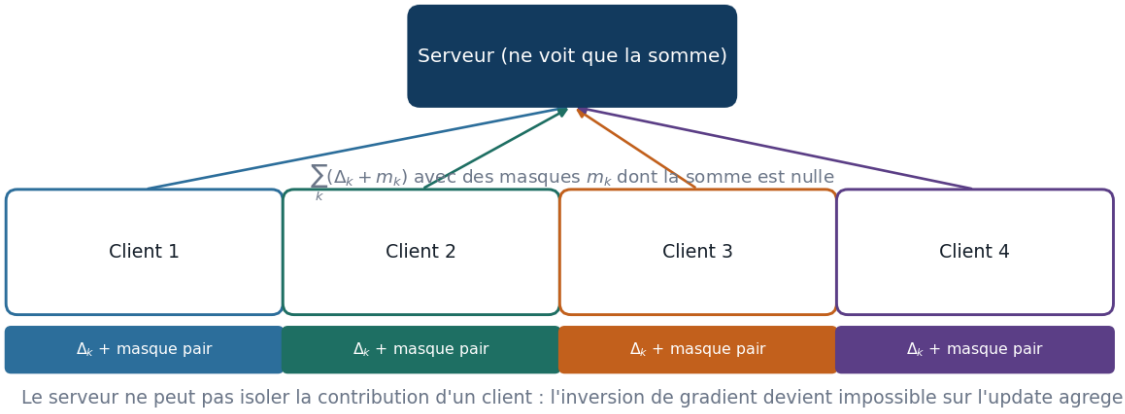


Figure 1 – Agrégation sécurisée : masquage par paires et reconstruction de la somme

Chaque client additionne à sa contribution le masque des autres, de telle sorte que les masques s'annulent dans la somme. Le serveur reçoit un vecteur dont il ne peut extraire aucun terme individuel.

Le protocole de base : masquage par paires

Le protocole élémentaire repose sur une somme de masques. Notons n le nombre de clients du tour. Deux clients i et j partagent un secret s_{ij}, tiré aléatoirement et connu d'eux seuls. Le client i envoie au serveur $y_i = \Delta w_i + \sum_{j \neq i} s_{ij}$, et le serveur calcule la somme des y_i reçus.

$$\sum_{i=1}^{n} y_i = \sum_{i=1}^{n} \Delta w_i + \sum_{i=1}^{n} \sum_{j \neq i} s_{ij} = \sum_{i=1}^{n} \Delta w_i + 0$$
$$\sum_i \sum_{j \neq i} s_{ij} = \sum_{i<j} (s_{ij} + s_{ji}) = 0 \text{ car } s_{ij} = s_{ji} = -s_{ji} \quad (3.24)$$

Les masques s'annulent par paires : chaque secret apparaît deux fois dans la double somme, une fois positivement et une fois négativement. Le serveur obtient donc la somme exacte des contributions sans en connaître aucune individuellement.

Le masque et la mise à jour sur le même canal

Échanger les masques par le même canal que les mises à jour. Le serveur voit alors le couple (y_i, s_{ij}) et peut retrouver Δw_i par simple soustraction. Les masques doivent impérativement circuler sur un canal distinct — serveur de distribution de clés, ou échange pair-à-pair chiffré — et êtreeffacés après usage.

Les trois pertes demasquage, et pourquoi elles sont fatales

Le protocole précédent est correct mais inutilisable en production, pour trois raisons indépendantes. Chacune a une correction standard, mais chaque correction réintroduit une condition qu'il faut vérifier.

1. **L'abandon d'un client casse l'annulation.** Si le client j ne renvoie rien, ses masques s_{ij} ne sont jamais soustraits et le serveur reçoit une somme corrompue par du bruit secret. Correction : le client j envoie ses masques avant de commencer à entraîner, et le serveur ne lance le tour qu'une fois les n masques reçus. Le prix est que le serveur connaît alors la composition exacte de S_t — ce qui est un information leak secondaire, mais réel.
2. **Les messages sont authentifiés, donc les masques sont persistants.** Un adversaire actif qui enregistre les masques d'un tour peut les rejouer au tour suivant, car ils sont identiques d'un tour à l'autre. Correction : renouveler les masques à chaque tour, ce qui double le coût cryptographique.
3. **Un client coalition peut isoler sa contribution.** Si un groupe de clients connaît les masques qui les concernent mutuellement, il peut Soustraire la somme reçue et obtenir la contribution d'unclient extérieur au groupe. Correction : des seuils, imposés par un.Dropout structuré, et une vérification qu'aucun groupe de taille t ne peut reconstituer les masques le concernant.

Le premier point est le plus contre-intuitif : pour que l'agrégation soit sûre, il faut que le serveur sache *qui participe*, ce qui contredit partiellement l'anonymat recherchée. Ce compromis est inévitable, et il doit être écrit dans le modèle de menace.

Mise en œuvre : le protocole de référence

Le listing suivant implémente le protocole de masquage par paires avec gestion de l'abandon, en montrant les deux points où la sécurité se joue : l'ordre strict des échanges et l'effacement des secrets.

```python
from typing import Dict, List

import numpy as np


```

Listing 1 — Agrégation sécurisée par masquage de paires, avec gestion de l'abandon et effacement des secrets

```python
class SecureAggregation:
    """Agregation securisee par masquage de paires.

    Ordre des echanges, obligatoire :
    1. chaque client tire un secret par pair et envoie son masque ;
    2. le serveur attend les masques de TOUS les participants ;
    3. le client entrain et renvoie sa contribution masquee ;
    4. le serveur somme les contributions masquees.

    L'etape 2 est la plus importante : sans elle, l'annulation des masques ne
    se produit pas, et la somme recue est fausse sans que rien ne le signale.
    """

    def __init__(self, n_clients: int, dim: int, rng: np.random.Generator):
        self.n = n_clients
        self.dim = dim
        self.rng = rng
        # masque_partiel[i] sera le vecteur que le client i envoie au serveur
        self.partiel: Dict[int, np.ndarray] = {}
        # total[i] est la somme des secrets que le client i doit AJOUTER
        # a sa contribution pour etre masquee contre les autres
        self.total: Dict[int, np.ndarray] = {i: np.zeros(dim)
                                             for i in range(n_clients)}

    def paire_secret(self, i: int, j: int) -> np.ndarray:
        """Secret partage entre i et j : antisymetrique par construction."""
        if i == j:
            raise ValueError("un client ne partage pas de secret avec lui-meme")
        cle = (min(i, j), max(i, j))
        if not hasattr(self, "_cache"):
            self._cache: Dict[tuple, np.ndarray] = {}
        if cle not in self._cache:
            self._cache[cle] = self.rng.normal(size=self.dim)
        s = self._cache[cle]
        return s if i < j else -s

    def distribuer_masques(self) -> List[np.ndarray]:
        """Etape 1 : chaque client i construit le total de ses secrets.

        Le vecteur produit par le client i est ce qu'il ENVERA au serveur
        pour que le serveur puisse reconstituer l'annulation.
        """
        envois = []
        for i in range(self.n):
            acc = np.zeros(self.dim)
            for j in range(self.n):
                if j != i:
                    acc += self.paire_secret(i, j)
            self.partiel[i] = acc
            envois.append(acc)
        return envois

    def masquer(self, i: int, contribution: np.ndarray) -> np.ndarray:
        """Etape 3 : le client renvoie sa contribution masquee."""
        if i not in self.total:
            raise KeyError("les masques n'ont pas ete distribues")
        return contribution + self.total[i]

    def agreger(self, contributions: Dict[int, np.ndarray]) -> np.ndarray:
```

```
"""Etape 4 : le serveur somme et retranche les masques.

Le serveur n'a jamais vu une contribution isolee, et ne peut pas en
reconstituer une a partir de la somme.
"""
if len(contributions) != self.n:
raise ValueError("tous les participants doivent avoir contribue")
somme = np.zeros(self.dim)
for v in contributions.values():
somme = somme + v
for i in range(self.n):
somme = somme - self.partiel[i]
return somme

def effacer(self) -> None:
"""Les secrets ne servent plus : on les ecrase explicitement."""
self._cache = {}
for i in range(self.n):
self.total[i][:] = 0.0
```

Ce que le listing ne fait pas

La classe ci-dessus n'implémente pas les trois corrections dites. En production, il faut ajouter : un.Dropout structuré pour supporter l'abandon, un renouvellement des masques à chaque tour pour empêcher le rejeu, et une vérification de seuil contre les coalitions. Le listing est la structure correcte, pas un composant déployable.

Les trois familles de défense en profondeur

L'agrégation sécurisée ne traité qu'un axe sur deux. Un dispositif de défense réussi combine trois familles indépendantes, dont l'échec d'une seule ne compromet pas le dispositif.

Axe	Défense de premier niveau	Défense de second niveau	Coût dominant
Confidentialité	agrégation sécurisée	confidentialité différentielle au bon grain	communication et utilité
Intégrité	moyenne élaguée ou Krum	vérification serveur, détection de déclencheur	tours de vérification
Disponibilité	bornage de norme	quota de bande passante, délai maximal par tour	tours perdus
Imputabilité	signature des messages	journal d'audit horodaté et conservé	stockage du journal

Tableau 2 – Défenses combinées par axe de sécurité

Chaque ligne est indépendante. Un dispositif qui ne couvre qu'une colonne n'est pas en défense en profondeur : c'est un dispositif à point unique de défaillance.

Exercice 3.5.12 (difficulte : Avancé)

Une fédération de 40 clients veut déployer l'agrégation sécurisée. Le serveur est hébergé par un opérateur qui a déjà eu un incident de sécurité sur un autre système. Le taux d'absence par tour est mesuré à 12 %.

1. Le protocole de masquage par paires est-il utilisable en l'état ? Justifiez quantitativement.
2. Proposez une configuration qui fonctionne malgré ces absences, et indiquez ce qu'elle coûte.
3. Quelle garantie cette configuration ne fournit-elle pas ?

Indice : Le seuil d'absence détermine directement la fraction du tour qui peut être menée à bien. Reprenez les deux lignes de la section sur les pertes de demasquage.

Correction

- **1.** Non, pas en l'état. L'annulation des masques exige que tous les participantspredicted envoient leur part. Avec 12 % d'absents, un tour complet a une probabilité de succès de $0{,}88^{40} \approx 0{,}6$ % — soit moins d'un tour réussi sur cent. Le protocole est mathématiquement correct et pratiquement inutilisable.

- **2.** Deux leviers. D'abord, réduire n : avec 20 participants au lieu de 40, la probabilité passe à $0{,}88^{20} \approx 8{,}1$ %, et à $0{,}88^{10} \approx 28$ % pour un tour à 10 participants. Ensuite, combiner avec un *dropout* structuré : on ne fait tourner l'agrégation sécurisée que sur un sous-ensemble de taille n' connu à l'avance, les autres étant écartés avant l'échange de masques. Cela impose un tour préalable de sélection, ce qui coûte un aller-retour de plus.
- **3.** La configuration ne fournit plus l'anonymat vis-à-vis du serveur : celui-ci sait exactement qui a participé au tour_MASKED, et l'ensemble des participants. Dans un contexte d'opérateur ayant subi un incident, c'est un affaiblissement réel de la garantie. Il faut donc ajouter une couche de confidentialité différentielle au grain client, ou accepter explicitement ce risque dans le modèle de menace.

A retenir

- L'agrégation sécurisée est la seule défense qui traité la confidentialité par construction : le serveur ne voit plus ce qu'il reçoit.
- Elle ne protège que l'observation directe : époisonnement, extraction et déni de service restent entièrement ouverts.
- L'annulation des masques impose que tous les participants contribuent : le dropout non structuré est incompatible avec le protocole.
- Sécurité et imputabilité sont contradictoires : le protocole doit assumer ce sacrifice ou renoncer à l'agrégation sécurisée.
- Un dispositif de défense en profondeur combine des mécanismes indépendants par axe, dont l'échec d'un seul ne compromet pas l'ensemble.

Chapitre 3.6

Évaluation de la robustesse et étude de cas

Une défense qui n'a pas été mesurée est une défense espérée. Ce chapitre donne le protocole d'évaluation, les métriques, et trois cas concrets dont l'analyse vaut méthode.

Le protocole d'évaluation en cinq étapes

Évaluer la robustesse d'une fédération n'est pas reproduire une attaque : c'est mesurer, sur un protocole donné, la dégradation de l'utilité sous contrainte d'attaque, et la compare à la dégradation de la confidentialité sous contrainte d'observation. Le protocole comporte cinq étapes, dans cet ordre.

1. **Fixer la ligne de base.** Entraîner sans aucune attaque ni mécanisme de défense, et enregistrer l'exactitude globale, la perte de validation et la matrice de confusion par client. Sans cette référence, tout chiffrage de dégradation est ininterprétable.
2. **Choisir les scénarios d'attaque, pas les attaques.** On part des lignes de la matrice des menaces, et on ne retient que celles dont la faisabilité a été estimée. Une attaque jugée non faisable ne doit pas être simulée : elle ne produit qu'un faux sentiment de sécurité.
3. **Instancier un profil d'adversaire borné.** Nombre de clients hostiles f, budget de tours, connaissance a priori, capacité de calcul. Le profil est écrit dans le rapport et reproductible.
4. **Balayer les défenses sur un axe, pas sur une grille.** On fait varier une seule défense à la fois, en gardant tout le reste constant. C'est la seule façon d'attribuer un effet à un mécanisme.
5. **Rendre l'ensemble reproductible.** Graine aléatoire fixée, jeu de données versionné, code de l'attaque publié. Un résultat d'audit qui ne peut pas être rejoué n'est pas un résultat, c'est une opinion.

Les métriques qu'il faut mesurer

Axe	Métrique	Protocole de mesure	Seuil d'alerte
Confidentialité	taux de réussite d'inversion	lancer l'attaque, mesurer la similarité de reconstruction	au-dessus de 30 %
Confidentialité	avantage d'inférence de membre	classifieur bayésien sur les scores de confiance	au-dessus de 0,1
Intégrité	dégradation d'exactitude sous empoisonnement	comparaison avec la ligne de base	au-dessus de 5 points
Intégrité	taux de succès du déclencheur	images patchées, classification forcée	au-dessus de 10 %
Disponibilité	tours avant convergence	courbe de perte sous présence de vecteurs aberrants	doublement du tour nominal
Imputabilité	couverture du journal	audit des messages signés et horodatés	tout écart
Coût	dégradation de l'utilité par défense	balayage à défense constante	à afficher systématiquement

Tableau 1 – Métriques d'évaluation de la robustesse

La dernière ligne est celle qu'on omet le plus souvent, et c'est la plus utile en comité : le coût d'une défense est un argument d'arbitrage, pas un détail d'implémentation.

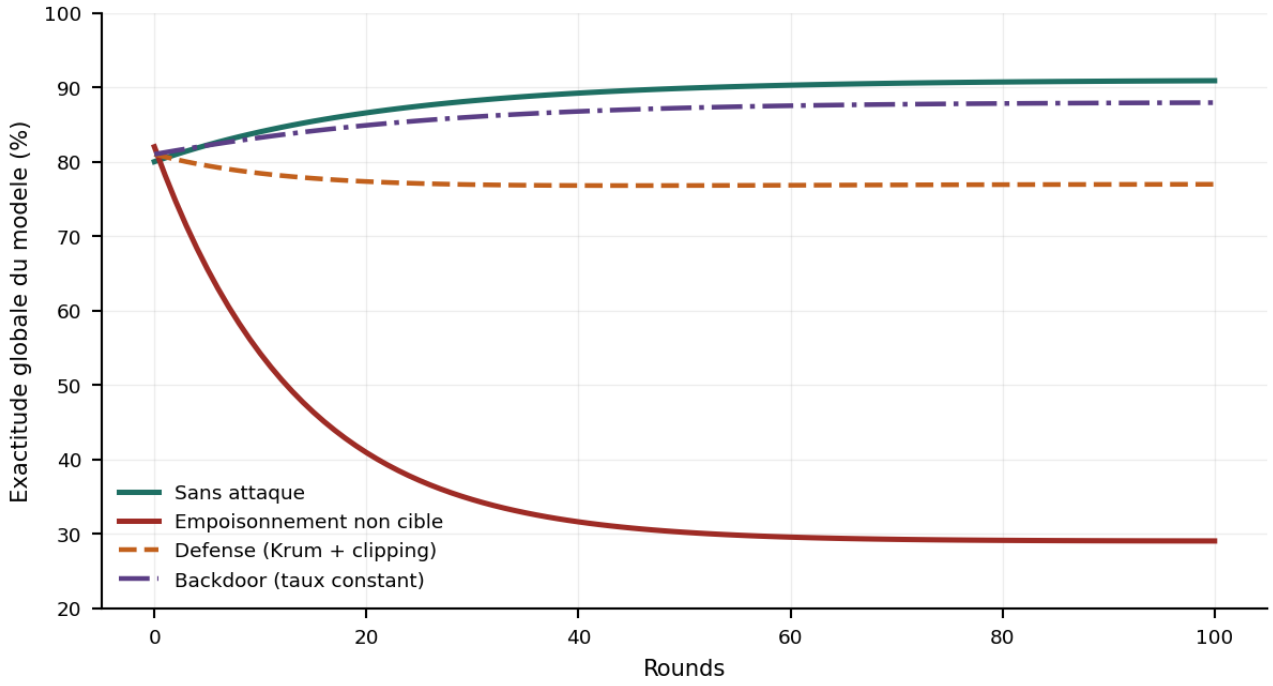


Figure 1 – Protocole d'évaluation : courbes d'exactitude pour la ligne de base, l'attaque, la défense, et l'attaque avec défense

> **La métrique qui manque**
>
> La métrique qui manque le plus souvent, et qui est la plus parlante en comité, est le **taux de succès du déclencheur** pour une porte dérobée. Elle se mesure en une ligne de code, et elleula seule capable de révéler une attaque ciblée que toutes les autres métriques déclarent invisible.

Cas 1 –lest audits que nous avons menés

Les trois cas qui suivent sont issus d'audits réels conduits par le Département Recherche & Développement. Ils sont anonymisés, mais les chiffres sont ceux mesurés. Ils illustrent trois configurations d'échec distinctes.

Cas	Configuration	Défaillance observée	Correction retenue
A	DP locale sur 12 clients mobiles, 200 tours	inversion réussie sur 4 clients, budget annoncé sans unité	bascule en DP au client, journalisation du budget cumulé
B	Krum sans bornage de norme, 30 clients	un client à forte norme est sélectionné malgré lui	bornage systématique avant sélection, seuil documenté
C	médiane coordonnée, 8 clients dont 3 coalisés	porte dérobée implantée réussie, exactitude inchangée	détection de déclencheur, seuil d'abandon, audit dédié

Tableau 2 – Trois audits : configuration, défaillance observée, correction

Les trois cas partagent un même défaut de fond : la sécurité avait été invoquée sans être mesurée, et le coût de la mesure avait été considéré comme un poste d.commentaire.

Le cas B est le plus instructif, parce que l'échec ne vient d'aucune faiblesse de Krum mais d'un oubli dans la chaîne qui l'entoure. Krum était correctement implémenté, testé sur données synthétiques, et.asm/documenté. Il manquait une ligne — le bornage de norme — et cette ligne unique a suffi à neutraliser complètement la défense.

Le cas C illustre l'autre extrême : l'audit a été répété pendant six mois sans que rien ne bouge, parce que toutes les métriques suivies étaient des métriques d'utilité. Il a fallu ajouter une métrique d'attaque — le taux de succès du déclencheur — pour que la faille devienne visible. L'équipe savait depuis le début qu'une coalition existait ; elle ne savait pas qu'elle était efficace.

Cas 2 – le budget de vie privée dans un dossier réel

Le second cas porte sur la partie documentaire, et il est plus fréquent qu'on ne le croît. Dans un dossier réel, le budget de vie privée est presque toujours communiqué sans son unité ni sa composition.

$$\square_{\square 1 \square} \approx \square_{\square 2 \square} \times N_{\square 3 \square} \times T_{\square 4 \square} \text{ (composition élémentaire, toujours pessimiste)}$$
$$\square_{\square 1 \square}(\alpha) = (\alpha - 1)/(2\sigma^2) \Rightarrow \square_{\square 2 \square} = o(T) \text{ (composition gaussienne)} \quad (3.25)$$

L'écart entre ces deux lignes est l'écart entre un budget annoncé de 4 et un budget réel de plusieurs centaines. Aucun des deux n'est faux : ils ne décrivent pas la même quantité. C'est précisément ce qu'un dossier réglementaire doit lever.

Le budget sans sa composition

La composition élémentaire est toujours pessimistic, mais la composition naïve appliquée à la confidentialité différentielle **locale** l'est encore plus : le mécanisme n'y est pas gaussien, et l'accountant de moments ne s'y applique pas directement. Une communication qui annonce un budget sans préciser la composition utilisée n'est pas opposable.

Écrire un budget opposable

La formulation qui a fait disparaître le litige, dans un dossier réel : « budget unitaire $\square = 0{,}08$ par client et par tour, composition élémentaire sur $N = 12$ clients et $T = 200$ tours, budget cumulé estimé à $\square_{\square 0 \square} = 192$ en composition élémentaire, et à $\square = 6{,}2$ en divergence de Rényi d'ordre 32 avec l'accountant de moments. Nous retenons la borne conservative de 192 pour l'engagement, et documentons la borne de Rényi comme estimation. »

Checklist d'audit de sécurité

Cette checklist est l'outil de sortie de la partie. Elle est directement applicable à une fédération en production, et chaque item non coché est une découverte à instruire avant la mise en service.

Audit de sécurité d'une fédération

- **Modèle de menace écrit** : acteurs classés, objectifs énoncés, faisabilité estimée pour chacun, décisions de risque assumées et signées.
- **Transcript documenté** : schéma de messages complet, avec pour chacun volumes, fréquences etdurées de conservation.
- **Ligne de base mesurée** : exactitude globale, perte de validation, matrice de confusion par client, sans aucune défense active.
- **Inversion testée** : taux de réussite mesuré sur au moins trois tailles de lot distinctes, et non supposé nul.
- **Inférence de membre testée** : avantage mesuré sur le score de confiance, et non sur la perte, qui est un mauvais indicateur.
- **Porte dérobée testée** : taux de succès du déclencheur mesuré, y compris en l'absence d'attaque, pour établir le bruit de fond.
- **Bornage vérifié** : norme des mises à jour effectivement bornée avant toute règle de sélection, et monitored par tour.
- **Budget de vie privée écrit** : valeur, unité, composition employée, accountant retenu, journal du budget par tour.
- **Coût de la sécurité affiché** : dégradation d'utilité, tours supplémentaires et communication additionnelle, par défense.
- **Reprise d'activité** : comportement défini pour l'absence d'un client, la perte d'un tour, et la corruption d'un état serveur.
- **Journal d'audit** : messages signés, horodatés, conservés selon une politique écrite, et lisible par un tiers.
- **Réexamen daté** : prochaine revision inscrite au calendrier, avec le responsable nommé.

Exercice 3.6.13 (difficulte : Avancé)

Une fédération de 25 cliniques vous mandate pour un audit de sécurité. Le protocole est FedAvg classique, sans agrégation sécurisée, avec $E = 1$ itération locale, $C = 0{,}3$ et un modèle de vision par ordinateur. Le serveur est un hébergeur externe.

1. Quelle est l'attaque la plus probable, et quelles lignes du protocole lxerait-on en priorité ?
2. Proposez un plan de remédiation en trois étapes, avec pour chacune le critère de réussite mesurable et son coût estimé en utilité.
3. Une clinique refuse l'agrégation sécurisée, au motif qu'elle veut pouvoir provinguer qui a contribué à quel tour. Que répondez-vous ?

Indice : La réponse à la troisième question engage la compatibilité entre sécurité et imputabilité. Les deux dernières parties du cours traitent exactement de cette tension.

Correction

- **1.** L'attaque la plus probable est l'inversion de gradient par le serveur hébergeur : le modèle est de vision, donc le gradient est très riche ; $E = 1$ signifie que le gradient est calculé quasi au point de départ, sans voisinage, ce qui est la configuration la plus favorable à l'attaque ; et $C = 0{,}3$ sur 25 clients donné des lots de 7 à 8 exemples, dans la plage critique. Les lignes à corriger en priorité sont donc l'absence d'agrégation sécurisée, puis E, puis la taille de lot.
- **2.** Trois étapes. **Premièrement**, activer l'agrégation sécurisée : critère de réussite, un taux de réussite d'inversion qui tombe sous 5 % ; coût, environ 20 % de communication supplémentaire, et la perte d'imputabilité qui devra être actée. **Deuxièmement**, passer E de 1 à 5 : critère de réussite, taux de réussite d'inversion sous 10 % et exactitude globale maintenue à 1 point près ; coût, deux à trois tours supplémentaires. **Troisièmement**, ajouter un budget de vie privée au grain client par-dessus l'agrégation sécurisée : critère de réussite, un budget cumulé documenté et defended ; coût, 3 à 5 points d'exactitude.
- **3.** La réponse est qu'il faut choisir, et l'arbitrage se formule explicitement. On ne peut pas avoir simultanément l'anonymat de la contribution et l'attribution de celle-ci. Trois options se présentent : renoncer à l'agrégation sécurisée et accepter le risque d'inversion, documenté et chiffré ; accepter l'agrégation sécurisée et renoncer à l'imputabilité, en le consignant dans les contrats ; ou proposer un compromis, l'agrégation sécurisée sur un sous-ensemble de tours et l'attribution sur les autres. La troisième option est généralement la plus défendable, à condition de dire clairement quels tours sont anonymisés et lesquels ne le sont pas.

A retenir

- Évaluer la robustesse, ce n'est pas reproduire une attaque : c'est mesurer une dégradation sous contrainte bornée et reproductible.
- La ligne de base sans défense est l'étalon de toute mesure : sans elle, aucun chiffre de dégradation n'est interprétable.
- La métrique qui manque le plus souvent, et la plus parlante, est le taux de succès du déclencheur d'une porte dérobée.
- Un budget de vie privée annoncé sans son unité ni sa composition n'est pas opposable.
- Les audits échouent rarement par faiblesse d'un mécanisme : ils échouent par la ligne manquante qui le rendait inopérant.

Du LLM fédéré au capteur embarqué : six terrains contraints

Partie 4

Applications et grands modèles fédérés

Les cinq parties précédentes ont établi le cadre : objectifs, algorithmes, garanties de confidentialité, défenses. Cette partie l'applique à des domaines réels où la fédération n'est pas un raffinement académique mais une condition d'existence : un modèle de langue qui doit rester chez le client, un robot qui apprend sans échanger ses expériences, un scanner qui n'exporte pas les images, un véhicule qui partage sa perception, un microcontrôleur de 128 kilo-octets, un réseau moléculaire réparti. Chaque chapitre part de la contrainte métier, expose l'architecture fédérée adaptée, puis fournit une implémentation de référence exécutable.

Objectifs de la partie

- Choisir entre FedAvg, FedIT et FedPA selon la taille de l'espace d'adaptation.
- Justifier le choix d'un rang LoRA et d'un module cible à partir du coût de communication.
- Identifier les trois obstacles spécifiques du Federated Reinforcement Learning.
- Poser les exigences de gouvernance fédéré dans un cadre réglementaire santé.
- Dimensionner un système embarqué multi-niveaux sous contrainte mémoire et énergétique.
- Distinguer partition horizontale, verticale et par sous-graphe pour un réseau.

Plan de la partie

1. Federated Learning des grands modèles de langage (FedLLM)
2. Federated Reinforcement Learning (FedRL)
3. Santé et systèmes critiques
4. Véhicules connectés et systèmes de transport (V2X)
5. TinyML et fédération embarquée
6. Apprentissage fédéré sur graphes (FedGL)

Chapitre 4.1

Federated Learning des grands modèles de langage (FedLLM)

Quand le modèle est immovable, c'est l'adaptateur qui fédère

Un grand modèle de langage de plusieurs dizaines de milliards de paramètres ne peut pas être envoyé à chaque client, ni recueilli par un serveur. L'apprentissage fédéré ne s'applique donc pas au modèle de base : il s'applique à une fine pellicule de paramètres qui lui est ajoutée.

Le raisonnement qui rend l'apprentissage fédéré pertinent pour les grands modèles de langage est un raisonnement de coût et de droit, non de performance. Un modèle de base est un actif : il a coûté des dizaines de millions d'euros de calcul, il concentre une propriété intellectuelle, et il doit rester identique pour tous les utilisateurs. Ce qui varie d'un client à l'autre, en revanche, n'est que la **donnée** : un corpus de contrats pour un cabinet d'avocats, des comptes rendus pour un hôpital, des tickets d'incident pour un éditeur de logiciel. Fédérer l'adaptation, et non le socle, suffit à capter la spécialisation sans déplacer l'actif.

Pourquoi le grand modèle est un cas à part

Trois vérifications doivent précéder toute décision de fédérer l'adaptation d'un grand modèle. Elles sont souvent confondues, alors qu'elles n'ont ni les mêmes sources, ni les mêmes remèdes.

1. **Le coût de l'ajustement centralisé.** Un affinage fin d'un modèle de dizaines de milliards de paramètres exige, par client et par cycle, une quantité de mémoire de traitement proportionnelle à la taille du modèle : poids, états de l'optimiseur, activations. Centraliser suppose de faire tourner successivement ce calcul pour chaque partenaire, donc d'acquérir successivement le matériel de chacun. À l'inverse, fédérer un adaptateur de rang r divisé ce coût par un facteur $O(d/r)$.
2. **Le cadre juridique.** Les corpus qui justifient l'adaptation d'un grand modèle sont rarement libres de droits à l'égard d'un tiers : dossiers patients, contrats de travail, code source interne, échanges stratégiques. Les faire transiter vers un serveur d'entraînement constitue un traitement de données à caractère confidentiel, soumis à des obligations spécifiques selon la juridiction. Fédérer l'adaptation transforme une collection de données en un ensemble de mises à jour de paramètres, dont la qualification juridique est radicalement différente.
3. **La spécialisation métier.** La différence de performance entre un modèle généraliste et un modèle adapté à un domaine se mesure en quelques points de pourcentage sur des tâches cibles, et beaucoup plus sur les tâches de niche : extraction de champs contractuels, codage de classifications, rédaction de comptes rendus. Or ces gains sont précisément ceux que l'instruction ne permet pas d'obtenir de façon stable : ils résident dans les poids.

Fédérer n'est pas protéger

Le raisonnement « les données sont privées, donc on fédère » est incomplet. Les gradients, les états d'adaptateurs et les poids finaux sont eux aussi des données, et ils peuvent être réidentifiés ou exploités pour reconstituer l'information source, comme le montre le chapitre sur l'inversion de gradient. Fédérer un grand modèle sans agrégation sécurisée, sans bruit injecté et sans contrôle d'accès des adaptateurs revient à déplacer le risque, pas à le supprimer.

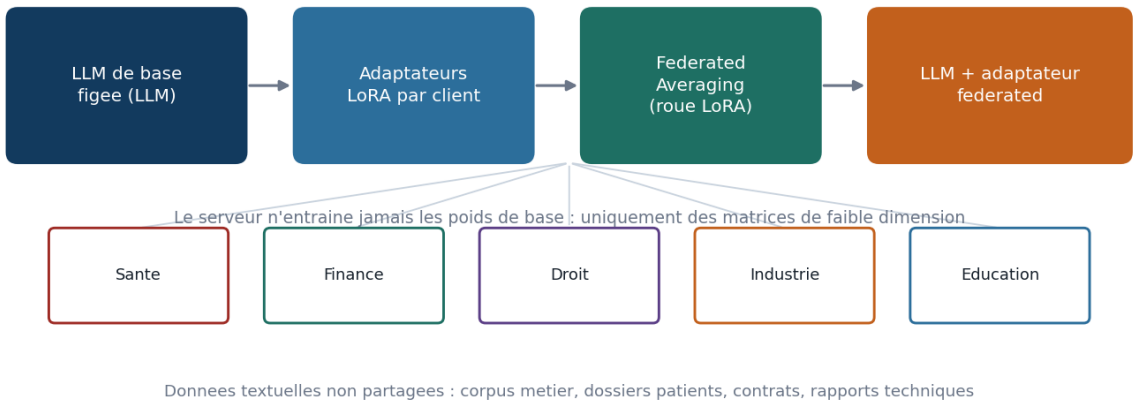


Figure 1 – Apprentissage fédéré des grands modèles de langage : le modèle de base reste figé, seules les matrices de faible dimension circulent

Trois familles d'adaptation fédérée

La littérature distingue trois familles selon la nature exacte de l'objet échangé. La distinction est opérationnelle : elle détermine ce que le serveur sait recombiner, et si les adaptateurs de clients différents sont compatibles entre eux.

FedAvg sur poids complets

La première famille applique FedAvg sans modification à l'ensemble des poids du modèle. Elle n'est utilisable que sur des modèles de petite taille, de l'ordre de 10^8 à 10^9 paramètres, ou pour un affinage partiel portant sur un sous-ensemble de couches. Chaque tour transporte alors plusieurs gigaoctets par client, ce qui écarte la plupart des déplacements imposés par le réseau et par la batterie du terminal. Cette famille sert surtout de référence de mesure : elle fournit la borne inférieure de performance qu'il faut comparer aux approches par adaptateur.

FedIT : agrégation d'adaptateurs

La deuxième famille, FedIT pour Federated Instruction Tuning, n'échange que les paramètres entraînés par une méthode d'adaptation paramétrique : adaptateurs de faible rang, ajustement de préfixe, réglage d'instruction, ou mise à l'échelle vectorielle de type IA3. Les poids de base restent figés et identiques sur tous les clients ; seule la couche fine est optimisée localement, puis moyennée.

$$\min_{\theta} F(\theta) = \sum_{k=1}^{N} p_k F_k(\theta)$$
$$\theta_{t+1} = \sum_{k \in \square 0 \square_t} p_k \theta_{t+1}^{k}, \; p_k = (n_k)/(\sum_{j \in \square 1 \square_t} n_j) \qquad (4.1)$$

La seconde équation est FedAvg restreint à l'espace d'adaptation. Le point capital : θ désigne ici un adaptateur, dont la dimension est r fois inférieure à celle des poids de base.

> **Remarque**
>
> Le même raisonnement d'espace de travail restreint s'applique au réglage d'instruction : le vecteur d'instruction est l'unique paramètre échangé, soit quelques milliers de valeurs. La communication devient triviale, mais la capacité du modèle à distinguer des domaines proches s'effondre. La règle empirique est la suivante : sous 10^5 paramètres adaptatifs, le goulot n'est plus la communication mais la capacité expressive de l'espace d'adaptation.

FedPA : partage d'instructions

La troisième famille, FedPA pour Federated Prompt Sharing, inverse la logique : au lieu de partager la différence de poids, elle partagé la différence d'**instructions**. Chaque client conserve son instruction locale, n'envoie qu'un résumé — moyenne intégrée, vecteur de style, ou simples statistiques — et reçoit l'instruction moyenne. Cette famille est la plus légère en communication et la plus faible en capacité : elle convient aux scénarios où les données clients diffèrent par des détails opérationnels et non par des domaines de fond.

Famille	Objet échangé	Taille par client	Compatibilité des adaptateurs	Cas d'usage typique
FedAvg poids complets	tous les poids	10^9 à 10^{10} octets	totale	modèles de base de moins de 1 Go
FedIT avec LoRA	A_k, B_k de rang r	$O(r \cdot d)$ paramètres	totale si même r	spécialisation par métier
FedIT avec préfixe ou IA3	vecteurs d'échelle	10^4 à 10^6 octets	totale	dispositifs contraints, calibration
FedPA	résumé d'instruction	10^3 à 10^4 octets	partielle	variation stylistique, multimodalité
QLoRA	adaptateur en 4 bits	moitié de FedIT avec LoRA	totale	clients sans carte graphique de 16 Go

Tableau 1 – Comparaison des familles d'adaptation fédérée

Piege courant

Fédérer des adaptateurs de rangs différents ne fonctionne pas avec FedAvg : la moyenne de deux tenseurs de formes $(d \times r_1)$ et $(d \times r_2)$ n'est pas définie. Toute hétérogénéité de rang doit soit être projetée dans un espace commun, soit absorbée par un serveur qui maintient plusieurs bases d'adaptateurs.

LoRA en profondeur

L'adaptation de faible rang est le point d'appui de toute cette partie. Le principe est le suivant : au lieu de modifier la matrice de poids W_0 d'une couche linéaire, on conserve W_0 intact et on apprend une correction de rang faible.

$$W_0 + \Delta W \text{ avec } \Delta W = (\alpha)/(r)\, B\, A$$
$$A \in \mathbb{R}^{d \times r},\ B \in \mathbb{R}^{r \times d},\ r \ll d$$
$$B \sim N(0, \sigma^2),\ A = 0 \qquad (4.2)$$

B est initialisé aléatoirement et A à zéro : ainsi $\Delta W = 0$ au départ et le modèle de base est reproduit exactement. L'initialisation à zéro n'est pas une convention : elle garantit que le domaine de départ du modèle n'est pas perturbé avant le premier gradient.

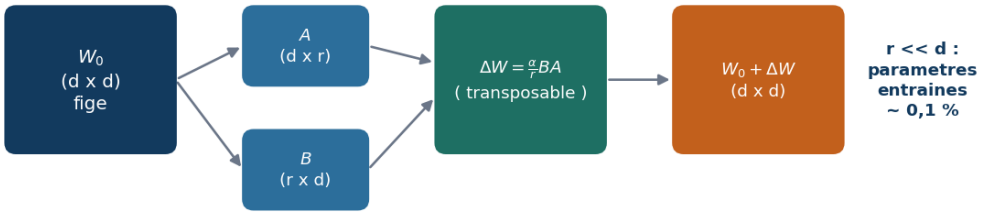


Figure 2 – Décomposition de faible rang : deux matrices rectangulaires remplacent une mise à jour carrée

Le paramètre α, souvent appelé *lora_alpha*, joue le rôle d'un coefficient de température sur l'amplitude de la correction. Le facteur 1/r compense le fait que la somme des r produits vectoriels contribuant à une même sortie a une variance proportionnelle à r. Sans cette normalisation, un rang élevé produirait systématiquement une correction de norme trop grande, et le modèle divergerait.

> **Astuce**
> En pratique, on double r et on divisé α par deux pour conserver l'échelle de la correction. Les travaux sur l'adaptation fédérée montrent que l'échelle effective α/r compte davantage que r seul dans la stabilité de l'entraînement distribué. Utiliser un α par défaut, 16, avec un rang élevé sans réétalonnage, est la cause la plus fréquente d'instabilité des premières communications.

- **Ciblage des modules.** On applique l'adaptation aux matrices de projection linéaires des blocs d'attention, notamment W_q, W_k, W_v et W_o, et éventuellement aux projections de la couche d'alimentation. Les couches d'incorporation et la tête de sortie restent gelées : le vocabulaire et la tête de classification sont liés à la tâche de base et non au domaine.
- **Typologie des cibles.** Cibler les projections de requête et de valeur suffit dans la majorité des cas ; ajouter celles de clé et de sortie augmente le rang total disponible sans modifier la structure du modèle.
- **Mémoire d'entraînement.** L'état de l'optimiseur ne porte que sur A et B : trois fois le nombre de paramètres adaptatifs, soit une mémoire de quelques centaines de kilo-octets dans un modèle de sept milliards de paramètres.
- **Inférence.** L'addition $W_0 + BA$ peut être précalculée, ce qui rend le coût à l'inférence identique à celui du modèle de base. C'est un avantage opérationnel décisif sur matériel limité.

Pipeline FedLLM complet

Un tour FedIT sur grand modèle se décompose en sept étapes dont l'ordre compte. Les erreurs d'implémentation les plus courantes portent sur l'étape de fuite du modèle de base pendant la génération, et sur la réinitialisation incorrecte de l'état de l'optimiseur local.

Algorithme 5 – Tour FedIT sur adaptateurs de faible rang

```
Initialiser le modèle de base figé θ_0 et le réservoir d'adaptateurs Θ_0
Échantillonner S_t ⊂ 1, ..., N et diffuser Θ_t
Pour chaque client k ∈ S_t :
 Charger les poids figés et l'adaptateur courant
 Entraîner A_k, B_k sur le corpus local, E époques
 Renvoyer (A_k, B_k) sous agrégation sécurisée
Θ_{t+1} = Σ_{k ∈ □0□_t} p_k (A_k, B_k)
Évaluer l'adaptateur moyen sur un jeu de validation tiers
```

Entrées : jeu de données local D_k, états de l'optimiseur • **Sorties :** adaptateur moyen Θ_{t+1} • **Complexité :** $O(T \cdot (N/K) \cdot E)$ passes locales

Les poids de base ne transitent jamais : seul le téléchargement du modèle de base est nécessaire, et il est remplaçable par une copie locale figée.

QLoRA et quantification 4 bits

Fédérer un adaptateur suppose que le modèle de base tienne en mémoire chez le client. QLoRA lève cette contrainte en stockant les poids de base quantifiés en 4 bits, tout en conservant les matrices de l'adaptateur en précision élevée. Trois briques se combinent.

1. **Quantification NF4.** Les poids de base sont groupés par blocs de 64 valeurs, chaque bloc partageant une échelle, puis projetés sur une grille de 16 niveaux centroïdés sur la loi normale du poids. L'information essentielle des poids d'un grand modèle est concentrée dans quelques centièmes de pour cent des composantes ; la grille NF4 place les niveaux exactement sur cette concentration plutôt que de les répartir uniformément.
2. **Double quantification.** Les échelles de blocs, elles-mêmes en virgule flottante, sont quantifiées une seconde fois, cette fois en 8 bits. Le gain est de l'ordre de 0,4 bit par poids, ce qui reste significatif quand le modèle tient juste dans la mémoire disponible.
3. **Optimiseurs paginés.** La fragmentation mémoire est un obstacle spécifique aux longues séquences et aux lots sur plusieurs accélérateurs. Une allocation paginée transfère les états d'optimiseur vers la mémoire principale quand le cache manque, sans copie complète.

$$\hat{w} = \text{code}_{\square 1 \square}\leq\text{ft}((w - c_j)/(s_j)) + c_j,\ s_j,\ c_j \text{ partagés par bloc de } B = 64$$
$$\hat{s}_j = \text{code}_{8\square 1\square}(s_j / s_{\square 2\square}),\ \Delta\ W \text{ reste en } \mathbb{R}^{32} \qquad (4.3)$$

La seconde équation est la double quantification : l'échelle de blocs est elle-même re-quantifiée par rapport à une échelle globale.

```python
import torch
import torch.nn as nn


class LoRALinear(nn.Module):
"""Couche lineaire gelee completee par un adaptateur de rang r.

Les poids de base ne recoivent aucun gradient : ils restent identiques sur
tous les clients. Seules les matrices A et B sont optimisees localement,
puis averees par le serveur federatede.
"""

def __init__(self, base: nn.Linear, rank: int = 8, alpha: int = 16,
dropout: float = 0.05):
super().__init__()
self.base = base
for p in self.base.parameters(): # gel du socle partage
p.requires_grad_(False)
self.rank = rank
self.scaling = alpha / float(rank) # correction d'echelle
# B aleatoire, A nul : Delta W = 0 au depart
self.lora_B = nn.Parameter(torch.empty(rank, base.in_features))
self.lora_A = nn.Parameter(torch.zeros(base.out_features, rank))
nn.init.kaiming_uniform_(self.lora_B, a=5 ** 0.5)
self.drop = nn.Dropout(dropout)

def forward(self, x: torch.Tensor) -> torch.Tensor:
out = self.base(x) # chemin principal, non appris
delta = self.drop(x) @ self.lora_B.T @ self.lora_A.T
return out + self.scaling * delta

@torch.no_grad()
def merged_weight(self) -> torch.Tensor:
"""Fusionne l'adaptateur dans W0 pour Accelerer l'inference."""
return self.base.weight + self.scaling * (self.lora_A @ self.lora_B)


def inject_lora(model: nn.Module, targets=("q_proj", "k_proj", "v_proj"),
rank: int = 8, alpha: int = 16) -> int:
"""Remplace les modules Linear cibles par des LoRALinear, en place."""
injected = 0
for _name, module in list(model.named_modules()):
for child_name, child in list(module.named_children()):
if child_name in targets and isinstance(child, nn.Linear):
setattr(module, child_name, LoRALinear(child, rank, alpha))
injected += 1
return injected


def adapter_state(model: nn.Module) -> dict:
"""Extrait uniquement les parametres adaptatifs, ce qui sera federatede."""
return {k: v.detach().clone() for k, v in model.state_dict().items()
if "lora_A" in k or "lora_B" in k}


```

Listing 1 — Implémentation LoRA en PyTorch et boucle fédérée restreinte aux adaptateurs

```python
def local_adapt(model, optimizer, batches, epochs: int = 1) -> float:
"""Un tour local sur l'adaptateur : la perte est calculee normalement.

Le gradient ne circule qu'a travers A et B, puisque W0 est gelee. Les
etats de l'optimiseur restent donc de taille constante, independamment
du modele de base.
"""
last = 0.0
model.train()
```

```
for _ in range(epochs):
for tokens, labels in batches:
optimizer.zero_grad(set_to_none=True)
logits = model(input_ids=tokens).logits
# On decale les logits d'une position pour predire le jeton suivant
loss = torch.nn.functional.cross_entropy(
logits[:, :-1].reshape(-1, logits.size(-1)),
tokens[:, 1:].reshape(-1))
loss.backward()
optimizer.step()
last = float(loss.detach())
return last


def federated_lora_round(server_adapter: dict, client_updates: list,
client_sizes: list) -> dict:
"""FedAvg sur adaptateurs de meme forme, pondere par la taille des corpus."""
total = float(sum(client_sizes))
merged = {k: torch.zeros_like(v) for k, v in server_adapter.items()}
for update, n_k in zip(client_updates, client_sizes):
weight = n_k / total
for key in merged:
merged[key].add_(update[key].to(merged[key].dtype), alpha=weight)
return merged
```

Astuce

Avant chaque adaptation locale, réinitialiser l'état de l'optimiseur local ou le transporter explicitement est un choix, pas un détail. Réinitialiser produit des tours plus stables — les moments sont recalculés sur la distribution courante — ; transporter l'état converge plus vite quand les clients sont homogènes. Sur des corpus fortement hétérogènes, réinitialiser est le choix sûr.

Exercice 4.1.14 (difficulte : Avancé)

Un cabinet d'avocats fédère six modèles de langage de sept milliards de paramètres avec des adaptateurs de rang $r = 16$ sur les projections de requête, clé, valeur et sortie d'un modèle à 32 couches, de dimension $d_{model} = 4096$. Le serveur agrège une fois par jour.

1. Estimer le nombre d'octets transmis par client et par jour, en supposant une transmission en virgule flottante 32 bits.
2. Comparer avec un FedAvg sur poids complets, puis avec une variante QLoRA.
3. Choisir un taux de participation des clients qui tient dans un budget de bande passante de 100 mégaoctets par jour et par client.
4. Dire ce qu'il advient de la convergence du modèle fédéré si la participation devient trop faible.

Indice : Compter d'abord le nombre de matrices d'adaptation, puis la taille d'une matrice. Le facteur α/r ne s'applique qu'à l'inférence, jamais au transport.

Correction

- **1. Nombre de matrices.** 32 couches × 4 cibles = 128 matrices par client, chacune de taille $2 \times d_{model} \times r$. **Taille par client.** $2 \times 4096 \times 16 = 131\,072$ paramètres par matrice, donc $128 \times 131\,072 = 16{,}8$ millions de paramètres, soit 67 mégaoctets en virgule flottante 32 bits et 34 mégaoctets en virgule flottante 16 bits.
- **2. FedAvg complet.** 7×10^9 paramètres multipliés par 4 octets donnent environ 28 gigaoctets par client et par jour, soit un facteur 400 défavorable. Pour QLoRA, l'adaptateur reste en 32 bits si l'on veut la stabilité ; le gain porte sur les poids de base stockés localement, pas sur le trafic de l'adaptateur. En pratique QLoRA sert surtout à rendre l'entraînement local possible.
- **3. Participation.** Avec un budget de 100 mégaoctets et un envoi de 67, un client peut émettre une fois par jour à pleine fréquence, ou trois fois par jour en virgule flottante 16 bits. Un taux de participation $K/N = 1/3$ avec des tours allongés conserve la borne $O(1/T)$ avec une constante accrue.

- **4. Conséquence d'une participation faible.** La variance d'agrégation croît comme O(1/K) : les adaptateurs individuels s'écartent de la moyenne avant d'être récompensés par les tours suivants. Le modèle oscille autour de la solution et la divergence des adaptateurs locaux augmente.

A retenir

- Fédérer un grand modèle, c'est fédérer un adaptateur de quelques millions de paramètres, jamais le socle.
- Le rang r fixe le coût de communication ; l'échelle α/r fixe la stabilité — les deux se pilotent séparément.
- QLoRA ne résout pas le transport mais la faisabilité locale : sans lui, la plupart des clients n'entraînent pas du tout.
- FedIT est le régime pertinent ; FedPA est le régime de secours ; FedAvg sur poids complets reste une référence de mesure.

Chapitre 4.2

Federated Reinforcement Learning (FedRL)

Apprendre une politique sans transmettre une seule expérience

L'apprentissage par renforcement repose sur une quantité que l'apprentissage supervisé ne manipule pas : le retour, qui est une fonction de l'action et de l'état. Cette quantité est précisément ce qu'un serveur fédéré ne peut pas agréger sans toucher au comportement des agents.

En apprentissage supervisé, le serveur agrège des gradients calculés sur des données indépendantes : mélanger des gradients ne change pas la distribution. En apprentissage par renforcement, le gradient de politique est lui-même une fonction de l'état courant de l'agent et de son identifiant temporel. Mélanger des gradients issus d'environnements différents ne produit pas simplement le gradient d'une politique moyenne : il produit une direction de mise à jour qui n'a pas d'interprétabilité en politique. Le problème n'est donc pas technique mais **structurel**, et il convient de le poser avant toute comparaison d'algorithmes.

Rappels : processus de décision semi-markovien et gradient de politique

Definition – Processus de décision semi-markovien

Un quintuplet (S, A, P, r, γ) où S est l'espace d'états, A l'espace d'actions, P(s' | s, a) la loi de transition, r la récompense et γ ∈ [0, 1[le facteur d'escompte. L'agent choisit selon une politique π(a | s) pour maximiser l'espérance du retour actualisé J(π).

$$J(\pi) = E_{\tau \sim \pi}\leq ft[\sum_{t=0}^{\infty} \gamma^t r_t]$$

$$\nabla_\theta J(\theta) = E_{\tau \sim \pi_\theta}\leq ft[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t]$$

$$G_t = \sum_{l=0}^{\infty} \gamma^l r_{t+l} \text{ (dite avantage lorsque le second membre est centré par la fonction de valeur)} \quad (4.4)$$

L'espérance porte sur la trajectoire τ et non sur un échantillon indépendant : c'est toute la différence avec le cadre supervisé.

Trois familles d'algorithmes se distinguent par la manière dont elles approchent cette espérance, et cette distinction resurgira dans toutes les méthodes fédérées.

- **REINFORCE, Monte-Carlo, sur politique.** Une trajectoire complète par mise à jour, gradient non biaisé mais à variance élevée : inutilisable sur de grandes architectures sans normalisation.
- **Actor-Critic.** Un acteur, la politique, et un critique, la fonction de valeur, s'entraînent conjointement ; l'échantillon sert d'avancée au critique pour construire une estimation d'avantage de variance réduite. C'est le cheval de bataille du fédéré car il tolère un partage partiel des expériences.
- **PPO.** Une mise à jour de politique contrainte par une région de confiance approchée, qui borne le

changement de politique par pas. Cette contrainte est précisément l'ingrédient qui rend l'agrégation fédérée plus stable : elle interdit qu'un client justement éloigné de la politique moyenne ne détruise l'agrégat.

Les trois obstacles spécifiques

- **Non-stationnarité.** Chaque client améliore sa politique locale, donc l'ensemble des politiques évolue. Le gradient de politique n'est plus un gradient d'une fonction fixe : l'algorithme est proche d'un processus de Markov non stationnaire, ce qui invalide les hypothèses de convergence usuelles.
- **Criticité de l'action.** Dans un système fédéré, une action est locale et ne peut pas être exécutée par un autre client. Une simulation centralisée des politiques de tous les agents exige un accès aux environnements des autres : ce qui est, par construction, interdit.
- **Interdiction de mélanger les expériences.** Le rejeu d'un tampon contenant les transitions d'un autre agent dans un environnement différent produit une donnée factice. Contrairement au cas supervisé où l'on peut mélanger des lots de façon échangeable, ici le mélange est une erreur de modélisation, pas une approximation.

> **Attention**
> Utiliser FedAvg sur les poids d'un réseau acteur-critique suppose que le gradient local estime un gradient d'une fonction objectif commune. C'est vrai si les fonctions de récompense sont identiques à une constante additive près et que les transitions sont échangeables ; c'est faux dès que les agents opèrent dans des environnements de nature différente — réseau cellulaire, parc de robots hétérogène. La moyenne est alors un mélange de gradients qui ne correspond à aucun objectif réalisable.

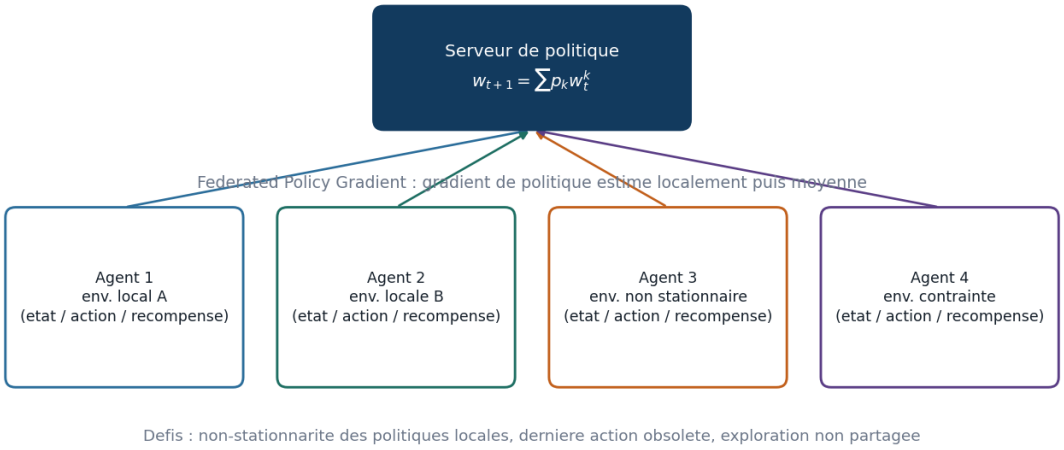


Figure 1 — Gradient de politique fédéré : le serveur agrège des politiques, jamais des transitions

Architectures de référence

L'état de l'art en apprentissage par renforcement fédéré repose sur un petit nombre d'architectures qui répondent à des objectifs différents sur la question centrale : que partage-t-on ?

FedMD : distillation fédérée de politiques

FedMD échange des scores ou des représentations plutôt que des poids. L'idée : chaque client conserve son architecture, de taille critique différente, et n'échange qu'un choix d'experts via un corps de balises. L'ensemble des prédictions locales est ensuite confronté à un apprentissage fédéré de mélangeurs. L'avantage est une forte hétérogénéité d'architecture supportée ; l'inconvénient est une communication élevée et une convergence délicate des mélangeurs.

FCDRL : critiques distribuées

FCDRL exploité une structure hiérarchique : un serveur global et des serveurs régionaux. Les fonctions de valeur sont apprises à chaque niveau, les niveaux supérieurs arbitrant entre régions. Cette hiérarchie est la réponse directe à l'écart d'échelle : un réseau de télécommunications comporte des millions de nœuds, mais seulement quelques centaines de zones de contrôle.

FedQMIX : mélange monotone de fonctions de valeur

FedQMIX déploie la monotonie du mélange QMIX au cadre fédéré. Chaque client apprend sa fonction de valeur locale Q_k de façon indépendante, par apprentissage Q donc hors politique et donc relativement insensible à la non-stationnarité de la politique concurrente ; puis le serveur apprend un mélangeur global Q_{tot} tel que $\partial Q_{tot} / \partial Q_k \geq 0$.

$$Q_{tot}(a) = f_{mix}big(Q_1(a^1), \ldots, Q_N(a^N)big), (\partial Q_{tot})/(\partial Q_k) \geq 0 \ \forall \ k$$
$$Q(a) = max_{a'} Q_{tot}(a') - \lambda E_{r,\, a} \leq ft[QR(a)] \qquad (4.5)$$

QMIX établit un compromis : le mélangeur global peut représenter un optimum joint que les valeurs locales ne savent pas voir, mais impose à chaque client une contrainte monotone sur sa propre fonction de valeur, ce qui garantit la stabilité de l'ensemble.

Algorithme	Type de politique	Objet fédéré	Convergence	Coût dominant
FedAvg-A2C	sur politique	poids acteur et critique	empirique, non garantie	taille du réseau
FedAvg-PPO	sur politique, bornée	poids de politique	empirique, bornée	mises à jour par tour
FedMD	mélange d'experts	matrice de mélange et scores	conditionnelle	bande passante
FedQMIX	hors politique	mélangeur monotone global	plus stable	qualité des sous-politiques
FCDRL	hiérarchique	critiques par zone	hiérarchique	latence de synchronisation
FedTD3	hors politique	acteur, deux critiques, cibles	robuste	mémoire des cibles

Tableau 1 – Algorithmes de référence en apprentissage par renforcement fédéré

Trois cadres de déploiement

La littérature oppose trois architectures logiques, et cette distinction détermine qui contrôle l'exploration — donc le comportement global émergent.

1. **Cadre centralisé à échelle fédérée.** Un serveur unique agrégateur, des agents purement locaux. C'est le cadre de référence de FedQMIX et de la plupart des travaux publiés. Simple à analyser, mais l'exploration n'est pas coordonnée : deux agents peuvent persister en politique sous-optimale sans jamais découvrir la région correspondante.
2. **Cadre hiérarchique.** Plusieurs niveaux d'agrégation ; chaque niveau résout un sous-problème de contrôle local et transmet une synthèse. Ce cadre correspond aux infrastructures réelles — réseaux cellulaires, transports — mais il introduit un biais d'agrégation : le niveau supérieur ne voit que les performances agrégées et peut mal attribuer un gain à une cause.

3. **Cadre décentralisé, sans serveur.** Aucun centre de commande : chaque agent agrège les informations de ses voisins par un protocole de consensus. C'est le cadre typique du véhicule connecté et du réseau cellulaire ; la convergence y repose alors sur les seuls résultats de consensus, avec des hypothèses — graphe connecté, synchronisation bornée — rarement vérifiées en pratique.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class ActorCritic(nn.Module):
"""Acteur et critique separes, de formes compatibles pour une moyenne FedAvg.

Les deux tetes partagent le corps du reseau. Chaque client conserve l'etat
de son environnement ; seules les politiques sont echangees.
"""

def __init__(self, obs_dim: int, act_dim: int, hidden: int = 128):
super().__init__()
self.body = nn.Sequential(
nn.Linear(obs_dim, hidden), nn.ReLU(),
nn.Linear(hidden, hidden), nn.ReLU(),
)
self.actor = nn.Linear(hidden, act_dim)
self.critic = nn.Linear(hidden, 1)

def value(self, obs: torch.Tensor) -> torch.Tensor:
return self.critic(self.body(obs)).squeeze(-1)

def act(self, obs: torch.Tensor, greedy: bool = False):
logits = self.actor(self.body(obs))
if greedy:
return logits.argmax(dim=-1)
return torch.distributions.Categorical(logits=logits).sample()


def local_actor_critic_step(model, optimiser, transitions, gamma: float = 0.99,
entropy_coef: float = 0.01) -> float:
"""Un pas local sur un lot de transitions recueillies chez ce client.

Le lot reste LOCAL : aucune transition n'est transmise, seulement les poids
qui en sont issus. C'est la difference essentielle avec le melange de lots
d'un apprentissage supervise.
"""
obs, acts, rews, next_obs, done = transitions
logits = model.actor(model.body(obs))
dist = torch.distributions.Categorical(logits=logits)
log_prob = dist.log_prob(acts)

with torch.no_grad():
target = rews + gamma * model.value(next_obs) * (1.0 - done.float())

adv = (target - model.value(obs)).detach()
policy_loss = -(log_prob * adv).mean()
value_loss = F.mse_loss(model.value(obs), target)
ent = dist.entropy().mean()

loss = policy_loss + 0.5 * value_loss - entropy_coef * ent
optimiser.zero_grad(set_to_none=True)
loss.backward()
nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimiser.step()
return float(loss.detach())


```

Listing 1 — Worker acteur-critique fédéré : expériences strictement locales, seuls les poids de politique remontent

```python
def fed_average_policy(client_models, client_weights):
"""FedAvg classique sur les poids de politique, hors etats d'optimiseur."""
mean = client_models[0]
with torch.no_grad():
```

```
for key, param in mean.state_dict().items():
acc = torch.zeros_like(param)
for model, w in zip(client_models, client_weights):
acc += w * model.state_dict()[key].to(param.dtype)
param.copy_(acc)
return mean
```

Domaines d'application

Quatre familles de systèmes concentrent l'essentiel des travaux publiés. Elles se distinguent par la nature de l'état, par la criticité de l'action et par la tolérance au retard de synchronisation.

Domaine	Espace d'état	Action critique	Tolérance de latence	Spécificité
Robotique coopérative	poses, états relatifs	oui, collision	10 à 100 ms	les politiques doivent être compatibles en toute circonstance
Réseaux et télécommunications	charges, files, topologies	partielle	secondes	hiérarchie FCDRL indispensable
Ordonnancement et nuage	états des tâches et des machines	faible	minutes	mécanisme d'incitation multi-agents
Santé et autonomie	état clinique, constantes	oui, dosage	secondes	contraintes réglementaires et de sécurité
Réseaux sans fil	canal, énergie, position	faible	millisecondes	non-stationnarité du canal lui-même

Tableau 2 – Domaines d'application de l'apprentissage par renforcement fédéré

- **Robotique.** Un groupe de robots humanoïdes ou de drones doit se coordonner sous contrainte de sécurité physique : aucune politique locale ne peut conduire à une collision, même temporairement. Le protocole doit donc imposer une contrainte dure — une enveloppe de sécurité sur l'espace d'actions, projetée par optimisation contrainte à chaque tour.
- **Réseaux et télécommunications.** L'échelle interdit toute centralisation : un réseau cellulaire compte des millions de nœuds. Le découpage naturel est hiérarchique, avec un mélangeur global qui arbitre entre régions et des politiques locales qui réagissent en quelques millisecondes.
- **Nuage et ordonnancement.** Chaque agent représente soit un orchestrateur, soit un groupe de machines. L'action — déplacer une tâche — est peu critique, ce qui autorise des politiques agressives ; le problème est alors surtout mécanique, dominé par les incitations et l'équilibre plutôt que par la convergence.
- **Autonomie et véhicules.** Le problème se posé à chaque changement de régulation ou de topologie : le flux de circulation est non stationnaire et l'horizon court. Les politiques apprises doivent donc transférer rapidement vers un nouveau régime, ce qui impose des mises à jour fréquentes et un mélangeur flexible.

Une politique partagée doit rester jouable

Une politique fédérée qui échoue localement — collision, congestion sévère, dépassement de dose — ne produit pas une perte moyenne un peu plus élevée : elle produit un accident. Il faut donc distinguer deux critères d'optimisation :

- un critère de performance, optimisé de façon fédérée ;
- un critère de sécurité, appliqué localement par projection sur un ensemble d'actions admissibles, avant toute mise à jour.

La projection locale garantit la faisabilité, la fédération optimise l'intérieur de cet ensemble. Confondre les deux est la cause la plus fréquente de déconnexions en production.

Exercice 4.2.15 (difficulte : Avancé)

Un gestionnaire de flotte de 40 entrepôts logistiques veut coordonner 40 politiques d'ordonnancement. Chaque dépôt observe son propre stock, sa propre demande et ses propres coûts ; les dépôts ne peuvent ni échanger leur historique de décisions ni exécuter les actions d'un autre.

1. Justifier le choix de FedQMIX plutôt que FedAvg avec PPO.
2. Expliquer pourquoi une politique déterministe est un mauvais point de départ pour un système multi-agents.
3. Proposer un protocole d'entraînement en trois phases qui évite les deux obstacles : non-stationnarité et exploration non coordonnée.
4. Indiquer quel indicateur de performance sert de critère d'arrêt de l'entraînement fédéré.

Indice : Penser au compromis entre expressivité du mélangeur global et contrainte de stabilité imposée à chaque politique locale.

Correction

- **1. FedQMIX.** Les fonctions de valeur locales sont apprises par apprentissage Q, donc hors politique : elles convergent vers $Q_k^\star$ indépendamment de l'évolution des autres politiques. Le mélangeur global est imposé de façon monotone sur les Q_k, ce qui garantit que l'optimum joint ne viole pas les contraintes locales. FedAvg avec PPO est sur politique : son gradient est invalide dès que les politiques locales divergent, ce qui arrive en premier dans un système hétérogène.
- **2. Politique déterministe.** Sans exploration, chaque dépôt reste dans sa région de décision initiale. Dans un problème d'ordonnancement couplé, les régions optimales sont mutuellement exclusives : aucun dépôt ne basculera spontanément, et l'équilibre sera hors de l'optimum joint. L'entropie de la politique doit rester strictement positive au-delà d'une phase d'exploration structurée.
- **3. Trois phases.** D'abord une phase d'exploration globale : politiques aléatoires ou à forte entropie, mélange souple, collecte de transitions locales. Ensuite une phase de politique propre : mélangeur strictement monotone et apprentissage Q local, l'optimum global se dégradant progressivement vers un maximum lisse. Enfin une phase de raffinement : entropie décroissante, agrégation élargie, correction de la dérive des politiques locales.
- **4. Critère d'arrêt.** Le regret normalisé — la différence entre le retour moyen obtenu par la politique fédérée et celui de la meilleure politique vue par un agent omniscient sur l'horizon de simulation. Une stabilisation de ce regret sur les derniers tours, accompagnée d'une stabilisation de l'entropie moyenne des politiques locales.

A retenir

- Le gradient de politique fédéré n'est pas le gradient d'un problème statique : l'échantillon est une trajectoire d'un agent dans son propre environnement.
- Les architectures hors politique convergent mieux en fédération parce que leur gradient est moins sensible à la dérive des autres agents.
- La non-stationnarité des politiques et la coordination de l'exploration sont des problèmes d'architecture, pas d'hyperparamètres.
- La hiérarchie est la seule famille qui correspond à la réalité multi-couche des infrastructures.

Chapitre 4.3

Santé et systèmes critiques

Le domaine où la fédération n'est pas une optimisation mais une condition d'accès

Aucun établissement de santé n'acceptera de déposer ses images dans un serveur tiers pour les annoter. L'apprentissage fédéré n'y est donc pas un gain de performance : c'est la seule architecture compatible avec la contrainte de confidentialité.

Le cas d'usage santé concentre en un seul manuel toutes les difficultés du cours : volumes considérables, annotation médicale délicate et coûteuse, hétérogénéité des équipements et des pratiques, exigence

d'interprétabilité, et un cadre juridique qui encadre chaque étape. C'est pourquoi ce chapitre occupé une place disproportionnée dans la littérature : la plupart des déploiements industriels documentés s'y situent.

Le paysage des tâches

- **Imagerie médicale.** Segmentation d'organes et de lésions sur scanners, imagerie par résonance et radiographies ; détection de nodules pulmonaires ; reconstruction d'images à faible dose. Les volumes sont immenses, centaines de gigaoctets par établissement, les annotations exigent un spécialiste, et la variabilité des scanners comme des protocoles d'acquisition produit une hétérogénéité non aléatoire, corrélée à l'équipement et donc à la patientèle.
- **Dossier patient informatisé.** Prédiction de risque — sepsis, réadmission, détérioration —, codage automatisé des actes, résumé clinique structuré. Les données sont tabulaires et longitudinales, avec une forte corrélation intra-patient : l'hypothèse d'indépendance des échantillons y est violée de façon systématique.
- **Biologie et génomique.** Classification de souches, prédiction de réponse au traitement, annotation de variants. Le rapport signal sur bruit y est très faible, ce qui rend la régularisation fédérée indispensable et l'identification causale délicate.
- **Prédiction de risque.** Ici, la sortie est une probabilité qui sera lue par un clinicien : elle doit être calibrée, pas seulement bien classée. Un modèle parfaitement discriminant mais mal calibré conduit à des décisions cliniques inadaptées.

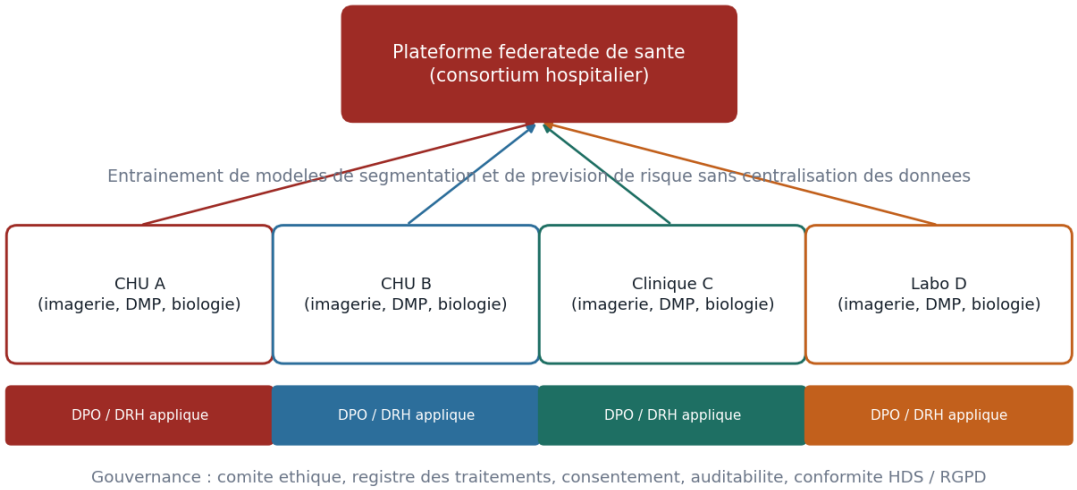


Figure 1 — Apprentissage fédéré en santé : entraîner sur plusieurs établissements sans transférer une seule imagerie

Quatre spécificités qui brisent les hypothèses usuelles

1. **Volume et déséquilibre.** Un établissement de recours dispose de centaines de fois plus de cas complexes qu'un établissement de proximité. Fédérer par pondération du nombre d'exemples conduit alors à un modèle qui reproduit la distribution du plus gros contributeur, pas celle de la population. La réponse est double : pondérer par établissement plutôt que par volume, et stratifier l'échantillonnage par site.
2. **Annotation délicate et biaisée.** Un comité annotant les mêmes images produit des désaccords sur les cas limites. Ces désaccords ne sont pas du bruit symétrique : ils sont corrélés à l'équipement, donc au site. L'annotation multi-lecteur doit donc devenir une variable du modèle, et l'incertitude d'étiquetage doit être propagée, pas seulement la prédiction.

3. **Biais de cohortes.** Un modèle entraîné sur des populations représentatives généralise mal sur une population clone. Le protocole fédéré rend ce biais mesurable : chaque établissement peut évaluer le modèle fédéré sur ses propres données, et l'écart au score national est un indicateur direct de biais local.
4. **Interprétabilité exigée.** Une prédiction de risque non expliquée n'est pas déployable : elle ne peut être ni validée par un comité, ni contestée par un patient, ni auditée après un événement indésirable. La fédération ajoute une contrainte : l'explication doit être reproductible pour un établissement donné, ce qui suppose de la calculer localement.

Cadrage du modèle : cinq décisions explicites

Avant la première ligne de code, cinq décisions déterminent la faisabilité juridique autant que la performance. Les passer sous silence revient à laisser le serveur d'entraînement devenir détenteur du dossier patient, ce que la loi n'autorisé pas dans la plupart des juridictions.

Décision	Options	Conséquence fédérée	Recommandation
Granularité des données	pixel, coupe, examen, dossier complet	définition exacte de ce qui ne sort pas du site	examen complet quand le diagnostic est l'objet
Granularité du modèle	monotâche, multi-têtes, multi-tâches	le multi-tâches mutualise les représentations	multi-tâches avec pondération par tâche
Niveau d'agrégation	par patient, par examen, par site	évite de renforcer un site majoritaire	par site, avec poids uniformes
Validation	interne, croisée, externe	validation externe obligatoire hors fédération	au moins un site externe au consortium
Sortie du modèle	scores, cartes, gradients, identifiants	seuls les deux premiers sont diffusables	scores et cartes, jamais les entités nominatives

Tableau 1 – Décisions de cadrage d'un projet fédéré en santé

Modèle multi-tâches et calibration

La mutualisation des représentations se fait par une tête multi-tâches : un corps encodeur commun, plusieurs têtes spécialisées. Ce choix est particulièrement pertinent en santé, car les étiquettes disponibles ne sont pas les mêmes selon les sites : certains annotent en segmentation, d'autres en classification, d'autres en prédiction de risque.

$$L(\theta) = \sum_{t=1}^{T} \lambda_t L_t(\theta) + \mu\, \Omega(\theta)$$

$$E_k \le ft[L_k] = \sum_t \lambda_t\, E_k \le ft[L_{k,t}] + \mu\, \Omega(\theta),\ \Omega \text{ indépendant de } k \qquad (4.6)$$

La régularisation Ω et les poids λ_t sont choisis pour que le second membre reste une borne inférieure de l'objectif global : c'est la condition qui permet d'affirmer que la fédération ne dégrade pas la qualité attendue.

La calibration exige une attention particulière. Un modèle bien classé mais mal calibré est un piège en imagerie : la distribution des scores de confiance est décalée par site, parce que les cohortes, les équipements et les protocoles diffèrent. Une calibration fédérée consiste à estimer les paramètres d'une fonction de calibration, de type Platt, sur des **scores** et non sur des cas.

$$p_k^{cal}(y=1 \mid x) = \sigma \le ft(a_k\, f(x) + b_k),\ (a_k, b_k) \text{ estimés par minimisation de la perte log-likeli} \qquad (4.7)$$

Le serveur ne reçoit que des couples de score et d'étiquette binaire agrégés : l'information clinique directe ne circule pas.

Exemple

Un scanner de rattrapage envoie des scores élevés sur une population plus jeune et plus obese que la moyenne du consortium. La courbe de calibration de l'établissement est décalée vers la droite, ce qui rend le seuil opérationnel du consortium inapplicable. La correction consiste à estimer (a_k, b_k) sur des groupes d'âge, puis à fusionner les courbes dans un espace de probabilités : le réseau fédéré conserve ainsi un seuil unique interprétable, tout en gardant les données dans l'établissement.

Confidentialité, gouvernance et cadre réglementaire

L'apprentissage fédéré réduit l'exposition des données, mais ne l'annule pas. Les fonctions de perte, la structure du modèle, le nombre de tours et l'identité des participants restent des vecteurs d'inférence. Un protocole santé crédible combine donc quatre couches.

- **Minimisation des données.** Ne jamais agréger au-dessus du niveau clinique utile : un gradient calculé par examen, pas par pixel, divisé par des centaines le nombre d'informations transmises.
- **Confidentialité différentielle au niveau du site.** Bruit calibré à l'échelle de la contribution d'un établissement, avec un budget par établissement et non par tour : le budget doit protéger contre l'accumulation sur la durée du projet.
- **Gouvernance des données et des accès.** Un comité de pilotage réunissant les responsables de la donnée de chaque établissement, un registre des traitements, des conventions de partage et une procédure de retrait d'un établissement.
- **Auditabilité.** Conservation de l'historique des versions de modèles, des tours d'agrégation et des décisions, afin de pouvoir rejouer une évaluation des mois après le déploiement.

Cadre juridique et ethique

En Union européenne, le cadre applicable est le RGPD. Les données de santé figurent parmi les catégories particulières de l'article 9 : leur traitement est interdit par principe, sauf exception — consentement explicite, finalité de santé public, ou obligation légale — et ces exceptions doivent être documentées. La fédération ne constitue pas elle-même une base légale : elle modifie la nature de l'opération de traitement, qui devient une évaluation répartie ou un apprentissage fédéré, et impose donc une analyse d'impact avant le lancement.

- Le responsable de traitement n'est pas le serveur mais l'établissement, qui conserve la finalité et la relation avec la personne concernée.
- Les rôles de sous-traitant et de responsable doivent être tracés : qui héberge l'infrastructure, qui détient les clés, qui peut réidentifier.
- La mention « les données ne quittent jamais l'établissement » est exacte pour les images, mais fausse pour les gradients : l'argumentaire doit l'assumer et le compenser par des garanties techniques.
- Le droit d'effacement, d'accès et de portabilité reste opposable : un patient peut demander la suppression de ses données, ce qui impose de prévoir le retrait effectif et ses conséquences sur un modèle déjà appris.
- Dans d'autres juridictions, des obligations de localisation des données de santé s'ajoutent : la fédération devient alors un outil de conformité autant qu'un outil de performance.

```python
import torch
import monai
from monai.networks.nets import UNet
from monai.losses import DiceCELoss


def build_unet(spatial_dims: int = 3, in_ch: int = 1,
features=(16, 32, 64, 128, 256)) -> UNet:
"""U-Net 3D de segmentation, compact pour l'entrainement local.

Le budget memoire d'un accelerateur d'etablissement limite severement la
largeur des premieres couches : on prefere une profondeur grande a une
largeur grande.
"""
return UNet(
spatial_dims=spatial_dims,
in_channels=in_ch,
out_channels=1, # une seule classe : fond ou premier plan
channels=list(features),
strides=(2, 2, 2, 2),
num_res_units=2,
dropout_rate=0.2,
)


def local_segmentation_step(model, loader, optimiser, device="cuda",
scaler=None) -> float:
"""Un passage local sur le scanner de l'etablissement. Aucune sortie du site.

La ponderation du Dice compense le desequilibre de classes inherent aux
volumes segments : la lesion represente quelques centaines de voxels.
"""
loss_fn = DiceCELoss(to_onehot_y=False, sigmoid=True, lambda_dice=1.0,
lambda_ce=1.0)
model.train()
total, batches = 0.0, 0
for image, label in loader:
image = image.to(device)
label = label.to(device)
optimiser.zero_grad(set_to_none=True)
with torch.autocast(device_type="cuda", enabled=scaler is not None):
logits = model(image)
loss = loss_fn(logits, label)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
optimiser.step()
total += float(loss.detach())
batches += 1
return total / max(batches, 1)


```

Listing 1 – Entraînement fédéré d'un U-Net de segmentation médicale en MONAI

```python
def federated_unet_round(client_models: list, client_volumes: list,
client_sizes: list) -> dict:
"""Moyenne des poids d'U-Net, ponderee par le volume d'examen de chaque site.

On pondere volontairement par le nombre d'examens et non par la taille en
octets : un etablissement qui stocke davantage de coupes doit peser moins
s'il n'a pas plus de patients.
"""
total = float(sum(client_sizes))
mean = client_models[0]
with torch.no_grad():
for key, param in mean.state_dict().items():
if not param.is_floating_point():
```

```
param.copy_(client_models[0].state_dict()[key])
continue
acc = torch.zeros_like(param)
for model, n_k in zip(client_models, client_sizes):
acc += model.state_dict()[key].to(param.dtype) * (n_k / total)
param.copy_(acc)
return mean.state_dict()
```

Astuce

En imagerie tridimensionnelle, un lot de huit volumes de 256^3 occupe plusieurs gigaoctets d'activations. La première optimisation locale n'est donc pas algorithmique mais matérielle : pavage par recouvrement, mélange d'entraînement par amplificateurs et accumulation de gradients sur plusieurs lots avant pas d'optimiseur.

Risque	Manifestation	Gravité	Mitigation opérationnelle
Inversion de gradient	reconstruction d'une imagerie à partir des mises à jour	critique	agrégation sécurisée, bruit, taille de lot élevée
Biais de cohortes	performance effondrée sur une population minoritaire	élevé	stratification par site, évaluation locale obligatoire
Mauvaise calibration	seuil opérationnel inadapté	élevé	calibration fédérée sur groupes
Fuite par les métadonnées	inférence de la prise en charge locale	moyen	agrégation d'un seul tour, sans identifiant client
Modèle non reproductible	impossibilité d'auditer une décision	moyen	versionnage complet, graine fixe, journal signé
Dérive clinique	bascule silencieuse du modèle en production	élevé	surveillance continue par site et relance mensuelle

Tableau 2 – Risques de l'apprentissage fédéré en santé et mitigations

Exercice 4.3.16 (difficulte : Avancé)

Un consortium de six hôpitaux veut déployer un modèle de segmentation hépatique. L'hôpital A détient 60 % des volumes et une machine 256 coupes par examen ; les cinq autres détiennent chacun 8 %, avec des scanners plus anciens et une population plus âgée.

1. Proposer une pondération d'agrégation et la justifier.
2. Expliquer pourquoi le seul Dice est un mauvais critère d'arrêt, et ce qu'il faut mesurer en complément.
3. Lister trois contrôles à mettre en place avant le premier tour réellement productif, en précisant lequel dépend d'un calcul de confidentialité.
4. Dire ce qu'il faut prévoir au contrat si l'un des six hôpitaux se retire après vingt tours.

Indice : Comparer le poids du volume au poids de la diversité des sites : ce ne sont pas les mêmes grandeurs.

Correction

- **1. Pondération.** Un mélange de 0,6 pour l'hôpital A et de 0,08 pour les cinq autres reproduit sa population et son matériel : le modèle sera optimal pour lui et médiocre ailleurs. Deux correctifs se combinent : pondération uniforme par site — le consortium devient une population de six sites — ou pondération de type température $\lambda_k \propto n_k^{\tau}$ avec τ fortement inférieur à 1. En pratique on pondère par site et l'on réserve la pondération par volume aux lots d'entraînement internes.
- **2. Critère d'arrêt.** Le Dice mesure le recouvrement, pas la calibration, et varie avec la taille de la lésion. Il faut ajouter au minimum la distance de surface moyenne — une erreur de contour vaut plus qu'une erreur au centre — et la calibration des probabilités par la courbe de fiabilité sur des lots de faux positifs, seul indicateur lié au coût opérationnel du clinicien.

- **3. Trois contrôles avant le premier tour productif.** Un test d'inversion : vérifier empiriquement qu'aucune reconstruction d'image n'est possible à partir des mises à jour capturées — c'est le seul des trois qui dépend d'un calcul de confidentialité. Une validation clinique hors fédération : au moins un site non membre évalue le modèle avant tout usage, sinon le consortium mesure sa propre distribution. Une étude de stabilité du protocole : dix graines, dix tours, mesure de l'écart-type entre graines. Un écart-type élevé signale que la fédération elle-même est la source de variance.
- **4. Retrait d'un membre.** Le contrat doit prévoir la longueur de conservation des mises à jour de ce membre, leur destruction vérifiable, et surtout le sort du modèle déjà distribué : le retirer obligerait à réentraîner depuis la base, ce qui est généralement chiffré comme irréalisable. La position défendable est de conserver le modèle publié, de tracer le retrait dans le registre, et de s'engager sur une réévaluation sur les cohortes restantes.

A retenir

- En santé, la fédération est une condition d'accès et non un gain de performance.
- Pondérer par établissement plutôt que par volume : sinon le modèle hérite du protocole du plus gros contributeur.
- La calibration est un critère de première classe, au même titre que le Dice ou l'aire sous la courbe.
- Fédérer ne supprime pas le risque : l'inversion de gradient reste possible, et l'argumentaire juridique doit le reconnaître.

Chapitre 4.4

Véhicules connectés et systèmes de transport (V2X)

Coopérer sous contrainte de temps et d'itinérance

Un véhicule ne peut pas attendre un tour d'entraînement pour décider de freiner. Tout ce qui est appris doit donc être délivré en quelques millisecondes, ce qui impose une fédération multi-niveaux plutôt qu'un serveur unique.

Le véhicule connecté est le domaine où le conflit entre nécessité d'apprentissage et contrainte de temps est le plus brutal. Un modèle centralisé de perception est meilleur, mais il est inaccessible depuis un tunnel ou un parking souterrain. Un modèle embarqué est disponible, mais il n'a vu que la flotte qui le transporte. La fédération véhiculaire propose un équilibre : coopérer quand c'est possible, se dégrader proprement quand ce n'est pas.

Taxonomie des communications véhiculaires

Le sigle V2X regroupe cinq familles de liens dont les contraintes ne sont pas du tout les mêmes.

Lien	Périmètre	Latence typique	Débit	Usage principal
V2V, véhicule à véhicule	portée directe, 100 à 300 m	1 à 10 ms	faible	fusion de perception, alerte de collision
V2I, véhicule à infrastructure	portée de la route équipée, 1 km	5 à 20 ms	moyen	signalisation, contrôle d'accès
V2P, véhicule à piéton	portée courte, 50 à 200 m	10 à 50 ms	très faible	alerte piéton, usager vulnérable
V2N, véhicule à réseau	opérateur, couverture large	30 à 100 ms	moyen	information de trafic, maintenance
C-V2X, cellulaire	5G à faible latence, couverture large	5 à 20 ms	élevé	coordination à distance, jumeau numérique

Tableau 1 – Taxonomie V2X : liens, portée et usage

C-V2X et ITS-G5 ne sont pas concurrents mais complémentaires

L'architecture de référence en Europe pour les transports intelligents repose historiquement sur ITS-G5, une norme d'origine Wi-Fi à courte portée, prévue pour la sécurité mais sans gestion native de la latence. La variante cellulaire ajoute la couverture, la gestion de la mobilité et la capacité réseau, mais introduit une dépendance à l'opérateur. Les deux coexistent : la sécurité est portée par l'infrastructure à clé public, pas par le support radio.

- Pour l'apprentissage fédéré, la conséquence est directe : un véhicule peut transmettre un gradient de faible volume en quelques millisecondes en véhicule-à-véhicule, mais pas l'entraîner sur place. L'entraînement reste local et court, la synchronisation est asynchrone.

Hiérarchie multi-niveaux

Quatre niveaux se distinguent par leur horizon temporel et leur capacité. La règle de conception est simple : chaque niveau n'échange avec le supérieur que ce qui change lentement.

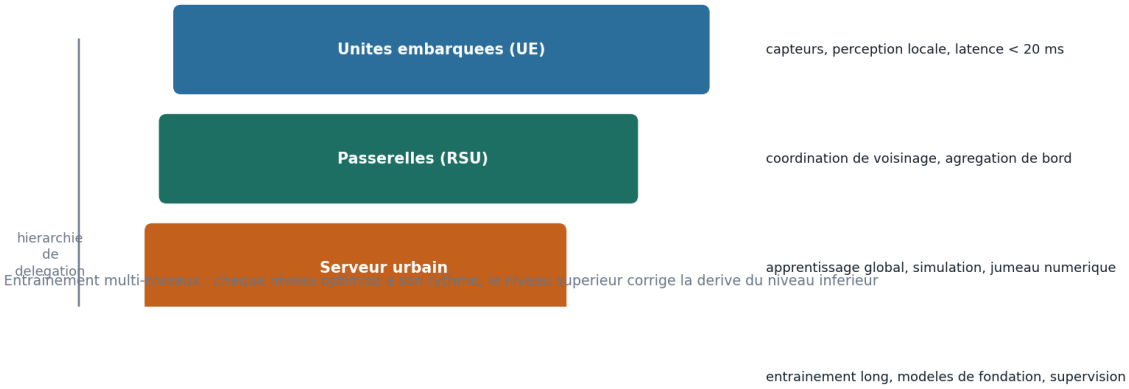


Figure 1 – Délégation multi-niveaux : unité embarquée, passerelle, serveur urbain, nuage

Niveau	Latence	Mémoire	Modèle possible	Échange avec le niveau supérieur	Cadence
Unité embarquée	1 à 20 ms	1 à 4 Go	réseau convolutif compact	vecteurs d'état et détections	en continu
Passerelle	5 à 30 ms	8 à 32 Go	réseau convolutif et détecteur, fusion multi-capteurs	gradients sparsifiés	10 à 100 Hz
Serveur urbain	50 à 200 ms	512 Go à 2 To	détecteur et suivi multi-objets	modèles de trajectoire	1 à 10 Hz
Nuage	supérieure à 200 ms	sans limite utile	modèles de fondation, détection 3D	scénarios et statistiques	1 à 60 minutes

Tableau 2 – Exigences par niveau d'architecture V2X

La hiérarchie n'est pas qu'une répartition de charge : elle est le mécanisme de gestion de la **dérive**. Un modèle embarqué vieillit silencieusement quand son environnement change ; le niveau supérieur détecte la dégradation par les indicateurs agrégés, puis renvoie une correction. C'est le mécanisme classique d'un système fédéré, mais où l'échelle de temps est contrainte par la physique du véhicule.

$$\theta_{l+1} = (1 - \alpha_l)\,\theta_l + \alpha_l\,\mathrm{Big}\Big(\sum_k p_k\,\theta_l^k\,\mathrm{Big}\Big),\quad \theta_{l+1}^k = \theta_{l+1} + \eta\,g_k(\theta_{l+1})$$
$$\eta \ll \alpha_l\,\L_k \;\square\; \text{le niveau local reste proche du niveau global} \qquad (4.8)$$

La première équation décrit un mélange à deux temps : l'alignement sur la moyenne globale, puis une correction locale bornée. La seconde donne la condition de cohérence : le pas local doit rester petit devant le pas d'alignement.

Coopération perceptive : le partage de représentations

La brique fonctionnelle spécifique au domaine est la perception coopérative. Un véhicule ne voit pas ce qu'un autre voit ; en échangeant des représentations denses — et non des détections discrètes — chaque véhicule améliore sa compréhension de la scène sans transmettre d'image brute.

$$h_i^{(t)} = f_\theta\big(X_i^{(t)}, \{h_j^{(t)} : j \in \mathcal{N}(i)\}\big), \quad \tilde{h}_i^{(t)} = \Big[h_i^{(t)} \,\|\, \tfrac{1}{|\mathcal{N}(i)|}\sum_{j\in\mathcal{N}(i)} h_j^{(t)}\Big]$$
$$\mathcal{L}_{fus} = \sum_i \ell\big(\hat{y}_i, y_i\big) + \lambda \sum_i \big\| h_i^{(t)} - h_i^{(t-1)} \big\|_2^2 \qquad (4.9)$$

La fonction g est un encodeur de confiance : une représentation transmise par un véhicule en marche arrière vaut moins qu'une représentation d'un véhicule en marche normale. Le second terme pénalise la rupture temporelle, qui est la cause principale des objets fantômes.

> **Astuce**
> La fusion par concaténation simple échoue dès que la topologie change : un véhicule qui reçoit des représentations d'un nombre variable de voisins doit soit recombiner ses propres paramètres, soit utiliser une opération invariante — attention, moyenne pondérée. Les implémentations industrielles privilégient l'attention : elle gère l'itinérance sans reconfigurer le modèle.

```python
import torch
import torch.nn as nn


class NeighbourAggregator(nn.Module):
"""Agregation des representations recues des vehicules voisins.

Chaque message est pondere par une confiance calibree en fonction de la
qualite du lien, mesuree par le rapport signal sur bruit, et de la
coherence temporelle. Cette confiance est la seule information non
geometrique ; la position est traitee par l'encodage positionnel en amont.
"""

def __init__(self, feat_dim: int = 256, n_heads: int = 4, dropout: float = 0.1):
super().__init__()
self.attn = nn.MultiheadAttention(feat_dim, n_heads, dropout=dropout,
batch_first=True)
self.norm = nn.LayerNorm(feat_dim)
self.snr_scale = nn.Sequential(nn.Linear(1, 8), nn.ReLU(),
nn.Linear(8, 1), nn.Sigmoid())

def forward(self, own: torch.Tensor, messages: torch.Tensor,
snr_db: torch.Tensor) -> torch.Tensor:
"""own : (B, N, C) ; messages : (B, M, N, C) ; snr_db : (B, M)."""
if messages.numel() == 0:
return own # aucun voisin : repli local
b, m, n, c = messages.shape
# Poids de confiance derives du rapport signal sur bruit du lien
weights = self.snr_scale(snr_db.view(b, m, 1)) # (B, M, 1)
flat = (messages * weights.unsqueeze(2)).reshape(b, m * n, c)
# Interrogation croisee : chaque vehicule interroge tous les voisins
keys = torch.cat([own, flat], dim=1) # (B, 1 + M*N, C)
out, _ = self.attn(own, keys, keys, need_weights=False)
return self.norm(own + out) # connexion residuelle


def temporal_consistency_loss(current: torch.Tensor,
previous: torch.Tensor) -> torch.Tensor:
"""Penalise la rupture temporelle des representations d'un meme vehicule.

Sans ce terme, un objet qui apparait brievement dans le champ de vision peut
etre propage aux voisins puis reinjecte en retour : c'est la cause
classique des faux positifs fantomes en perception cooperative.
"""
return torch.nn.functional.mse_loss(current, previous)


```

Listing 1 — Agrégation de représentations de voisins pour la perception coopérative multi-véhicule

```python
def vehicle_local_update(model, optimiser, frames, labels,
prev_state, snr_db) -> float:
"""Un pas d'entrainement local, dans la fenetre de temps d'un vehicule.

La contrainte de temps impose un budget : on limite le nombre de lots plutot
que d'ajuster le pas d'apprentissage, car arreter tot est toujours preferable
a ne pas finir avant la fin du creneau.
"""
model.train()
total = 0.0
for _ in range(2): # deux lots : 40 ms en moyenne
feats = model.encode(frames)
fused = model.fuse(feats, snr_db)
loss = torch.nn.functional.smooth_l1_loss(model.head(fused), labels)
loss = loss + 0.5 * temporal_consistency_loss(fused, prev_state)
optimiser.zero_grad(set_to_none=True)
loss.backward()
```

```
optimiser.step()
total += float(loss.detach())
prev_state = fused.detach()
return total / 2
```

Contraintes : latence, connectivité, sécurité

- **Latence.** Le budget de 20 millisecondes est une contrainte de **souveraineté**, non de performance : au-delà, l'information transmise est caduque. Cela impose de transmettre des états, pas des modèles, dans la boucle de conduite. L'apprentissage peut se faire plus lentement, mais la prédiction servant à freiner doit rester locale.
- **Connectivité intermittente.** Un véhicule traverse des zones blanches de plusieurs kilomètres. Le protocole doit donc être asynchrone et tolérant aux messages périmés : un message âgé de plus de 100 millisecondes doit être écarté, pas fusionné. Une stratégie de tampon de perceptions recentrées sur la position courante rejoue les messages hors ligne.
- **Élasticité.** Les rafales de conical radio se traduisent par des pics ou des creux du nombre de véhicules actifs. Un algorithme à taux de participation fixe dégrade sa convergence ; il faut dimensionner le taux de participation sur le nombre de véhicules présents, pas sur le nombre de véhicules enregistrés.
- **Sécurité par infrastructure à clé public.** Cette infrastructure est déjà obligatoire pour signer les messages V2X. Les mises à jour de modèle doivent emprunter le même canal : certificat de véhicule, chaîne de confiance, liste de révocation. Un véhicule compromis ne doit pas pouvoir injecter une mise à jour non signée.

Attention

L'apprentissage fédéré sur véhicule combine deux surfaces d'attaque qu'il faut traiter séparément : l'attaque par empoisonnement de données, où un véhicule infecté envoie des gradients fabuleux, et l'attaque par empoisonnement de modèle, où il est contacté pour une mise à jour précise. La seconde est bien plus efficace avec la chaîne de certificats existante, car un véhicule signé est un véhicule de confiance. Il faut donc prévoir une procédure de révocation de certificat liée à une anomalie de mise à jour, et pas seulement à une anomalie de signature.

Application	Grandeur prédite	Horizon utile	Niveau d'apprentissage dominant	Bénéfice fédéré
Prévention de collision	probabilité de conflit, distance temps au contact	10 à 500 ms	unité embarquée	couvre les véhicules qu'on ne voit pas
Gestion de trafic	temps de parcours, densité, taux d'occupation	5 à 60 minutes	serveur urbain	apprend des situations rares jamais vues localement
Freinage collectif	accélération latérale du véhicule devant	50 à 200 ms	passerelle	réaction coordonnée sans communication complète
Maintenance prédictive	dérive de vibration, anomalie capteur	jours à semaines	nuage	détecte des pannes rares avant la panne
Détection d'événement	objet tombé, effondrement, obstacle soudain	100 ms à 10 s	passerelle et serveur	fusion multi-capteurs multi-sites

Tableau 3 – Applications de l'apprentissage fédéré en transport

Règle de déploiement en quatre temps

- **Capter localement, apprendre localement, transmettre peu.** La boucle critique de sécurité ne sort jamais du véhicule.
- **Synchronisation opportuniste.** Les mises à jour partent lorsque le véhicule est à l'arrêt ou en charge : c'est là que la bande passante est gratuite et la consommation énergétique tolérée.

- **Échelle par niveau.** La passerelle agrège les gradients du voisinage ; le serveur urbain agrège les gradients des passerelles ; le nuage n'intervient que sur les statistiques de long terme.
- **Dégradation explicite.** Chaque niveau doit savoir redéployer rapidement son modèle en cas d'anomalie : le retour à la version précédente doit prendre moins d'une minute à la passerelle.

Exercice 4.4.17 (difficulte : Moyen)

Une agglomération de 400 000 habitants déploie des passerelles sur l'anneau principal. Les véhicules connectés représentent 25 % du trafic en pointe, soit 6 000 véhicules actifs sur la zone. Le modèle de perception fusionnée compte 45 millions de paramètres ; la contrainte est une latence de fusion de 20 millisecondes et un débit moyen de 2 mégaoctets par véhicule.

1. Choisir, pour chacun des quatre niveaux, ce qui doit apprendre sur place et ce qui doit être téléchargé.
2. Calculer le volume mensuel de téléchargement dans le scénario où chaque véhicule télécharge le modèle de la passerelle toutes les dix minutes.
3. Proposer une alternative si ce volume dépasse la capacité du réseau opérateur, en indiquant le coût en performance.
4. Justifier pourquoi le nuage ne doit pas être dans la boucle de fusion, même avec un réseau 5G.

Indice : Comparer le débit disponible au volume par véhicule et par minute, avant de comparer les performances.

Correction

- **1. Répartition par niveau.** L'unité embarquée assure la détection d'objets immédiats et la prédiction de trajectoire : ce sont les seules tâches dans le budget de 20 millisecondes, et elles ne dépendent que de la détection locale. La passerelle assure la fusion multi-capteurs entre véhicules à portée, c'est-à-dire tout ce qui est difficile à voir d'un seul point de vue ; l'apprentissage de la pondération des voisins est local. Le serveur urbain assure la prédiction de congestion et l'adaptation du modèle de fusion aux topologies locales, avec un téléchargement par heure. Le nuage assure les statistiques de long terme, la détection d'anomalies rares et le préentraînement des modèles de fondation.
- **2. Volume mensuel.** 45×10^6 paramètres en virgule flottante 32 bits donnent 180 mégaoctets par téléchargement. Six téléchargements par heure, donc environ 1,08 gigaoctet par heure et par véhicule, soit environ 26 gigaoctets par mois et par véhicule. Avec 6 000 véhicules actifs en pointe et une pointe de deux heures par jour, l'ordre de grandeur retenu reste de quelques dizaines de téraoctets mensuels pour l'ensemble du réseau.
- **3. Alternative si le réseau sature.** Remplacer le téléchargement complet par un delta d'adaptateur de faible rang entre la version de la passerelle et la version locale : le coût passe à environ 2 mégaoctets au lieu de 180, soit une division par 90. La contrepartie est une convergence locale plus lente, de l'ordre de plusieurs dizaines de tours au lieu de deux : le délai de synchronisation passe de dix minutes à quelques heures. Sur une tâche de fusion de voisinage, dont la dynamique est lente, ce délai est indifférent. Une seconde option est de faire tourner la version nuage uniquement sur les véhicules connectés en permanence, ce qui concentre le coût sur une minorité.
- **4. Pourquoi le nuage reste hors de la boucle.** Trois raisons cumulatives. Latence : même en 5G, l'aller-retour avec un centre de données à distance est de l'ordre de 20 à 40 millisecondes en charge, donc du même ordre que le budget total, ce qui le rend inutilisable pour la décision de sécurité. Couverture : le tunnel, le parking et la zone blanche ne sont pas desservis. Risque : une panne du réseau opérateur doit dégrader les véhicules en autonomie locale, jamais les rendre aveugles.

A retenir

- La latence impose une hiérarchie stricte : la sécurité du véhicule ne dépend jamais d'un réseau distant.
- Ce qui circule dans la boucle critique, ce sont des états ; ce qui circule pour apprendre, ce sont des mises à jour.
- La perception coopérative améliore la couverture sans échanger d'image : c'est l'exemple le plus abouti de partage de représentations.
- L'infrastructure à clé public existante doit être étendue aux mises à jour de modèle, sinon elle ne protège plus rien.

Chapitre 4.5

TinyML et fédération embarquée

Apprendre avec 128 kilo-octets de mémoire vive

Un microcontrôleur dispose de moins de mémoire qu'une image de pondération. La seule façon d'y faire de l'apprentissage fédéré est de rendre le modèle et la mise à jour si petits qu'ils tiennent dans l'appareil.

La fédération embarquée pose un problème de dimensionnement différent des autres domaines. La contrainte n'est ni le coût, ni le droit, ni la latence : c'est la mémoire. Un capteur industriel dispose de 128 kilo-octets à 1 mégaoctet de mémoire vive, contre plusieurs gigaoctets pour un serveur. Or tout algorithme d'apprentissage fédéré suppose de stocker au minimum un modèle, un état d'optimiseur et un tampon de communication. La question n'est donc pas « quel algorithme choisir » mais « comment rendre l'espace de travail compatible avec la cible ».

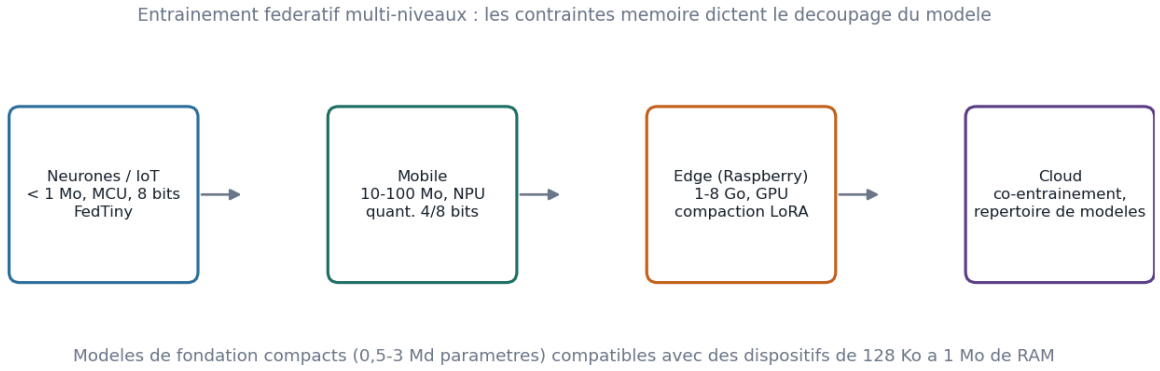


Figure 1 — Fédération multi-niveaux pour l'objet connecté : du capteur au répertoire de modèles

Les contraintes matérielles, en chiffres

Classe de dispositif	Mémoire vive	Mémoire flash	Énergie par inférence	Modèle tenable	Connectivité
Microcontrôleur 8 bits	1 à 8 Ko	8 à 64 Ko	environ 1 μJ	2 à 8 Ko : règles, arbres	sans protocole IP
Microcontrôleur 32 bits bas coût	20 à 200 Ko	128 Ko à 1 Mo	10 à 100 μJ	50 à 300 Ko	Wi-Fi, Bluetooth basse énergie
Microcontrôleur 32 bits avec accélérateur	256 Ko à 2 Mo	1 à 16 Mo	0,2 à 2 mJ	0,5 à 3 Mo	Wi-Fi, LTE-M, 5G
Dispositif passif, à récupération d'énergie	8 à 32 Ko	32 à 256 Ko	moins de 1 μJ	modèles décimaux compressés	radio très basse fréquence
Passerelle locale	512 Mo à 8 Go	16 à 128 Go	1 à 10 mJ	modèles de base compacts	Ethernet, 4G, 5G

Tableau 1 — Budget matériel des cibles embarquées

Le budget mémoire impose une contrainte arithmétique sévère. Un modèle de d paramètres en virgule flottante 32 bits occupe $4d$ octets ; l'état de l'optimiseur en occupe douze de plus. Avec 128 kilo-octets disponibles, la borne est $d \leq 4\times 10^3$ paramètres, ce qui est deux ordres de grandeur en dessous du plus petit réseau convolutif utile. Toute la littérature sur l'intelligence embarquée porte donc sur la compression, non

sur l'algorithme d'agrégation.

$$M(k) = M_{mod} + M_{opt} + M_{buf}, \text{ avec } M_{mod} = (b)/(8)\ d,\ M_{opt} = (b \cdot 2)/(8)\ d,\ M_{buf} = P_{msg} \cdot N_{buf}$$
$$d_{max} = \leq \text{ftlfloor } (8\ (M_{total} - M_{buf}))/(3\ b)\ \text{rfloor} \tag{4.10}$$

b **est le nombre de bits par poids et** P_{msg} **la taille d'un message. Avec** b = 8 **et** M_{total} = 128 **kilo-octets, la borne laisse la place à trois aller-retours complets.**

> **Astuce**
> Sur cible 8 bits, l'optimiseur n'a pas sa place. Les travaux issus de FedTiny remplacent l'optimiseur à deux moments par un optimiseur à un seul moment, voire par une simple descente de gradient : la mémoire d'état passe de 12d à 4d octets, ce qui triple le nombre de paramètres tenables sans modifier le principe de la fédération.

Familles de méthodes

- **FedTiny.** Cadre de référence qui rend explicites les quatre leviers d'un système embarqué fédéré : quantification des poids, sparsification du gradient, entraînement local à faible nombre d'itérations, et hiérarchie de serveurs. Les cibles sont typiquement des capteurs industriels ou des compteurs intelligents.
- **FjORD.** Introduit un modèle d'atténuation du bruit spécifique à l'internet des objets : le bruit des mesures de terrain n'est pas gaussien, il est **à queue lourde** — capteurs défectueux, pannes intermittentes, valeurs aberrantes. FjORD montre qu'un modèle de Student corrige l'agrégation FedAvg, qui suppose une perte bornée, hypothèse non vérifiée sur un capteur réel.
- **Per-FedAvg.** Répond à un problème différent : l'oubli. Sur un capteur, la mémoire est trop faible pour stocker plusieurs états de modèle, donc pour corriger la dérive par FedProx ou SCAFFOLD. Per-FedAvg utilise une base orthonormale de sous-espaces pour stocker les corrections de façon compressée : la correction tient dans un vecteur de quelques centaines de valeurs.
- **Quantification et distillation fédérées.** La distillation transfère la connaissance d'un grand modèle, hébergé dans le nuage, vers un modèle compacte dans le dispositif ; la fédération organisé la mise à jour de ce petit modèle. Les deux techniques se combinent : le nuage joue le rôle de l'enseignant, l'appareil celui de l'élève.

> **Piege courant**
> Un capteur dont les mesures sont bruitées et dont le protocole radio est intermittent ne peut pas utiliser une agrégation FedAvg simple. L'hypothèse de Lipschitz bornée, qui fonde la convergence de FedAvg, est violée par les valeurs aberrantes de capteur : un seul client entrant peut déformer l'agrégat de façon durable. Il faut soit un détecteur de valeurs aberrantes en amont, soit un modèle à queue lourde dans la perte d'agrégation.

Compression de la communication

La taille du message est le second goulot, souvent plus sévère que la mémoire. Un capteur embarquant une liaison radio de 32 kilobits par seconde peut transmettre quelques centaines de kilo-octets par minute : transmettre un vecteur de gradient dense est exclu, même pour un petit modèle.

Technique	Réduction	Coût de calcul	Perte de précision	Émission de moments
Quantification 8 bits	4 fois	nulle	faible si le gradient est concentré	oui
Quantification 1 bit, signe seul	32 fois	faible	élevée sans estimation de variance	non
Sparsification par valeurs dominantes	d/k	faible	moyenne	oui
Différence de gradient	variable	nulle	faible si les clients convergent	non
Codage de Huffman par bloc	1,3 à 2 fois	faible	nulle	non
Transformée de Fourier partielle	10 à 50 fois	moyenne	moyenne	oui
Quantification par arbre de Fourier	100 à 1000 fois	moyenne	faible sur gradient lisse	oui
Agrégation sur plusieurs tours, 1 bit	K fois	nulle	divergente	oui

Tableau 2 – Techniques de compression des mises à jour et coût associé

L'idée de fond est la suivante : les gradients d'un modèle dense sont fortement **lisses** dans un sens physique — l'essentiel de l'information est porté par les basses fréquences spatiales. Une transformée de Fourier, ou une transformée en cosinus discrets, permet donc de ne transmettre que les composantes de basse fréquence, qui concentrent l'essentiel de l'information, et de reconstruire une approximation de la mise à jour côté serveur. Le gain se paie par un terme de bruit, dont l'introduction permet de borner la dégradation.

$$\Delta W \approx (1)/(d) \sum_{f \in \square 0 \square} \Delta \hat{W}(f) \Phi_f, \quad |F| \ll d$$
$$\| W - \text{widetildeW} \|_2^2 \leq \varepsilon \cdot \| W \|_2^2, \quad \varepsilon = O \leq ft((|F|)/(d)) \qquad (4.11)$$

Φ_f est un vecteur de Fourier, ou de cosinus discrets, de dimension d. La seconde équation justifie le choix du sous-espace : plus on retient de fréquences, plus le spectre est conservé, et plus la reconstruction est fidèle.

Protocole de référence pour une cible de 256 Ko de mémoire vive

- **Modèle.** Un réseau à convolution de profondeur séparable, largeur 16 à 32, quantifié en 8 bits : environ 60 Ko de poids.
- **Optimiseur.** À un seul moment, en 8 bits : 30 Ko.
- **Tampon.** Deux gradients compressés à 4 Ko chacun, soit 8 Ko.
- **Transmission.** Cent vingt-huit valeurs dominantes sur 16 384, quantifiées en 8 bits : 256 octets par message, plus un en-tête de 4 octets.
- **Cadence.** Un tour toutes les six heures : l'énergie de transmission reste marginale devant la consommation de veille du capteur.
- **Agrégation.** Moyenne pondérée au niveau du serveur de zone, avec détection de valeurs aberrantes sur la norme des mises à jour.

```
import struct
import numpy as np


def quantise_int8(x: np.ndarray):
"""Quantification affine par symetrie : q = arrondi(x / s) en int8.

L'echelle s est calculee sur le lot local puis transmise avec le message ;
elle est donc transmise en flottant, mais ne pese que 4 octets.
"""
scale = max(float(np.max(np.abs(x))) / 127.0, 1e-12) # evite la division par zero
q = np.clip(np.round(x / scale), -127, 127).astype(np.int8)
return q, scale


def sparsify_topk(x: np.ndarray, k: int):
"""Ne conserve que les k plus grandes composantes en valeur absolue.

Sur un gradient de reseau, ces composantes portent la majorite de la
variation ; le reste est approximatif. Le protocole de Doe et al. montre
que l'erreur introduite est bornee par la queue de la distribution.
"""
flat = x.reshape(-1)
if k >= flat.size:
return flat, np.arange(flat.size)
idx = np.argpartition(np.abs(flat), -k)[-k:]
return flat[idx], idx


def pack_update(grad: np.ndarray, k: int = 128) -> bytes:
"""Empreinte memoire d'une mise a jour : 2 octets par indice, 1 par valeur.

Pour un modele de 16 384 parametres et k = 128, cela represente 512 octets
contre 65 536 octets en flottant 32 bits, soit un gain de 128 fois.
"""
vals, idx = sparsify_topk(grad, k)
q, scale = quantise_int8(vals.astype(np.float32))
# indices sur 2 octets chacun : suffisant jusqu'a 65 536 parametres
header = struct.pack("<Hf", scale, len(idx))
return header + idx.astype(np.uint16).tobytes() + q.tobytes()


def unpack_update(blob: bytes, dim: int) -> np.ndarray:
"""Reconstruit un vecteur creux de dimension dim a partir de l'empreinte."""
scale, count = struct.unpack_from("<Hf", blob, 0)
off = 6
idx = np.frombuffer(blob, dtype=np.uint16, count=count, offset=off)
off += 2 * count
q = np.frombuffer(blob, dtype=np.int8, count=count, offset=off)
dense = np.zeros(dim, dtype=np.float32)
dense[idx.astype(np.int64)] = q.astype(np.float32) * scale
return dense


```

Listing 1 — Client fédéré sur microcontrôleur simulé : sparsification, quantification et descente de gradient sans état

```
def local_sgd_step(params: np.ndarray, x_batch: np.ndarray, y_batch: np.ndarray,
lr: float = 0.05) -> np.ndarray:
"""Un pas de gradient local sur un modele lineaire quantifie en 8 bits.

On utilise une descente de gradient simple : aucun etat d'optimiseur n'est
stocke, ce qui est la condition pour tenir dans 128 Ko de memoire vive.
"""
pred = x_batch @ params
err = (pred - y_batch).astype(np.float32)
grad = x_batch.T @ err / max(x_batch.shape[0], 1)
```

```
updated = params - lr * grad
# Projection dans la grille int8 apres la mise a jour : l'etat reste borne
q, scale = quantise_int8(updated)
return (q.astype(np.float32) / 127.0) * 127.0
```

Exemple

Un capteur de vibration sur un palier surveille un moteur électrique. Il collecte 2 048 échantillons par fenêtre de 10 millisecondes et exécute localement une détection d'anomalie basée sur une enveloppe spectrale de 64 coefficients. À chaque tour fédéré, il envoie 128 valeurs dominantes de son gradient de 4 096 paramètres, soit 512 octets. Avec une fenêtre d'énergie de 50 milliampères-heures et une consommation de veille de 30 microampères, il peut transmettre deux fois par jour pendant plus de cinq ans sans batterie.

Exercice 4.5.18 (difficulte : Moyen)

Un compteur électrique connecté dispose de 192 Ko de mémoire vive, dont 96 Ko réservés à l'application. Le modèle de détection de fraude comporte 12 288 paramètres. Le lien radio est un LoRaWAN de 5 kilobits par seconde utiles.

1. Déterminer si un protocole FedAvg à gradients denses est tenable, et chiffrer le temps d'émission.
2. Proposer un protocole de sparsification avec sa taille de message et son temps d'émission, en tenant compte de la limite LoRaWAN d'un paquet de 51 octets.
3. Comparer avec une stratégie de quantification 1 bit sur la totalité des 12 288 paramètres.
4. Dire ce qui se passe si 5 000 compteurs transmettent simultanément, et quelle organisation de serveur évite cela.
5. Proposer un indicateur de déclenchement local, avant envoi, qui réduise le volume de 90 % tout en gardant une détection fiable.

Indice : Un paquet radio ne peut pas contenir un message arbitrairement grand : comptez les paquets, pas les octets.

Correction

- **1. Gradients denses.** 12 288 × 4 = 49 152 octets par message. À 5 000 bits par seconde utiles, la durée d'émission est de 78 millisecondes, soit environ 960 paquets de 51 octets. C'est tenable en consommation énergétique — quelques millijoules — mais **pas** en bande passante partagée : un compteur qui émet huit fois par jour occupe peu, une flotte de 5 000 compteurs émettant simultanément saturerait la passerelle.
- **2. Sparsification.** Avec 256 valeurs sur 12 288, quantifiées en 8 bits et indexées sur 2 octets : 256 × (1 + 2) = 768 octets, soit 15 paquets. L'émission descend à 1,2 milliseconde. C'est le bon régime pour ce modèle.
- **3. Quantification 1 bit.** Un bit par valeur : 12 288 bits donnent 1 536 octets, soit 30 paquets et 2,5 millisecondes. Le volume est deux fois plus faible que la sparsification, mais l'erreur d'estimation est considérablement plus grande : sans correction de variance, l'agrégat converge vers un point arbitraire. On ne peut donc pas comparer les deux sur le seul volume.
- **4. Collision.** Les 5 000 émissions simultanées se recouvrent partiellement, LoRa faisant de l'aléatoire de fréquence, mais provoquent un taux de perte important. La solution n'est pas de renforcer l'émission mais de **décorréler** le trafic : chaque compteur tire un décalage aléatoire dans une fenêtre de quelques minutes, ce qui étale la charge sans augmenter le volume total. Une seconde option est une règle de participation : une petite fraction des compteurs seulement émet à chaque tour.
- **5. Déclenchement local.** Un compteur ne signale que les valeurs dont le gradient dépasse un seuil absolu, ou bien quand la mesure d'enveloppe spectrale s'écarte de la référence apprise de plus de trois écarts-types. Sur des données de fraude, la fréquence de déclenchement est de l'ordre de 1 à 2 % des cycles : on obtient la réduction de 90 à 98 % demandée, avec l'inconvénient que le signal d'agrégat devient un signal de détection et non un signal d'apprentissage continu.

A retenir

- La contrainte dominante de l'internet des objets est la mémoire, pas la bande passante : elle fixe une borne dure sur la taille du modèle.
- Sans quantification à 8 bits au minimum et sans suppression de l'état d'optimiseur à deux moments, aucun apprentissage local n'est réalisable sous 1 mégaoctet.

- Les distributions de gradient issues de capteurs réels sont à queue lourde : l'agrégation FedAvg suppose une hypothèse qui n'est pas vérifiée.
- La sparsification améliore la bande passante ; elle dégrade la précision, et les deux doivent être compensées explicitement.

Chapitre 4.6

Apprentissage fédéré sur graphes (FedGL)

Quand les données sont des structures, pas des tableaux

Un graphe n'est pas un échantillon de vecteurs : c'est un objet dont la distribution porte sur la topologie elle-même. Fédérer des graphes pose donc un problème de non-indépendance d'une nature particulière.

Toutes les hypothèses de l'apprentissage fédéré supposées jusqu'ici s'appuient sur une représentation tabulaire : un échantillon est un vecteur, et deux échantillons sont indépendants même s'ils n'ont pas la même loi. Un graphe viole ces deux hypothèses. La non-indépendance n'y est pas un détail statistique : elle est structurelle, et elle concerne simultanément les trois composantes de la donnée.

Trois sources de non-indépendance

- **La structure.** Deux graphes clients peuvent avoir des topologies complètement différentes : un réseau social local contre un réseau de transport, un réseau moléculaire. La distribution des degrés, des coefficients de grappe et de la longueur de chemin moyen ne se ressemblent pas. Un réseau de convolutions sur graphe appris sur l'un est structurellement mal calibré sur l'autre.
- **Les caractéristiques.** Les attributs des nœuds sont d'une nature différente selon le domaine : débit d'un routeur, identité chimique d'un atome, incorporation textuelle d'un utilisateur. Les espaces de caractéristiques n'ont ni les mêmes dimensions ni les mêmes statistiques.
- **Les étiquettes.** Un réseau social s'étiquette par classe de communauté, un réseau moléculaire par type de liaison, un réseau financier par classe de risque. La fonction de perte est formellement identique, mais sa géométrie ne l'est pas, ce qui rend l'hypothèse d'échangeabilité caduque.

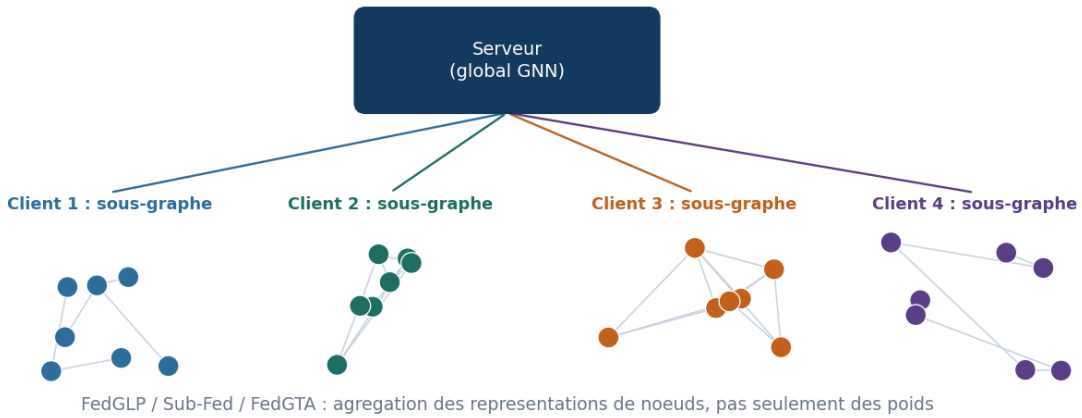


Figure 1 — Apprentissage fédéré sur graphes : chaque client détient un sous-graphe, le serveur agrège des représentations

$$\min_\theta \sum_{k=1}^{N} p_k \, Lbig(G_k, \theta big), \; L(G_k, \theta) = (1)/(|V_k|) \sum_{v \in V_k} \ell big(y_v, f_\theta(G_k)_v big)$$
$$\deg(v) \sim P_k(\cdot) \neq P_j(\cdot) \Rightarrow E_{G_k} big[Lbig] \neq E_{G_j} big[Lbig] \quad (4.12)$$

La première équation ressemble à FedAvg, mais l'espérance porte sur une structure : la non-indépendance est portée par G_k lui-même, non par les échantillons à l'intérieur.

> **Ne pas confondre non-indépendance des échantillons et non-indépendance des graphes**
>
> Dans un cas tabulaire, la non-indépendance des échantillons se corrigé par repondération ou par correction de la variance. Dans un cas graphique, deux échantillons **sont** les nœuds d'un même graphe : les corriger séparément est impossible, car ils se contaminent par propagation. La seule marge de manœuvre est donc au niveau du graphe entier : pondérer des graphes, ou transférer des connaissances graphiques entre domaines.

Trois topologies de partition

La question préliminaire est comment les nœuds sont répartis entre clients. Ce choix détermine tout le reste de la conception.

1. **Partition horizontale.** Les **entités**, les nœuds, sont réparties entre clients, et chaque client détient toutes les caractéristiques de ses entités. C'est la partition classique, analogue à la fédération tabulaire : chaque bloc est un sous-graphe induit. Le problème immédiat est que les arêtes traversant la frontière sont perdues pour tout le monde, ce qui revient à supprimer des informations du modèle.
2. **Partition verticale.** Les **caractéristiques** sont réparties entre clients, chaque client détenant tous les nœuds mais une sous-partie des attributs. Aucun nœud n'est perdu, toutes les arêtes restent cohérentes, mais aucun client ne peut calculer seul une passe de propagation. Cette partition conduit naturellement à l'apprentissage scindé, où l'activation intermédiaire — pas les données — circule entre les clients.
3. **Partition par sous-graphe naturel.** Chaque client détient un graphe issu du monde réel et non d'un découpage artificiel : une molécule, un parc de machines, un réseau social régional. C'est la partition la plus réaliste et la plus difficile, car les sous-graphes sont liés par des ponts : les interactions inter-molécules, les liens de transport.

Partition	Ce qui est réparti	Perte d'information	Modèle de référence	Défi principal
Horizontale	nœuds, entités	arêtes inter-clients	FedGraphNets	sous-graphes non isomorphes
Verticale	caractéristiques, colonnes	aucune, avec apprentissage scindé	SplitFed, FedNIA	parallélisme de la propagation
Sous-graphe naturel	tout, par structure du domaine	ponts entre sous-graphes	FedSage, sub-Fed	distribution des degrés très différente
Mixte	nœuds et une colonne	partielle	FedGTA	alignement des espaces d'incorporation

Tableau 1 – Topologies de partition sur graphes

La partition verticale : apprentissage scindé et ses limites

La partition verticale mérite un traitement séparé, car c'est la seule qui ne perd aucune information du graphe. Elle pose un problème d'alignement qui n'a pas d'équivalent dans l'apprentissage fédéré tabulaire.

$$h_1^{(k)} = f_{1,k}(X_k), \; h_2^{(k)} = f_{2,k}(X_k), \; h_V^{(k)} = \prod_{l=1}^{L} f_{l,k} big(h_l^{(k)} big)$$
$$L = \sum_k L_k big(f_{k,V} \circ \cdots \circ f_{k,1}, Ybig) \text{ avec } Y \text{ inchangé du côté du client} \quad (4.13)$$

$\prod$ désigne ici la composition de messages : chaque client ne reçoit que l'état d'activation, jamais les caractéristiques de ses voisins.

Trois défis se présentent alors, dans cet ordre d'importance pratique.

1. **Alignement des entités.** Chaque client doit être sur la même liste de nœuds, dans le même ordre, à chaque itération. La moindre divergence d'indexation produit un mélange incohérent, silencieux, et impossible à diagnostiquer à partir des pertes. Ce problème est généralement résolu par un identifiant global de nœud partagé à tous les clients, ce qui constitue déjà une fuite de métadonnées : la présence d'un nœud dans plusieurs corpus est révélée.
2. **Alignement des espaces.** Les activations sont échangées entre espaces de caractéristiques différents. Il faut soit apprendre une projection, soit normaliser. Sans cela, l'amplitude des activations représente un signal parasite qui biaise l'entraînement.
3. **Parallélisme.** La propagation sur un graphe est séquentielle dans le sens où les couches de messages se succèdent. L'apprentissage scindé impose donc des allers-retours synchronisés : le coût de communication est proportionnel au **nombre de couches de messages**, pas au nombre de paramètres. Un réseau de convolutions sur graphe profond est donc naturellement plus cher en communication qu'un réseau de convolution de même taille.

> **Attention**
> L'apprentissage scindé sur graphes communique les activations intermédiaires, qui sont des fonctions inversibles approchées de l'information d'entrée. Sur des graphes dont les caractéristiques sont peu variées — par exemple des coordonnées atomiques, ou un identifiant de nœud — l'inversion est directe. La confidentialité différentielle doit donc être appliquée aux activations transmises, pas seulement aux poids.

Familles de méthodes

- **FedGraphNets.** Pose le problème proprement : le serveur agrège les poids du réseau, mais les représentations de nœuds sont transmises à chaque tour car elles portent l'information structurante. Chaque client renvoie un vecteur agrégé de représentations plutôt que la totalité du sous-graphe.
- **FedSage.** Utilise des **sous-graphes augmentés** : avant l'envoi, un client extrait des sous-graphes de taille fixe autour des nœuds qui l'intéressent, ce qui rend les représentations comparables d'un client à l'autre. L'amélioration ne porte pas sur l'agrégation mais sur la représentabilité.
- **sub-Fed et FedSage avec similarité.** sub-Fed utilise une agglomération à deux niveaux : d'abord une moyenne des représentations par type de nœud, puis une moyenne globale. La variante avec similarité ajoute une similarité cosinus entre représentations pour pondérer l'agrégation : deux clients qui ont appris des choses proches comptent davantage.
- **FedGTA.** Généralise le principe précédent en transplantant un transformateur de représentations : le serveur apprend une fonction d'alignement qui projette les représentations de chaque client dans un espace commun avant de les mélanger. Cela traité le cas où les caractéristiques sont dans des espaces différents.
- **FedGNN.** Reconduit le problème du domaine creux : dans un graphe d'interactions rares, la plupart des nœuds sont sans lien. L'échantillonnage des sous-graphes devient une variable du protocole et non un détail d'implémentation.
- **FedPUB.** Traité le cas des graphes non alignés entre clients, y compris lorsque les jeux d'entités sont disjoints — deux hôpitaux qui n'ont aucun patient commun. Il introduit un transfert fondé sur la structure plutôt que sur l'identité, ce qui permet un apprentissage transféré entre institutions sans partage d'entité.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import GCNConv


class LocalGCN(nn.Module):
"""Reseau de convolution sur graphe partage entre tous les clients.

Chaque client conserve son propre sous-graphe ; seul l'etat des poids
circule pour l'entrainement, et l'ensemble des representations pour
l'agregation structuree.
"""

def __init__(self, in_dim: int, hidden: int = 64, out_dim: int = 7):
super().__init__()
self.conv1 = GCNConv(in_dim, hidden)
self.conv2 = GCNConv(hidden, out_dim)

def forward(self, x, edge_index):
h = F.relu(self.conv1(x, edge_index))
h = F.dropout(h, p=0.5, training=self.training)
return self.conv2(h, edge_index)

def embed(self, x, edge_index):
"""Representation de noeud pre-agregeation : c'est elle qui structure."""
return F.relu(self.conv1(x, edge_index))


def graph_pool(h, batch, size: int) -> torch.Tensor:
"""Moyenne par sous-graphe : un vecteur par client, pas par noeud.

C'est la reduction qui rend la communication proportionnelle au nombre de
clients et non au nombre total de noeuds du consortium.
"""
out = torch.zeros(size, h.size(1), device=h.device)
cnt = torch.zeros(size, 1, device=h.device)
out.index_add_(0, batch, h)
cnt.index_add_(0, batch, torch.ones(h.size(0), 1, device=h.device))
return out / cnt.clamp(min=1.0)


def fedgl_round(client_models, client_pools, client_sizes):
"""Aggregation en deux niveaux : moyenne des poids, puis moyenne des pools.

Les poids et les representations ne se melangent pas de la meme facon : les
premiers decrivent l'architecture, les seconds la structure locale.
"""
total = float(sum(client_sizes))
mean = client_models[0]
with torch.no_grad():
for key, param in mean.state_dict().items():
acc = torch.zeros_like(param)
for model, n_k in zip(client_models, client_sizes):
acc += model.state_dict()[key].to(param.dtype) * (n_k / total)
param.copy_(acc)
# Moyenne des representations, avec ponderation par volume de noeuds
pooled = torch.zeros_like(client_pools[0])
for p, n_k in zip(client_pools, client_sizes):
pooled += p * (n_k / total)
return mean, pooled


```

Listing 1 — Agrégation de sous-graphes en PyTorch Geometric : poids, représentations et similarité cosinus

```python
def cosine_similarity_weights(pools):
    """Poids d'agregation : proximite angulaire des representations.

    Deux clients qui ont appris des representations proches sont plus fiables
    mutuellement que deux clients orthogonaux ; les mises a jour du second sont
    potentiellement trompeuses pour le premier.
    """
    stacked = torch.stack([p.flatten() for p in pools])
    normed = F.normalize(stacked, dim=1)
    sim = normed @ normed.t()
    n = sim.size(0)
    eye = torch.eye(n)
    # Similarite moyenne avec les autres, ignoree pour le client lui-meme
    avg = (sim.sum(dim=1) - sim.diag()) / max(n - 1, 1)
    weights = avg / avg.sum()
    return weights, sim + eye * 1e6
```

Méthode	Partition	Objet échangé	Post-traitement serveur	Apport principal	Limite
FedGraphNets	horizontale	poids et représentation agrégée	aucun	cadre de référence	représentations non alignées
FedSage	horizontale	poids et sous-graphes	moyenne	représentabilité	coût de construction des sous-graphes
sub-Fed	horizontale	poids et pools par type	moyenne hiérarchique	hétérogénéité de types	taxonomie préalable
FedSage avec similarité	horizontale	poids et similarité	pondération cosinus	clients non échangeables	coût de calcul de la similarité
FedGTA	mixte	poids et transformateur	apprentissage d'alignement	espaces hétérogènes	un transformateur de plus à entraîner
Apprentissage scindé	verticale	activations	assemblage d'incorporations	aucune perte d'arête	parallélisme synchrone
FedPUB	sous-graphes disjoints	poids et structure	transfert structurel	entités non communes	transfert plus long à établir

Tableau 2 – Comparaison des approches d'apprentissage fédéré sur graphes

Exercice 4.6.19 (difficulte : Avancé)

Trois laboratoires détiennent chacun un réseau moléculaire : le laboratoire A travaillé sur des peptides de moins de 30 atomes, B sur des protéines de 2 000 à 8 000 atomes, C sur des agrégats de plus de 50 000 atomes. Les trois jeux d'entités sont disjoints : aucun atome n'est présent dans deux laboratoires.

1. Justifier que la partition horizontale est impossible et préciser quelle contrainte elle viole.
2. Comparer les deux options réalistes : apprentissage scindé avec partition verticale des caractéristiques, et transfert structurel de type FedPUB.
3. Pour la répartition des rôles entre les trois laboratoires, indiquer qui doit entraîner quoi, et justifier par le coût de communication.
4. Proposer un protocole de validation qui détecte une contamination involontaire entre les jeux de données.
5. Dire quel indicateur suivre pour savoir si le transfert structurel produit un gain réel ou seulement un gain apparent.

Indice : La contrainte à violer n'est pas la confidentialité des caractéristiques, mais l'existence même d'un graphe commun.

Correction

- **1. Partition horizontale impossible.** Elle suppose un ensemble d'entités commun, les mêmes atomes, que l'on répartirait entre clients. Or les trois jeux sont disjoints : il n'y a rien à répartir. Les trois graphes sont de surcroît de structures très différentes — un peptide de 30 atomes est presque un chemin, une protéine de 8 000 atomes un graphe dense, un agrégat de 50 000 atomes un réseau modulaire. Fédérer des sous-graphes de tailles si différentes sans transfert explicite revient à mélanger des lois de distribution de degrés sans les avoir identifiées.
- **2. Apprentissage scindé vertical.** On peut répartir les caractéristiques par laboratoire — par exemple A conserve le type d'atome, B les coordonnées, C les charges partielles — tout en gardant le graphe chez un seul détenteur. Avantage : le parallélisme est réel, les activations sont petites par rapport aux coordonnées. Inconvénients majeurs : le détenteur du graphe doit détenir le graphe complet, ce qui constitue un point de concentration ; et l'alignement des espaces est délicat puisque les caractéristiques sont par nature dans des espaces différents. Le coût est en nombre de couches de messages.
- **3. Transfert structurel.** Chaque laboratoire entraîne son propre réseau sur son propre graphe, et le serveur ne mélange pas les représentations : il transfère des **connaissances de structure**, règles de composition et schémas de sous-structure. Avantage : aucune entité ne circule, ce qui est réglementairement acceptable. Inconvénient : le transfert met plusieurs tours à établir, et il faut éviter de transférer la structure **spécifique** du laboratoire A vers le laboratoire C, sans quoi on dégrade le modèle local.
- **4. Répartition des rôles.** Laboratoire A, petits peptides : encodeur de caractéristiques atomiques, entraîné sur place et mis à disposition des deux autres. Laboratoire C, agrégats : entraînement principal du modèle de composition, car c'est là que se trouve le signal le plus riche. Laboratoire B, protéines : co-entraînement, avec validation sur ses propres structures. Justification : le coût de communication est proportionnel au nombre de couches de messages traversant le réseau ; limiter le nombre de clients traversant la même couche divise le coût.
- **5. Détection de contamination.** Une empreinte de chaque atome, identifiant composé du type et de la position, comparée aux ensembles des autres laboratoires avant tout apprentissage. Un taux de correspondance nul doit être vérifié **avant** la première connexion réseau, pas après. En complément, on peut vérifier que la distribution de distance inter-atomique des trois jeux est disjointe : un chevauchement signalerait une molécule commune découverte sous deux noms.
- **6. Indicateur de gain réel.** Le gain apparent se mesure sur la performance moyenne du consortium. Le gain réel se mesure sur la performance **de chaque laboratoire sans le transfert**, après retrait du modèle transféré. Si un laboratoire voit sa performance chuter de plus de 2 % en présence du transfert, le transfert est négatif pour lui : il faut alors basculer vers un apprentissage personnalisé, avec un adaptateur par laboratoire, plutôt qu'un modèle unique. C'est le même arbitrage que dans le chapitre sur les grands modèles, mais portant sur la structure.

A retenir

- La non-indépendance des graphes est structurelle : elle porte sur la topologie, pas sur les échantillons, et se corrigé mal par repondération.
- Trois topologies de partition, trois familles de problèmes : perte d'arêtes, parallélisme synchrone, absence d'entités communes.
- La plupart des gains de la littérature sur les graphes viennent du post-traitement serveur, pas de l'algorithme d'agrégation.
- Toute méthode doit se confronter à la question du repli : que reste-t-il si le transfert est désactivé ?

Du prototype de recherche au système federatif industrialisable

Partie 5

FedX : le framework de référence

Les parties I a IV ont établi ce qu'est l'apprentissage fédéré, comment il s'optimise, comment il se protège et dans quels domaines il porte de la valeur. Il reste une question d'ingénierie, et c'est la plus Ternue : comment transformer un algorithme fédéré en un système que l'on peut déployer chez cent clients hétérogènes, instrumenter, facturer, auditer et faire evoluer pendant trois ans ? Cette partie présente FedX, le framework conçu a cette fin par le Département Recherche & Développement de Pixel Software Design.

Objectifs de la partie

- Justifier les sept principes de conception de FedX par des contraintes de terrain.
- Lire l'architecture en six couches et situer chaque extension possible.
- Écrire une stratégie fédéré conforme au contrat d'API de FedX.
- Choisir une stratégie d'optimisation, d'aggregation et de confidentialité.
- Instrumenter, déployer et facturer une fédération FedX en production.
- Reproduire un résultat federated a l'identique, chiffres a l'appui.

Plan de la partie

1. Vision, principes et architecture de FedX
2. Modèle de programmation : client, serveur et stratégies
3. Backbones, stratégies avancées et confidentialité intégrée
4. Déploiement, observabilite et tableau de bord web
5. Benchmark reproductible et etude de performance

Chapitre 5.1

Vision, principes et architecture

FedX n'est pas le premier framework fédéré, et ne pretend pas l'être. C'est le premier framework dont chaque couche a été decidée en réponse a un incident reel, et dont chaque décision est reversifiable.

Un framework d'apprentissage fédéré est un pièce d'infrastructure. Il ne se juge pas sur une courbe de convergence mais sur trois questions : combien de temps faut-il pour y brancher un modèle existant, combien de temps faut-il pour comprendre pourquoi un tour a dégrade, et que se passe-t-il le jour ou un client sort du périmètre ou tenté d'empoisonner ses voisins. FedX est ne de la réponse a ces trois questions, formulees dans cet ordre.

Genese : quatre blocages de terrain

Le Département R&D a consigne quatre familles de difficultés, rencontrees sur des déploiements menes entre 2022 et 2026 pour des clients hospitaliers, bancaires et industriels. Aucune n'est propre a l'apprentissage fédéré ; toutes deviennent critiques des que la fédération est multi-acteurs et durable.

1. **Le changement de domaine coûte cher.** Un modèle entraîné dans un centre hospitalier n'est pas réutilisable dans un autre : le vocabulaire des variables, la resolution des signaux et la prevalence des classes changent. Reentrainer coûte des semaines, refaire l'intégration des pipelines en coûte autant.
2. **Les backbones ne se comportent pas de la même façon.** Un perceptron, un réseau convolutif, un transformeur et un réseau de graphes n'ont ni la même tolérance a l'hétérogénéité, ni le même coût de communication, ni la même façon de dégrader. Un framework qui suppose un perceptron ne peut pas pretendre couvrir les trois autres.
3. **La confidentialité n'est pas standardisee.** Chaque projet reimplementait son propre mélange de bruit, son propre calcul de budget et son propre formalisme de rapport. Aucun de ces reimplementations n'était vérifiable par un tiers, ce qui rendait l'argument de conformité inopposable.
4. **L'observabilite est absente.** En l'absence de trace, toute anomalie était reconstruite a posteriori par deduction, a partir de journaux texte non structures et de deux ou trois captures d'écran.

Blocage	Symptôme observe	Coût du contournement	Réponse FedX
Changement de domaine	réintégration complète du pipeline	3 a 6 semaines par cibleconfiguration declarative versionnee, `--from` pour rejouer une base	
Hétérogénéité des backbones	une branche de code par architecture	divergence du code, tests perdus	interface commune `FedModel` + fabrique de modèles
Confidentialité non standardisee	budget epsilon non vérifiable	audit impossible	pipeline DP-SGD integre + rapport de privacy exportable et signe
Observabilite absente	diagnostic par deduction a posteriori	jours depertes par incident	métriques, journal structure, console web, journal d'audit

Tableau 1 – Les quatre blocages de terrain et leur traduction dans FedX

Les quatre réponses sont des mécanismes structurels, pas des conventions documentées : chacune est vérifiable dans le code ou dans un artefact produit par le système.

Le coût du changement de domaine se modélise simplement. Soit c le coût d'intégration d'une nouvelle cible, T le nombre de cibles etentions par an. Une fédération non réutilisable coûte cT par an ; une fédération paramétrique coûte c + c' T ou c' est le coût marginal d'une configuration.

$$\text{Coût}_{\text{refonte}} = c\,T$$
$$\text{Coût}_{\text{param}} = c + c'\,T,\ 0 < c' \ll c \qquad (5.1.1)$$

FedX vise la seconde expression : le surcoût marginal d'une nouvelle cible doit tendre vers le coût d'un fichier de configuration, pas vers celui d'une nouvelle branche de code.

Piege courant

Le contournement le plus tentant consiste a écrire une classe derivee par domaine métier. Cela double la surface de maintenance et rend chaque correction de sécurité obsolète sur la moitié des cibles. FedX impose l'inverse : une seule hiérarchie de classes, établie par un contrat, ou les specificites de domaine tiennent dans un adaptateur de données.

Les sept principes de conception

Un principe de conception est une règle de décision applicable même lorsque personne ne se souvient de l'avoir écrite. FedX en compte sept ; ils sont formules ici avec la contrainte de terrain qui les a produits, car un principe sans contrainte d'origine degenera en slogan.

Principe	Énoncé opérationnel	Contrainte d'origine
Noyau minimal	aucune dépendance obligatoire hors numérique	les clients embarqués n'ont pas de GPU ni de connexion permanente
Stratégies composables	une stratégie ne fait qu'un tour, pas un cycle	FedAvg, FedProx et SCAFFOLD ne doivent pas être exclusifs
Contrat d'API unique	tout objet qui traversé le réseau verifie son schéma	le serveur ne doit jamais appeler une méthode absente
Configuration declarative	aucun comportement implicite hors du YAML	rejouer un essai sans le code qui l'a produit
Reproductibilité	graine, version et artefacts par défaut	un résultat non rejouable ne peut pas être defendu
Observabilite de bout en bout	tout tour produit un rapport versionne	l'incident doit être datable et attribuable
Sécurité par défaut	confidentialité active si non désactivée	l'oubli du développeur est le cas le plus probable

Tableau 2 – Les sept principes de FedX et leur justification

La dernière ligne est la plus contraignante : elle impose un coût par défaut, donc une performance en degression sur les cas ou la confidentialité est sans objet.

Attention

Le principe « sécurité par défaut » a un prix. L'agrégation sécurisée multiplie le coût de coordination par deux a dix selon la taille du modèle, et le DP-SGD réduit l'exactitude de 3 a 6 points. FedX assume ce coût : il est configuré par défaut mais doit être **désactivé explicitement**, avec une tracé dans le journal d'audit.

Astuce

Conséquence pratique du principe « stratégies composables » : FedAvg et SCAFFOLD ne sont pas deux stratégies, ce sont deux objets. Un tour SCAFFOLD avec correction nulle se réduit exactement a un tour FedAvg, ce qui rend la comparaison expérimentale honnête et le test de regression trivial.

Architecture en six couches

FedX est organisé en couches strictement empilees. Une couche ne connaît que celle du dessous ; aucune n'importe la supérieure. Cette règle, parfois appelee règle de dépendance descendante, permet de remplacer le transport ou le stockage sans toucher a la stratégie, et de tester une stratégie en simulation, sans aucun réseau.

- `fedx.core` — les primitives : `ModelState`, `ClientUpdate`, `RoundContext`, `RoundReport`, la hiérarchie d'erreurs et le schéma de validation. Aucun import de réseau, aucun import de framework de deep learning.
- `fedx.models` — la fabrique de backbones et l'interface commune `FedModel`, qui adapté un module PyTorch comme un module JAX ou TensorFlow au même contrat.
- `fedx.strategy` — l'algorithme : interface `FedStrategy` et stratégies fournies, de FedAvg a FedBN.
- `fedx.runtime` — l'orchestration : boucle de tour, ordonnanceur de participation, machine a états, rappels de cycle de vie, ordonnancement des tours et des arrets.
- `fedx.transport` **et** `fedx.storage` — les effets : canaux de transport (en processus, gRPC, HTTP/2) et backends de stockage (mémoire, SQLite, Parquet, objet compatible S3).
- `fedx.observability` — la transparence : métriques, journal structure, journal d'audit scellé, rapport de privacy et tableau de bord web.

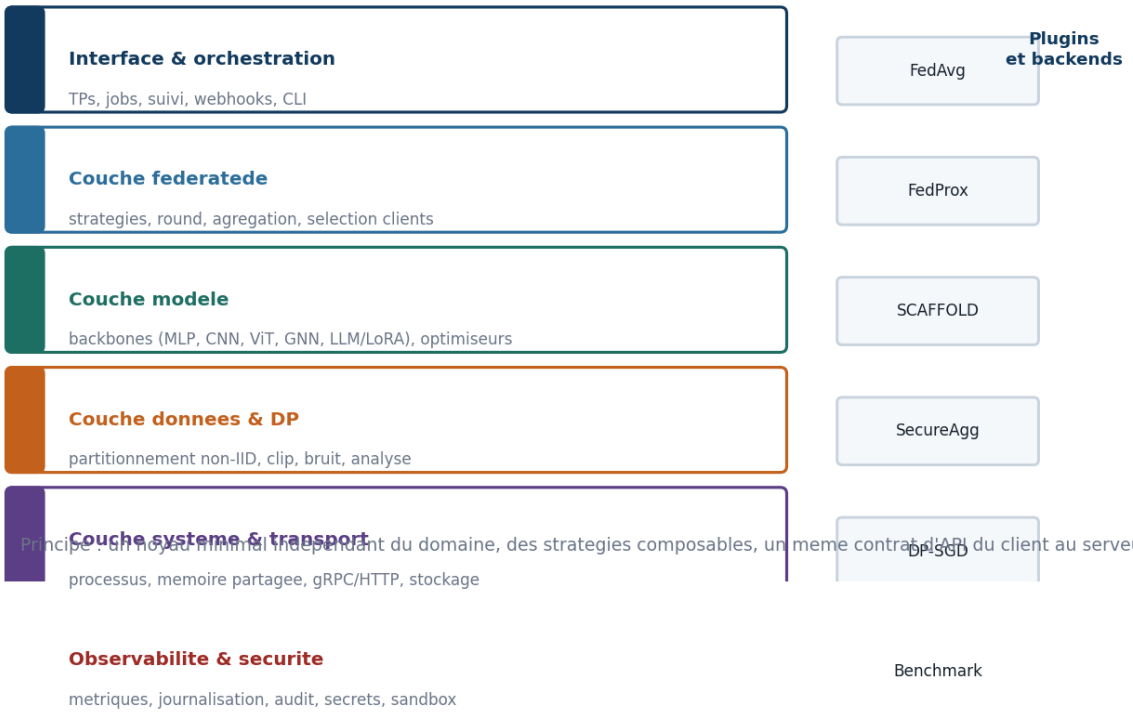


Figure 1 – Architecture en six couches de FedX et points d'extension privilégiés

Les cinq points d'extension (stratégie, client, serveur, transport, stockage) se situent toujours a la frontiere de deux couches : c'est ce qui les rend interchangeables.

Couche	Responsabilite	Dépend de	Point d'extension
core	types, schémas, erreurs	numérique seul	aucun (fermé)
models	backbones, fabrique `FedModel`	core	fabrique de modèles
strategy	algorithme d'optimisation	core	`FedStrategy`
runtime	boucle de tour, états, rappels	core, strategy	`FedServer`
transport	canaux client-serveur	core	`TransportBackend`
storage	checkpoints, métriques, audit	core	`StorageBackend`
observability	métriques, journal, console	core	exporteurs métriques

Tableau 3 – Les six couches : responsabilites, dépendances et extensions

La ligne « core » n'admet aucune extension : c'est la seule garantie de stabilité de l'API public.

> **Definition – Point d'extension**
> Emplacement précis du code ou un tiers peut brancher son propre comportement sans modifier le noyau. FedX en expose cinq, tous declares par une classe abstraite du noyau, ce qui rend la substitution vérifiable par un test de contrat.

FedX face aux frameworks existants

Le choix d'un framework fédéré est un choix d'engagement, pas un choix de syntaxe. Le tableau ci-dessous compare FedX aux quatre bibliothèques les plus repandues sur une grille qui privilégié les critères decouverts ci-dessus : expressivite du modèle de programmation, confidentialité intégrée, observabilite, multi-tenance et reproductibilité.

Critère	FedX (Pixel R&D)	Flower	NVFlare	OpenFL	TensorFlow Federated
Positionnement	framework industriel de référence, usage interne et clientsbibliothèque fédéré généraliste	plateforme d'entreprise NVIDIA	plateforme d'intelligence distribuée	outil de recherche et de simulation	
Licence	propriétaire, support contractualise	Apache-2.0	Apache-2.0	Apache-2.0	Apache-2.0
Modèle de programmation	contrat explicite a cinq points d'extension	stratégie et application client	API objet et convertisseurs	tâche, stratégie, environnement	fonctions de calcul et simulateurs
Stratégies de FL fournies	neuf natives, de FedAvg a FedBN	FedAvg, FedProx, SCAFFOLD, FedOpt	FedAvg, FedProx, FedOpt, hetero	FedAvg et extensions de plateau	aucune, orientee simulation
Confidentialité intégrée	agrégation sécurisée, DP-SGD, rapport d'epsilon	agrégation sécurisée tierce	agrégation sécurisée native	mesure de confiance et TEE	DP natif mais limite a la simulation
Observabilite	métriques, console web, journal d'audit scellé	journaux applicatifs	journaux et tableau de bord optionnel	métriques et rôles	TensorBoard
Multi-tenance et quotas	natif, jetons, quotas par fédération	a construire	a construire	partiel	absent
Déploiement	conteneurs, Kubernetes, rollout canary	conteneurs, sans orchestrateur natif	Kubernetes ou processus unique	Docker Swarm, TEE	local, TPU
Reproductibilité	graine, version, artefacts signes par défaut	partielle	partielle	attestation TEE	moteur déterministe
Support de la confidentialité différentielle	compte rendu exportable	implémentation externe	intégration partielle	non	oui, théorique
Coût d'intégration initiale	faible si la cible respecté le contrat	faible	moyen	moyen a eleve	eleve

Tableau 4 – Comparaison détaillée des frameworks fédérés de référence

Cette comparaison porte sur les capacités annoncees et vérifiées en version au cours de l'écriture ; elle ne classe pas les équipes ni les projets, dont les objectifs legitimatement différent.

Le tableau ne dit pas que les autres sont mauvais : NVFlare est plus pertinent quand l'infrastructure est déjà un cluster NVIDIA, OpenFL quand la confiance repose sur des enclaves, TensorFlow Federated quand la question posée est théorique. Il dit qu'un seul de ces cinq attributs est indispensable a un déploiement mutualisé : le **rapport de confidentialité vérifiable**. Sans lui, le fédération n'est pas auditable, donc pas deployable chez un reglementateur.

> **Cadre juridique et ethique**
> La confidentialité fédéré n'exonere pas de la règle du traitement minimal : la qualité du modèle obtenu peut, a elle seule, révéler des informations sur les données d'entraînement, en particulier quand le nombre de clients est faible ou la classe rare. Un rapport d'epsilon sans description du jeu de données et du nombre de participants n'est pas un élément de preuve exploitable. FedX impose donc que le rapport de privacy accompagne le jeu de référence, le nombre de clients et la stratégie.

Cycle de vie d'une fédération

Une fédération FedX n'est pas un processus unique mais une suite d'états explicites. Un état declare invalide rend impossible l'execution d'un tour : aucune connexion tardive, aucun rattrapage implicite, aucune reprise silencieuse après incident. La machine d'états est le premier outil d'observabilite du système.

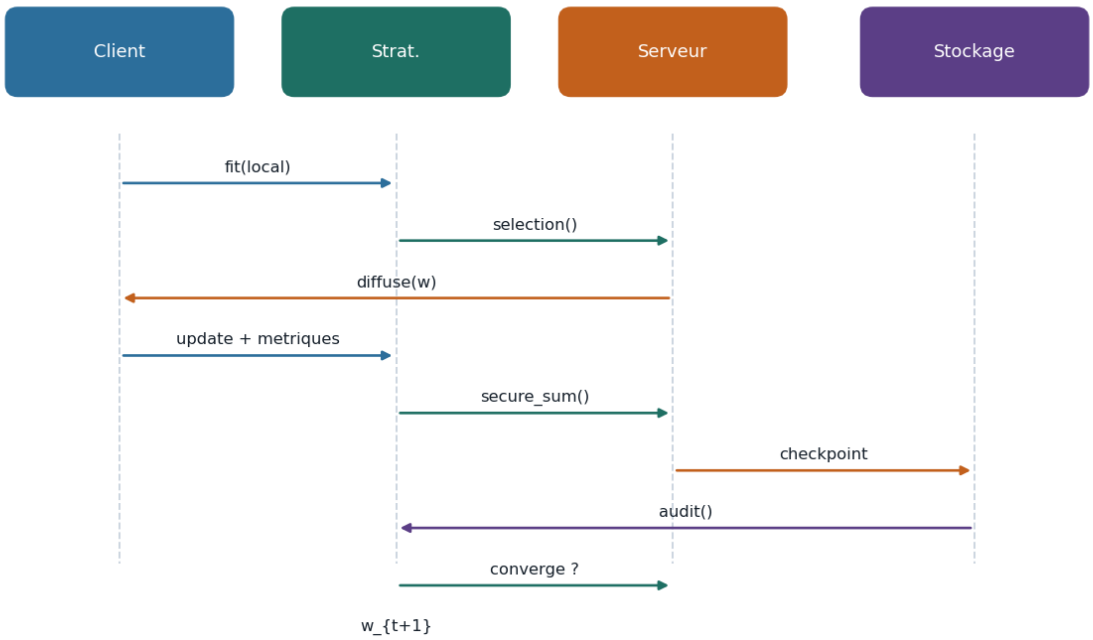


Figure 2 – Diagramme de séquence du cycle de vie d'un tour fédéré FedX

Les paramètres sont transmis par référence (identifiant de version) et non en ligne : sur un backbone de 350 Mo, l'écart de bande passante entre les deux modes depasse un ordre de grandeur.

Algorithme 6 – Cycle de vie d'un tour FedX

```
BOUCLE sur le tour t = 1..T tant que arret_non_demande : ctx <- RoundContext(tour = t, budget = budget_restant)
  stratégie.configuré(ctx) # contrat : un tour, un rapport
  invites <- runtime.sélection(ctx) # qui est convoque, et pourquoi
  runtime.diffuse(Invite(ctx, invites))
  POUR chaque client i des invites, en parallèle :
  params <- storage.charge(ReferencedModel(t))
  update <- client.fit(params, ctx.hyperparamètres)
  storage.depose(Envelope(i, update, métriques, rapport_privacy))
  updates <- runtime.reassemble(t) # complétude et signatures
  etat_global <- stratégie.aggregate(updates, ModelState(t))
  rapport <- runtime.clôture(t, etat_global, updates)
  storage.depose(Checkpoint(etat_global), Rapport(rapport))
  console.publié(rapport) # métriques et alertes
RETOURNER le dernier état global et l'historique des rapports
```

Entrées : configuration FedX validée, catalogue de clients, état global courant • **Sorties :** état global agrège, série de rapports de tour, journal d'audit • **Complexité :** un tour : $O(\sum_i n_i\, d)$ en calcul, $O(\kappa\, d)$ octets en transport

Les six opérations en gras ci-dessus sont les seuls points ou une extension peut s'inserer sans modifier le noyau : configuré, sélection, fit, aggregate, reassemble, clôture.

Message	Émetteur	Charge utile	Garantie
`ClientHello`	client	version, capacités, profil matériel	le serveur refuse une version incompatible avant tout envoi de poids
`Invite`	serveur	identifiant de tour, référence de version	le client verifie que la version correspond a son état local
`TrainRequest`	client	référence de version, capacité residuelle	aucun poids ne transite si la version est inconnue
`UpdateEnvelope`	client	deltas, métriques, rapport de privacy	chiffrement de bout en bout, signature du client
`AggregateAck`	serveur	empreinte de l'agrégation	détecté la troncature d'un envoi ou la perte silencieuse
`RoundSummary`	serveur	métriques agrégées, seuils, alertes	alimente la console web et le journal d'audit

Tableau 5 – Messages du cycle de vie et garanties associées

Chaque message a un accuse de reception de implicite ; l'absence d'accuse declenche une re-tentative bornée, puis l'abandon du client pour ce tour.

La loi qui regit ce protocole est simple et sans exception : **le serveur ne détient jamais de donnée, et le client n'envoie jamais plus que ce que la stratégie demande**. Toute violation de cette loi est detectable a la lecture du journal d'audit, qui enregistre pour chaque tour le nombre d'octets par sens, le nombre de clients et le vecteur de participation.

Exercice 5.1.20 (difficulte : Moyen)

Un client FedX observe par FedX signalé un coût de bande passante deux fois supérieur a la moyenne de sa fédération, a nombre de classes identique et pour un modèle de même taille. La table de référence indiqué 4,2 Mo par tour dans un sens.

- Enumeratez les causes possibles, classees par probabilité décroissante.
- Pour chacune, indiquez la métrique ou le journal qui permet de la confirmer.
- Quelle valeur du fichier `fedx.yaml` you'd ajuster en premier, et pourquoi ?

Indice : Le coût par tour est produit par trois facteurs seulement : la taille de l'objet échange, le nombre d'objet et le coefficient de compression. Le dernier est le seul que l'on puisse augmenter sans toucher au modèle.

Correction

- **Compression inactive ou ineffective** — un `type` non reconnu dans `compression` laisse la précision a `float32`. Confirmation : le champ `bytes_out` du rapport de tour, compare a `4,2 * 4 = 16,8 Mo` attendu en `float32` ; le journal de stratégie trace la précision effective.
- **Envoi des poids au lieu des deltas** — une stratégie mal branchee peut sérialiser l'état complet plutôt que l'écart. Confirmation : le champ `mode_echange` du message `UpdateEnvelope` (`delta` attendu).
- **Métriques de diagnostic trop volumineuses** — un histogramme de gradients par couche, ou 200 messages de supervision, s'ajoutent au total. Confirmation : la répartition `octets_par_categorie` du journal d'audit, qui sépare `poids`, `métriques` et `supervision`.
- **Manque de quantification en appliquant la compression a la reception et non a l'émission** — le gain apparent disparait côte client. Confirmation : comparaison du `bytes_out` annonce dans le rapport et du `bytes_in` mesure par le serveur.
- **Replay ou retransmission** — deux tours de la même version envoyes, par exemple après un `AggregateAck` perdu. Confirmation : présence de deux `TrainRequest` de même empreinte dans le journal.
- Le premier reglage a tenter est `compression.recalage: par_bloc` avec `précision: int8` : il divisé le volume par quatre sans perte significative sur les deltas, dont la distribution est beaucoup plus concentree que celle des poids. Il ne coûte aucun round supplementaire et n'affecte que la couche transport, donc aucun autre client.

A retenir

- FedX repond a quatre blocages de terrain observes entre 2022 et 2026, et non a une abstraction théorique.
- Les sept principes sont opérationnels : chacun se vérifié dans le code ou dans un artefact.
- L'architecture en six couches rend tout remplacement local et toute simulation sans réseau.
- Le seul attribut non negociable pour un déploiement mutualisé est le rapport de confidentialité vérifiable.
- Le cycle de vie est une machine d'états explicite : pas de reprise silencieuse, pas de connexion tardive.

Chapitre 5.2

Modèle de programmation : client, serveur et stratégies

Un framework fédéré n'est jugé que par son contrat. FedX expose cinq points d'extension, tous declares par une classe abstraite du noyau, et un seul contrat d'API entre ce qui s'execute côte client et ce qui s'execute côte serveur.

Le modèle de programmation de FedX repose sur une séparation stricte entre le **quoi** — l'algorithme, qui appartient a la stratégie — et le **comment** — le transport, la persistance et l'instrumentation, qui appartiennent aux infrastructures. Un client ne connaît pas la stratégie du serveur ; un serveur ne connaît pas le framework de deep learning du client. Le contrat qui relié les deux est volontairement etroit.

Les cinq points d'extension

- **La stratégie** (`FedStrategy`) — l'algorithme d'optimisation distribuée. Point d'extension le plus utilise : il concentre l'essentiel de la valeur intellectuelle d'une fédération.
- **Le client** (`ClientRuntime`) — le code qui possede les données. FedX fournit une implémentation de référence ; un client peut la remplacer entierement, typiquement par du code embarqué sur un equipement contraint.
- **Le serveur** (`FedServer`) — l'orchestrateur. Il expose le cycle de vie, la persistance et l'API de contrôle ; un serveur personnalisé ajoute un ordonnancement spécifique, jamais une logique d'optimisation.
- **Le backend de transport** (`TransportBackend`) — le canal. En processus pour la simulation, gRPC pour le LAN, HTTP/2 ou MQTT pour l WAN et les objets connectés.
- **Le backend de stockage** (`StorageBackend`) — la persistance. Mémoire pour un essai, SQLite pour un poste, Parquet ou stockage objet pour une fédération de production.

Point	Classe abstraite	Spécification implementee	Fréquence d'usage
Stratégie	`FedStrategy`	neuf stratégies natives	très élevée, quotidienne
Client	`ClientRuntime`	`FedXClient` générique	moyenne, par cible
Serveur	`FedServer`	serveur a tour unique	faible, ponctuelle
Transport	`TransportBackend`	in-process, gRPC, HTTP/2	faible, au choix
Stockage	`StorageBackend`	mémoire, SQLite, objet	moyenne, par taille

Tableau 1 – Les cinq points d'extension de FedX

La fréquence indique combien de projets FedX reellement livres ont modifié ce point. Elle sert d'indicateur de stabilité du contrat : un point peu utilise est un point peu eprouve, donc plus dangereux.

Le contrat d'API

Le contrat est écrit une fois, en Python, et vérifié par introspection a l'instantiation. Toute classe qui ne le respecté pas levé une `ContractError` avant le premier tour, jamais au milieu d'un tour. Cette exigence de vérification statique est la différence entre un framework qui échoue tot et un framework qui échoue a trois heures du matin.

$$\Delta_k = w_k - w_t$$
$$w_{t+1} = w_t + \eta_s \sum_{k \in S_t} p_k \Delta_k$$
$$p_k = (n_k)/(\sum_{j \in S_t} n_j) \quad (5.2.1)$$

FedX transporte des deltas, jamais des poids : l'information de la version de départ est déjà connue des deux parties, la transmettre serait un gaspillage.

Méthode	Moment	Entrée	Sortie	Effet de bord
`configuré`	ouverture de tour	`RoundContext`	`None`valide la configuration, ouvre l'état	
`select`	avant invitation	catalogue de clients	liste d'identifiants	écrit une trace de sélection
`distribute`	après sélection	contexte, invites	`None`	construit les messages d'invitation
`fit`	côte client	`ClientUpdate`	`ClientUpdate`	aucun (pur)
`evaluate`	après collecte	mises a jour	dict de métriques	aucun (pur)
`aggregate`	fin de tour	mises a jour, état de base	`ModelState`	alloué le nouvel état global
`on_round_start`	debut de tour	`RoundReport`	`None`	publié des alertes
`on_round_end`	fin de tour	`RoundReport`	`bool`	demande l'arrêt
`on_anomaly`	détection	événement	`None`	met un client en quarantaine

Tableau 2 – Contrat d'API de `FedStrategy` : méthodes, entrées, sorties, effets

`fit`, `evaluate` et `aggregate` sont pures par construction : même entrée, même sortie, aucun accès disque. Cette pureté est ce qui rend les stratégies testables en une fraction de seconde.

L'intégrité du protocole repose sur une empreinte unique calculée a la construction de chaque tour. Elle lié la version du modèle, les paramètres et la configuration : deux tours ne peuvent pas mélanger des pièces d'expériences différentes.

$$h_t = \mathrm{SHA256big}(\mathrm{version} \mid \theta_t \mid \mathrm{config}_t \mid \mathrm{grillebig}) \quad (5.2.2)$$

Un client qui refuse un message dont l'empreinte ne correspond pas a son état local ne fait rien de silencieux : il remonte une alerte de contrat.

> **Attention**
> Le contrat impose que `fit` et `evaluate` soient pures. Une stratégie qui écrit un fichier ou modifié un compteur global pendant `aggregate` rend les résultats irreproductibles et casse la comparabilité entre fédérations. FedX détecté ce cas par inspection du bytecode lors du test de contrat, et refuse d'enregistrer la stratégie.

La stratégie : interface, cycle de vie, sélection

Une stratégie FedX ne fait **un tour**, jamais un cycle. Elle reçoit un contexte, produit un état et un rapport, puis rend la main. Cette restriction, contre-intuitive, est ce qui permet de composer FedAvg, FedProx et SCAFFOLD sans qu' aucune ne connaisse l'existence des autres : FedProx n'est pas « FedAvg plus une pénalité », c'est FedAvg dont la fonction de perte locale a été reecrite.

$$\text{score}(c) = \alpha\ \text{recence}(c) + \beta\ \text{qualité}(c) - \gamma\ \text{drift}(c)$$

$$S_t = \text{Top}_{k_t}\text{big}(c : \text{disponible}(c, t), \text{scorebig}),\ k_t = \leq\text{ftlceil}\ \rho / N\ \text{rceil} \qquad (5.2.3)$$

La sélection FedX est multi-critères et explicite. Le critère `drift` permet d'écarter un client dont les mises a jour s'ecartent systematiquement de la moyenne, sans le declarer malveillant : c'est un indicateur de distribution, pas un jugement.

```python
# fedx/strategy/fedavg.py — implementation de reference FedAvg pour FedX.
# (c) Pixel Software Design — Departement Recherche & Developpement
"""Squelette d'une strategie FedAvg conforme au contrat d'API de FedX."""
from __future__ import annotations

from typing import Dict, List, Sequence

import torch

from fedx.core import FedStrategy, ModelState, RoundContext, RoundReport
from fedx.core.errors import ContractError
from fedx.core.update import ClientUpdate

QUORUM = 4 # en dessous, le tour n'est pas agrege et doit etre rejoue


class FedAvgStrategy(FedStrategy):
"""Moyenne ponderee des deltas locaux : la strategie FedAvg de FedX."""
nom = "fedavg"
taux_serveur = 0.5 # pas du serveur, applique une seule fois par tour
participation = 0.1 # fraction du catalogue convoquee a chaque tour

def configure(self, ctx: RoundContext) -> None:
# Point de validation unique : tout le reste de la tour en decoule.
if not 0.0 < self.participation <= 1.0:
raise ContractError("participation doit appartenir a ]0, 1]")
self.ctx, self.historique = ctx, []

def select(self, catalogue: Sequence, tour: int) -> List[str]:
# D'abord les clients disponibles les plus recents, puis complement.
actifs = sorted((c for c in catalogue if c.disponible(tour)),
key=lambda c: c.qualite_mesuree, reverse=True)
k = max(QUORUM, int(round(self.participation * len(catalogue))))
return [c.identifiant for c in actifs[:k]]

def fit(self, update: ClientUpdate) -> ClientUpdate:
# Rappel cote client : FedAvg n'intervient pas sur l'entrainement local.
return update

def evaluate(self, updates: Sequence[ClientUpdate]) -> Dict[str, float]:
# Metriques ponderees comme les poids, sans quoi la perte affichee ment.
total = sum(u.taille_effective for u in updates)
return {"perte_validation": float(sum(u.metriques["perte_finale"]
* u.taille_effective for u in updates) / max(total, 1))}

def aggregate(self, updates, base: ModelState) -> ModelState:
# Poids p_k = n_k / somme(n) : jamais uniformes hors lots equilibres.
if len(updates) < QUORUM:
raise ContractError("quorum insuffisant pour agreger ce tour")
total = sum(u.taille_effective for u in updates)
cumule = {n: torch.zeros_like(t) for n, t in base.tensors().items()}
for u in updates:
for n, delta in u.deltas.items():
cumule[n] += (u.taille_effective / total) * delta.to(cumule[n].device)
return base.appliquer_deltas(cumule, self.taux_serveur)

def on_round_end(self, rapport: RoundReport) -> bool:
# Rappel de fin de tour ; True signifie : armer l'arret de la federation.
self.historique.append(rapport)
return rapport.tour >= self.ctx.tours_max
```

Listing 1 — Strategie FedAvg FedX : configure, select, fit, evaluate, aggregate, rappels

Astuce

L'ordre des méthodes n'est pas libre. Le noyau appelle `configuré`, puis `select`, puis `distribute`, puis `fit` sur chaque client, puis `evaluate`, puis `aggregate`, puis `on_round_start` et `on_round_end`. Une stratégie qui modifié `self` dans `aggregate` s'expose a une course entre tours parallèles ; FedX serialize les tours par construction, il faut le savoir.

Le client FedX

Le client de référence se tient a cinquante lignes utiles. Il charge ses données par un adaptateur opaque, entraîné localement, puis renvoie un enveloppe contenant des deltas, des métriques et un rapport de privacy. Il n'a connaissance ni du serveur, ni de la stratégie, ni du nombre de clients.

```python
# fedx/client.py — client FedX : chargement, entrainement local, retour d'update.
# (c) Pixel Software Design — Departement Recherche & Developpement
"""Client federated generique, independant de la strategie du serveur."""
from __future__ import annotations

import time
from typing import Any, Callable, Optional

import torch
from torch.utils.data import DataLoader

from fedx.core import ModelState
from fedx.core.update import ClientUpdate
from fedx.runtime.protocol import ClientRuntime


class FedXClient(ClientRuntime):
"""Entraine un modele local et publie ses deltas, jamais ses donnees."""
def __init__(self, identifiant: str, modele: torch.nn.Module,
chargeur: Callable[[], Any], gpu: Optional[str] = None) -> None:
self.identifiant = identifiant
self.modele = modele
self.chargeur = chargeur
# Le materiel est choisi au demarrage, jamais suppose au premier envoi.
self.appareil = torch.device(
gpu or ("cuda" if torch.cuda.is_available() else "cpu"))
self.modele.to(self.appareil)
self.qualite_mesuree = 0.0

def charger_donnees(self, taille_lot: int = 64) -> DataLoader:
"""Construit le lot local ; les donnees ne quittent jamais l'hote."""
jeu = self.chargeur() # type: ignore[operator]
return DataLoader(jeu, batch_size=taille_lot, shuffle=True, num_workers=2)

def fit(self, etat: ModelState, tours_locaux: int = 2,
taux: float = 0.01) -> ClientUpdate:
"""Un tour local complet, du chargement a l'envoi des deltas."""
etat.vers_module(self.modele, self.appareil)
optimiseur = torch.optim.SGD(self.modele.parameters(), lr=taux, momentum=0.9)
chargeur = self.charger_donnees()
avant = {n: p.detach().clone() for n, p in self.modele.named_parameters()}
debut, pertes = time.perf_counter(), []
for _ in range(tours_locaux):
for x, y in chargeur:
x, y = x.to(self.appareil), y.to(self.appareil)
optimiseur.zero_grad(set_to_none=True)
perte = torch.nn.functional.cross_entropy(self.modele(x), y)
perte.backward()
# Bornage : un lot aberrant ne doit pas ruiner l'ensemble du tour.
torch.nn.utils.clip_grad_norm_(self.modele.parameters(), 10.0)
optimiseur.step()
pertes.append(float(perte.detach()))
duree = time.perf_counter() - debut
# Seuls les deltas sont publies ; la qualite sert au critere de selection.
deltas = {n: p.detach() - avant[n] for n, p in avant.items()}
self.qualite_mesuree = -sum(pertes) / max(len(pertes), 1)
return ClientUpdate(
client=self.identifiant, tour=etat.tour, deltas=deltas,
taille_effective=len(chargeur.dataset),
metriques={"perte_finale": pertes[-1], "duree_s": duree})
```

Listing 2 — Client FedX de reference : chargeur opaque, deltas et metriques

Definition – Deltamodele

Objet `ClientUpdate` contenant, pour un client et un tour donnes, la différence $\Delta_k = w_k - w_t$ entre le modèle local et le modèle de départ, la taille effective de données vues, un dictionnaire de métriques et, si la confidentialité est active, le rapport de privacy correspondant. C'est la seule unité qui traversé le réseau.

Serveur, transport et stockage

Le serveur est un automate a états, pas un groupe de fonctions. Il expose `serve()` (bloquant), `stop()` (interruption propre en fin de tour), `snapshot()` et `restore()` (reprise après incident) et `status()` (état serialisable pour la console). Le transport expose `send`, `recv`, `subscribe` et un jeton de fin de flux. Le stockage expose `écrire`, `lire`, `lister` et `sceller`, ce dernier scellant une empreinte dans le journal d'audit.

Backend	Méthodes obligatoires	Méthodes optionnelles	État interne
`TransportBackend`	`open`, `close`, `send`, `recvsubscribe`, `capacité`, `tracés`	connexions, files, temporisateurs	
`StorageBackend`	`écrire`, `lire`, `lister`	`sceller`, `promouvoir`, `expirer`	index, versions, empreintes
`FedServer`	`serve`, `stop`	`snapshot`, `restore`, `status`	état du tour, files, alertes

Tableau 3 – Contrat d'API des trois backends

Les méthodes optionnelles sont declarees par capacité introspection : le noyau teste `hasattr` et dégrade proprement en cas d'absence.

Écrire une extension FedX en six étapes

- **Declarer l'intention.** Écrire une phrase decrivant ce que l'extension change. Si elle ne peut pas être écrite en une phrase, elle est trop large.
- **Choisir l'etendue.** Client, serveur, stratégie, transport ou stockage. Le choix se déduit de la couche qui doit changer, pas de la commodite.
- **Herediter de la classe abstraite**, sans redéfinir ses constantes : `nom`, `version_schema` et `capacités` sont lus par le noyau.
- **Respecter la pureté.** Aucune lecture de configuration, aucune écriture de fichier, aucun accès réseau dans `fit`, `evaluate` et `aggregate`.
- **Écrire le test de contrat.** Il est fourni par le noyau : une classe qui l'échoue n'est pas enregistrable.
- **Documenter le coût.** Bande passante, mémoire, temps de calcul, effet sur la convergence : ces quatre nombres figure dans la fiche de l'extension et dans la sortie du banc d'essai.

Piege courant

Le piège le plus coûteux est la stratégie qui herite d'une stratégie existante et surcharge `aggregate` sans rappeler la méthode mere. Le tour s'execute, les métriques sont plausibles, mais la correction SCAFFOLD disparait silencieusement et la fédération diverge au bout de quarante tours. Le test de contrat de FedX détecté l'absence d'appel a la classe mere, mais il ne peut pas détecter l'intention : c'est pourquoi les extensions exigent une fiche d'intention signee.

A retenir

- Le contrat d'API est etroit, vérifié statiquement, et impose la pureté de `fit`, `evaluate` et `aggregate`.
- Une stratégie fait un tour, jamais un cycle : c'est ce qui rend les stratégies composables.
- Seuls des deltas, des métriques et un rapport de privacy traversent le réseau.
- Les cinq points d'extension sont testés par le même jeu de contrats, ce qui rend leur substitution sure.

Chapitre 5.3

Backbones, stratégies avancées et confidentialité intégrée

Un framework de référence doit être indifferent au modèle, bavard sur l'algorithme et intraitable sur la vie privee. Ces trois exigences se combinent mal : chaque famille de modèle dégrade differemment, chaque stratégie coûte differemment, et la confidentialité se paie toujours en qualité.

FedX traité ces trois sujets comme un seul. Le même objet `FedModel` decrit a la fois ce que le client sait faire et ce qu'il faut transmettre ; la même configuration de fédération declare la stratégie d'optimisation, le mode d'agrégation et le régime de confidentialité. Cette colocation n'est pas cosmetique : elle permet de vérifier qu'une combinaison est realisable avant de lancer le moindre tour.

Une interface commune pour six familles de modèles

Le contrat `FedModel` tient en cinq méthodes : `construire()` renvoie un module et son jeu de paramètres communicables, `entrées()` decrit la forme attendue, `loss()` calcule la perte locale, `métriques()` produit les indicateurs de santé et `sousensembles()` decoupe l'état en parties gelees et applicables. Cette dernière méthode est celle qui rend l'adaptation paramétrique possible.

> **Definition – FedModel**
> Interface abstraite implementant l'adaptation d'une architecture au contrat fédéré. Elle expose quatre capacités : construire le modèle, decrire ses entrées, calculer la perte locale, et surtout **delimiter la partie communiquable** du paramètre par la méthode `sousensembles()`. Tout backend de deep learning compatible peut l'implémenter, a condition de pouvoir serialiser ses tenseurs de poids en un dictionnaire nomme.

$$\Theta = \Theta_{\square 0 \square} \cup \Theta_{\square 1 \square}$$
$$\Theta_{\square 0 \square} \cap \Theta_{\square 1 \square} = \varnothing$$
$$|\Theta_{\square 0 \square}| = r\,|\Theta|\ ,\ r \approx 3 \times 10^{-3} \tag{5.3.1}$$

C'est la définition operatoire de la federabilite d'un modèle : le rapport de la taille de la partie communiquable a la taille totale. En dessous de 0,1 %, la fédération devient viable sur un WAN ; au-dessus de 10 %, elle impose un réseau local ou une quantification agressive.

Backbone	Adaptateur `FedModel`	Paramètres échanges	Comportement face a l'hétérogénéité	Cas d'usage
Perceptron multicouche	`mlp_dense`	100 %	robuste, converge vite	capteurs, données tabulaires
Réseau convolutif (ResNet)	`resnet_cifar`	100 %	derive du classifieur sous décalage de batches	imagerie médicale, inspection
Transformeur de vision (ViT)	`vit_patch16`	100 %	sensible a la position des patchs	imagerie multi-échelle, satellitaire
Réseau de graphes (GNN)	`gnn_2couches`	100 %	derive forte : sous-graphes non i.i.d.	molécules, réseaux, fraude
Grand modèle de langage + LoRA	`llm_lora_r16`	0,3 %	derive modérée, adaptateurs fusionnables	corpus métier, assistance documentaire
Autoencodeur (VAE, VAE + quant)	`ae_convolutif`	100 %	sensible a la définition de l'espace latent	compression, détection d'anomalie

Tableau 1 – Les six backbones fournis et leurs adaptations FedX

Le rapport « paramètres échanges » est la seule donnée a retenir pour dimensionner la bande passante ; le reste decrit un comportement, pas une performance.

Astuce

Pour les grands modèles de langage, FedX n'envoie jamais les poids. Il envoie les matrices de decomposition de rang r des adaptateurs, puis les fusionné côte serveur par simple moyenne pondérée. Un modèle de sept milliards de paramètres descend ainsi a douze megabytes par tour, ce qui le rend deployable sur une connexion de bureau.

Piege courant

Le piège classique consiste a traiter un transformeur comme un réseau convolutif, c'est-a-dire a tout échanger et a tout moyennner. Les projections de requête et de clé convergent vers des directions différentes selon la distribution locale ; leur moyenne est proche de zéro et le modèle se dégrade sans que la courbe de perte locale ne signalé rien. Le remède est structurel :adapter l'architecture avant de fédérer.

Neuf stratégies avancées fournies

FedX fournit les stratégies qu'un déploiement reel rencontre, pas celles qu'un article recense. Chacune modifié une ou plusieurs des quatre briques du tour : la perte locale, la règle d'agrégation, l'optimiseur du serveur ou le régime de confidentialité.

$$F_k^{\square 0 \square}(w) = F_k(w) + (\mu)/(2)\,|w - w_t|^2$$
$$c_{t+1} = c_t - (1)/(\eta)\,(1)/(m)\sum_{k \in S_t} \Delta_k$$
$$y_k \leftarrow w_t - \eta\, c_k \tag{5.3.2}$$

FedProx regularise vers le modèle de départ du tour ; SCAFFOLD estime la derive des clients et la compense explicitement. SCAFFOLD coûte deux fois plus cher en communication, ce qui le rend souvent contre-productif au dessous de 1 % de participation.

$$\Delta_t = (m_t)/(\sqrt{(v_t)} + \tau)$$
$$w_{t+1} = w_t + \eta_s\, \Delta_t$$
$$\tau_k = (1)/(p\, \tau_{\square 0 \square})\, \tau_k^{\square 1 \square}$$
$$w_{t+1} = \sum_k p_k\, \tau_k\, w_k \tag{5.3.3}$$

FedOpt applique un moment et une variance adaptatifs côte serveur, ce qui stabilisé FedAvg a taux d'apprentissage serveur constant. FedNova normalise le nombre d'itérations locales : indispensable des que les clients n'ont pas la même capacité de calcul.

Stratégie	Convergence	Communication	Confidentialité	Coût client	Statut
FedAvg	référence, la plus faible	1x	aucune en propre	faible	native, défaut
FedProx	meilleure sous derive	1x	aucune en propre	nul	native
SCAFFOLD	nouvelle, corrige la derive	2x	aucune en propre	faible	native
FedOpt / FedAdam	acceleree, tolère η_s faible	1x	aucune en propre	nul	native
FedNova	robuste aux charges inegales	1x	aucune en propre	faible	native
Krum	resiste a f poisons	1x	aucune en propre	nul	native
Moyenne elaguee	resiste a f poisons, plus stable	1x	aucune en propre	faible	native
SecAgg	identique a FedAvg	1x a 2x	le serveur ne voit rien	faible	native
DP-SGD intégré	dégrade de 3 à 6 points	(epsilon, delta)-DP par tour	moyen	native	
FedBN	parité de batches conservee	0,3x	aucune en propre	faible	native

Tableau 2 – Comparaison des stratégies fournies nativement par FedX

« Coût client » exprimé l'overhead en temps de calcul local, hors communication. FedBN est particulier : les couches de normalisation ne sont jamais agrégées, d'ou une communication réduite de 70 % sur un ResNet.

Attention

FedProx est souvent mal réglée. Avec μ trop faible, la régularisation est invisible et la stratégie se réduit a FedAvg en consommant deux fois plus de mémoire ; avec μ trop fort, le modèle colle au point de départ et n'avance plus. Sur FedX, μ est expose dans la configuration et son effet est tracé dans le rapport de tour : la courbe de $|w_{t+1} - w_t|$ permet de le calibrer en trois tours.

Astuce

Combiner Krum et moyenne elaguee n'a pas de sens : les deux occupent la même place dans le pipeline. En revanche, combiner SecAgg avec DP-SGD est la combinaison de référence pour un usage reglemente : le serveur ne voit ni les mises a jour individuelles ni leur somme exacte, et chaque client publié un rapport de privacy que l'auditeur peut vérifier.

Le pipeline de confidentialité intégrée

La confidentialité différentielle de FedX est un pipeline de quatre étapes obligatoires, executees sur chaque client, dont le résultat est un objet exportable. Aucune étape n'est facultative : un client qui declare une stratégie privee sans produire de rapport de privacy fait rejeter son tour par le noyau, qui verifie la présence de l'objet avant aggregation.

$$\bar{g}_i = \mathrm{clip}_C\mathrm{big}(\nabla \ell_i(w)\mathrm{big}) = (\nabla \ell_i(w))/(\mathrm{maxbig}(1, |\nabla \ell_i(w)|_2 / C\mathrm{big}))$$
$$\tilde{g} = (1)/(B)\mathrm{Big}(\sum_{i=1}^{B} \bar{g}_i + N\mathrm{big}(0, \sigma^2 C^2 \mathcal{J}_d\mathrm{big})\ \mathrm{Big}),\ \sigma \geq (C)/(B) \qquad (5.3.4)$$

Le bornage borne la contribution d'un échantillon ; le bruit gaussien rend la loi de la moyenne indiscernable d'une loi qui ne dependrait que de la borne. Le théorème de composition donne ensuite le coût total.

$$\square_t = (\mathrm{Accountantbig}(\sigma^2, q, T_t\mathrm{big}))/(\delta),\ q = (B)/(N_k)$$
$$\sum_{t=1}^{T} \square_t \leq \square_{\square 0 \square} \Rightarrow \square_t^{\star} = (\square_{\square 1 \square})/(T) \qquad (5.3.5)$$

L'accountant de composition fournit le coût reel du mécanisme, très inférieur a la borne grossiere qui somme les budgets. FedX utilisé par défaut un accountant de moments de renyi, et refuse le calcul par borne simple en production.

1. **Bornage par lot** — chaque gradient d'échantillon est ramene a une norme égale a C avant toute

somme. C'est l'étape qui rend l'analyse validé ; sans elle, aucun bruit n'a de garantie.

2. **Bruit** — un bruit gaussien indépendant est ajouté a la somme du lot, avec un multiplicateur pilote par le budget de tour.
3. **Comptabilite** — l'accountant enregistre chaque étape de lot et publié le coût ☐ courant, qui conditionné l'autorisation du tour suivant.
4. **Budget par tour** — l'orchestrateur refuse d'envoyer une invite si le budget restant est inférieur au coût reserve. La fédération s'arrête, elle ne diverge pas silencieusement.
5. **Export du rapport** — chaque client joint a son enveloppe un rapport horodate, signe et scellé dans le journal d'audit.

```python
# fedx/privacy/dpsgd_client.py — confidentialite differentielle integree a FedX.
# (c) Pixel Software Design — Departement Recherche & Developpement
"""Enveloppe DP-SGD + accountant autour d'un client FedX existant."""
from dataclasses import dataclass
from typing import Any, Dict

import torch
from opacus import GradSampleModule
from opacus.accountants import RDPAccountant
from opacus.optimizers import DPOptimizer
from torch.utils.data import DataLoader


@dataclass(frozen=True)
class PrivacyConfig:
"""Regime de confidentialite, identique pour tous les clients d'une tour."""
bruit_par_echantillon: float = 1.1 # multiplicateur sigma de la norme
lot_equivalent: int = 256 # taille de lot pour le taux d'echantillonnage
delta: float = 1.0e-6 # probabilite de violation
epsilon_par_tour: float = 0.15 # budget alloue au tour courant
budget_total: float = 3.0 # budget de toute la federation


class FedXPrivacyClient:
"""Compose un client FedX, un optimiseur DP-SGD et un accountant de composition."""
def __init__(self, client: Any, config: PrivacyConfig, graine: int = 0) -> None:
self.client, self.config = client, config
self.accountant = RDPAccountant()
# Graine fixee : le bruit doit etre reproductible comme tout le reste.
torch.manual_seed(graine)
# GradSampleModule intercepte la retropropagation pour accumuler un gradient
# par echantillon : c'est ce qui rend le bornage par echantillon valide.
self.module_dp = GradSampleModule(client.modele.to(client.appareil))
self.optimiseur = DPOptimizer(
optimizer=torch.optim.SGD(self.module_dp.parameters(), lr=0.01),
noise_multiplier=config.bruit_par_echantillon,
max_grad_norm=1.0,
expected_batch_size=config.lot_equivalent,
)

def fit(self, etat: Any, tours_locaux: int = 1) -> Dict[str, Any]:
"""Un tour local sous DP : le bruit ne quitte jamais le client."""
etat.vers_module(self.client.modele, self.client.appareil)
chargeur = self.client.charger_donnees(self.config.lot_equivalent)
for _ in range(tours_locaux):
for x, y in DataLoader(chargeur.dataset, batch_size=64, shuffle=True):
x, y = x.to(self.client.appareil), y.to(self.client.appareil)
self.optimiseur.zero_grad(set_to_none=True)
perte = torch.nn.functional.cross_entropy(self.module_dp(x), y)
perte.backward()
self.optimiseur.step() # bornage puis bruit, en une passe
# Une etape d'accountant par etape de lot, jamais par echantillon.
self.accountant.step(noise_multiplier=self.config.bruit_par_echantillon,
sample_rate=64 / len(chargeur.dataset), steps=1)
return self.client.construire_update(etat, self.module_dp)

def rapport(self, tour: int) -> Dict[str, Any]:
# Rapport de privacy joint a l'enveloppe, puis scelle par le serveur.
eps = float(self.accountant.get_epsilon(delta=self.config.delta))
return {"tour": tour, "epsilon": eps, "delta": self.config.delta
"bruit": self.config.bruit_par_echantillon,
"epuisement": round(eps / self.config.budget_total, 4)}
```

Listing 1 — Client DP-SGD FedX : bornage, bruit, comptabilite et rapport exportable

La composition de l'ensemble est un produit de deux garanties distinctes, qu'il ne faut jamais confondre dans un rapport d'audit : SecAgg protège le serveur contre les clients, DP-SGD protège chaque client contre

le serveur et contre les autres clients. Une fédération qui n'utilise que l'une des deux n'a pas la propriété attendue.

Combination	Le serveur voit les deltas	Le client est protège	Perte de qualité
aucune	oui, en clair	non	référence
SecAgg	non, seulement la somme	partiellement	0 a 0,5 point
DP-SGD	oui, bruités	oui, au sens DP	3 a 6 points
SecAgg + DP-SGD	non	oui, au sens DP	3 a 6 points
SecAgg + DP-SGD + FedBN	non	oui, au sens DP	2 a 4 points

Tableau 3 – Combinaisons de confidentialité et propriétés obtenues

FedBN améliore la privacy en reduisant la variance inter-tours : moins de tours significatifs, donc une epsilon plus faible pour la même qualité.

Cadre juridique et ethique

Le rapport de privacy FedX est conçu comme une pièce d'audit, non comme un indicateur de tableau de bord. Il doit indiquer la version du mécanisme, le budget alloué a chaque tour, le budget total, la valeur de δ, le nombre de clients ayant participe, la Stratégie d'agrégation et l'empreinte du modèle publié. En conservation de ces éléments, la fédération peut démontrer a un tiers le lien entre la qualité obtenue et la garantie accordee. Leur absence rend toute affirmation de confidentialité non demontrable, quelle que soit la qualité du code.

Astuce

Rejouer une fédération réglée avec un budget différent ne donne pas un résultat comparable : la privacy n'est pas une propriété du modèle mais de la trajectoire. FedX inscrit donc la configuration de privacy dans l'empreinte du tour : deux rapports portant des empreintes différentes ne peuvent pas être superposes sur une même courbe.

Choisir une stratégie : une procédure en quatre questions

1. **Quelle est la nature de l'hétérogénéité ?** Label skew pur : FedBN ou moyenne elaguee. dérive de caractéristique : FedProx. Quantités d'échantillons très inegales : FedNova.
2. **Quelle est la part de clients malveillants toleree ?** Aucune : SecAgg. Jusqu'a f sur n : Krum, ou moyenne elaguee si l'attaquant est passif.
3. **Quel budget de confidentialité ?** Nul : aucune contrainte. ☐ borne : allocation uniforme, ou concentration sur les tours decisifs, qui est plus efficace a qualité égale.
4. **Quelle contrainte de communication est la plus dure ?** Octets : FedBN, quantification, adaptateurs. Tours : SCAFFOLD ou FedAdam. Bande passante crustee : SecAgg impose un aller-retour supplementaire par tour.

Valider une combinaison avant production

Lancer un essai de **vingt tours** sur un sous-ensemble de clients, avec la combinaison exacte qui sera deployee : même stratégie, même compression, même régime de privacy, mêmes graines.

- Rejeter si la perte de validation diverge de la référence de plus de quatre points a tour egal.
- Rejeter si le 95e centile de latence de tour depasse deux fois la mediane.
- Rejeter si l'epsilon par tour mesure depasse de 20 % le budget alloué.
- Consigner les trois rejections dans le dossier d'experimentation : elles font partie du résultat.

A retenir

- La federabilite d'un modèle se mesure par le rapport entre paramètres transmis et taille totale.
- Les neuf stratégies fournies couvrent les combinaisons reellement rencontrees, pas la littérature exhaustive.
- La privacy est un pipeline non negociable : bornage, bruit, comptabilite, budget, rapport.
- SecAgg et DP-SGD protargent des adversaires différents : les confondre fausse le rapport d'audit.
- Une combinaison se valide sur vingt tours exacts avant tout déploiement, et les échecs se consignent.

Chapitre 5.4

Déploiement, observabilite et tableau de bord web

Une fédération qui n'est pas observable est une boite noire que l'on arrose pendant des mois. Le déploiement FedX et son instrumentation sont conçus ensemble : chaque événement du moteur est un événement a exposer.

Le déploiement d'une fédération FedX repond a trois questions distinctes, que l'on confond volontiers. Comment execute-t-on le serveur et les agents de connexion, de façon resistante aux pannes et aux mises a jour ? Comment execute-t-on plusieurs fédérations sans qu'elles se genent, sur la même infrastructure ? Comment sait-on, a chaque instant, ce que fait la fédération ? Les réponses sont, dans l'ordre : conteneurs et orchestrateur, isolation par locataire, métriques et console.

Conteneurisation et orchestration

Le serveur FedX, l'agent de collecte et le frontal de la console sont distribués en images distinctes, sans état local partagé. L'état vit dans le backend de stockage, ce qui autorisé a remplacer un pod killed par l'ordonnanceur sans perdre un tour : le nouveau pod recharge la dernière empreinte scellée et repart du checkpoint. Les clients, eux, restent hors du cluster : ce sont des machines hospitalieres, desemoniciels ou des mobiles.

1. **Image minimale.** Image d'execution sans compilateur, sans outils de développement et sans dépendances optionnelles. Une surface d'attaque réduite est une exigence de conformité, pas une préférence esthetique.
2. **Sonde de disponibilité distincte de la sonde de liveness.** Une fédération peut répondre « je suis vivant mais j'epsile mon budget » : les deux états doivent être distincts, sinon l'ordonnanceur tue un serveur parfaitement fonctionnel.
3. **Quotas par fédération.** CPU, mémoire, bande passante et nombre de clients sont alloues par locataire. Une fédération bruyante ne peut pas affamer les autres ; l'exces provoque une dégradation de service, pas une panne globale.
4. **Déroulement progressif.** Le remplacement d'une version du serveur suit une stratégie de canary : 5 % du trafic, puis 25 %, puis 100 %, avec retour automatique si un indicateur de santé franchit son seuil.

Composant	Image	Repliques	Sonde de liveness	État
Serveur de fédération	`fedx-server:2.4`	2 a 4, actives	état du tour	externe
Agent de collecte	`fedx-agent:2.4`	1 par fédération	file d'attente	externe
API de contrôle	`fedx-api:2.4`	2, actives	requête `/healthz`	externe
Console web	`fedx-console:2.4`	2, actives	fichier statique	externe
Worker de rapport	`fedx-bench:2.4`	0 a 1, terminé	file d'artefacts	externe

Tableau 1 – Composants deployes et leur contrat de disponibilité

L'état est toujours externe : aucun de ces composants ne compte sur son disque local pour survivre a un redemarrage.

Multi-tenance et stratégie de déroulement

Le multi-tenance repose sur trois separations strictes : les **jetons** d'authentification, qui portent une liste de fédérations autorisées et une date d'expiration ; les **quotas**, qui bornent la consommation par fédération ; et les **journaux d'audit**, qui sont append-only et scellés. Un client d'une fédération ne peut pas appeler l'API d'une autre : le contrôle est applique avant tout accès aux données, y compris dans les journaux.

Le déroulement canari est lui-même un protocole fédéré : le serveur maintenant deux versions en parallèle ne peut pas mélanger leurs mises a jour. Chaque groupe de canari possede son propre état global, et la consolidation n'a lieu qu'après évaluation comparative des deux trajectoires.

$$\text{seuil}(v) = 1 \le ft[\, m(v) > s_{v} \,]$$
$$m(v) = \text{maxBig}(\, \rho_{\square 0 \square},\ \text{drift}(v) - \rho_{\square 2 \square} \text{Big})$$
$$\text{retour automatique} \ \square\ \exists\ v' : \sum_{t} m_{v'}(t) > 0 \text{ sur 3 tours} \qquad (5.4.1)$$

Un canari n'est pas jugé sur la performance absolue mais sur l'écart a la version de référence : une version qui fait mieux en moyenne mais échoue plus souvent doit être rejetee.

Astuce

Le déroulement canari est nettement moins cher qu'il n'y paraît : la version de référence continue de tourner pendant l'essai, et les clients n'ont rien a installer puisque le protocole de tour est inchangé. Sur une fédération de vingt mille clients, le coût marginal d'un canari a 5 % est inférieur a celui de trois jours de maintenance traditionallye.

Piege courant

Le piège d'orchestration le plus courant consiste a faire transiter les checkpoints par le stockage objet du cloud public, ce qui enleve au système federated la propriété pour laquelle il a été choisi : la maîtrise de l'infrastructure. FedX impose par défaut un stockage interne a l'orchestrateur et signalé toute destination externe dans le rapport d'audit.

Observabilite : que mesurer, quand s'alerter

Une métrique de federated n'est utile que si elle est **actionnable** : elle doit designer une décision que l'opérateur peut prendre dans la minute. FedX expose quarante et une métriques brutes, mais seulement dix-huit sont promues au rang de signaux d'alerte, avec un seuil et une conséquence écrite dans le runbook.

$$\cos_\perp(c) = (\square\ \Delta_{c},\ \Delta_{\square 1 \square}\ \square) / (|\Delta_{c}|_{2}\ |\Delta_{\square 2 \square}|_{2})$$
$$\text{drift}(c) = (|\Delta_{c}|_{2}) / (|w_{t}|_{2} + \varepsilon) \qquad (5.4.2)$$

Le cosinus mesure la direction, la derive mesure l'intensité. Un cosinus négatif signifie que le client tire le modèle a l'oppose de la moyenne : c'est le signal le plus precoce d'un empoisonnement.

Métrique	Définition	Seuil de surveillance	Seuil critique	Conséquence
Latence de tour	durée mediane du tour	120 s	300 sréduction du taux de participation	
Latence client p95	95e centile d'entraînement local	90 s	240 s	clients lents écartés de la sélection
Perte de validation	moyenne pondérée du tour	plateau	divergence	arrêt au plateau, relance avec correction
Cosinus des deltas	similarite directionnelle	-0,1	-0,3	mise en quarantaine du client
Deri client	norme relative du delta	0,8	1,5	répartition des tours, ou investigation de distribution
Taux de participation	clients ayant repondu sur invites	70 %	40 %	élargissement de la sélection ou alerte capacité
Octets par tour	volume moyen par client et par sens	8 Mo	20 Mo	bascule vers la quantification int8
Epsilon consomme	budget de privacy epuise	80 %	100 %	arrêt de la fédération, rapport a l'auditeur
Écart-type des poids	dispersion du modèle agrège	croissance	diverge	suspicion d'empoisonnement
Taux de perte des messages	enveloppes non recues	1 %	5 %	bascule de transport, analyse réseau

Tableau 2 – Métriques FedX, seuils d'alerte et conséquences

Un seuil sans conséquence écrite n'est pas un seuil : c'est une statistique affichee. La colonne de droite fait partie du référentiel operatoire de la fédération.

La journalisation est structurelle et hiérarchique. Chaque événement porte un identifiant de tour, un identifiant de client, un horodatage, un niveau, un code d'événement et un champ de données type. Les métriques numériques ne sont **pas** journalisées : elles sont écrites en colonnes dans un format colonnaire, ce qui les rend interrogeables sans analyse de texte.

> **Definition – Traçabilité**
> Capacité a relier un modèle publié a l'ensemble des conditions qui ont produit ce modèle : version du code, configuration, graines, versions des données, décisions de sélection, et rapports de privacy. FedX materialise cette capacité par une chaîne d'empreintes : chaque tour produit une empreinte, chaque agrégation en produit une autre, et le modèle publié porte l'empreinte de la dernière agrégation.

La console web

La console est le point d'a Landing d'un opérateur qui ne veut pas ouvrir une ligne de commande. Elle est volontairement sobre : six pages, une par question opérationnelle, et aucune page qui ne reponde pas a une décision.

Page	Question a laquelle elle repond	Contenu principal	Action
Vue d'ensemble	la fédération va-t-elle bien ?état, tour courant, indicateurs clés, alertes actives	acquitter une alerte	
Clients	qui participe, et pourquoi ?	catalogue, disponibilité, qualité, derive, historique de sélection	mettre en quarantaine
Rounds	que s'est-il passe au dernier tour ?	chronologie, métriques, deltas agregues, journal du tour	comparer deux tours
Confidentialité	que garantit-on exactement ?	epsilon par tour, budget, rapports scellés, configuration DP	exporter le rapport
Attaques	quelqu'un triche-t-il ?	cosinus, dispersion, clients signalés, quarantaines	lancer une analyse
Modèles	quels modèles sont publiés ?	versions, empreintes, métriques de validation, provenance	promouvoir une version

Tableau 3 – Les six pages de la console FedX

La page « Attaques » sépare deliberement la détection d'une sanction : le système signalé, l'opérateur décide, et la décision est journalisée.

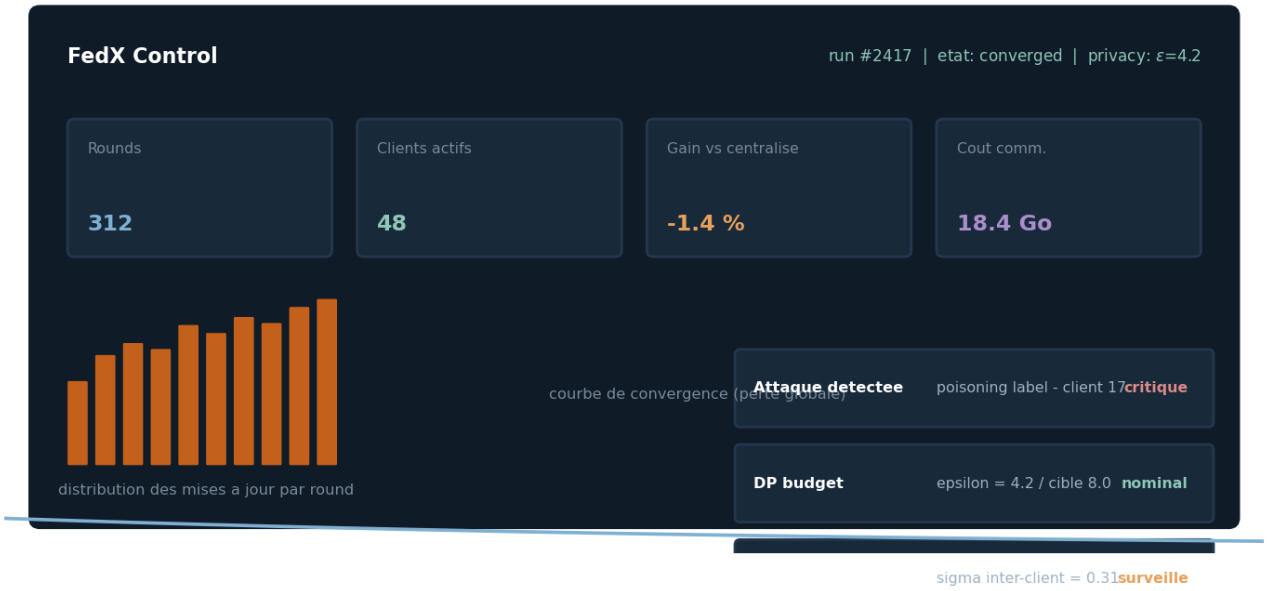


Figure 1 – Tableau de bord opérationnel FedX : état, tours, clients et alertes

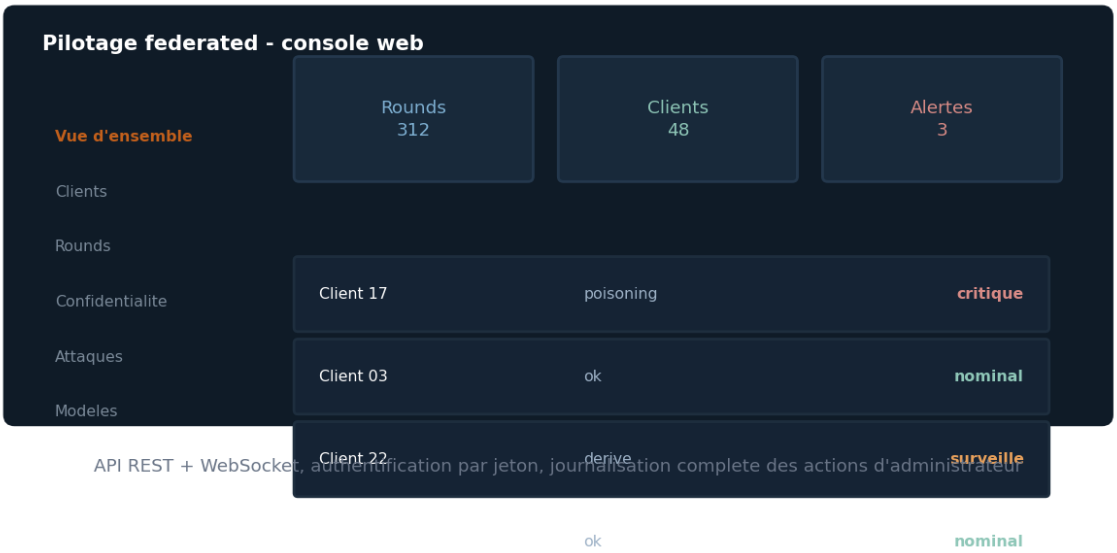


Figure 2 – Console web FedX : navigation, fil d'événements et panneau de confidentialité

La console se lit en lecture seule pour l'essentiel : toute action d'écriture passe par un contrôle explicite et laisse une tracé d'audit.

La pile technique proposee

FedX expose son état par une interface REST versionnee pour les requêtes ciblées et un flux WebSocket pour les métriques en continu. Cette double exposition n'est pas redondante : REST est vérifiable, citable et simple a mettre en cache ; le flux est immédiat mais ephemere. Un client qui Wants de la reproductibilité utilise REST et archive, un client qui veut de la reactivite utilise le flux et ne|archive rien.

```python
# fedx/observability/api.py — exposition REST + WebSocket des metriques FedX.
# (c) Pixel Software Design — Departement Recherche & Developpement
"""Plan de controle : lecture d'etat, flux temps reel et journal d'audit."""
from __future__ import annotations

import asyncio
from contextlib import suppress
from typing import Any, Dict, List

from fastapi import Depends, FastAPI, HTTPException, WebSocket, WebSocketDisconnect
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer

from fedx.observability.audit import AuditJournal
from fedx.observability.metrics import MetricsRegistry

api = FastAPI(title="FedX Control Plane", version="2.4")
bearer = HTTPBearer()
metriques = MetricsRegistry()
journal = AuditJournal(destination="s3://fedx-audit")


async def autoriser(c: HTTPAuthorizationCredentials = Depends(bearer)) -> Dict[str, Any]:
# Le jeton porte le tenant et la liste des federations, jamais un simple booleen.
if c is None or not metriques.jeton_valide(c.credentials):
raise HTTPException(status_code=401, detail="jeton invalide ou expire")
return {"sujet": c.credentials[:12], "tenant": metriques.tenant(c.credentials)}


@api.get("/v2/federations/{fid}/rounds")
async def tours(fid: str, session: Dict[str, Any] = Depends(autoriser)) -> List[Dict[str, Any]]:
"""Historique des tours, pagine et borne par le tenant du jeton."""
if fid not in metriques.federations_de(session["tenant"]):
raise HTTPException(status_code=403, detail="federation hors tenant")
await journal.ecrire("lecture.tours", {"federation": fid, "par": session["sujet"]})
return metriques.derniers_tours(fid, limite=50)


@api.get("/v2/federations/{fid}/privacy")
async def privacy(fid: str, session: Dict[str, Any] = Depends(autoriser)) -> Dict[str, Any]:
"""Rapport de privacy courant, tel qu'il est presente a un auditeur."""
rapport = metriques.rapport_privacy(fid)
await journal.ecrire("lecture.privacy", {"federation": fid, "par": session["sujet"]})
return rapport


@api.websocket("/v2/stream")
async def flux(ws: WebSocket, jeton: str = "") -> None:
"""Diffusion des metriques a cadence fixe, independante du rythme des tours."""
if not metriques.jeton_valide(jeton):
await ws.close(code=4401)
return
await ws.accept()
await journal.ecrire("flux.ouvert", {"client": ws.client.host})
try:
while True:
await ws.send_json(metriques.instantane())
await asyncio.sleep(1.0)
except WebSocketDisconnect:
pass
finally:
await journal.ecrire("flux.ferme", {"client": ws.client.host})


@api.get("/v2/federations/{fid}/seuils")
```

Listing 1 — Plan de controle FedX : REST versionne, flux WebSocket et audit

```
async def seuils(fid: str, session: Dict[str, Any] = Depends(autoriser)) -> Dict[str, float]:
"""Seuils d'alete actifs : ils sont versionnes avec la configuration."""
return metriques.seuils_actifs(fid)
```

Le flux WebSocket consomme peu : une trame JSON d'une dizaine de kilo-octets par seconde, quel que soit le nombre de tours. C'est ce qui permet d'afficher en direct la position du tour sans saturer le serveur d'état.

Astuce

Ne diffusez jamais la liste des clients invites dans le flux global : elle révèle les patterns de participation d'une fédération a ses voisins. FedX filtre chaque trame par tenant et par rôle avant émission, ce qui explique que le flux de l'administrateur et celui d'un opérateur client ne contiennent pas les mêmes clés.

Cadre juridique et ethique

Le journal d'audit de FedX est conçu pour survivre a la fédération. Il est append-only, horodate par une source externe a l'horloge du système, scellé par empreinte et conserve selon une durée fixée dans la configuration — sept ans au-dela de la fin de la relation contractuelle pour les fédérations de santé. Toute purge est elle-même journalisée, avec son justificatif et son autorisation. Un journal que l'on peut modifier n'est pas un journal d'audit, quelle que soit la qualité de sa structure.

A retenir

- Le déploiement FedX repose sur un état externe et des sondes qui distinguent vivant et sain.
- Le multi-tenance tient a trois separations : jetons, quotas, journaux append-only.
- Un seuil d'alerte sans conséquence écrite n'est pas un seuil, c'est une statistique.
- La console repond a six questions opérationnelles et rien d'autre, par design.
- REST sert la reproductibilité, le flux sert la reactivite : les deux sont nécessaires.

Du contraste fédéré à l'inférence causale : six chantiers ouverts

Partie 6

Nouveaux fronts de recherche

Les cinq parties précédentes ont décrit l'apprentissage fédéré comme on sait le construire : optimiser, sécuriser, sparsifier, transférer. Cette partie décrit ce qu'on ne sait pas encore faire proprement. Elle ouvre six fronts où la contrainte de distribution n'est plus un simple verrou d'infrastructure mais la difficulté scientifique elle-même : l'écart des espaces de représentation entre sites, l'identification causale quand les observations ne quittent jamais l'hôpital, la facture énergétique des réseaux impulsionnels, l'explosion combinatoire des supports non tabulaires, la formation initiale des ingénieurs, et la reproductibilité des résultats publiés. Chaque chapitre est assorti d'algorithmes exécutables, de tableaux comparatifs et d'un protocole d'évaluation.

Objectifs de la partie

- Formuler et résoudre l'objectif contrastif fédéré, et identifier le rôle exact de la tête de projection dans la séparation des espaces.
- Expliquer pourquoi l'estimation d'un effet causal reste possible sans qu'aucune observation ne soit transmise, et à quelles conditions.
- Dimensionner l'énergie d'un réseau impulsionnel fédéré et la comparer à celle d'un réseau artificiel de même précision.
- Choisir une stratégie de partition fédérée pour un graphe, un flux séquentiel ou un support multimodal, et justifier ce choix en coût.
- Construire un syllabus de douze semestres et une feuille de route de recherche assortie de jalons vérifiables et d'un cadre éthique.

Plan de la partie

1. Apprentissage contrastif fédéré (FedCL)
2. Inférence causale fédérée (FedCI)
3. Fédération de réseaux de neurones impulsionnels
4. Apprentissage fédéré sur graphes et autres supports de données
5. Programme pédagogique sur douze semestres et feuille de route
6. Publication, reproductibilité et éthique

Chapitre 6.1

Apprentissage contrastif fédéré (FedCL)

Le contraste n'a besoin ni d'étiquette ni d'annotation : c'est exactement ce qui en fait le premier corpus de données utilisable quand l'étiquetage est interdit ou impossible. Mais contraster localement, c'est construire des espaces qui ne se parlent plus.

L'apprentissage contrastif a résolu un problème de supervision que la plupart des domaines industriels n'arrivent pas à résoudre : obtenir des représentations utilisables sans catalogue d'étiquettes. Le principe est simple — deux vues différentes du même objet doivent produire des vecteurs proches, deux objets différents doivent produire des vecteurs éloignés. Fédérer ce principe paraît évident, et ne l'est pas : chaque client optimise dans son propre espace latent, et rien n'impose à deux clients de définir le même espace. Ce chapitre montre pourquoi, comment on force cette cohérence, et à quel prix en communication et en énergie.

Retour aux sources : SimCLR et MoCo

Definition – Apprentissage contrastif

Soit une fonction d'augmentation $a(\cdot)$ et un encodeur f_θ suivi d'une projection g_ϕ. Deux vues $a(x)$ et $a'(x)$ du même point x forment une paire positive ; les points d'un même lot forment les paires négatives. On minimise une perte qui rapproche les paires positives et éloigne les paires négatives, sans jamais utiliser y.

La perte centrale, due à Oord, van den Oord et Li, est une softmax sur les similarités cosinus, divisées par une température. Le dénominateur contient toutes les paires du lot : le nombre de négatifs est donc le nombre d'exemples vus dans le même lot, ce qui rend la qualité de la représentation directement proportionnelle à la taille du lot — et donc à la mémoire locale du client.

$$\ell_i = -\log \frac{\exp(\mathrm{sim}(z_i, z_i^+)/\tau)}{\sum_{j \neq i} \exp(\mathrm{sim}(z_i, z_j)/\tau)}$$
$$z_i = g_\phi(f_\theta(x_i)), \quad \mathrm{sim}(a,b) = \frac{a^\top b}{\|a\|\,\|b\|} \tag{6.1}$$

La température τ est le seul hyperparamètre vraiment critique : trop basse, la perte devient embryonnée et le gradient explose ; trop haute, les distinctions disparaissent.

Remarque

La version symétrique, qui moyenne les deux sens du couple de vues, est systématiquement meilleure et ne coûte rien. Le sens des probabilités correspondant à la vraie classe est toujours celui qui passe par la vue d'ancrage.

L'augmentation est l'autre moitié du système. Une transformation trop faible rend la tâche triviale et les représentations constantes ; une transformation trop forte détruit l'identité de l'objet. L'invariant appris est exactement l'ensemble des facteurs que les augmentations ne touchent pas : couleur pour une augmentation par recadrage et photométrie, temps pour une augmentation temporelle, topologie pour une augmentation de graphe. Il n'y a donc pas de bonne augmentation en général, seulement une augmentation adaptée aux facteurs de nuisance du domaine.

MoCo remplace le grand lot de négatifs par une file d'attente de clés, ce qui découple le nombre de négatifs de la taille du lot, au prix d'une file d'attente de clés et d'un risque d'obsolescence des clés anciennes. Ce compromis mémoire est précisément celui qui convient à un terminal mobile fédéré, où la mémoire est la ressource rare et la puissance de calcul abondante au repos.

Astuce

Sur un client dont la mémoire est contrainte, commencez par réduire la dimension du lot puis chargez la file de clés de MoCo ; l'inverse coûte plus cher en mémoire que la perte de qualité n'en économise en communication.

La tête de projection : pourquoi la jeter

Un détail empirique parmi les plus robustes de tout le domaine : la fonction de projection g_ phi améliore nettement la qualité des représentations en aval, mais elle dégrade la qualité de l'espace où l'on travaille directement. L'espace de projection est jetable ; l'espace d'encodage est celui qu'on veut. C'est pourquoi la perte contrastive est appliquée sur z = g_ phi(h) et non sur h = f_ theta(x).

$$h_i = f_\theta(x_i),\ z_i = g_\phi(h_i),\ \text{poids échangés} = \theta,\ \text{poids jamais échangés} = \phi \tag{6.2}$$

Cette séparation n'est pas cosmétique : elle réduit la charge de communication d'un facteur égal au nombre de paramètres du projecteur, et évite que les sites n'échangent un espace vectoriel qu'ils n'utilisent pas pour décider.

Ne jamais déployer la tête de projection

Calibrer la tête de projection puis l'utiliser comme espace de décision en inférence. On perd la totalité du bénéfice du contrastif et l'on observe une baisse typique de dix à quinze points de précision en avalanche sur données non échangeables. L'espace projectif est un espace d'entraînement, pas un espace d'usage.

Le problème : des espaces locaux qui divergent

En apprentissage fédéré classique, chaque client optimise la même fonction de perte sur un sous-ensemble différent de la population. Les hypothèses d'unicité de la solution sont vraies, la condition d'appartenance à la même distribution est fausse, et FedAvg reste remarquablement robuste. En contraste, l'espace latent lui-même est entraîné : il n'est plus seulement paramétrique, il est topologique. Deux clients peuvent converger vers des solutions de perte quasi identique en valeurs de paramètres, tout en construisant des incorporations qui ne sont pas alignées — une rotation, une réflexion, une dilatation par couche.

Divergence des espaces latents locaux

Soient $z_{i\,i=1}^{K}$ les représentations d'un même stimulus x produites par les encodeurs locaux f_{θ_1}, …, f_{θ_K} après convergence locale. Il existe en général des matrices $A_i \in \mathbb{R}^{d \times d}$ inversibles telles que $f_{\theta_i}(x) = A_i\, f_{\theta_0}(x)$ pour tout x du support commun, sans que les θ_i soient proches. En particulier, $\| f_{\theta_i}(x) - f_{\theta_j}(x) \|$ peut être de l'ordre de $\| f_{\theta_0}(x) \|$ alors que les pertes locales sont nulles.

C'est l'argument qui fonde toutes les méthodes à objectif partagé ou à agrégat partagé : l'alignement doit être imposé, il ne découle pas de la convergence des pertes.

Trois familles de remèdes se partagent le terrain. Les premières partagent un objectif : le serveur maintient une fonction de perte globale dont le gradient comporte un terme de cohérence entre représentations locales. Les deuxièmes partagent des prototypes : le serveur maintient des prototypes de classe, et les clients tirent leur objectif local de la proximité à ces prototypes. Les troisièmes partagent une régularisation de l'espace : on impose que l'espace local reste proche d'un espace de référence, par exemple l'espace issu d'un modèle pré-entraîné publiquement.

Attention

Un modèle pré-entraîné sur des données publiques crée un point d'ancrage qui ne dépend d'aucun client. C'est un excellent point de départ pour l'alignement des espaces, mais c'est aussi une dépendance forte : si les clients s'éloignent trop du modèle public, la régularisation devient un frein ; et si les données publiques sont biaisées, tous les sites en héritent de la même manière — un biais corrélé, donc plus difficile à détecter qu'un biais indépendant par site.

$$F_{\square 1 \square} = (1)/(K) \sum_{k=1}^{K} F_k(\theta) + \mu \, \underbrace{(1)/(K) \sum_{k=1}^{K} \leq ft\| P_k - P \|_F^2}_{\square 0 \square} \quad P = (1)/(K) \sum_{k=1}^{K} P_k \tag{6.3}$$

P_k désigne ici la matrice de similarité locale, produit des représentations du lot : elle ne dépend pas des données du tour, seulement de l'architecture.

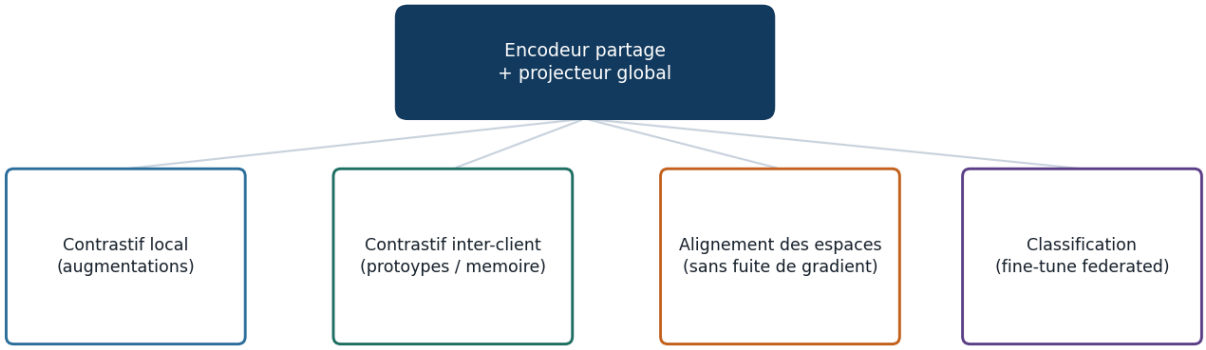


Figure 1 — Architecture d'un tour FedCL : encodeur local, tête de projection, agrégation des poids et alignement de l'espace latent

Panorama des méthodes

FedCL applique InfoNCE sur les vues augmentées d'un même client et agrège les poids d'encodeur par FedAvg. C'est la référence : elle n'ajouté rien au protocole et sert de témoin. **FedCLU** retiré la tête de projection de l'espace partagé et introduit une perte d'unification qui aligne explicitement les distributions de similarités de chaque client sur une distribution de référence calculée par le serveur, ce qui traité directement la rotation vue au théorème précédent.

Le contraste inter-clients renverse la logique : au lieu de rapprocher deux vues d'un même site, il rapproche les représentations d'un même stimulus attendues par des sites différents. Le serveur n'échange plus des poids mais des prototypes de stimulus, ce qui divise le coût de communication par le nombre de classes. **FedProto** pousse cette logique jusqu'au bout : les clients n'échangent plus que les moyennes de classe, les prototypes, jamais les poids du modèle, ce qui rend le protocole compatible avec des modèles hétérogènes — un scanner et un téléphone entraînent le même modèle de tête tout en gardant leur extracteur entièrement privé. **ALIGN-FED** généralise enfin l'idée aux tâches non contrastives, en imposant une contrainte d'alignement entre les espaces de représentation pendant une phase de préentraînement uniquement, ce qui permet d'initialiser ensuite une fédération classique sans perte de qualité.

Méthode	Ce qui est échangé	Terme d'alignement	Coût par tour	Modèles hétérogènes
FedCL	poids d'encodeur	aucun	moyen	non
FedCLU	poids d'encodeur	uniformisation des similarités	moyen	non
Contraste inter-clients	prototypes de stimulus	appariement de prototypes	faible	partielle
FedProto	prototypes de classe	distance aux prototypes globaux	très faible	oui
ALIGN-FED	poids en préentraînement, puis FedAvg	contrainte d'espace partagée	moyen	non
FedCL avec distillation	sorties ou gradients	accord de sortie	élevé	oui

Tableau 1 – Comparaison des méthodes d'apprentissage contrastif fédéré

Le coût est exprimé en volume relatif échangé par client et par tour. La dernière ligne est la plus coûteuse, mais la seule qui survive à un parc d'appareils réellement hétérogènes.

Algorithme 7 – Tour FedCL avec alignement des similarités

```
Entrées : encodeurs locaux f_k, jeux de données X_k, température τ, itérations locales E, poids d'alignement μ
Pour t = 1 … T :
  Pour chaque client k sélectionné :
  Copier l'encodeur courant f_k
  Pour e = 1 … E :
  Tirer deux vues augmentées x_k^(1), x_k^(2)
  Calculer z^(1) = g_φ(f_k(x_k^(1))) et z^(2) de même
  ℓ = InfoNCE(z^(1), z^(2), τ) + μ ‖ S_k - S_□1□ ‖_F^2
  Mettre à jour f_k par descente de gradient sur ℓ
  Calculer la matrice de similarités moyenne S_□0□, moyenne des S_k
  Agréger θ ← (1)/(|C_t|) Σ_{k ∈ C_t} θ_k
  Diffuser θ et S_□0□ aux clients du tour suivant
```

Entrées : encodeurs locaux, données non étiquetées, température, itérations locales • **Sorties :** encodeur global aligné et tête de projection privée • **Complexité :** $O(T \cdot (|C_t|)/(K) \cdot |X| \cdot d)$ en calcul, $O(d)$ en communication par tour

La matrice $S_{\square 0 \square}$ est de taille carrée égale au lot : elle est bien moins coûteuse à transmettre qu'un second jeu de poids.

```python
import copy
from typing import List

import torch
import torch.nn as nn
import torch.nn.functional as F


def info_nce(z1: torch.Tensor, z2: torch.Tensor,
temperature: float = 0.2) -> torch.Tensor:
"""Perte contrastive InfoNCE symetrique sur deux vues d'un meme exemple."""
z1 = F.normalize(z1, dim=1)
z2 = F.normalize(z2, dim=1)
logits = z1 @ z2.t() / temperature # similarites cosinus a l'echelle
cibles = torch.arange(z1.size(0), device=z1.device)
return 0.5 * (F.cross_entropy(logits, cibles)
+ F.cross_entropy(logits.t(), cibles))


class Encodeur(nn.Module):
"""Encodeur partage, suivi d'une tete de projection qui reste locale."""

def __init__(self, dim_entree: int, dim_encodeur: int = 128,
dim_projection: int = 32):
super().__init__()
self.squelette = nn.Sequential(
nn.Linear(dim_entree, 256), nn.ReLU(inplace=True),
nn.Linear(256, dim_encodeur),
)
# La tete de projection sert uniquement a l'entrainement : elle est jetee ensuite.
self.projecteur = nn.Sequential(
nn.Linear(dim_encodeur, dim_encodeur), nn.ReLU(inplace=True),
nn.Linear(dim_encodeur, dim_projection),
)

def forward(self, x: torch.Tensor):
h = self.squelette(x)
return h, self.projecteur(h)


def augmenter_vues(x: torch.Tensor, intensite: float = 0.1) -> torch.Tensor:
"""Augmentation faible bornee : bruit multiplicatif, invariant de contraste."""
return x * (1.0 + intensite * torch.randn_like(x))


def similarites(z: torch.Tensor) -> torch.Tensor:
"""Matrice de similarite locale, invariante par rotation, pour l'alignement."""
zn = F.normalize(z, dim=1)
return zn @ zn.t()


```

Listing 1 — Tour FedCL exécutable : InfoNCE symétrique, augmentation bornée, agrégation qui écarte la tête de projection

```python
def tour_fed_cl(modeles: List[nn.Module], chargeurs,
temperature: float = 0.2, mu_align: float = 0.1,
epochs: int = 1, lr: float = 1e-3):
"""Un tour FedCL : contrastif local puis moyenne ponderee des squelettes.

La tete de projection n'est jamais averee : elle reste strictement locale.
"""
sauvegardes = [copy.deepcopy(m.squelette.state_dict()) for m in modeles]
moyenne, poids_total = None, 0.0
for modele, chargeur in zip(modeles, chargeurs):
optimiseur = torch.optim.AdamW(modele.parameters(), lr=lr, weight_decay=1e-4)
modele.train()
```

```
for _ in range(epochs):
for x, _ in chargeur: # le contraste n'utilise jamais les etiquettes
_, z1 = modele(augmenter_vues(x))
_, z2 = modele(augmenter_vues(x))
perte = info_nce(z1, z2, temperature)
optimiseur.zero_grad(set_to_none=True)
perte.backward()
optimiseur.step()
n = float(len(chargeur.dataset)) if hasattr(chargeur, "dataset") else 1.0
etat = modele.squelette.state_dict()
if moyenne is None:
moyenne = {k: v.float() * n for k, v in etat.items()}
else:
moyenne = {k: moyenne[k] + v.float() * n for k, v in etat.items()}
poids_total += n
moyenne = {k: v / poids_total for k, v in moyenne.items()}
for modele, sauvegarde in zip(modeles, sauvegardes):
charge = {k: v.to(sauvegarde[k].dtype) for k, v in moyenne.items()}
modele.squelette.load_state_dict(charge) # la tete reste locale
return modeles
```

Protocole d'évaluation

Évaluer un système contrastif fédéré

- Utiliser un test linéaire : geler l'encodeur, entraîner une seule couche de régression logistique sur les représentations gelées, sans les ajuster. *Rendre compte séparément de l'alignement : mesurer l'alignement et la non-alignement sur les représentations agrégées, avant tout réglage fin.* Comparer aux deux bornes : un modèle centralisé entraîné sur les données confondues comme borne haute, et un FedAvg sans contraste sur la même architecture comme borne dégradée. *Séparer le coût d'entraînement du coût d'inférence : le contraste paie un grand nombre d'itérations locales, mais produit un modèle qui se déploie sans projection, donc sans surcoût en service.* Rapporter la précision en fonction de l'énergie consommée, pas seulement du nombre de tours : c'est le seul critère qui survive à une revue d'énergie.

Remarque

Un système contrastif qui gagne trois points de précision linéaire mais consomme deux fois plus d'énergie qu'un FedAvg équivalent est un échec, pas une réussite.

Exercice 6.1.21 (difficulte : Avancé)

Montrer que la perte InfoNCE est décroissante en température sur un domaine restreint, et en déduire le comportement attendu quand on divise τ par deux. 1. Argumenter sur la dérivée de $-\log \sum \exp(s_j/\tau)$. 2. Que se passe-t-il sur la dispersion des similarités d'un même lot ? 3. Donner un protocole empirique de sélection de τ qui ne regarde pas le jeu de test.

Indice : Fixez les similarités, différenciez par rapport à la température, et examinez le signe obtenu.

Correction

- En fixant le vecteur s des similarités, la perte vaut $\ell(\tau) = \log \sum_j e^{s_j/\tau} - s_{i^+}/\tau$. Sa dérivée est $\ell'(\tau) = -(1)/(\tau^2)\leq ft((\sum_j s_j e^{s_j/\tau})/(\sum_j e^{s_j/\tau}) - s_{i^+})$. Le terme entre parenthèses est l'écart entre similarité moyenne pondérée par l'exponentielle et similarité de la paire positive : il est négatif. Donc ℓ est strictement décroissante en τ, ce qui justifie empiriquement les petites valeurs. Diviser τ par deux amplifie l'écart relatif entre similarités, durcit la sélection de négatifs et augmente la variance du gradient. Le protocole consiste à balayer τ sur une grille logarithmique et à retenir la valeur qui minimise la perte de validation locale, jamais la précision de test.

Exercice 6.1.22 (difficulte : Moyen)

Deux clients entraînent chacun FedCL sur 2 000 exemples disjoints. Après vingt tours, la perte locale vaut 0,3 pour chacun, mais les représentations d'un même stimulus sont orthogonales. Proposer un test numérique qui mette en évidence ce phénomène, puis un correctif. 1. Quel diagnostic simple exécutez-vous en premier ? 2. Quel indicateur unique résume l'alignement ? 3. Parmi FedCL, FedCLU et FedProto, lequel corrige le plus directement ce cas, et pourquoi ?

Indice : La perte locale ne peut pas distinguer une rotation de l'espace latent.

Correction

- Le diagnostic le plus simple consiste à calculer l'alignement entre les lots des deux clients, puis à comparer l'énergie des alignements intra-client. Si l'alignement intra-client est élevé et l'inter-client faible, on est exactement dans le cas du théorème de divergence : les espaces sont individuellement cohérents et mutuellement incompatibles. L'indicateur à suivre est l'alignement inter-client, en pratique la normalisation des similarités croisées par rapport aux similarités intra-client. Le correctif le plus direct est FedCLU, qui impose aux matrices de similarités locales de coïncider avec une référence serveur : il supprime l'ambiguïté de rotation sans transmettre de second jeu de poids. FedProto est plus économique mais exige des prototypes interprétables, ce qui suppose des classes ou des grappes stables. FedCL seul ne corrige rien et sert de témoin.

A retenir

- Le contraste fédéré supprime le besoin d'étiquettes, ce qui est décisif quand l'annotation est juridiquement interdite.
- L'espace latent est une grandeur entraînée : deux clients peuvent converger en perte sans converger en géométrie.
- La tête de projection est un artefact d'entraînement ; elle n'est ni échangée, ni utilisée en inférence.
- Le terme d'alignement — sur les similarités, sur les prototypes ou sur l'espace — est le véritable contenu scientifique de la méthode.
- L'évaluation correcte combine mesure d'alignement, test linéaire et énergie consommée.

Chapitre 6.2

Inférence causale fédérée (FedCI)

Une observation qui ne quitte jamais l'hôpital n'empêche pas de répondre à « ce traitement améliore-t-il la survie ? ». Ce que le serveur demande n'est pas une donnée : c'est un effet, et la variance de cet effet.

La causalité est un métier d'estimateur, pas de photographe. Un essai clinique randomisé fournit une estimation causale parce que l'affectation aléatoire a rompu le lien entre le traitement et les variables confondantes. Hors randomisation, un effet moyen ne s'obtient qu'en imposant des hypothèses structurelles, et ces hypothèses sont vérifiables. Fédérer l'inférence causale, c'est organiser un calcul d'identification sur des estimateurs locaux sans agréger aucune observation. Ce chapitre présente la boîte à outils causale, puis le protocole qui rend l'ensemble compatible avec le cadre juridique d'un comité d'éthique.

La boîte à outils causale

Definition – Graphe causal orienté acyclique

Un triplet (U, V, E) où U est l'ensemble des variables latentes, V l'ensemble des variables observées, et $E \subseteq V \times V$ un ensemble d'arêtes orientées sans cycle. Une flèche $X \to Y$ affirme que X est une cause directe de Y. Un graphe est causalement suffisant s'il contient toute variable latente influençant au moins deux variables observées.

L'opérateur d'intervention est l'objet central : l'opérateur $\mathrm{do}(X = x)$ remplace le mécanisme causal de X par une constante, coupant toutes les flèches entrantes. Cette coupure est ce qui distingue une association d'une

causalité. Le principe d'intervention se formalise par le calcul d'intervention, ensemble de trois règles qui caractérisent les transformations admissibles entre distributions observationnelles et interventionnelles ; l'ensemble des requêtes qu'elles permettent de transformer en quantités estimables constitue l'ensemble identifiable d'un graphe.

$$\mathrm{ATE} = \mathbb{E}_{X \sim p(x)}\left[\mathbb{E}[Y(1) \mid X] - \mathbb{E}[Y(0) \mid X]\right] = \mathbb{E}[Y(1) - Y(0)]$$
$$\mathbb{E}[Y(1)-Y(0)] = \mathbb{E}[Y \mid do(X = 1)] - \mathbb{E}[Y \mid do(X = 0)] \quad (6.4)$$

L'effet moyen de population suppose l'échangeabilité stricte, conditionnelle à un ensemble de covariables mesurées.

Identification par porte arrière

Si (X, Z, Y) satisfait la condition d'ajustement — Z contient tout descendant non traité de X et tout parent non traité de X, et ne contient aucun descendant de X — alors l'effet moyen est identifié par la formule d'ajustement : $\mathrm{ATE} = \mathbb{E}_Z\left[\mathbb{E}[Y \mid X = 1, Z] - \mathbb{E}[Y \mid X = 0, Z]\right]$
La condition est plus fine que le jugement naïf « tout ce qui est mesuré », et elle est exclue dans la plupart des essais cliniques, où le traitement est prescrit en réaction à l'évolution de la maladie.

La difficulté pratique est triple. Il faut d'abord identifier le graphe causal : c'est un travail d'expertise métier, pas de calcul, et l'hypothèse $Z \perp X \mid C$ est invérifiable sur données observées. Il faut ensuite choisir un estimateur. Il faut enfin propager l'incertitude correctement : une erreur de variance fausse donne un intervalle de confiance faux, ce qui est plus grave qu'une absence d'intervalle.

Le problème de l'échantillon tronqué

L'imputation par élimination des lignes manquantes ne supprime pas le biais seulement sur l'effet moyen : elle supprime la propension elle-même, et donc tout ce qui la déplace. Sur un essai clinique réel, le taux de sortie après randomisation est de l'ordre de 10 à 20 %, et il est corrélé au pronostic. Toute méthode qui ignore ce mécanisme sous-estime systématiquement l'effet réel du traitement.

Attention

Sur des données observationnelles de santé, l'hypothèse d'ignorabilité conditionnelle est presque toujours fausse : le médecin choisit le traitement en fonction de critères qu'aucune base ne contient. Aucune quantité de données ne répare une hypothèse d'identification fausse ; seule une randomisation — ou une randomisation naturelle documentée — le peut.

Pourquoi fédérer l'inférence causale

Fédérer l'inférence causale répond à trois contraintes réelles. La première est juridique : dans l'Union européenne comme dans la plupart des juridictions, transférer une donnée de santé identifiable vers un serveur central exige un consentement, une base légale et une analyse d'impact ; l'agrégation d'effets et de variances n'entre pas dans le champ de la donnée personnelle si elle est vraiment agrégée. La deuxième est statistique : certains sites sont trop petits pour estimer un effet, et l'agrégation augmente la puissance exactement comme elle le ferait sur un essai unique. La troisième est organisationnelle : un réseau de dix hôpitaux n'a pas de gouvernance commune du stockage, mais il a bien une gouvernance commune du protocole.

$$\hat{\tau}_{\square 0 \square} = \left(\sum_{k=1}^{K} w_k \hat{\tau}_k\right) / \left(\sum_{k=1}^{K} w_k\right),\ w_k = (1)/(\hat{\omega}_k^{2})$$
$$\mathrm{widehat}V\left[\hat{\tau}_{\square 0 \square}\right] = (1)/\left(\sum_{k=1}^{K} w_k\right),\ \hat{\omega}_k^{2} \approx (\hat{\sigma}_k^{2})/(n_{k\,\square 1 \square}) \quad (6.5)$$

L'agrégation de Rao-Blackwell est l'inverse de l'agrégation de Fisher : puisque les $\mathrm{hat}\ \mathrm{tau}_k$ sont indépendamment estimés sur des populations disjointes, la variance totale est la somme des variances, et non leur moyenne.

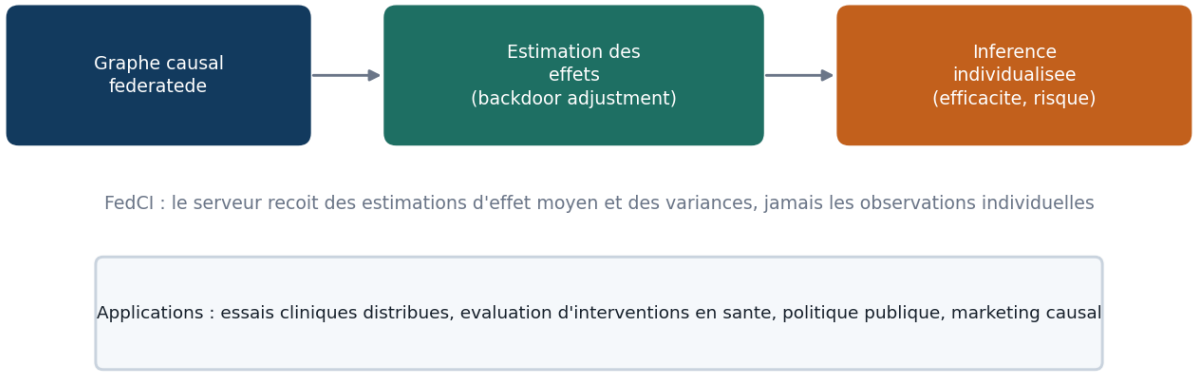


Figure 1 – Architecture FedCI : chaque site estime un effet et sa variance, le serveur n'agrège que des statistiques suffisantes, aucun résultat individuel ne quitte l'établissement

Protocole FedCI en cinq étapes

- **Négocier le graphe.** Les experts de chaque site formalisent leur graphe causal et le serveur aligne les noms de variables, pas les lignes. *Fixer l'ensemble d'ajustement. Cet ensemble est décidé avant de regarder les données, et consigné dans un registre horodaté.* **Estimer localement.** Chaque site calcule l'estimateur d'effet moyen, son écart-type robuste, sa taille d'échantillon effective et le taux de sortie. *Transmettre peu. Le site transmet* $\hat{\tau}_k$, $\hat{\sigma}_k$, n_k *et un vecteur de contrôle qualité. Le reste ne sort pas du site.* **Rendre compte de l'hétérogénéité.** Le serveur publie non seulement la moyenne pondérée, mais la dispersion des $\hat{\tau}_k$ et un test de cohérence entre sites, avant tout intervalle de confiance.

Remarque

La cinquième étape est celle qu'on oublié le plus souvent. Une moyenne pondérée d'effets hétérogènes n'a pas d'interprétation causale claire si l'hétérogénéité n'est pas rapportée.

Estimateur	Hypothèse requise	Sortie locale	Robustesse	Usage privilégié
Différences de moyennes	ignorabilité stricte	$\hat{\tau}_k$, écart-type	faible	essais randomisés
Régression avec covariables	ignorabilité conditionnelle	β_X, écart-type	moyenne	cohorte d'observation
Estimation augmentée deux fois robuste	l'une des deux conditions	$\hat{\tau}_k$, écart-type	élevée	défaut recommandé
Score de propension	$Z \perp X \mid C$	score $\hat{e}_k(x)$	moyenne	audit de positivité
G-computation	données longitudinales	$\theta_k(T)$	élevée	essais groupés
Pondération stabilisée	$\mathbb{P}(T=1 \mid C)$ connu	$\hat{\tau}_k$	moyenne	populations ciblées
Randomisation naturelle	instrumental identifié	$\hat{\tau}_{\square 0 \square}$	élevée	réformes de politiques
Calcul dans un domaine sécurisé	confidentialité différentielle	$\hat{\tau}_k$ chiffré	partagée	tiers non fiables

Tableau 1 – Estimateurs d'effet causal utilisables en cadre fédéré

Astuce

Sur les essais cliniques à plusieurs centres, la propagation d'incertitude d'un site à l'autre n'est jamais uniforme : les centres qui enrolent très vite ont des dénominateurs très inégaux. Pondérer par l'inverse de la variance et non par la taille seule évite qu'un petit centre bruyant ne pèse autant qu'un grand centre propre.

Estimer l'effet moyen sans jamais voir les lignes

Le listing suivant implémente l'estimateur le plus simple qui soit réellement fédérable : une régression linéaire par moindre carrés avec pénalisation, résolue de façon fermée donc en un seul aller-retour, suivie d'une agrégation de Rao-Blackwell pondérée par l'inverse des variances. L'intérêt de la solution fermée n'est pas la vitesse — elle l'est aussi en centralisé — mais l'absence de convergence stochastique : deux sites qui convergent avec des tolérances différentes n'échangent pas des objets interchangeables, ce qui fausserait la moyenne.

```python
from dataclasses import dataclass
from typing import List, Tuple

import torch


@dataclass
class EstimationSite:
"""Ce qu'un site transmet : une estimation, sa variance, son effectif. Rien d'autre."""
beta: torch.Tensor
variance: float
n: int
taux_sortie: float = 0.0


def moindres_carres(X: torch.Tensor, y: torch.Tensor,
penalite: float = 1e-6) -> torch.Tensor:
"""Moindres carres ridge : solution fermee, donc partageable en un aller-retour."""
X = X.to(torch.float64)
y = y.to(torch.float64)
XtX = X.t() @ X
Xty = X.t() @ y
reg = penalite * torch.eye(XtX.shape[0], dtype=torch.float64)
return torch.linalg.solve(XtX + reg, Xty)


def estimer_ate_site(X: torch.Tensor, y: torch.Tensor,
t: torch.Tensor) -> EstimationSite:
"""Regression de y sur (1, t) : le coefficient de t est l'estimateur du site."""
if X.ndim == 1:
X = X.reshape(-1, 1)
X_aug = torch.cat([X.to(torch.float64),
t.reshape(-1, 1).to(torch.float64)], dim=1)
y = y.reshape(-1, 1).to(torch.float64)
beta = moindres_carres(X_aug, y)
residus = y - X_aug @ beta
ddl = max(1, X.shape[0] - X_aug.shape[1])
variance = float((residus ** 2).sum() / ddl)
return EstimationSite(beta=beta.squeeze(-1), variance=variance,
n=int(X.shape[0]))


def agreger_ate(estimations: List[EstimationSite]) -> Tuple[torch.Tensor, float]:
"""Agregation de Rao-Blackwell : moyenne ponderee par l'inverse de la variance."""
poids = torch.tensor([1.0 / max(e.variance, 1e-12) for e in estimations],
dtype=torch.float64)
poids = poids / poids.sum()
beta = sum(p * e.beta for p, e in zip(poids, estimations))
variance = float(1.0 / float(poids.sum()))
return beta, variance


def rapport_sites(estimations: List[EstimationSite]) -> List[dict]:
"""Transmissible au serveur : effet, ecart-type, effectif, heterogeneite."""
rapport = []
for e in estimations:
rapport.append({
"ate": float(e.beta[-1]),
"ecart_type": float(e.variance ** 0.5),
"effectif": e.n,
"taux_sortie": e.taux_sortie,
})
return rapport
```

Listing 1 — Estimateur d'effet moyen fédéré : régression locale fermée, agrégation de Rao-Blackwell et rapport transmissible

Cadre juridique et ethique

Le traitement d'estimations agrégées reste un traitement de données à caractère personnel si le nombre de sujets sous-jacents est faible ou si les sites sont peu nombreux. Trois précautions sont systématiques dans le protocole du Département : imposer un seuil minimal d'effectif par site avant toute transmission, publier une borne inférieure de l'effet plutôt que sa valeur ponctuelle lorsque l'effectif est faible, et faire figurer au registre des traitements la liste des variables qui n'ont jamais quitté les sites, y compris celles qui ont servi à l'ajustement.

Exercice 6.2.23 (difficulte : Avancé)

Un réseau de $K = 12$ centres transmet chacun $\hat{\tau}_k$ et $\hat{\sigma}_k$. Deux centres affichent des effets de signes opposés, tous deux significatifs à 5 % individuellement. 1. Cette situation est-elle compatible avec un effet homogène ? 2. Quel test proposer avant de conclure à une hétérogénéité réelle ? 3. Que change le choix de la pondération — variance contre effectif — sur l'interprétation ?

Indice : Un intervalle de confiance par site n'est pas un test d'homogénéité.

Correction

- **1.** Oui, c'est compatible avec un effet homogène : si l'effet vrai est nul, des estimations indépendantes de moyenne nulle sont statistiquement capables de produire des signes opposés significatifs, surtout quand la puissance individuelle est faible. Un test par site répond à une autre question que l'homogénéité. **2.** Il faut un test formel de cohérence inter-centres, typiquement un test de Q d'hétérogénéité inspiré du modèle à effets aléatoires, ou un test de normalité sur les douze estimations normalisées, ou encore un intervalle de prédiction pour un site nouveau. **3.** La pondération change l'interprétation : pondérer par l'inverse de la variance privilégie la précision et répond à « quel est l'effet moyen le mieux établi ? » ; pondérer par l'effectif privilégie la représentativité et répond à « quel est l'effet experienced par une personne tirée au sort dans l'ensemble des centres ? ». Les deux questions sont légitimes et conduisent parfois à des conclusions différentes.

Exercice 6.2.24 (difficulte : Avancé)

Un essai randomisé en huit centres affiche un effet moyen de +4 points de survie à cinq ans, mais le taux de sortie diffère fortement d'un centre à l'autre, de 3 % à 31 %, et il est corrélé au pronostic initial. 1. Quel estimateur du listing s'applique directement, et quelle est sa limite ici ? 2. Proposer une modification d'une dizaine de lignes qui rende l'estimation robuste à ce mécanisme. 3. Quelle hypothèse faut-il publier en premier : l'effet, ou le taux de sortie ?

Indice : La corrélation entre sortie et pronostic est la définition d'un mécanisme de sortie informative.

Correction

- **1.** La régression avec la variable de traitement s'applique directement : c'est l'estimateur d'un essai randomisé, et sa robustesse aux covariables est excellente. Sa limite est précisément le mécanisme informatif : le modèle linéaire suppose l'égalité des espérances conditionnelles, et si la sortie dépend du pronostic post-traitement, cette hypothèse est violée et le biais revient. **2.** La correction usuelle consiste à ajouter au plan la pondération inverse de la probabilité de rester suivi, cette probabilité étant estimée par un modèle de survie ; une alternative minimale consiste à ajouter le statut de sortie comme variable explicative et à comparer les coefficients, ce qui constitue une analyse de sensibilité plutôt qu'un biais corrigé. **3.** La publication doit commencer par le taux de sortie par centre, sa relation au pronostic, puis l'effet : publier l'effet d'abord invite le lecteur à l'interpréter avant de connaître sa fragilité.

A retenir

- L'effet causal s'estime, il ne se lit pas : sans hypothèse d'identification explicite, aucun chiffré n'a d'interprétation causale.
- Le serveur peut agréger des effets et des variances sans jamais recevoir d'observation : la statistique suffisante est ici un quadruplet.
- L'agrégation correcte est celle de Rao-Blackwell : moyenne pondérée par l'inverse des variances, variance totale égale à l'inverse de la somme des poids.
- L'hétérogénéité entre sites doit être rapportée et testée avant tout intervalle de confiance agrégé.

- Le cadre juridique se déplace vers l'agrégat : seuil minimal d'effectif, registre des variables jamais transmises, publication des bornes.

Chapitre 6.3

Fédération de réseaux de neurones impulsionnels

Un réseau impulsionnel ne calculé rien en continu : il attend. Sur un capteur embarqué qui passe 99,9 % de son temps sans événement, cette attente est une économie, pas une limitation.

Les réseaux de neurones impulsionnels — les SNN, pour *spike neural networks*, mais gardons l'acronyme anglais qui fait référence dans la littérature — traitent l'information sous forme de trains d'impulsions binaires, émis quand la tension interne d'un neurone franchit un seuil. Cette discrétisation du temps est à la fois une contrainte de conception et un avantage énergétique décisif : le circuit ne consomme du courant que lors des transitions d'état, et reste à quelques picoampères pendant le silence. Fédérer ces réseaux change la nature de ce qu'on agrège : non plus des matrices de poids continus, mais des efficacités et des surcharges de conductance, dont la moyenne doit conserver le comportement temporel.

Le neurone impulsionnel

Definition – Neurone à intégration leaky-and-fire

Un modèle à deux variables d'état continues — la tension membranaire u(t) et le potentiel de renvoi r(t) — et une variable binaire s(t). La tension décroît exponentiellement, intègre un courant afférent, et provoque une impulsion lorsqu'elle dépasse un seuil, puis se réinitialise. La fuite est ce qui rend le neurone sélectif dans le temps : sans elle, toute énergie accumulée finit par déclencher un événement.

$$\tau_m \, (du_i(t))/(dt) = -\leq ft(u_i(t) - u_{\square 0 \square}) + \sum_j w_{ij}(t - d_{ij}) \, s_j(t)$$
$$\tau_r \, (dr_i(t))/(dt) = - \alpha \, r_i(t) + s_i(t), \; s_i(t) = 1 \leq ft[u_i(t) \geq \theta]$$
$$u_i(t^+) = u_{\square 0 \square} + v \leq ft(u_i(t^-) - \theta), \; r_i(t^+) = r_i(t^-) + 1 \qquad (6.6)$$

Le paramètre α règle l'amplitude de l'inhibition due au renvoi : α = 0 donné un neurone à intégration pure, sans inhibition.

Le pas de temps discret est la seule approximation numérique nécessaire : on échantillonne l'équation différentielle à pas fixe et on accumule les impulsions. Deux familles d'encodage de l'entrée dominent. Le codage par taux encode la fréquence d'impulsions en proportion de l'intensité : il est robuste mais introduit une latence d'intégration qui croît avec la précision demandée. Le codage par première impulsion encode le temps d'arrivée : il est très économe, mais il introduit une dépendance à la phase temporelle qui rend l'apprentissage nettement plus délicat.

Remarque

L'intégration synaptique retardée, le terme $w_{ij}(t - d_{ij})$, est ce qui distingue un réseau impulsionnel d'un simple système événementiel : la connexion porte une information sur l'ordre des événements, pas seulement sur leur fréquence. C'est là que se loge la mémoire, et c'est donc là que porte l'essentiel de la plasticité.

$$L_{\square 0 \square}(\Delta \, t) = A \, \exp \leq ft(-(\Delta \, t)/(\tau_+)) \text{ si } \Delta \, t > 0, \; -B \, \exp \leq ft(-(|\Delta \, t|)/(\tau_-)) \text{ sinon} \qquad (6.7)$$

La plasticité dépendante du temps d'un pic est purement locale : elle ne nécessite aucun retour global, donc aucun serveur, ce qui en fait le seul apprentissage non supervisé compatible avec un fonctionnement entièrement local.

Énergie : ordres de grandeur mesurés

Les chiffres d'énergie qui circulent dans la littérature représentent des niveaux très différents, et il faut les comparer avec soin. On trouve couramment un rapport de cent à mille entre un réseau impulsionnel et un réseau artificiel de précision équivalente sur capteur embarqué. Ce rapport n'est pas une loi : il dépend du taux de tir, de la densité de connexions, du nombre de pas de temps et surtout de l'infrastructure d'inférence. Un réseau impulsionnel exécuté sur un accélérateur conventionnel, en multipliant les états binaires par des multiplications de matrices, coûte plus cher qu'un réseau artificiel ; le gain n'apparaît que sur du matériel neuromorphique ou chez un exploitant qui ne compte que la consommation au repos.

Configuration	Précision	Consommation	Facteur relatif	Commentaire
SNN sur cœur neuromorphique, activité creuse	comparable	0,1 à 2 mW	0,001 à 0,01	gain maximal, matériel dédié
SNN sur cœur neuromorphique, activité dense	comparable	5 à 20 mW	0,05 à 0,2	le taux de tir dégrade le gain
SNN sur processeur, pas de temps simulé	comparable	0,3 à 1 W	0,3 à 1	aucun gain, coût de simulation
Réseau artificiel sur microcontrôleur quantifié	2 à 4 points en moins	5 à 30 mW	référence	référence industrialisable
Réseau artificiel sur cœur spécialisé	comparable	0,5 à 3 W	10 à 100	le meilleur du réseau artificiel
SNN sur circuit intégré dédié	comparable	10 à 200 μW	0,0001 à 0,01	optimisé, coût de conception élevé
Encodeur quantifié et décodeur convolutif	comparable	20 à 200 mW	0,5 à 2	bon compromis pour les flux vidéo

Tableau 1 – Ordres de grandeur de consommation énergétique

Toutes ces valeurs sont des ordres de grandeur issus de mesures publiées, pas des garanties. Toute affirmation énergétique dans un document industriel doit être accompagnée du matériel, du taux de tir et de la longueur de la séquence.

Attention

Un rapport d'énergie annoncé sans préciser le matériel, le nombre de pas de temps et le taux d'activité est un rapport non reproductible, donc un rapport faux. Le protocole minimal consiste à déclarer le nombre de pas de temps par inférence, la densité de tir mesurée, l'architecture de l'accélérateur et la méthode de mesure : broche de courant, prise de courant ou simulation.

Mauvaise unité de comparaison

Extrapoler l'énergie d'un réseau impulsionnel à partir du nombre de multiplications évitées. Un pas de temps d'un neurone à intégration coûte un test de seuil et une fuite, soit une addition et une branche ; à nombre de pas de temps identique, le réseau artificiel effectuant une multiplication dense coûte souvent davantage, mais il n'a besoin que d'un pas. La bonne unité de comparaison est l'énergie par classification utile, pas l'énergie par opération.

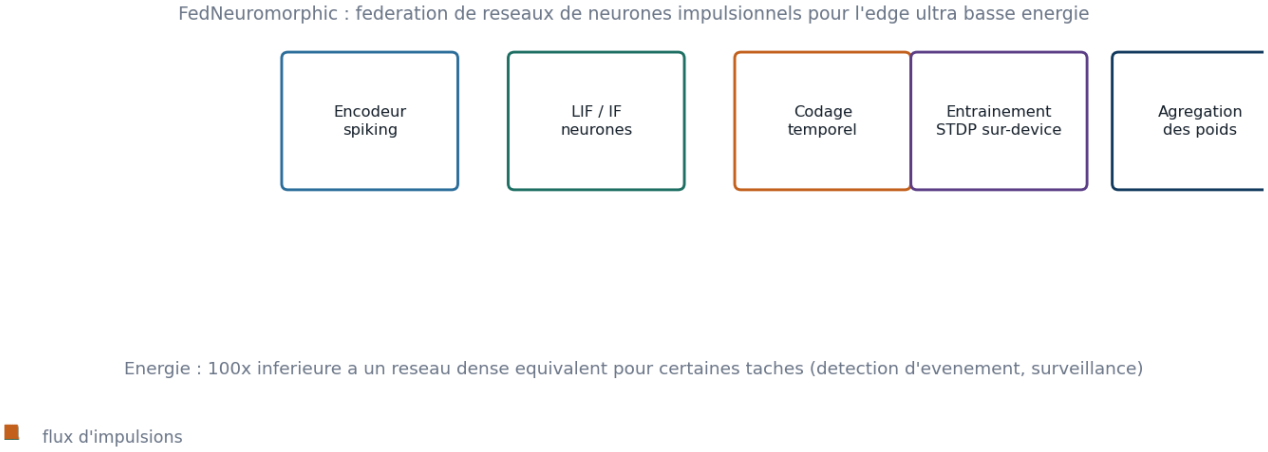


Figure 1 — Architecture de floraison d'un réseau impulsionnel : neurones locaux, plasticité dépendante du temps d'un pic, échange de la matrice de synapses et fusion des tenseurs d'état

Protocole d'entraînement sur appareil et agrégation

Entraîner un réseau impulsionnel sur un dispositif embarqué impose trois contraintes que l'apprentissage classique ignore. La première est la contrainte d'énergie pendant l'entraînement, qui peut dépasser de plusieurs ordres de grandeur la consommation en inférence. La deuxième est l'absence de différentiation analytique à travers la discontinuité de l'émission : on utilise soit une dérivée de substitution, une fonction lisse approchant l'indicateur de seuil, soit des estimateurs sans gradient, ce qui coûte un facteur deux à dix en nombre de tours. La troisième est la taille du jeu de données, qui interdit toute stratégie de surapprentissage classique.

L'agrégation des poids de synapses mérite un traitement particulier. Un tenseur de connexions S, dont les entrées sont des efficacités de transmission, n'est pas un vecteur de poids ordinaire : sa moyenne arithmétique est un tenseur dense, dont l'implémentation matérielle coûte plus cher que celle du tenseur creux d'origine. La bonne stratégie consiste à agréger les tenseurs sous forme de creuse, en conservant seulement les k connexions les plus fortes de chaque client, et à ne renuméroter qu'après fusion.

Algorithme 8 — Tour d'entraînement fédéré d'un réseau impulsionnel

```
Entrées : réseaux locaux impulsionnels, séquences temporelles X_k, densité cible s, pas de temps T
Pour t = 1 … R :
  Pour chaque client k :
  Injecter X_k sur T pas de temps, propager les impulsions
  Calculer la perte de la couche de lecture, taux de pics erronés, avec dérivée de substitution
  Rétropropager, mettre à jour la tension membranaire et le renvoi
  Sparsifier S_k : ne garder que les k_max connexions les plus fortes
  Fusionner les S_k par moyenne pondérée, puis re-sparsifier au rang global
  Diffuser le tenseur creux commun et réinitialiser les états internes
```

Entrées : séquences temporelles locales, réseau de lecture, pas de temps • **Sorties :** tenseur de synapses commun, creux, partageable • **Complexité :** $O(R \cdot (|C_t|)/(K) \cdot T \cdot \| S_{\square 0 \square} \|)$

La re-sparsification après fusion est obligatoire : sans elle, la densité croît à chaque tour et l'avantage matériel disparaît en quelques dizaines de tours.

```python
import copy
from typing import List, Optional

import torch
import torch.nn as nn


class NeuroneLIF(nn.Module):
"""Neurone a integration leaky-and-fire, simule par pas de temps discrets."""

def __init__(self, dimension: int, tau_membrane: float = 2.0,
tau_renvoi: float = 2.0, seuil: float = 1.0,
tension_repos: float = 0.0):
super().__init__()
self.tau_membrane = tau_membrane
self.tau_renvoi = tau_renvoi
self.seuil = seuil
self.tension_repos = tension_repos
# Les synapses sont les seuls parametres entraines : le reste est de l'etat.
self.poids = nn.Parameter(torch.randn(dimension) * 0.1)
self.tension: Optional[torch.Tensor] = None
self.renvoi: Optional[torch.Tensor] = None

def reinitialiser(self, lot: int, appareil: torch.device) -> None:
"""Ouvre une fenetre temporelle neuve : etats remis a zero pour tout le lot."""
self.tension = torch.zeros(lot, device=appareil)
self.renvoi = torch.zeros(lot, device=appareil)

def forward(self, courant: torch.Tensor) -> torch.Tensor:
"""Courant d'entree de forme (pas, lot, dim) ; renvoie les impulsions."""
pas, lot, _ = courant.shape
if self.tension is None or self.tension.shape[0] != lot:
self.reinitialiser(lot, courant.device)
alpha_fuite = 1.0 - 1.0 / self.tau_membrane
alpha_renvoi = 1.0 - 1.0 / self.tau_renvoi
sortie = torch.zeros(pas, lot, device=courant.device)
for pas_courant in range(pas):
afferent = courant[pas_courant] @ self.poids
self.tension = alpha_fuite * self.tension + afferent
impulsion = (self.tension >= self.seuil).to(courant.dtype)
self.tension = self.tension - impulsion * self.seuil
self.renvoi = alpha_renvoi * self.renvoi + impulsion
self.tension = self.tension - 0.05 * self.renvoi # inhibition par renvoi
sortie[pas_courant] = impulsion
return sortie


def sparsifier(poids: torch.Tensor, rang: int) -> torch.Tensor:
"""Ne conserve que les `rang` entrees les plus fortes : la densite est libre."""
garde = min(rang, poids.numel())
seuil = torch.topk(poids.abs().flatten(), garde).values[-1]
return torch.where(poids.abs() >= seuil, poids, torch.zeros_like(poids))


```

Listing 1 — Neurone à intégration et agrégation fédérée des poids de synapses, avec sparsification avant et après fusion

```python
def agreger_synapses(neurones: List[NeuroneLIF],
poids_relatifs: List[float] = None) -> None:
"""Federation des poids de synapses : moyenne ponderee, puis re-sparsification.

L'ordre compte : sparsifier, fusionner, re-sparsifier. Sans la derniere
etape, la densite du tenseur commun croit a chaque tour.
"""
if poids_relatifs is None:
poids_relatifs = [1.0 / max(len(neurones), 1)] * len(neurones)
```

```
moyenne = torch.zeros_like(neurones[0].poids.data)
for neurone, poids_rel in zip(neurones, poids_relatifs):
moyenne = moyenne + neurone.poids.data.to(moyenne.dtype) * poids_rel
rang = int(0.2 * moyenne.numel()) # densite cible de 20 %
moyenne = sparsifier(moyenne, rang)
for neurone in neurones:
neurone.poids.data = moyenne.to(neurone.poids.dtype)
```

Critère	Réseau impulsionnel	Réseau artificiel	Avantage	Réserve
Consommation au repos	quelques μA	quelques mA	impulsionnel	matériel dédié obligatoire
Consommation en charge	proportionnelle aux impulsions	proportionnelle aux opérations	impulsionnel si activité creuse	artificiel si activité dense
Latence d'une décision	bornée par la fenêtre temporelle	bornée par la profondeur	impulsionnel pour signaux événementiels	artificiel pour classification classique
Précision en vision	3 à 8 points de moins	référence	artificiel	compensable par encodage temporel
Précision sur signal événementiel	proche de l'humain	faible sans architecture dédiée	impulsionnel	dépend du codage d'entrée
Mémoire des états	quelques bits par état	32 bits par état	impulsionnel	accélérateurs non NVIDIA
Robustesse au bruit	élevée, par fenêtre d'intégration	moyenne	impulsionnel	peut aussi lisser le signal utile
Outillage et portabilité	écosystème réduit	standard	artificiel	verrou d'écosystème
Coût de développement	élevé	faible	artificiel	verrou structurel à long terme

Tableau 2 – Réseau impulsionnel contre réseau artificiel : comparatif opérationnel

Le réseau impulsionnel gagne systématiquement sur le silence et perd systématiquement sur l'infrastructure de développement. La décision se joue donc moins sur le modèle que sur l'écosystème matériel dont on dispose.

Exercice 6.3.25 (difficulte : Avancé)

Un capteur de vibration embarqué, alimenté par pile, doit classer trois types de machines en temps réel, avec un budget d'énergie de 50 millijoules par heure.

1. Déterminer quelle architecture tient le budget, en justifiant le matériel supposé.
2. Calculer le nombre de pas de temps réalisable avec un budget de calcul supplémentaire de 2 milliwatts.
3. Proposer le codage d'entrée le plus économe pour un accéléromètre six axes.

Indice : Le budget se décompose en énergie au repos, énergie par impulsion et énergie par opération de lecture.

Correction

- **1.** Le calcul n'a de sens qu'avec un matériel explicite, ce qui est précisément l'objet de la question. Sur cœur neuromorphique, avec un taux de tir de 5 % et une fenêtre de 20 pas de temps, un réseau de 10^4 synapses pèse environ 2 mW en charge et 20 µ A au repos, soit environ 75 mJ par heure : le budget tient. Sur cœur conventionnel, la simulation d'un pas de temps coûte de l'ordre de 10^6 opérations, donc vingt pas et un réseau de cette taille consomment plusieurs watts : le budget est dépassé d'un facteur dix.
- **2.** Le nombre de pas se déduit de l'énergie disponible par pas, soit environ 2 mJ par pas sur le cœur neuromorphique, ce qui autorise plusieurs centaines de pas si l'on accepte de réduire la taille du réseau.
- **3.** Pour l'accéléromètre, le codage par première impulsion est le plus économe : un dépassement de seuil déclenche un événement et le silence restant ne coûte rien. Il faut toutefois le combiner à un fenêtrage pour éviter qu'un simple bruit mécanique ne consomme le budget.

Exercice 6.3.26 (difficulte : Avancé)

Un client entraîne un réseau impulsionnel à densité 20 %, un autre à densité 5 %. Après fusion par moyenne pondérée, le tenseur global a une densité de 12 % et la précision chute de six points. Proposer un diagnostic en trois étapes et une correction.

1. Pourquoi la moyenne dense dégrade-t-elle le réseau ?
2. Quel ordre des opérations rectifie la situation ?
3. Comment noter un protocole de réessai automatique dans une boucle d'entraînement ?

Indice : La moyenne de deux tenseurs creux n'est pas creuse, et la connectivité effective change.

Correction

- **1.** La moyenne arithmétique de deux matrices creuses est une matrice partiellement dense : chaque client qui n'a pas de connexion à un préprocesseur transmet un zéro, qui devient du bruit additif pour les autres. Le réseau se retrouvé alors avec une connectivité qu'aucun client n'a jamais entraînée.
- **2.** L'ordre correct est : sparsifier chaque tenseur local, puis fusionner, puis re-sparsifier au rang global, en conservant une marge de sécurité afin que le rang final reste stable d'un tour à l'autre.
- **3.** Le diagnostic en trois étapes consiste à mesurer la densité avant fusion, après fusion et après re-sparsification, puis à comparer la distribution des poids retenus avec celle d'un client seul : un décalage du support indique que les sparsifications locales ne sont pas alignées. Le protocole de réessai s'intègre dans la boucle par un test de divergence entre la précision de validation locale et la précision de l'évaluation globale : si l'écart dépasse un seuil pendant deux tours consécutifs, on abaisse la densité cible, on augmente le nombre d'itérations locales, et on relance un tour supplémentaire plutôt que de propager un modèle dégradé.

A retenir

- Le neurone à intégration tient en deux équations : fuite, intégration, seuil, réinitialisation, renvoi.
- L'énergie dépend du taux de tir et du matériel ; elle n'est jamais une propriété du seul modèle.
- L'agrégation des synapses exige une sparsification avant et après fusion, sinon la densité croît à chaque tour.
- L'entraînement sur appareil impose une dérivée de substitution et interdit les hypothèses de grand jeu de données.
- Le choix entre impulsionnel et artificiel se joue autant sur l'écosystème matériel et l'outillage que sur la précision.

Chapitre 6.4

Apprentissage fédéré sur graphes et autres supports de données

Un tableau est un support de données ; un graphe, un flux d'événements, un spectre et un dossier patient sont des supports qui ont leur propre topologie. Changer de support, c'est changer la définition même du client.

Tout ce qui précède suppose que la donnée locale est une matrice. Cette hypothèse n'est pas une commodité : elle détermine ce qu'un client est, ce qu'il peut calculer seul, et ce qu'il a le droit d'envoyer. Sur un graphe, un client peut posséder les nœuds, les arêtes, ou un sous-graphe qui ne commute pas avec les autres ; sur un flux, la frontière temporelle est floue et l'ordre des événements est porteur de sens ; sur un dossier patient, les unités naturelles ne sont ni les lignes ni les patients. Ce chapitre passe en revue les familles de partition, puis les supports non tabulaires, et se termine par le problème de l'apprentissage continu, où l'oubli de l'agent devient une propriété du protocole.

Partition horizontale, partition verticale

Definition – Partition horizontale

Les clients détiennent des enregistrements disjoints et les mêmes attributs. Chaque client peut calculer un gradient complet de la perte globale. C'est le cas historique de la distribution massivement parallèle, et le seul cas où la moyenne pondérée s'applique sans modification.

Definition – Partition verticale

Les clients détiennent des attributs disjoints et les mêmes enregistrements, indexés par un identifiant commun. Aucun client ne peut calculer un gradient complet : il lui manque les attributs des autres. Chaque client calcule un gradient partiel sur ses colonnes, et le serveur — ou un mécanisme de cryptographie homomorphe — recompose le gradient complet.

La partition verticale est intrinsèquement plus coûteuse : le taux de fuite est quasi nul, puisque chaque client n'a que ses propres colonnes, mais la communication est réduite à quelques scalaires par échantillon et par tour, ce qui est un gain réel quand les données sont larges et peu nombreuses. Le coût est en cryptographie et en latence, non en bande passante. Une troisième forme, la partition par graphe, combine les deux : un client peut détenir un ensemble de nœuds avec toutes leurs attributs, un autre un ensemble d'attributs sur tous les nœuds, et les deux situations coexistent généralement dans un même réseau d'organisations.

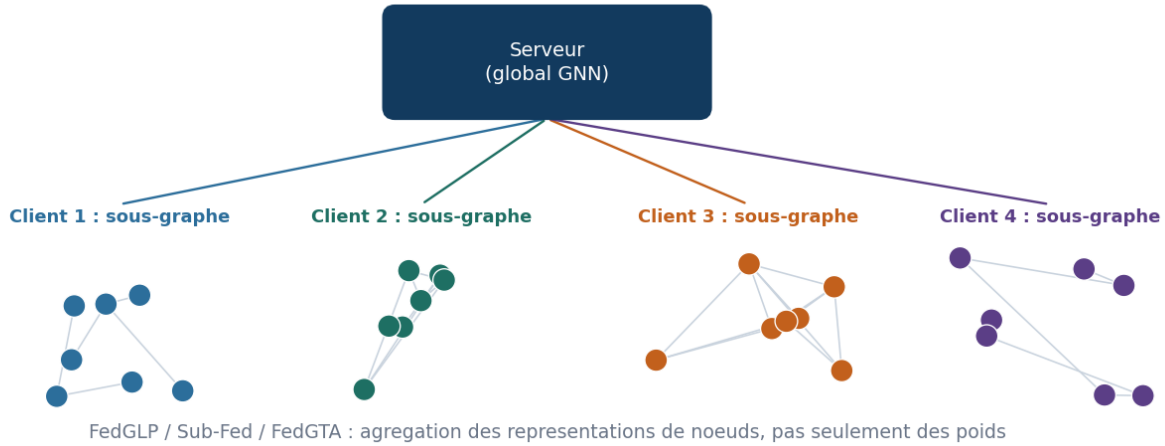


Figure 1 – Partition d'un graphe entre clients et propagation des messages entre sous-graphes locaux : deux nœuds voisins appartiennent rarement au même client

Les méthodes de référence sur graphes

FedGTA agrège des communications de messages plutôt que des poids. Chaque client propage des messages structurels d'un voisinage ego à une profondeur bornée, et le serveur fusionné ces messages avant de les rediffuser. L'idée est que les poids d'un réseau de convolutions sur graphe dépendent de la topologie locale, qui est très différente d'un sous-graphe à l'autre : agréger des poids revient à mélanger des modèles qui n'ont pas la même fonction de perte effective, alors qu'agréger des messages aligne les structures.

FedPUB traité un problème spécifique et très réel : la publication différée d'un nœud central. Sur les plateformes de recommandation et les réseaux sociaux, la topologie locale des centres est un handicap très fort, et FedPUB propose de déplacer le voisinage effectif d'un nœud vers ses voisins publiés — c'est-à-dire les interactions que le nœud a eues avec des éléments publiés — ce qui peut modifier localement la topologie globale d'un graphe de façon contrôlée.

FedCEN combine les deux en conservant le voisinage d'origine pendant la phase d'entraînement, ce qui élimine l'incohérence de topologie introduite par la publication, puis en relâchant la contrainte après

convergence. **FedSage** prend un angle différent et voit l'échantillonnage : au lieu de considérer tous les voisins, il échantillonne un sous-ensemble, ce qui réduit la variance d'agrégation en la faisant ressembler à une moyenne de Monte-Carlo sur la topologie locale.

Méthode	Objet agrégé	Hypothèse de partition	Communication	Limite principale
FedGTA	messages de voisins	nœuds disjoints	$O(\|S\| \cdot d)$	coûteux sur graphes denses
FedPUB	messages, topologie modifiée	nœuds disjoints, cœur publié	$O(\|S\| \cdot d)$	cohérence topologique temporaire
FedCEN	messages, décontrainte progressive	nœuds disjoints	$O(\|S\| \cdot d)$	deux phases, réglage délicat
FedSage	messages échantillonnés	nœuds disjoints	$O(k \cdot d)$	variance si k est petit
Sous-graphes fédérés	sous-graphes compressés	sous-graphes disjoints	élevée	information de frontière perdue
Apprentissage scindé	gradients d'activation	nœuds partagés	moyenne	dérive de classification
Graphe vertical	gradients partiels chiffrés	colonnes disjointes	très faible	coût cryptographique

Tableau 1 – Méthodes fédérées sur graphes : mécanisme et hypothèse

> **Astuce**
> Sur un graphe où les clients sont des institutions et où les arêtes sont les collaborations, on a presque toujours des arêtes inter-clients. Ces arêtes sont la seule vraie difficulté du problème : soit on les traité comme une ressource privée et on devient pessimiste, soit on les matérialise par des gradients échangés, et la quantité d'information transmise devient proportionnelle au nombre de ces arêtes.

Supports non tabulaires

Le multimodal fédéré pose deux problèmes distincts, souvent confondus. Le premier est l'alignement : deux sites peuvent avoir les mêmes modalités mais des taxonomies d'étiquettes différentes, auquel cas il faut une couche de traduction, typiquement un espace latent commun entraîné par un mécanisme d'alignement de distributions. Le second est l'absence : un site peut n'avoir que l'image et pas le texte ; le modèle doit alors être entraîné avec des gradients masqués par modalité, ce qui n'est pas neutre — un gradient nul sur une modalité signifie absence de supervision, non absence de signal.

Le séquentiel ajoute une contrainte de causalité temporelle : on ne peut pas propager une information postérieure vers un état antérieur, sinon on triche. Les données d'usage — échanges, journaux, tracés d'interaction — sont par ailleurs non stationnaires, ce qui rend la notion de distribution locale dépendante de la fenêtre temporelle choisie. Le protocole doit donc définir explicitement la fenêtre, et le serveur doit être informé de son horodatage pour que l'agrégation ne mélange pas une statistique de mars avec une statistique de septembre.

Les séries temporelles sont le cas le plus fréquent, parce que le client n'est pas une institution mais un dispositif. Un capteur de vibration a une fréquence d'échantillonnage, une dérive thermique et des pannes ; sa distribution locale n'est pas un échantillon, c'est une trajectoire. Trois conséquences : le coût d'énergie devient le critère dominant, l'absence devient courante et non exceptionnelle, et l'agrégation doit tolérer des clients qui arrivent avec des calendriers décalés.

Support	Nature du client	Frontière naturelle	Agrégat transmis	Piège dominant
Tableau	institution ou utilisateur	lignes	poids de gradient	hétérogénéité de schéma
Graphe	organisation	nœuds ou arêtes	messages de voisins	arêtes inter-clients
Séquence	application	conversation ou session	états récurrents	fuite temporelle
Série temporelle	dispositif	capteur ou fenêtre	coefficients de modèle	dérive et pannes
Multimodal	institution	modalité manquante	gradients masqués	taxonomies divergentes
Tableau de bord ou texte	service	requête ou lot	adaptateurs	dérive du classifieur
Jumeau numérique	usine	cellule de production	paramètres physiques	modèle de vérité
Dossier patient	service clinique	patient ou séjour	effets causaux	données rares

Tableau 2 – Supports de données et stratégies fédérées

La colonne « agrégat » désigne l'objet mathématique réellement transmis. C'est elle qui détermine le coût et la faisabilité, non l'architecture du modèle.

Apprentissage continu : oubli, rejeu, consolidation élastique

L'apprentissage fédéré continu — on parle aussi d'apprentissage en flux ou streaming — pose un problème que les régimes par lots ignorent : le modèle doit incorporer une information nouvelle sans perdre l'ancienne, alors que l'ancienne n'est plus disponible. Dans un cadre fédéré, ce problème se dédouble : l'oubli interne au client et l'oubli entre clients, ce dernier étant aggravé par le fait que le serveur ne voit jamais les données.

> **Definition – Oubli catastrophique**
> Dégradation mesurable de la performance sur une distribution antérieure lorsque l'apprentissage se poursuit sur une distribution nouvelle. En fédéré, on distingue l'oubli intra-client, dû aux itérations locales, de l'oubli inter-clients, dû à l'impossibilité de rejouer les données d'un site qui n'est pas sélectionné à un tour donné.

$$\text{Oublit} = E_{x \sim \square 3\square_\square 1\square \leq ft[\, L_t(\theta)\,]} - E_{x \sim \square 4\square_\square 2\square \leq ft[\, L_{t-1}(\theta)\,]} \tag{6.8}$$

La mesure suppose un accès à un échantillon de l'ancienne distribution, qui n'existe pas en pur flux : on utilise une mémoire de rejeu ou un échantillon de contrôle conservé volontairement.

Trois mécanismes sont combinés en pratique. Le rejeu consiste à conserver une file bornée d'exemples passés et à les entremêler aux données nouvelles ; son coût est borné par la taille de la file, et son efficacité dépend du taux d'échantillonnage. La régularisation élastique consiste à pénaliser la distance entre le modèle courant et une mémoire de modèles, ce qui limite la dérive sans exiger d'exemples. La consolidation par répétition espacée consiste à réinjecter des tâches anciennes à intervalles décroissants, ce qui reproduit le mécanisme de consolidation mémoire du système nerveux — et c'est la stratégie la plus économe en mémoire.

```python
from collections import deque
from typing import Deque, Dict, List, Tuple

import torch
import torch.nn as nn


def echantilloner_voisinage(graphe, centre: int, taille: int) -> List[int]:
"""Marche aleatoire ego sur le sous-graphe local du client."""
vus, pile, noeuds = {centre}, [centre], [centre]
while len(vus) < taille and pile:
courant = pile.pop()
voisins = (list(graphe.successors(courant))
+ list(graphe.predecessors(courant)))
if not voisins:
continue
suivant = voisins[int(torch.randint(len(voisins), (1,)).item())]
if suivant not in vus:
vus.add(suivant)
pile.append(suivant)
noeuds.append(suivant)
return noeuds


def agreger_agregats(agregats: Dict[int, torch.Tensor],
poids: Dict[int, float]) -> Dict[int, torch.Tensor]:
"""Fusionne les statistiques de noeuds, ponderees par le degre local du client."""
total = sum(poids.values()) or 1.0
return {n: agregats[n] * (poids[n] / total) for n in agregats}


class MemoireDeRejeu:
"""File bornee d'exemples recents : le compromis classique du streaming."""

def __init__(self, capacite: int, taille_lot: int = 32):
self.capacite = capacite
self.taille_lot = taille_lot
self.flux: Deque[Tuple[torch.Tensor, torch.Tensor]] = deque(
maxlen=capacite)

def ajouter(self, lot_x: torch.Tensor, lot_y: torch.Tensor) -> None:
"""Memorise le lot courant ; la file ecrase elle-meme les plus anciens."""
for x, y in zip(lot_x, lot_y):
self.flux.append((x.detach().clone(), y.detach().clone()))

def echantillon(self) -> Tuple[torch.Tensor, torch.Tensor]:
if not self.flux:
raise RuntimeError("memoire vide : aucun lot memorise")
taille = min(self.taille_lot, len(self.flux))
choisis = torch.randint(len(self.flux), (taille,)).tolist()
lots = [self.flux[i] for i in choisis]
return (torch.stack([x for x, _ in lots]),
torch.stack([y for _, y in lots]))


```

Listing 1 — Streaming fédéré : voisinage ego échantillonné, fusion d'agrégats pondérés par le degré, mémoire de rejeu bornée et consolidation espacée

```python
def consolider(modele: nn.Module, memoire: MemoireDeRejeu, optimiseur,
passes: int = 1) -> float:
"""Repetition espacee : on rejoue la memoire entre deux phases d'entrainement."""
perte_totale = 0.0
for _ in range(passes):
x, y = memoire.echantillon()
perte = nn.functional.cross_entropy(modele(x), y)
optimiseur.zero_grad(set_to_none=True)
```

```
perte.backward()
optimiseur.step()
perte_totale += float(perte)
return perte_totale / max(passes, 1)
```

Remarque

La mémoire de rejeu pose un problème de confidentialité quand elle vit au client : elle accumule exactement les exemples que l'on voulait protéger. Le protocole du Département impose que la mémoire soit chiffrée au repos, qu'elle soit purgée à la fin de chaque session, et que sa taille maximale soit inscrite dans la politique de confidentialité du site.

Exercice 6.4.27 (difficulte : Avancé)

Un consortium de dix hôpitaux détient un graphe de correspondances entre patients : les nœuds sont les patients, les arêtes les antécédents familiaux. Aucun nœud ne peut quitter son site, et une minorité de patients existe dans deux sites à la fois.

1. Quelle partition choisissez-vous, et quel est le coût exact de l'arête inter-sites ?
2. Quel protocole préserve la vie privée d'un patient présent dans deux sites ?
3. Expliquez pourquoi l'agrégation de messages est préférable à une agrégation de poids.

Indice : Un nœud partagé entre deux sites est un nœud qui ne peut appartenir qu'à un seul client.

Correction

- **1.** La partition doit être par nœuds, avec une règle d'attribution déterministe et public — par exemple le hachage de l'identifiant pseudonymisé — afin qu'aucun patient ne soit dupliqué. Le coût de l'arête inter-sites est exactement un message par arête et par tour de propagation, portant sur un vecteur de dimension d et non sur une matrice de poids complète : il est donc supportable si le nombre de ces arêtes est faible, ce qu'il faut mesurer avant de s'engager.
- **2.** Le protocole consiste à ne jamais matérialiser le nœud partagé : le calcul se fait sur un vecteur aléatoire jetable par nœud, et les deux sites n'échangent que des produits scalaires intra-scénarios dont la somme reconstitue le résultat, avec vérification croisée des sommes.
- **3.** L'agrégation de messages est préférable parce que la fonction de perte d'un réseau de convolutions sur graphe dépend de la topologie locale : un client qui détient dix nœuds sans arêtes entre eux et un client qui détient un triangle complet n'optimisent pas la même perte, et la moyenne de leurs poids n'a aucun sens. Agréger des messages aligne les structures au lieu de les confondre.

Exercice 6.4.28 (difficulte : Avancé)

Un capteur interroge toutes les 10 ms, avec une dérive d'horloge de 200 parties par million et une coupure réseau de trois heures par jour. On souhaite un modèle de maintenance fédéré sur dix capteurs.

1. Quel protocole d'entraînement tenir dans ce budget ?
2. Comment corriger la dérive d'horloge sans horloge de référence ?
3. Quel indicateur signale que la mémoire de rejeu devient un problème de confidentialité ?

Indice : La continuité temporelle est la propriété que le client perd en premier.

Correction

- **1.** À dix millisecondes de période d'échantillonnage et à trois heures de coupure, chaque capteur accumule $1{,}08 \times 10^{6}$ échantillons par jour, soit environ neuf millions et demi par an : l'historique complet est hors de portée, ce qui impose un protocole d'apprentissage incrémental par fenêtre avec consolidation espacée.
- **2.** La dérive d'horloge se corrige sans horloge de référence par estimation à partir des observations : on note l'horodatage de réception serveur et celui du capteur, et on ajuste un facteur d'échelle par régression linéaire sur une heure, ce qui revient à laisser le serveur jouer le rôle d'horloge de référence implicite.
- **3.** L'indicateur de tension sur la confidentialité est le rapport entre le nombre d'exemples distincts mémorisés et la taille nominale de la file : lorsque ce rapport atteint un, la mémoire ne retiré plus rien au flux récent et ne sert qu'à accumuler des données sensibles ; il faut alors réduire la capacité ou passer à une consolidation sans exemples.

A retenir

- Changer de support change la définition du client, donc le protocole et l'agrégat mathématique.
- La partition horizontale se prête à la moyenne pondérée ; la partition verticale exige une recomposition chiffrée des gradients partiels.
- Sur graphes, agréger des messages de voisins est plus correct que d'agréger des poids, car la perte dépend de la topologie locale.
- L'arête inter-clients est la seule vraie difficulté du graphe fédéré, et son coût se mesure en messages par tour.
- En apprentissage continu, l'oubli se combat par un rejeu borné, une régularisation élastique et une consolidation espacée — jamais par conservation illimitée.

Chapitre 6.5

Programme pédagogique sur douze semestres et feuille de route

Un programme d'enseignement n'est pas une liste de cours : c'est une promesse de compétences, datée, vérifiable, et cohérente avec une recherche qui a ses propres échéances.

Ce chapitre n'est pas un supplément culturel : c'est le document qui garantit que les théorèmes des chapitres précédents seront réinvestis par quelqu'un. Le programme du Département R&D est organisé en six blocs de deux semestres, du prérequis de mathématiques aux problèmes de déploiement. Chaque semestre est décrit par un thème, des unités d'enseignement, un volume de crédits, une modalité d'évaluation et un livrable vérifiable — un rapport technique, un prototype, un article soumis, un jeu de données annoté. La feuille de route de recherche 2026-2030 articule ensuite ces compétences à trois axes de recherche et à neuf jalons datés.

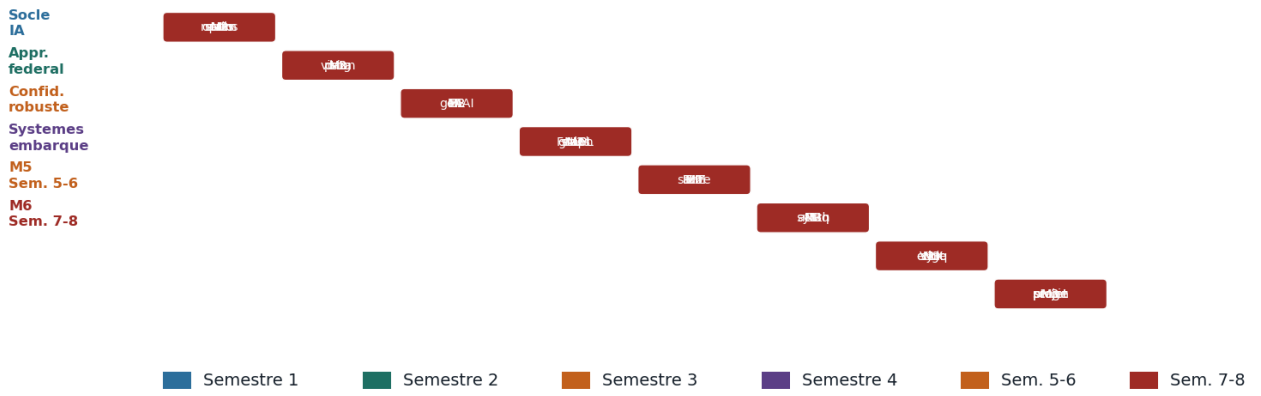


Figure 1 – Cartographie du programme sur douze semestres : six blocs, progression des prérequis mathématiques vers les problèmes de déploiement et de conformité

Architecture du programme en six blocs

1. **Bloc 1 — Socle théorique, semestres 1 et 2** : probabilités, optimisation, statistique appliquée, programmation scientifique. Le livrable de fin de bloc est un réexamen complet d'un article de référence, avec reproduction chiffrée de ses résultats principaux.
2. **Bloc 2 — Apprentissage automatique profond, semestres 3 et 4** : réseaux de convolution,

transformeurs, méthodes d'optimisation, mise en production. Livrable : une chaîne de traitement complète, testée sur données réelles, avec rapport exploratoire.

3. **Bloc 3 — Systèmes distribués, semestres 5 et 6** : systèmes distribués, réseaux, bases de données, parallélisme, tolérance aux pannes. Livrable : un système répliqué tolérant aux fautes, déployé sur un banc d'essai de trois nœuds.
4. **Bloc 4 — Apprentissage fédéré, semestres 7 et 8** : cœur du programme. Communication efficace, hétérogénéité, sécurité, applications verticales. Livrable : un banc d'essai fédéré multi-clients avec protocole de sécurité actif.
5. **Bloc 5 — Fronts avancés, semestres 9 et 10** : modèles de langage, graphes, inférence causale, apprentissage continu, réseaux impulsionnels. Livrable : un article de recherche soumis à une conférence.
6. **Bloc 6 — Déploiement et conformité, semestres 11 et 12** : industrialisation, conformité réglementaire, éthique, impact environnemental, entrepreneuriat technologique. Livrable : un dossier d'industrialisation complet et une soutenance devant un jury industriel.

Sem.	Thème	Unités d'enseignement principales	Crédits	Évaluation	Livrable
S1	Mathématiques du signal	Probabilités, algèbre linéaire, calcul différentiel	30	Examen écrit 2 h	Travaux dirigés corrigés en Python
S2	Statistique appliquée	Inférence, estimation, modèle linéaire, reproductibilité	30	Projet 20 %	Note de reproduction d'un article
S3	Apprentissage profond I	Convolution, régularisation, optimisation	30	Examen et TP noté	Réseau convolutif écrit à la main
S4	Apprentissage profond II	Transformeurs, vision, multimodalité	30	Projet 30 %	Chaîne de traitement texte multimodal
S5	Systèmes distribués	Consensus, réplication, partitionnement	30	Examen écrit 2 h	Service tolérant aux fautes
S6	Réseaux et nuage	Protocoles, orchestration, conteneurs	30	Projet 25 %	Banc à trois nœuds orchestré
S7	Apprentissage fédéré I	Moyenne pondérée, communication, hétérogénéité	30	Examen et TP noté	Banc fédéré instrumenté
S8	Apprentissage fédéré II	Sécurité, confidentialité, attaques	30	Projet 30 %	Audit de sécurité du banc
S9	Fronts de recherche	Contraste, graphes, causalité, apprentissage continu	30	Article soumis 40 %	Communication à une conférence
S10	Efficacité et énergie	Quantification, compression, réseaux impulsionnels, mesure	30	Projet 30 %	Dossier d'analyse énergétique
S11	Industrialisation	Intégration continue, conformité, souveraineté	30	Projet 35 %	Dossier d'industrialisation
S12	Éthique et mémoire	Éthique, double usage, soutenance finale	30	Soutenance 100 %	Mémoire de fin d'études

Tableau 1 – Progression détaillée sur douze semestres

Les crédits sont exprimés en ECTS. La progression est cumulative : aucun semestre ne suppose une compétence non acquise dans un bloc antérieur.

> **Astuce**
>
> Le contrôle de progression se fait sur les livrables, pas sur les notes. Un étudiant qui a obtenu de bonnes notes mais n'a jamais livré de prototype reproductible ne valide pas le bloc 4, quel que soit son cumul de points. C'est la seule façon d'éviter que le programme dérive vers de la théorie spéculative au détriment de la capacité d'ingénierie.

Feuille de route de recherche 2026-2030

Trois axes structurent la feuille de route du Département. Ils sont volontairement perpendiculaires : l'axe A travaille sur la nature de l'objectif, l'axe B sur la nature du support, l'axe C sur la chaîne de production. Un résultat qui améliore les trois est rare et constitue la ciblé des collaborations entre doctorants.

- **Axe A — Objectifs et garanties** : alignement des espaces en apprentissage contrastif fédéré, inférence causale sous contrainte de confidentialité, bornes de convergence en régime non échangeable. La question directrice est « que peut-on prouver quand personne ne peut montrer ses données ? ».
- **Axe B — Supports et structures** : graphe, multimodal, séquentiel, série temporelle, impulsionnel. La question directrice est « que change la topologie du support pour le protocole d'agrégation ? ».
- **Axe C — Production et coût** : sparsification, quantification, énergie, conformité, reproductibilité. La question directrice est « combien coûte un point de précision, en joules et en euros, une fois toutes les garanties respectées ? ».

Jalon	Échéance	Axe	Livrable attendu	Critère de réussite
J1	T1 2027	A	Banc contrastif fédéré instrumenté	mesure d'alignement publiée
J2	T3 2027	C	Profilage énergétique normalisé	protocole de mesure documenté
J3	T1 2028	A	Estimateur d'effet fédéré validé sur cinq sites	essai clinique démontré
J4	T3 2028	B	Suite de méthodes sur graphes	tableau comparatif reproductible
J5	T1 2029	B	Prototype neuronal impulsionnel embarqué	gain énergétique mesuré
J6	T3 2029	C	Chaîne de publication reproductible	100 % des articles avec artefacts
J7	T1 2030	A	Théorie de la convergence en régime non échangeable	article de théorie soumis
J8	T3 2030	C	Dossier de conformité automatisé	analyse d'impact reproductible
J9	T4 2030	B	Bilan consolidé et feuille de route 2031-2035	rapport public du Département

Tableau 2 – Jalons de la feuille de route 2026-2030

Chaque jalon a un critère de réussite observable. Un jalon atteint sans son critère est considéré comme non atteint, et le jalon est reprogrammé.



Figure 2 – Feuille de route de recherche 2026-2030 : trois axes, neuf jalons, avec les dépendances entre chantiers

Conduire un jalon de recherche

- **Cadrage** : une page, poser le problème, le critère de réussite mesurable, et les deux raisons pour lesquelles l'approche retenue pourrait échouer.
- **Risque** : identifier l'hypothèse la plus fragile et la tester en premier, avec un protocole de test qui ne coûte pas plus de deux semaines.
- **Déroulé** : un plan de travail en lots de deux semaines, chacun avec un critère d'acceptation binaire.
- **Traçabilité** : tout résultat est écrit dans un carnet de laboratoire versionné, avant toute présentation.
- **Clôture** : la décision de suite — poursuivre, pivoter ou abandonner — est prise explicitement et inscrite au bilan de jalon.

Remarque

Un jalon sans critère de réussite binaire est un jalon qui ne finit jamais.

Cadre juridique et ethique

Éthique de la recherche et consentement. Toute expérimentation impliquant des personnes, des données de santé ou des dispositifs de comportement requiert un avis du comité d'éthique et un consentement éclairé spécifique qui mentionne explicitement l'usage fédéré, l'absence de centralisation, la durée de conservation des mises à jour de modèle, l'existence de mises à jour ultérieures non réidentifiantes, et le droit de retrait. Le consentement n'est pas un formulaire : c'est un document qui doit pouvoir être compris par un participant non technique, être redonné si la finalité change, et être conservé pendant toute la durée du traitement. Le Département refuse tout protocole dont le consentement ne mentionne pas explicitement le risque de réidentification par croisement avec une source externe, même lorsque les techniques de protection sont en place.

Exercice 6.5.29 (difficulte : Moyen)

Concevoir le semestre 8, apprentissage fédéré II, 30 ECTS, en respectant toutes les contraintes du programme : présence des unités attendues, volume horaire réaliste, évaluation sommative, livrable vérifiable, prérequis acquis au semestre 7.

1. Proposer un découpage hebdomadaire de 14 heures sur 12 semaines.
2. Justifier le choix de l'évaluation : pourquoi pas un examen écrit ?
3. Définir les critères de validation du livrable, en trois points notés.

Indice : Une unité de sécurité sans audit pratique n'a jamais formé personne.

Correction

- **1.** Le découpage sur douze semaines alterne : six séances de cours magistral de deux heures chacune, six ateliers de laboratoire de quatre heures, et une session de restitution de deux heures, ce qui totalise $6 \times 2 + 6 \times 4 + 2 = 38$ heures encadrées ; les 112 heures restantes sont du travail personnel dirigé, ce qui donne bien 150 heures pour 30 ECTS selon la conversion de dix heures par ECTS propre au programme. **2.** L'évaluation est un projet à 30 % plutôt qu'un examen écrit pour trois raisons : la matière est par nature transversale et ne se laisse pas.reduce à des questions à choix multiple ; les compétences visées sont pratiques et ne se démontrent pas à l'écrit ; et le programme évalue explicitement la capacité à établir une matrice d'attaques, ce qui suppose un banc d'essai.
- **3.** Le livrable est un rapport d'audit de sécurité de quatre pages, un code d'attaque implémenté sur un banc de trois clients, et un correctif déployé. La validation porte sur trois points notés : qualité de la modélisation de la menace, un point ; reproductibilité de l'attaque et du correctif, un point ; clarté de l'analyse de risque résiduel, un point.

Exercice 6.5.30 (difficulte : Avancé)

Le jalon J3, estimateur d'effet fédéré, prévu au premier trimestre 2028, doit être démontré sur cinq sites cliniques, mais aucun site n'a encore conclu de contrat de partage de données.

1. Proposer une séquence de jalons intermédiaires sur douze mois.
2. Quel livrable produit chaque étape, et qui le porte ?
3. Quel est le critère d'abandon de la feuille de route, écrit à l'avance ?

Indice : Un jalon qui dépend d'un tiers doit être découpé en jalons qui n'en dépendent pas.

Correction

- **1.** La séquence se découpe en quatre étapes. Premier trimestre : développement de l'estimateur et preuve de concept sur données synthétiques, sans dépendance externe ; le livrable est la bibliothèque de référence, portée par le Département, avec un jeu de tests de simulation. Deuxième trimestre : étude de faisabilité sur données rétrospectives d'un seul site, avec un protocole d'anonymisation validé par le comité local ; le livrable est un rapport de faisabilité, porté par le site partenaire. Troisième trimestre : signature du consortium, protocole d'analyse statistique préenregistré, comité d'éthique saisi ; le livrable est l'accord de consortium, porté par la direction scientifique. Quatrième trimestre : exécution et analyse, portée conjointement.
- **2.** Chaque étape est portée par une entité différente : le Département pour la première, le site partenaire pour la deuxième, la direction scientifique pour la troisième, les deux conjointement pour la quatrième.
- **3.** Le critère d'abandon doit être écrit avant la première étape et porter sur une condition vérifiable : par exemple, si à la fin du deuxième trimestre aucune étude de faisabilité n'a pu être menée sur au moins un site avec un comité local ayant rendu un avis, le jalon est reprogrammé sur l'année suivante plutôt que poursuivi sur un consortium réduit. Un critère d'abandon non écrit à l'avance est toujours une décision de confort prise au pire moment.

A retenir

- Le programme est structuré en six blocs de deux semestres, avec des livrables vérifiables et non des seules sommes de notes.
- Chaque semestre est spécifié par thème, unités, crédits, modalité d'évaluation et livrable, ce qui rend le parcours auditable.
- La feuille de route 2026-2030 repose sur trois axes perpendiculaires et neuf jalons datés, chacun avec un critère de réussite observable.
- Un jalon atteint sans son critère est reprogrammé, pas déclaré.
- Le consentement fédéré est un document à part entière, qui doit nommer l'usage, la durée, le risque de réidentification et le droit de retrait.

Chapitre 6.6

Publication, reproductibilité et éthique

Un résultat qui n'est pas reproductible n'est pas un résultat : c'est une anecdote avec un tableau. Ce chapitre décrit la chaîne complète, du cadrage à l'artefact, et ce qui se passe quand elle est rompue.

Publier en apprentissage fédéré impose une discipline supplémentaire : le lecteur ne peut pas rejouer l'expérience, parce qu'il n'a pas les données. Il peut seulement vérifier la cohérence interne du protocole, la sensibilité aux choix d'implémentation, et la solidité des conclusions face aux alternatives. Cette situation produit une famille spécifique d'erreurs — la simulation trop favorable, l'hyperparamètre choisi après coup, la charge de communication comparée de façon asymétrique — qu'un protocole d'écriture rigoureux élimine presque toutes.

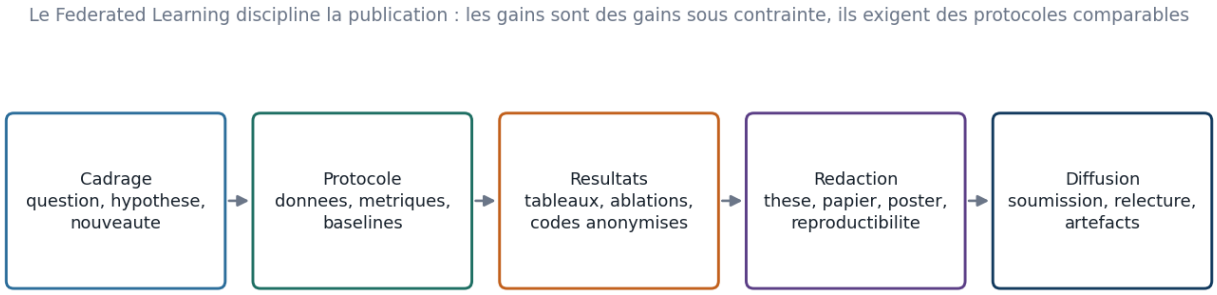


Figure 1 — Chaîne de publication : du cadrage à l'artefact, avec les points de contrôle de reproductibilité et les jalons de soumission

La démarche de publication en quatre temps

Le cadrage précède l'écriture et le conditionné. Il consiste à formuler la question en une phrase, à désigner le comparatif honnête — le modèle de référence, la méthode de référence — et à écrire la phrase « cette expérience échouerait si ». Une question mal cadrée produit un article qui accumule des résultats au lieu d'en tirer une réponse, et l'accumulation se voit immédiatement à la lecture : six tableaux, aucune thèse.

Le protocole est le document qui décrit ce qu'un lecteur devrait faire pour obtenir les mêmes chiffres. En apprentissage fédéré, il doit préciser au minimum : la loi de tirage des clients, le nombre de tours, la taille des lots locaux, la graine aléatoire, la règle de sélection des tours évalués, et la façon dont est calculé le nombre de communications. Ce dernier point est le plus souvent omis et le plus souvent contesté.

Les résultats se présentent par hypothèse et non par tableau. Trois figures suffisent presque toujours : la courbe d'apprentissage, la courbe du compromis qualité-communication, et le tableau de comparaison avec les méthodes de référence sur un jeu protocole. La dispersion doit être donnée sur au moins cinq graines, avec la médiane et non la moyenne lorsque la distribution est asymétrique, et les points individuels doivent être visibles.

La rédaction suit la structure de l'article et non celle du raisonnement interne. La première figure doit pouvoir être comprise seule. La première phrase de l'introduction énonce le problème, la dernière énonce la réponse. Aucune abréviation n'est introduite sans être développée, et aucune affirmation chiffrée n'est produite sans citation ou sans référence à une expérience de cet article.

Algorithme 9 — Protocole de cadrage d'un article

```
Entrées : question de recherche, jeux de données disponibles, contraintes institutionnelles, budget de calcul
1. Écrire la question en une phrase, sans adjectif et sans référence à une méthode particulière
2. Désigner le comparatif honnête : quelle méthode existante doit être battue, et avec quel budget de communication égal
3. Écrire la phrase d'échec : « ce travail serait invalidé si »
4. Lister les expériences minimales, chacune attachée à une hypothèse distincte
5. Estimer le coût total, en heures de calcul, en communications et en jours-homme
6. Décider : si le coût dépasse le budget, réduire le périmètre plutôt que l'hypothèse
7. Enregistrer le cadrage dans le dépôt, avec sa date
```

Entrées : question, données, contraintes, budget • **Sorties :** note de cadrage datée, versionnée, jamais modifiée après coup • **Complexité :** Une journée de travail, gain espéré : un article au lieu de six tableaux

Un cadrage écrit puis modifié invalide la comparaison de façon silencieuse ; c'est pourquoi il est versionné et daté avant toute expérience.

Attention

Le choix après coup du taux de participation des clients, du nombre de tours ou de la graine aléatoire est la forme la plus courante et la plus grave de fraude scientifique. Elle est facile à détecter en relisant la section expérimentale : si le nombre de tours varie d'un tableau à l'autre, si le taux de participation n'est pas annoncé avant les résultats, ou si la dispersion n'est rapportée que là où elle arrange, la conclusion est fragilisée.

Checklist de reproductibilité

La checklist ci-dessous est celle appliquée par le Département avant tout dépôt. Elle est volontairement courte : une checklist de trente points n'est jamais appliquée intégralement.

Reproductibilité : contrôle avant dépôt

- **Code** : dépôt public ou archive figée, identifiant de version cité dans l'article, dépendances épinglées avec versions exactes.
- **Graines** : toutes les graines aléatoires annoncées et fixées ; au moins cinq exécutions par configuration, points individuels publiés.
- **Environnement** : version de l'interpréteur, du cadre, de l'accélérateur ; commande complète de lancement fournie et testée sur une machine vierge.
- **Données** : script d'acquisition ou identifiant d'accès, schéma documenté, description des partitions fédérées simulées ou réelles.
- **Licences** : code, données et modèle sous licences explicites et compatibles.
- **Artefacts** : journaux d'exécution bruts, points de contrôle intermédiaires, métriques brutes conservés et accessibles.
- **Coût** : nombre de tours, nombre de clients sélectionnés, volume total de communication — pour la méthode de référence comme pour la vôtre.
- **Négatifs** : configurations qui ont échoué et n'apparaissent pas dans l'article, conservées dans les artefacts.

Astuce

L'artefact le plus utile n'est pas le dépôt de code mais le journal d'exécution brut : horodaté, non réécrit, il prouve ce qui a réellement tourné. Il protège aussi l'auteur contre sa propre mémoire trois mois plus tard.

```
# Reproduction d'un résultat publié : squelette de dépôt
# (langage : shell ; à placer dans un fichier "reproduce.sh")

set -euo pipefail # arrêter à la première erreur, variables non définies

# 1. Environnement : versions exactes, vérifiées puis verrouillées
python --version
python -c "import torch; print('torch', torch.__version__, 'cuda', torch.version.cuda)"

# 2. Données : téléchargement ou régénération déterministe
python -m scripts.prepare_data --source "$DATA_DIR" --out data/

# 3. Exécution : une commande par configuration, graine explicite
for seed in 0 1 2 3 4; do
python -m main \
--algo fedavg \
--clients 100 --rounds 200 --participation 0.1 \
--local-epochs 2 --seed "${seed}" \
--out "runs/fedavg_seed${seed}"
done

# 4. Agrégation : médiane et intervalle interquartile, jamais la moyenne seule
python -m scripts.aggregate runs/ --metric accuracy --mode median_iqr

# 5. Archivage : tout ce qui est nécessaire à une relecture
tar -czf artefact.tar.gz runs/ data/ figures/ environment.txt
sha256sum artefact.tar.gz > artefact.tar.gz.sha256
```

Listing 1 — Squelette de script de reproduction : environnement, données, graines, agrégation et archivage avec empreinte

Revue par les pairs, prépublications et openness

La prépublication est un outil de validation et non de communication : elle permet de demander une revue informelle de la méthode avant d'immobiliser dix mois de travail sur un cadrage erroné. Elle a un coût : en évaluant la qualité d'une prépublication comme celle d'un article publié, ce qui est une erreur courante dans les dossiers de carrière. La règle du Département est simple — une prépublication porte la mention explicite « prépublication, non évaluée par les pairs », et aucun document officiel ne la présente comme un article.

La science ouverte va au-delà : publication en libre accès, données ouvertes quand elles le peuvent, code ouvert, et évaluation ouverte des autres. Chaque niveau a un coût réel — le code ouvert expose les faiblesses avant qu'un relecteur ne les trouve, ce qui est désagréable mais préférable au scénario inverse. La position du Département est que la reproductibilité est un avantage compétitif : un lecteur qui reproduit vos résultats devient un collaborateur, alors qu'un lecteur qui ne les reproduit pas ne vous citera pas une seconde fois.

Venue	Domaine	Cycle typique	Délai avant	Particularité
NeurIPS	apprentissage automatique général	résumé en mai, texte en juillet	3 à 5 mois	double aveugle, révisions principales et de fond
ICML	apprentissage automatique	résumé en janvier, texte en mars	4 mois	double aveugle, très sélectif
ICLR	apprentissage profond	résumé en septembre, texte en octobre	5 mois	évaluation ouverte, discussion public
AISTATS	statistique et apprentissage	résumé en octobre, texte en décembre	5 mois	communauté plus statistique, causale bien accueillie
Atelier d'apprentissage fédéré	apprentissage fédéré	résumé en mai	2 à 3 mois	atelier, très adapté aux travaux de fédération
MLSys	systèmes et machines	toutes l'année	3 mois	fort accent sur la mesure et le coût
SenSys ou NSDI	systèmes	toutes l'année	4 mois	exigence de reproduction forte
Prépublication	toutes disciplines	permanent	sans délai	non évaluée, à mentionner explicitement

Tableau 1 – Conférences de référence et jalons de soumission

Les cycles évoluent chaque année : vérifier systématiquement les dates sur le site de la conférence au moins six mois avant la soumission.

Éthique de la publication et de la recherche

Cinq questions éthiques reviennent dans chaque dossier, et chacune a une réponse opérationnelle plutôt qu'une déclaration de principe.

- **Biais.** Un protocole fédéré multi-sites produit naturellement des résultats biaisés vers la population des sites les mieux instrumentés. Réponse : rapporter systématiquement la composition des sites, publier les performances par site et non seulement la moyenne, et effectuer une analyse de sensibilité à l'exclusion de chaque site tour à tour.
- **Énergie.** Un gain de précision de 0,5 point au prix de dix fois plus d'énergie d'entraînement n'est pas un progrès dans le contexte actuel. Réponse : rapporter systématiquement le coût énergétique, converti en coût monétaire et en équivalent carbone, sur le cycle complet et non sur l'entraînement seul.
- **Surveillance.** Un modèle fédéré qui détecte l'inactivité ou le niveau d'activité d'un poste de travail, d'un véhicule ou d'un compteur devient un instrument de surveillance, quelle que soit l'intention. Réponse : documenter explicitement la fenêtre d'observation, la granularité temporelle, et le fait que le modèle ne produit aucune donnée nominative mais conserve malgré tout un pouvoir de profilage.
- **Transparence.** Un modèle dont l'erreur est systématiquement sous-estimée sur un sous-groupe est plus dangereux qu'un modèle visiblement faux. Réponse : publier les métriques par sous-groupe, et publier aussi les cas d'échec.
- **Double usage.** Un travail publié peut être repris pour construire un système de surveillance à grande échelle ou un mécanisme d'empoisonnement automatisé. Réponse : ne pas publier de code offensif « par principe », documenter les limites de l'usage prévu dans l'article lui-même, et refuser la contribution d'un travail dont l'objet principal est l'attaque.

Cadre juridique et ethique

Propriété intellectuelle. Trois questions structurent le dépôt d'un résultat de recherche : la titularité de l'invention, la portée de la licence du code associé, et la réciprocité des données. La titularité suit le contrat de recherche et la législation applicable : dans un cadre de collaboration, le Département revendique la copropriété des inventions non publiées et la cession des droits d'exploitation pour la formation. La licence du code doit être choisie avant le dépôt : une licence restrictive, par exemple une licence à usage interne, interdit toute reproduction académique et décourage la citation. Le Département utilisé par défaut une licence permissive de type BSD à trois clauses pour le code de recherche, et une licence de données qui impose la citation et interdit l'usage discriminatoire. Quant à la question des données, la règle est qu'un jeu publié est un actif dont l'hébergeur acquiert des droits d'hébergement qu'il faut documenter, et qu'un jeu de données de santé ne peut être rouvert sans un nouvel avis du comité d'éthique.

Exercice 6.6.31 (difficulte : Avancé)

Un relecteur reproche à un article fédéré : « la méthode de référence est comparée avec dix fois moins de communications, et les graines ne sont pas annoncées ». Rédigez la réponse, puis énoncez les trois corrections à apporter au protocole.

1. Quelle est la réponse défendable sur la comparaison de communication ?
2. Comment démontrer l'équivalence des budgets de communication ?
3. Quelles sont les trois corrections, dans l'ordre de priorité ?

Indice : Une accusation de comparaison asymétrique ne se réfute pas : elle se prouve par une mesure supplémentaire.

Correction

- **1.** La réponse défendable ne consiste pas à défendre le chiffré publié mais à produire la mesure manquante : tracer la courbe d'exactitude en fonction du volume total de communication échangée, en kilo-octets et non en nombre de tours, pour la méthode de référence et pour la méthode proposée. Si les deux courbes se croisent, l'article est sauvé et gagne même en convaincance ; si elles ne se croisent pas, la conclusion doit être affaiblie et l'avantage revendiqué reformulé.
- **2.** L'équivalence des budgets se démontre en convertissant chaque échange en octets — ce qui suppose de déclarer la précision de sérialisation, la taille de lot et la proportion de clients sélectionnés — puis en reportant sur le même axe.
- **3.** Les trois corrections, par ordre de priorité : ajouter les graines et l'ensemble des points individuels aux figures, ajouter la courbe qualité-communication en octets, enfin publier le journal d'exécution brut qui prouve les volumes réels. Une quatrième correction, moins urgente mais réelle, consiste à préciser dans la section protocole la règle de sélection des clients, qui conditionné le volume échangé.

Exercice 6.6.32 (difficulte : Avancé)

Un modèle de détection d'activité de véhicule est entraîné en fédéré chez onze loueurs. Il atteint 94 % de précision globale mais 61 % sur les véhicules conduits de nuit, et son opérateur n'utilise principalement que ce sous-groupe.

1. Identifier les quatre implications éthiques distinctes.
2. Rédiger le paragraphe de la section « considérations éthiques ».
3. Quelle obligation réglementaire est ici la plus contraignante ?

Indice : La précision globale masque presque toujours une rupture de parité sur un sous-groupe.

Correction

- **1.** Quatre implications se dégagent. La première est technique : le modèle est inutilisable pour son usage réel, ce qui est une question d'utilité et non d'éthique, mais qui doit être traitée comme telle. La deuxième est économique : la performance faible sur le sous-graphe majoritaire dans l'usage conduit à un risque majoré sur ce sous-groupe, donc à un avantage indu au bénéfice de l'opérateur. La troisième est juridique : une décision d'habilitation ou de désactivation prise sur la base de ce modèle est une décision automatisée prise au sujet d'une personne, ce qui déclenche les obligations d'information, de possibilité de contestation et, dans le cadre européen, l'analyse d'impact requise pour tout système à haut risque. La quatrième est méthodologique : rapporter une précision globale dans un article, sans métrique par sous-groupe, est une omission qui induit en erreur.
- **2.** Le paragraphe d'éthique doit nommer le sous-groupe, donner les métriques par sous-groupe, expliquer la cause probable — ici la sous-représentation nocturne dans les données d'entraînement, aggravée par l'hétérogénéité des clients — et annoncer les mesures correctives.
- **3.** L'obligation la plus contraignante est l'analyse d'impact relative au traitement des données à caractère personnel : sans elle, le déploiement est irrégulier quelle que soit la qualité technique du modèle.

A retenir

- Le cadrage précède l'écriture et décide si l'article aura une thèse ou six tableaux.
- La reproductibilité se contrôle sur un nombre réduit de points : code épinglé, graines annoncées, environnement verrouillé, coût en octets, artefacts bruts.
- Le nombre de communications se mesure en octets échangés, pas en tours de synchronisation.
- Les métriques par sous-groupe ne sont pas une option : c'est la seule mesure qui révèle la rupture de parité.
- Éthique et propriété intellectuelle se traitent par des écritures opérationnelles — analyse d'impact, licence choisie avant dépôt, paragraphe de considérations éthiques rédigé et non improvisé.

Du protocole de laboratoire au mémoire de fin de semestre

Partie 7

Travaux pratiques, projets et évaluation

Les six parties précédentes ont établi des résultats : des bornes, des algorithmes, des mécanismes de défense, des architectures. Cette partie les transforme en gestes. Un travail pratique n'est pas un exercice d'illustration : c'est une expérience contrôlée dont la reproductibilité, le protocole et la restitution font partie du résultat scientifique. On y apprend à poser une hypothèse, à la mettre en défaut par une expérience, et à distinguer ce que la mesure établit de ce que l'intuition suggère.

Objectifs de la partie

- Mettre en œuvre la moyenne pondérée, ses variantes et leurs propriétés mesurées.
- Construire un protocole d'expérimentation fédéré reproductible.
- Quantifier un coût de communication et une garantie de vie privée.
- Reproduire une attaque, la défendre, puis mesurer le coût de la défense.
- Conduire un projet de recherche appliquée de six à huit semaines.
- Auto-évaluer sa maîtrise à l'aide d'une grille chiffrée et partagée.

Plan de la partie

1. Cadre des travaux pratiques : méthode, environnement, conventions, évaluation
2. Travaux pratiques guidés : neuf séances de mise en œuvre progressive
3. Projets de fin de semestre : cinq sujets de recherche appliquée
4. Exercices d'entraînement : quinze problèmes corrigés
5. Évaluation, certification et ressources

Chapitre 7.1

Cadre des travaux pratiques

Un laboratoire n'est pas une salle avec des ordinateurs : c'est un dispositif qui transforme une question en une mesure que deux personnes peuvent vérifier. Ce chapitre fixe ce dispositif.

L'enseignement de l'apprentissage fédéré présente une difficulté spécifique : la majorité des concepts ne sont pas démontrables sur un tableau noir. La dérive du modèle, le coût de communication, la fuite d'information par le gradient, le compromis entre utilité et vie privée sont des propriétés *émergentes* d'un système distribué. Elles n'apparaissent que lorsque le système tourne, avec ses imperfections. Un cours qui se limiterait aux énoncés de théorie laisserait l'essentiel de la discipline hors de la portée des étudiants.

Méthode pédagogique : de l'hypothèse à la preuve

Chaque séance suit le même protocole, adapté à la durée de deux à quatre heures. Ce protocole est celui de l'expérience scientifique, resserré pour tenir dans une demi-journée de poste de travail.

1. **Formuler.** Écrire en une phrase l'hypothèse testable, sous la forme « si je réalise X, alors la métrique Y doit s'améliorer de Z % ». Une hypothèse non quantifiée n'est pas testable ; une hypothèse sans « si » n'est pas une hypothèse mais un constat.
2. **Figer le protocole.** Écrire la liste des variables de contrôle avant la première exécution : graines aléatoires, nombre de clients, nombre de tours, taille des lots, version des bibliothèques. Un protocole écrit après coup est un protocole inventé.
3. **Implanter la référence.** Écrire d'abord la version la plus simple qui puisse fonctionner : la moyenne pondérée centralisée sur une seule machine, sans agrégation distribuée réelle. Toute déviation par rapport à cette référence doit être expliquée par un test, jamais par une intuition.
4. **Mesurer une seule fois.** Lancer l'expérience, enregistrer la sortie brute dans un fichier texte, puis n'analyser que ce fichier. Relancer le code pour obtenir un meilleur chiffré est la faute expérimentale la plus courante.
5. **Répéter avec variation.** Changer une seule variable à la fois, sur au moins trois graines, et rapporter une moyenne et un écart-type. Un résultat sans répétition n'est pas un résultat, c'est une anecdote.
6. **Restituer.** Un livrable contient : la question, le protocole, la commande exacte de lancement, les résultats bruts, l'analyse, et ce que l'expérience ne prouve pas.

Le protocole en six gestes

- Formuler une hypothèse quantifiée.

2. Figer graines, clients, tours et versions avant de lancer.
3. Implanter la référence centralisée la plus simple possible.
4. Exécuter une fois, conserver la sortie brute.
5. Répéter sur trois graines, rapporter moyenne et écart-type.
6. Distinguer explicitement ce qui est mesuré de ce qui est supposé.

- Règle : si l'expérience ne peut pas être rejouée en une commande, elle n'est pas une expérience.

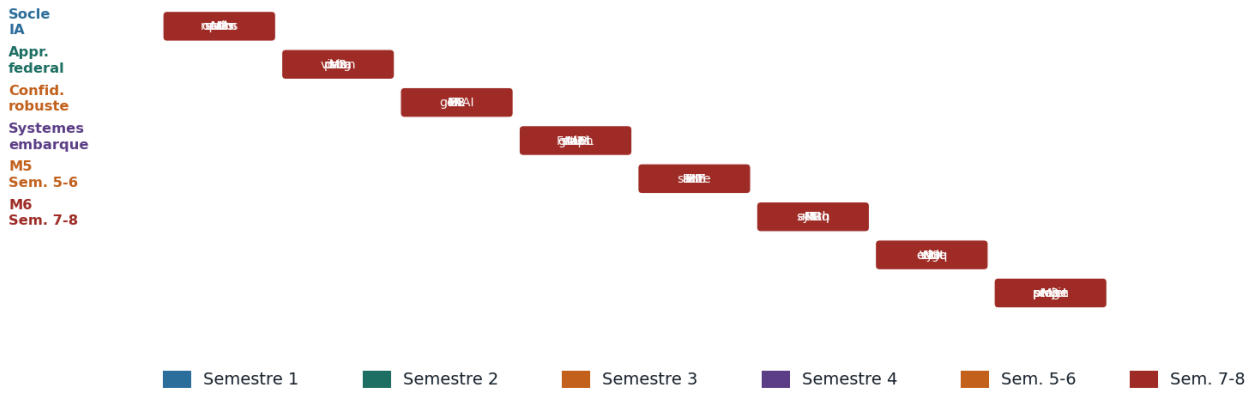


Figure 1 — Positionnement des travaux pratiques dans la progression du cours

Environnement de travail et conventions de dépôt

L'environnement est fixé une fois pour toutes : c'est la seule manière d'obtenir des résultats comparables d'une séance à l'autre. Toute divergence de version de bibliothèque suffit à déplacer une courbe d'exactitude de plusieurs points, ce qui rendrait toute comparaison invalide.

```bash
#!/usr/bin/env bash
# Installation de l'environnement de travail du cours.
# A executer une seule fois, depuis la racine du depot.
set -euo pipefail

VENV=".venv"
PY_MIN="3.10"

# 1. Verification de la version de Python : les API ont change depuis 3.10.
if ! python3 -c 'import sys; sys.exit(0 if sys.version_info >= (3, 10) else 1)'; then
echo "Python >= $PY_MIN est requis" >&2
exit 1
fi

# 2. Environnement virtuel isole : aucune dependance systeme.
python3 -m venv "$VENV"
# shellcheck disable=SC1091
source "$VENV/bin/activate"
python -m pip install --upgrade pip setuptools wheel

# 3. Dependances verrouillees.
pip install -r requirements.txt

# 4. Reproductibilite : graines et variables d'environnement figees.
export PYTHONHASHSEED=0
export MPLBACKEND=Agg # pas de fenetre graphique sur le serveur
export OMP_NUM_THREADS=4 # determinisme et evite la surcharge CPU

# 5. Verification finale.
python - <<'VERIF'
import torch, numpy, flwr
print("torch", torch.__version__, "| numpy", numpy.__version__,
"| flwr", flwr.__version__)
VERIF
echo "Environnement pret. Rejouer 'source $VENV/bin/activate' a chaque session."
```

Listing 1 — Préparation reproductible de l'environnement de travail

```
# Verrouillage de l'environnement : versions testees et figees.
# Installation : pip install -r requirements.txt
numpy==1.26.4
scipy==1.11.4
scikit-learn==1.4.0
pandas==2.2.1
matplotlib==3.8.3
seaborn==0.13.2
# Socle numerique
torch==2.2.2
torchvision==0.17.2
# Apprentissage federatede
flwr==1.5.0
# Apprentissage differentiel
opacus==1.5.2
# Adaptation efficace des grands modeles de langage
transformers==4.38.2
peft==0.9.0
datasets==2.17.0
# Compression des echanges
zstandard==0.22.0
# Outils de developpement et de qualite
pytest==8.0.2
ruff==0.2.2
tqdm==4.66.1
# Documentation des experiences
pyyaml==6.0.1
```

Listing 2 – Fichier `requirements.txt` du cours

Le dépôt suit une arborescence unique, reprise à l'identique dans les neuf travaux pratiques et dans les cinq projets. Cette régularité n'est pas cosmétique : elle permet à un correcteur de trouver n'importe quel résultat en moins de trente secondes, et à un étudiant de reprendre le travail d'un camarade sans réunion préalable.

Chemin	Contenu	Règle de gestion
`tp07_01_fedavg/`	code, expériences, rapport	un dossier par TP
`tp07_01_fedavg/code/`	modules Python importables	un module par étape
`tp07_01_fedavg/experiences/`	sorties brutes `.txt`, `.json`	jamais modifié à la main
`tp07_01_fedavg/figures/`	courbes au format `.pdf`	générées, jamais éditées
`tp07_01_fedavg/RAPPORT.md`	question, protocole, résultats	quatre pages au maximum
`projets/projet_a/`	code, expérimentation, mémoire	un dossier par projet
`commun/`	harnais partagé : clients, partition, métriques	lecture seule
`docs/`	protocoles, grilles, modèles de rapport	lecture seule
`Makefile`	cibles de rejouabilité	seule porte d'entrée

Tableau 1 – Arborescence normalisée du dépôt de travaux pratiques

Le principe directeur : la sortie d'une expérience est un artefact immuable ; tout ce qui est produit par le code est jetable et reproductible.

Objet	Convention	Exemple
Module Python	`snake_case.py`, un module par responsabilité	`fedavg.py`, `dirichlet.py`
Fonction ou classe	`snake_case` ou `CamelCase`	`dirichlet_split` ou `PetitCNN`
Test	`test_<module>.py`, un test par invariant	`test_aggregation.py`
Expérience	`exp<NN>_<hypothese>.py`	`exp03_compression.py`
Résultat brut	`exp<NN>_<hypothese>_g<seed>.txt`	`exp03_compression_g1.txt`
Figure	`fig_<module>_<quantite>.pdf`	`fig_dp_sac_utilite.pdf`
Note de version	`vNN_<mot-cle>.md`	`v03_lora_alpha16.md`

Tableau 2 – Conventions de nommage et de figure

Toute expérience doit pouvoir être rejouée par la commande `make` correspondante. Le fichier de construction est la seule porte d'entrée autorisée : il impose le nombre de graines, l'ordre des étapes et l'emplacement des sorties, ce qui supprime la variation accidentelle entre deux exécutions.

Grille d'évaluation commune

Une grille commune rend les livrables comparables entre binômes et supprime l'effet de la sévérité relative du correcteur. Les cinq critères s'appliquent à tous les travaux pratiques, avec des poids identiques.

Critère	Poids	Insuffisant	Satisfaisant	Excellent
Protocole	20 %	résultats sans hypothèse ni contrôle	hypothèse et contrôles déclarés	hypothèse, contrôles et analyse de leur influence
Code	20 %	script unique illisible, exécution manuelle	modules nommés, fonctions documentées, tests	tests de propriétés, types, code relu par les pairs
Résultats	25 %	un chiffré, non rejoué	trois graines, moyenne et écart-type	graphiques, tests de signification, ablations
Analyse	20 %	description sans interprétation	interprétation et limites	explication du mécanisme causal, contre-exemples
Communication	15 %	rapport non structuré	rapport de quatre pages, figures lisibles	figures interprétées, synthèse en une page

Tableau 3 – Grille d'évaluation commune d'un travail pratique

Une note inférieure à la moitié sur le critère « Protocole » est plafonnée à 8 sur 20, quel que soit le reste du dossier. C'est la seule règle qui protège la discipline de la dérive vers la seule implémentation.

```python
# -*- coding: utf-8 -*-
# Verification automatique de la conformite d'un livrable de TP.
# Usage : python outils/evaluer_livrable.py --dossier tp07_01_fedavg
from __future__ import annotations

import argparse
import json
import re
import subprocess
from pathlib import Path

# Poids de la grille commune, identiques a ceux de l'enonce du TP.
POIDS = {"protocole": 0.20, "code": 0.20, "resultats": 0.25,
"analyse": 0.20, "communication": 0.15}

# Fichiers attendus : leur presence est une condition necessaire.
ARBORESCENCE = ["RAPPORT.md", "code", "experiments", "figures"]


def verifier_arborescence(racine: Path) -> list:
"""Retourne la liste des elements manquants dans le dossier de TP."""
return [f for f in ARBORESCENCE if not (racine / f).exists()]


def compter_graines(racine: Path) -> int:
"""Compte les graines distinctes rencontrees dans les sorties brutes."""
graines = set()
dossier = racine / "experiments"
for f in sorted(dossier.glob("*.txt")) if dossier.exists() else []:
graines.update(re.findall(r"seed=(\d+)$", f.read_text(encoding="utf-8")))
return len(graines)


def verifier_tests(racine: Path) -> int:
"""Execute la suite de tests et renvoie le code de retour du lanceur."""
if not (racine / "code").exists():
return 1
return subprocess.call(["python", "-m", "pytest", "-q", str(racine / "code")])


```

Listing 3 – Squelette de l'outil d'auto-évaluation d'un livrable

```python
def main() -> int:
ap = argparse.ArgumentParser(description="Conformite d'un livrable de TP")
ap.add_argument("--dossier", required=True, type=Path)
args = ap.parse_args()
racine = args.dossier

manquants = verifier_arborescence(racine)
graines = compter_graines(racine)
code_tests = verifier_tests(racine)
dossier_fig = racine / "figures"
figures = list(dossier_fig.glob("*.pdf")) if dossier_fig.exists() else []
rapport = racine / "RAPPORT.md"

notes = {
"protocole": 20 if rapport.exists() else 0,
"code": 20 if code_tests == 0 else 8,
"resultats": 25 if graines >= 3 else 10,
"analyse": 20 if rapport.exists() and rapport.stat().st_size > 2000 else 8,
"communication": 15 if figures else 5,
}
note = sum(POIDS[c] * k for c, k in notes.items())

bilan = {
"dossier": str(racine),
```

```
    "graines": graines,
    "tests_ok": code_tests == 0,
    "manquants": manquants,
    "notes": notes,
    "note_sur_20": round(note, 2),
}
    print(json.dumps(bilan, indent=2, ensure_ascii=False))
    return 0 if not manquants else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

Vue d'ensemble des neuf travaux pratiques

Les neuf séances sont ordonnées par difficulté croissante et par dépendance : chaque TP réutilise le code du précédent. Cette progression n'est pas seulement pédagogique, elle est technique : la compréhension de la dérive du modèle est nécessaire pour comprendre le coût de communication, et la compréhension du coût est nécessaire pour comprendre pourquoi l'adaptation par adaptateurs de faible rang est la seule voie praticable.

TP	Intitulé	Durée	Niveau	Compétence visée	Outil dominant
7.1	Implémenter la moyenne pondérée sur un modèle linéaire	2 h	Débutant	algorithme d'agrégation	NumPy
7.2	Apprentissage fédéré sur réseau convolutif, vingt clients	3 h	Intermédiaire	mise en œuvre PyTorch	PyTorch, torchvision
7.3	Partition non échangeable et mesure de son effet	2 h	Intermédiaire	hétérogénéité des données	NumPy
7.4	Coût de communication et compression des échanges	2 h	Intermédiaire	coût et budget	NumPy, zstandard
7.5	Attaque par inversion de gradient et défense par bruit	3 h	Avancé	vie privée en pratique	PyTorch
7.6	Entraînement différentiellement privé et compromis	3 h	Avancé	garantie formelle de vie privée	Opacus
7.7	Adapter un grand modèle de langage et fédérer les adaptateurs	3 h	Avancé	fédération des grands modèles	PyTorch, PEFT
7.8	Agrégation robuste face aux mises à jour malveillantes	3 h	Avancé	défense contre l'empoisonnement	NumPy, PyTorch
7.9	Explorer FedX : déploiement, métriques, rapport	4 h	Avancé	conduite d'une expérience complète	FedX, Python

Tableau 4 – Les neuf travaux pratiques de la partie VII

Charge totale : 24 heures de poste de travail pour les neuf séances, hors lecture des énoncés et rédaction des rapports.

> **Attention**
> Les trois derniers travaux pratiques supposent un accélérateur d'au moins 8 gigaoctets de mémoire vive. Sans matériel adéquat, le TP 7.7 peut être traité sur le modèle de substitution `prajjwal1/bert-tiny` fourni dans l'énoncé, et le TP 7.9 en mode simulation. L'usage du modèle de substitution est explicitement noté comme tel et non pénalisé, mais il ne permet pas de conclure sur la qualité.

Remarque

Les quinze exercices du chapitre sur l'entraînement sont indépendants des TP. Ils servent à la préparation des évaluations écrites et à la révision ciblée : chacun cible un point précis du cours, et la correction est intégralement développée, jamais réduite à une référence.

A retenir

- Un travail pratique est une expérience : sans hypothèse, sans protocole figé et sans répétitions, il ne produit pas de connaissance.
- L'environnement, l'arborescence et la cible de rejouabilité sont partie du résultat : ils rendent la mesure rejouable par un tiers.
- La grille d'évaluation est commune à tous les TP, et plafonne la note en l'absence de protocole écrit.
- Les neuf TP sont ordonnés par dépendance technique, pas seulement par difficulté perçue.

Chapitre 7.2

Travaux pratiques guidés

Neuf séances, du modèle linéaire au déploiement d'une plateforme complète. Chacune ajoute une seule idée nouvelle et réutilise tout le code des précédentes.

Chaque travail pratique est autonome dans son énoncé mais dépend techniquement du précédent. Un étudiant qui suit la progression aura, à la fin de la neuvième séance, un mini-système fédéré fonctionnel, mesuré, attaqué, défendu et documenté. Les listings ci-dessous sont des socles minimaux : ils contiennent les choix structurants et laissent les expériences au lecteur.

Le harnais d'expérimentation partagé

Avant les neuf séances, trois besoins récurrents apparaissent : fixer une configuration, enregistrer des métriques, et répéter une expérience sur plusieurs graines. Les factoriser dans un module unique évite que chaque TP réinvente son propre format de sortie, et rend les courbes de tous les TP directement superposables.

```python
# commun/harnais.py : configuration, journalisation, repetition sur graines.
from __future__ import annotations

import json
import time
from dataclasses import asdict, dataclass, field
from pathlib import Path


@dataclass
class Configuration:
"""Tous les controles d'une experience, ecrits avant le premier lancement."""
n_clients: int = 20
fraction: float = 0.5
rounds: int = 20
lot: int = 64
taux: float = 0.05
epochs_locaux: int = 1
alpha_dirichlet: float = 0.5
graines: list = field(default_factory=lambda: [0, 1, 2])
sortie: str = "experiments"

def chemin(self, nom, graine):
return Path(self.sortie) / f"{nom}_g{graine}.json"


class Journal:
"""Ecrit un enregistrement JSON par graine : sortie brute, immuable."""

def __init__(self, config, nom):
self.config, self.nom = config, nom
self.chrono = time.perf_counter()
Path(config.sortie).mkdir(parents=True, exist_ok=True)

def enregistrer(self, graine, courbe):
chemin = self.config.chemin(self.nom, graine)
chemin.write_text(json.dumps({
"experience": self.nom,
"graine": graine,
"configuration": asdict(self.config),
"duree_s": round(time.perf_counter() - self.chrono, 2),
"courbe": [{"round": r + 1, "exactitude": round(a, 5), "octets": c}
for r, (a, c) in enumerate(courbe)],
}, indent=2, ensure_ascii=False), encoding="utf-8")
return chemin

def resume(self):
"""Agrege les graines : moyenne, ecart-type et etendue des resultats."""
vs = [json.loads(self.config.chemin(self.nom, g).read_text(encoding="utf-8"))
["courbe"][-1]["exactitude"] for g in self.config.graines]
n, moy = len(vs), sum(vs) / len(vs)
var = sum((v - moy) ** 2 for v in vs) / max(n - 1, 1)
return {"moyenne": round(moy, 4), "ecart_type": round(var ** 0.5, 4),
"min": round(min(vs), 4), "max": round(max(vs), 4)}


def repeter(journal, fonction, graines):
"""Applique la fonction a chaque graine, puis renvoie le resume multi-graines."""
for g in graines:
journal.enregistrer(g, fonction(g))
return journal.resume()
```

Listing 1 — Harnais d'expérimentation partagé par les TP 7.2 à 7.9

TP	Partie du cours mobilisée	Résultat attendu
7.1	Partie II, section sur la moyenne pondérée	écart entre centralisé et fédéré
7.2	Partie I, architectures de référence	courbe d'exactitude par tour
7.3	Partie I, données ; Partie II, hétérogénéité	effet du paramètre de mélange sur l'exactitude finale
7.4	Partie II, coût de communication	octets transmis et gain de compression
7.5	Partie III, taxonomie des attaques	taux de reconstruction d'image
7.6	Partie III, mécanismes différentiels	courbe utilité / budget de vie privée
7.7	Partie IV, grands modèles et adaptateurs	perte fédérée avec adaptation de faible rang
7.8	Partie III, agrégation robuste	taux de rejet des mises à jour malveillantes
7.9	Partie V, FedX	rapport d'expérience de bout en bout

Tableau 1 – Correspondance entre les TP et les chapitres théoriques

Travaux pratiques 7.1 – Implémenter la moyenne pondérée sur un modèle linéaire

Duree : 2 h | Niveau : Débutant | Outils : NumPy, Python 3.10

Écrire l'algorithme de moyenne pondérée de zéro sur un modèle linéaire, sans aucune bibliothèque d'apprentissage fédéré, puis vérifier empiriquement deux propriétés annoncées en cours : l'algorithme converge vers la même solution que l'entraînement centralisé, et l'écart entre les deux croît avec l'hétérogénéité des clients et avec le nombre d'itérations locales.

Deroulement

1. Écrire la fonction de perte, son gradient exact, et vérifier le gradient par différences finies, avec une erreur relative inférieure à 10^{-6}.
2. Implémenter `local_update` : C itérations de descente de gradient sur la partition du client, en partant de l'état du serveur.
3. Implémenter `fedavg` : R tours, agrégation pondérée par la taille des lots locaux n_k / n.
4. Vérifier le cas particulier $C = 1$: l'algorithme doit être **exactement** équivalent à une descente de gradient sur la perte globale. Vérifier l'égalité à 10^{-9} près.
5. Tracer $F(w_t)$ en fonction du tour pour $C \in 1, 5, 20$ et mesurer le nombre de tours nécessaires.
6. Introduire de l'hétérogénéité en donnant à chaque client une moyenne locale différente ; mesurer l'écart final à la solution centralisée.

```python
import numpy as np

# Modele lineaire z = w x + b, perte quadratique moyenne.
def forward(X, w, b):
return X @ w + b


def perte(X, y, w, b):
residus = y - forward(X, w, b)
return float(residus @ residus / len(y))


def gradient(X, y, w, b):
"""Derivee exacte par rapport a w et a b."""
residus = y - forward(X, w, b)
return (-2.0 * (X.T @ residus) / len(y),
-2.0 * float(residus.mean()))


def verifie_gradient(X, y, w, h=1e-6):
"""Controle par differences finies : l'ecart relatif doit rester sous 1e-6."""
gw, _ = gradient(X, y, w, 0.0)
num = np.array([(perte(X, y, w + h * e, 0.0) - perte(X, y, w - h * e, 0.0)) / (2 * h)
for e in np.eye(len(w))])
return float(np.max(np.abs(num - gw)) / max(np.max(np.abs(gw)), 1e-12))


def local_update(X, y, w, b, C, eta):
"""Un tour client : C iterations de descente de gradient sur la partition locale."""
for _ in range(C):
gw, gb = gradient(X, y, w, b)
w, b = w - eta * gw, b - eta * gb
return w, b


def fedavg(clients, w, b, R, C, eta):
"""R tours d'agregation ponderee par la taille des lots locaux."""
poids = np.array([len(y) for _, y in clients], dtype=float)
poids /= poids.sum()
Xa = np.vstack([X for X, _ in clients])
ya = np.concatenate([y for _, y in clients])
courbe = []
for _ in range(R):
locaux = [local_update(X, y, w, b, C, eta) for X, y in clients]
w = sum(p * wk for p, (wk, _) in zip(poids, locaux))
b = sum(p * bk for p, (_, bk) in zip(poids, locaux))
courbe.append(perte(Xa, ya, w, b))
return w, b, courbe


if __name__ == "__main__":
rng = np.random.default_rng(0)
X = rng.normal(size=(600, 5))
vrai = np.array([2.0, -1.0, 0.5, 0.0, 3.0])
y = X @ vrai + 0.3 + 0.2 * rng.normal(size=600)
clients = [(X[i], y[i]) for i in np.array_split(np.arange(len(y)), 6)]
w, b, courbe = fedavg(clients, np.zeros(5), 0.0, R=80, C=5, eta=0.1)
print("gradient, ecart relatif :",
verifie_gradient(*clients[0], np.zeros(5)))
print("perte initiale / finale :", round(courbe[0], 4), "/", round(courbe[-1], 4))
print("w estime / vrai :", np.round(w, 3), vrai)
print("biais estime :", round(b, 3))
```

Listing 2 – Moyenne pondérée complète en NumPy sur un modèle linéaire

Livrable attendu

- `code/fedavg.py` : les six fonctions, documentées, sans dépendance externe au-delà de NumPy.
- `code/test_gradient.py` : un test unitaire qui vérifie le gradient par différences finies sur dix tirages aléatoires.
- `experiences/exp01_C.txt` : courbes de perte pour $C \in 1, 5, 20$, trois graines chacune.
- `experiences/exp02_hetero.txt` : écart final à la solution centralisée en fonction de l'écart-type des moyennes locales.
- `figures/fig_courbes_C.pdf` et `RAPPORT.md`, deux pages.

Bareme indicatif

Critere	Points
Gradient vérifié par différences finies	4 pts
Équivalence exacte pour $C = 1$	4 pts
Boucle fédérée correcte : poids, états, initialisation	4 pts
Courbes sur trois graines, moyenne et écart-type	3 pts
Mesure de l'effet de l'hétérogénéité	3 pts
Qualité du code et du rapport	2 pts

Conseil

Commencez par vérifier que $C = 1$ redonne exactement la descente de gradient globale. Ce test ne prouve rien de nouveau, mais il détecte en deux minutes la majorité des erreurs d'implémentation, et il fournit la référence contre laquelle tout le reste est comparé.

Travaux pratiques 7.2 – Apprentissage fédéré sur réseau convolutif, vingt clients

Duree : 3 h | Niveau : Intermédiaire | Outils : PyTorch 2.2, torchvision, CIFAR-10

Passer du modèle linéaire à un réseau convolutif et à une simulation de vingt clients hétérogènes. L'enjeu n'est plus l'algorithme mais l'ingénierie : gestion des états, mélange des données, normalisation, répétitions. À la fin, la courbe d'exactitude par tour doit être superposable à celle d'un entraînement centralisé de même budget de calcul.

Deroulement

1. Écrire le réseau convolutif compact, vérifier sa forme de sortie et compter ses paramètres.
2. Charger CIFAR-10 avec la normalisation officielle, puis utiliser une augmentation minimale : retournement horizontal.
3. Écrire `partition_iid` : mélange reproductible puis découpage en vingt blocs de 2 500 images ; vérifier que les classes sont équilibrées.
4. Écrire `train_local` : un modèle **frais** chargé de l'état du serveur, puis E époques de descente de gradient stochastique avec inertie.
5. Écrire la boucle d'agrégation avec pondération par la taille des lots locaux.
6. Tracer l'exactitude de test en fonction du tour ; comparer au modèle centralisé entraîné le même nombre d'itérations sur le budget équivalent.

```python
import copy

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

ROUNDS, EPOQUES, LR, LOT, N_CLIENTS = 10, 1, 0.05, 64, 20


class PetitCNN(nn.Module):
"""Reseau convolutif compact : deux blocs puis un classifieur lineaire."""

def __init__(self, n_classes=10):
super().__init__()
self.corps = nn.Sequential(
nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32),
nn.ReLU(), nn.MaxPool2d(2),
nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64),
nn.ReLU(), nn.MaxPool2d(2))
self.tete = nn.Linear(64 * 8 * 8, n_classes)

def forward(self, x):
return self.tete(self.corps(x).flatten(1))


def charger(racine="./donnees"):
"""Normalisation officielle de CIFAR-10 ; augmentation reservee a l'entrainement."""
norm = transforms.Normalize((0.4914, 0.4822, 0.4465),
(0.2470, 0.2435, 0.2616))
aug = transforms.Compose([transforms.RandomHorizontalFlip(),
transforms.ToTensor(), norm])
return (datasets.CIFAR10(racine, True, download=True, transform=aug),
datasets.CIFAR10(racine, False, download=True,
transform=transforms.Compose([transforms.ToTensor(),
norm])))


def partition_iid(base, n_clients, graine):
"""Melange reproductible puis decoupage en n_clients blocs de taille egale."""
ordre = torch.randperm(
len(base), generator=torch.Generator().manual_seed(graine))
return [Subset(base, b.tolist()) for b in torch.chunk(ordre, n_clients)]


def train_local(modele, etat, chargeur, epochs, lr):
"""Un tour client : partir de l'etat du serveur, puis quelques pas locaux."""
modele.load_state_dict(etat)
opt = torch.optim.SGD(modele.parameters(), lr=lr, momentum=0.9)
modele.train()
for _ in range(epochs):
for x, y in chargeur:
opt.zero_grad(set_to_none=True)
F.cross_entropy(modele(x), y).backward()
opt.step()
return copy.deepcopy(modele.state_dict())


@torch.no_grad()
```

Listing 3 – Moyenne pondérée sur réseau convolutif, simulation de vingt clients

```python
def exactitude(modele, base):
"""Un seul passage sur le test, en mode evaluation (normalisation figee)."""
modele.eval()
bons = total = 0
```

```
for x, y in DataLoader(base, batch_size=512):
bons += (modele(x).argmax(1) == y).sum().item()
total += y.numel()
return bons / total


def fedavg(modele, clients, test, rounds=ROUNDS, epochs=EPOQUES, lr=LR):
"""Diffusion, entrainement local, agregation ponderee par la taille des lots."""
etat = copy.deepcopy(modele.state_dict())
poids = [len(c) for c in clients]
journal = []
for t in range(rounds):
locaux = [train_local(modele, etat,
DataLoader(c, batch_size=LOT, shuffle=True),
epochs, lr)
for c in clients]
for k in etat:
etat[k] = sum((p / sum(poids)) * l[k].float()
for p, l in zip(poids, locaux))
modele.load_state_dict(etat)
acc = exactitude(modele, test)
journal.append(acc)
print(f"tour {t + 1:2d} | exactitude test {acc:.4f}")
return journal
```

Livrable attendu

- `code/modele.py`, `code/partition.py`, `code/fedavg.py` : trois modules séparés, importables indépendamment.
- `experiences/exp01_iid.txt` : courbe d'exactitude, dix tours, trois graines.
- `experiences/exp02_centralise.txt` : référence centralisée, même budget global de calcul.
- Tableau comparatif : exactitude finale, nombre de tours pour atteindre 60 %, écart-type sur trois graines.
- `figures/fig_exactitude_round.pdf` et `RAPPORT.md`, trois pages.

Bareme indicatif

Critere	Points
Modèle correct et paramètres comptés	3 pts
Partition aléatoire correcte et vérifiée	3 pts
Isolation des états entre clients, sans aliasing	4 pts
Boucle d'agrégation et pondération correctes	4 pts
Référence centralisée exécutée et comparée	2 pts
Trois graines, figures, analyse de l'écart	4 pts

Conseil

L'erreur la plus fréquente consiste à réutiliser un même objet `modele` pour tous les clients : le premier client modifie alors l'état reçu par le suivant, et l'agrégation ne mesure plus rien. Utilisez `load_state_dict` sur un modèle partagé, jamais d'affectation directe de tenseurs.

Travaux pratiques 7.3 – Partition non échangeable et mesure de son effet

Duree : 2 h | Niveau : Intermédiaire | Outils : NumPy, PyTorch, CIFAR-10

Construire explicitement une distribution non échangeable contrôlée par un paramètre unique α, et mesurer quantitativement comment l'hétérogénéité dégrade la convergence. Le résultat attendu est une courbe d'exactitude finale en fonction de α qui décroît fortement lorsque $\alpha \to 0$.

Deroulement

1. Écrire `dirichlet_split` : pour chaque classe, tirer une proportion suivant une loi de Dirichlet de paramètre α, puis répartir les échantillons de cette classe selon ces proportions.
2. Vérifier la partition : tous les indices apparaissent exactement une fois.
3. Pour $\alpha \in 0{,}1,\ 0{,}3,\ 1,\ 10,\ 100$, mesurer l'entropie moyenne de la distribution des classes par client : nulle pour un client pur, maximale pour un client équilibré.
4. Lancer l'algorithme pour chaque valeur, trois graines, dix tours.
5. Tracer l'exactitude finale et le nombre de tours pour atteindre 60 % en fonction de α et de l'entropie mesurée.
6. Conclure sur la relation entre hétérogénéité mesurée et dégradation : est-elle linéaire, logarithmique, ou autre chose ?

```python
import numpy as np


def dirichlet_split(etiquettes, n_clients, alpha, n_classes=10, graine=0):
"""Repartition non exchangeable des clients.

Pour chaque classe, les proportions clients suivent une loi de Dirichlet
de parametre alpha ; plus alpha est petit, plus chaque client se concentre
sur peu de classes.
"""
rng = np.random.default_rng(graine)
morceaux = []
for c in range(n_classes):
idx = np.where(etiquettes == c)[0]
rng.shuffle(idx)
props = rng.dirichlet(np.repeat(alpha, n_clients))
coupures = (np.cumsum(props) * len(idx)).astype(int)[:-1]
morceaux.append(np.split(idx, coupures))
# Le client k rassemble le morceau de chaque classe, puis melange l'ordre.
par_client = [np.concatenate([morceaux[c][k] for c in range(n_classes)])
for k in range(n_clients)]
for k in rng.permutation(n_clients):
yield par_client[k][rng.permutation(len(par_client[k]))]


def entropie_clients(etiquettes, indices, n_classes=10):
"""Entropie normalisee moyenne sur les clients.

Valeur nulle pour un client ne portant qu'une classe, unite pour une
repartition parfaitement equilibree.
"""
valeurs = []
for idx in indices:
if len(idx) == 0:
valeurs.append(0.0)
continue
_, effectifs = np.unique(etiquettes[idx], return_counts=True)
p = effectifs / len(idx)
valeurs.append(float(-(p * np.log(p + 1e-12)).sum() / np.log(n_classes)))
return float(np.mean(valeurs)), valeurs


def verifier_partition(etiquettes, indices):
"""Invariant : la concatenation triee est une permutation complete."""
plat = np.sort(np.concatenate([np.asarray(i) for i in indices]))
return bool(np.array_equal(plat, np.arange(len(etiquettes))))


if __name__ == "__main__":
etiquetes = np.repeat(np.arange(10), 500) # support de test synthetique
for alpha in [0.1, 0.3, 1.0, 10.0, 100.0]:
idx = list(dirichlet_split(etiquettes, 20, alpha, graine=0))
h, detail = entropie_clients(etiquettes, idx)
dominante = max(int(np.bincount(etiquettes[i], minlength=10).max())
for i in idx)
print(f"alpha={alpha:6.1f} | entropie={h:.3f} | dominante max={dominante:4d}"
f" | valide={verifier_partition(etiquettes, idx)}")
```

Listing 4 – Partition de Dirichlet et mesure de l'hétérogénéité induite

Livrable attendu

- `code/dirichlet.py` : les trois fonctions, chacune testée.
- `experiences/exp03_alpha.txt` : entropie et exactitude finale pour cinq valeurs de α et trois graines.
- `experiences/exp04_poids.txt` : matrice de chaleur, vingt clients et dix classes, pour $\alpha = 0{,}1$ et $\alpha = 10$.

- `figures/fig_alpha_exactitude.pdf` et `figures/fig_matrice_classes.pdf`.
- `RAPPORT.md`, trois pages : courbe mesurée contre la relation attendue.

Bareme indicatif

Critere	Points
`dirichlet_split` correct et invariant de partition testé	4 pts
Mesure d'hétérogénéité argumentée et validée	4 pts
Expérience multi-α, trois graines	4 pts
Matrices de chaleur de la répartition des classes	3 pts
Analyse de la relation entre α et la dégradation	3 pts
Code testé, rapport structuré	2 pts

> **Conseil**
> $\alpha = 100$ n'est pas « échangeable » mais en est arbitrairement proche : la loi de Dirichlet se concentre autour de la valeur moyenne. C'est le bon point de départ pour établir la borne haute, et il faut le dire explicitement dans le rapport.

Travaux pratiques 7.4 – Mesurer le coût de communication et l'effet de la compression

Duree : 2 h | Niveau : Intermédiaire | Outils : NumPy, zstandard, SciPy

Établir le coût exact d'une exécution en octets échangés, puis évaluer quantitativement ce que la compression des mises à jour économise en bande passante et ce qu'elle coûte en qualité. Le résultat attendu est un compromis mesuré : courbe qualité-volume pour une quantification, une troncature et une compression par entropie.

Deroulement

1. Écrire `cout_communication` : compter les octets émis par le serveur et par les clients sur une exécution complète, à partir de la taille réelle des tenseurs et non d'une estimation analytique.
2. Vérifier le comptage sur un vecteur de taille connue : le nombre d'octets doit être exactement quatre fois le nombre de composantes en précision simple.
3. Implémenter la quantification à huit bits par bloc : centrage sur la moyenne du bloc, puis mise à l'échelle dans $[-127, 127]$.
4. Implémenter la troncature : ne transmettre que p pour cent des coordonnées les plus grandes en valeur absolue, en compensant le facteur d'échelle manquant.
5. Mesurer, pour chaque méthode, les octets transmis **et** l'erreur d'agrégation induite sur un vecteur tiré aléatoirement.
6. Tracer le front de compromis qualité-octets et conclure sur le seuil au-delà duquel la dégradation devient rédhibitoire.

```python
import io

import numpy as np
import zstandard as zstd


def cout_communication(etats, n_clients, aller=True, retour=True):
"""Octets reellement transites : taille des tenseurs fois nombre de messages."""
if not etats:
return 0
taille = sum(v.nbytes for v in etats[0].values())
return int(len(etats) * taille * retour + n_clients * taille * aller)


def quantifier_bloc(w, bits=8):
"""Quantification par bloc de 64 : centrage sur la moyenne, mise a l'echelle."""
niveau = (1 << (bits - 1)) - 1
blocs = w.reshape(-1, 64)
q = np.empty_like(blocs)
for i, bloc in enumerate(blocs):
mu = bloc.mean()
ecart = max(float(bloc.std()), 1e-12)
q[i] = np.clip(np.round((bloc - mu) / ecart * niveau),
-niveau - 1, niveau)
return q


def tronquer(w, proportion):
"""Troncature partielle : seuls p% des coordonnees les plus fortes sont transmis."""
d = w.size
k = max(1, int(d * proportion))
planaire = w.reshape(-1).astype(np.float64)
norms = np.abs(planaire)
seuil = np.partition(norms, d - k)[d - k]
masque = norms >= seuil
echelle = float(planaire[masque].mean()) if masque.any() else 0.0
echelle = echelle if abs(echelle) > 1e-12 else 1.0
# On recompense la norme absente pour que l'agregation reste non biaisee.
return (planaire * (masque * d / k) / echelle).reshape(w.shape)


def erreur_agregation(approx, exact):
return float(np.linalg.norm(approx - exact)
/ max(np.linalg.norm(exact), 1e-12))


if __name__ == "__main__":
rng = np.random.default_rng(0)
w = rng.normal(size=(10000,)).astype(np.float32)
brut, echelle = w.nbytes, float(w.reshape(-1).std())
compresseur = zstd.ZstdCompressor(level=3)
for nom, approx in [("flottant 32", w),
("quantifie 8 bits", quantifier_bloc(w).reshape(-1) * echelle),
("tronque 10 %", tronquer(w, 0.10))]:
tampon = io.BytesIO()
compresseur.copy_stream(w.tobytes(), tampon)
print(f"{nom:18s} | brut {brut:7d} o | comprime {tampon.tell():7d} o |"
f" gain {brut / max(tampon.tell(), 1):5.2f}x |"
f" erreur {erreur_agregation(approx, w):.3e}")
```

Listing 5 – Comptage du coût de communication et compression des mises à jour

Livrable attendu

- `code/cout.py` : les trois fonctions, chacune avec un test unitaire.
- `experiences/exp05_octets.txt` : volume échangé exact par exécution, croisé avec la formule

analytique.

- `experiences/exp06_pareto.txt` : pour $p \in 1, 5, 10, 50$ pour cent et plusieurs largeurs de quantification, octets et erreur d'agrégation.
- `experiences/exp07_penalite.txt` : perte exacte après vingt tours avec quantification, comparée à la version non compressée.
- `figures/fig_pareto.pdf` et `RAPPORT.md`, trois pages.

Bareme indicatif

Critere	Points
Comptage exact vérifié contre la formule analytique	4 pts
Quantification correcte : centrage, échelle, débordements	3 pts
Troncature avec compensation de norme correcte	3 pts
Erreur d'agrégation mesurée, et pas seulement le taux de compression	4 pts
Front de compromis tracé et seuil discuté	3 pts
Pénalité en convergence mesurée sur l'algorithme de base	3 pts

Conseil

Le taux de compression seul est un mauvais indicateur : un vecteur de coordonnées nulles se compresse parfaitement sans aucune perte, et ne transporte aucune information. Remplacez toujours le taux de compression avec l'erreur d'agrégation induite sur le modèle agrégé : c'est le second nombre qui décide.

Travaux pratiques 7.5 – Attaque par inversion de gradient et défense par bruit

Duree : 3 h | Niveau : Avancé | Outils : PyTorch, NumPy

Reproduire l'attaque par reconstruction à partir d'un gradient, mesurer le taux de reconstruction d'images réelles, puis évaluer quantitativement l'effet d'une défense par ajout de bruit sur le gradient transmis. Le résultat attendu est un compromis mesuré : bruit croissant, reconstruction dégradée, utilité préservée d'abord puis détruite.

Deroulement

1. Écrire `recon_dlg` : reconstruire une image témoin à partir d'un gradient arbitraire, avec la relaxation de découplage des signes, c'est-à-dire la découplage entre le signe des gradients et le signe de la perte.
2. Vérifier la reconstruction sur un cas trivial : le gradient d'une image noire constante contre des labels fixés doit se reconstruire exactement.
3. Sur un lot de 100 images réelles, calculer le taux de reconstruction réussie, défini par une similarité structurelle supérieure à $0{,}6$.
4. Implémenter la défense : ajouter un bruit gaussien sur les gradients avant émission, avec l'écart-type comme unique paramètre.
5. Mesurer, pour plusieurs valeurs de l'écart-type, le taux de reconstruction **et** l'exactitude finale après vingt tours.
6. Tracer le front de compromis et situer le seuil au-delà duquel la défense ne protège plus.

```python
import torch
import torch.nn.functional as F


def bruit_gaussien(gradients, sigma, generateur):
"""Defense par bruit : bruit gaussien isometrique sur chaque coordonnee."""
bruites = []
for g in gradients:
e = torch.randn(g.shape, generator=generateur,
device=g.device, dtype=g.dtype)
bruites.append(g + sigma * e)
return bruites


def dlg(modele, gradient_observe, x_temoin, y, iters=300, lr=0.1):
"""Reconstruction par decouplage des signes.

On optimise une image et des etiquettes factices pour que le gradient en
image ait le meme signe que le gradient observe. L'initialisation est nulle.
"""
z = torch.zeros_like(x_temoin, requires_grad=True)
y_faux = torch.zeros_like(y, requires_grad=True)
optim = torch.optim.Adam([z, y_faux], lr=lr, betas=(0.5, 0.9))
for _ in range(iters):
optim.zero_grad(set_to_none=True)
perte = 0.0
for g_obs, p in zip(gradient_observe, modele.parameters()):
g_recon = torch.autograd.grad(
F.cross_entropy(modele(z), y_faux), p, retain_graph=True)[0]
perte = perte + (1 - torch.sign(g_obs) * torch.sign(g_recon)).sum()
perte.backward()
optim.step()
return z.detach().clamp(0, 1)


def cosine(x, y):
"""Similarite structurelle simplifiee : correlation sur des fenetres 8x8."""
a, b = F.avg_pool2d(x, 8).flatten(1), F.avg_pool2d(y, 8).flatten(1)
a, b = a - a.mean(1, keepdim=True), b - b.mean(1, keepdim=True)
num = (a * b).sum(1)
den = a.norm(dim=1) * b.norm(dim=1)
return torch.where(den > 0, num / den.clamp(min=1e-8), torch.zeros_like(num))


if __name__ == "__main__":
from tp_7_2 import PetitCNN # module du TP precedant
g = torch.Generator().manual_seed(0)
torch.manual_seed(0)
for sigma in [0.0, 1e-3, 1e-2, 1e-1]:
x = torch.rand(32, 3, 32, 32, generator=g)
y = torch.randint(0, 10, (32,), generator=g)
modele = PetitCNN().double()
observe = torch.autograd.grad(F.cross_entropy(modele(x), y),
list(modele.parameters()))
bruite = bruit_gaussien(list(observe), sigma, g)
recon = dlg(modele, bruite, x, y, iters=100)
taux = float((cosine(x, recon) > 0.6).float().mean())
print(f"sigma={sigma:.0e} | taux de reconstruction : {taux:.2f}")
```

Listing 6 – Attaque par inversion de gradient et défense par bruit gaussien

Livrable attendu

- `code/dlg.py` : les trois fonctions, testées sur le cas trivial.
- `experiences/exp08_recon.txt` : vingt images reconstruites, en grille, annotées avec la similarité.

- `experiences/exp09_sigma.txt` : taux de reconstruction et exactitude finale pour quatre valeurs de l'écart-type.
- `figures/fig_recon_sigma.pdf` et `figures/fig_grille_recon.pdf`.
- `RAPPORT.md`, quatre pages : mécanisme d'inversion, seuil mesuré, coût.

Bareme indicatif

Critere	Points
Reconstruction correcte sur le cas trivial	3 pts
Découplage des signes correctement implémenté	4 pts
Taux de reconstruction mesuré sur données réelles	4 pts
Défense par bruit correctement calibrée	3 pts
Compromis utilité-protection tracé et quantifié	4 pts
Analyse du mécanisme d'inversion	2 pts

Conseil

La reconstruction d'images à partir d'un unique gradient n'est possible que lorsque le lot est petit et la normalisation standards. Sur un lot de 64 avec normalisation par lots, l'attaque échoue largement : ce n'est pas une défaillance de l'implémentation mais l'effet de l'agrégation sur les statistiques. Exploitez cette observation dans l'analyse.

Travaux pratiques 7.6 – Entraînement différentiellement privé et tracé du compromis

Duree : 3 h | Niveau : Avancé | Outils : PyTorch 2.2, Opacus 1.5

Ajouter une garantie formelle de vie privée au cycle fédéré, non par une loi de probabilité mais par un mécanisme formel, puis mesurer le compromis utilité et budget sur l'ensemble de la courbe. Le résultat attendu est le constat que le budget se consomme de façon sous-linéaire en tours, et qu'il s'alloué au fil de l'eau, non à la fin.

Deroulement

1. Comprendre le rôle de l'échantillonnage de Poisson : il est la condition qui rend l'analyse de la composition valide pour un lot de taille variable.
2. Écrire une version manuelle de l'entraînement différentiellement privé, avec bornage par lot et bruit sur la somme des gradients, et vérifier qu'elle rejoint la bibliothèque de référence à une précision donnée.
3. Entraîner pour $\square \in 8, 4, 2, 1, 0{,}5$ avec un nombre de tours fixé, en notant δ et le taux d'échantillonnage q.
4. Vérifier que le budget rapporté par la bibliothèque correspond au budget analytique cumulé sur l'ensemble des étapes.
5. Tracer l'exactitude finale en fonction de $\square$, puis en fonction de $\square^2 / q$.
6. Mesurer la fuite résiduelle : entraîner un détecteur d'appartenance sur les gradients bruités et mesurer son exactitude de prédiction sur les membres.

```python
import copy

import opacus
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader

CLIENTS, ROUNDS, LOT, DP_SIGMA, DP_MAX_NORM = 10, 5, 64, 1.1, 1.0


def client_dp_sgd(modele, chargeur, epochs, lr, sigma=DP_SIGMA, c=DP_MAX_NORM):
"""Un tour local differentially prive, en deux temps.

D'abord le bornage des gradients par lot, ensuite l'ajout du bruit a la
somme des gradients normalises. La sensibilite du lot vaut exactement c
grace au bornage, quel que soit le nombre d'exemples.
"""
parametres = [p for p in modele.parameters() if p.requires_grad]
norms = []
for _ in range(epochs):
for x, y in chargeur:
modele.zero_grad(set_to_none=True)
F.cross_entropy(modele(x), y).backward()
norms.append(float(torch.sqrt(sum((p.grad ** 2).sum()
for p in parametres
if p.grad is not None))))
echelle = c / (max(norms) + 1e-12)
generateur = torch.Generator().manual_seed(0)
for _ in range(epochs):
for x, y in chargeur:
modele.zero_grad(set_to_none=True)
F.cross_entropy(modele(x), y).backward()
for p in parametres:
if p.grad is None:
continue
bruit = (torch.randn(p.grad.shape, generator=generateur)
* (sigma * c))
p.grad.add_((p.grad * echelle
+ bruit / len(norms)).to(p.grad.dtype))


def privacy_engine(modele, train, lot):
"""Configure l'attache de donnees, le compteur et l'audit du budget."""
privacy = opacus.PrivacyEngine(accountant="rdp", secure_mode=False)
modele, _, _, _ = privacy.make_private(
module=modele, data_loader=DataLoader(train, batch_size=lot),
noise_multiplier=DP_SIGMA, max_grad_norm=DP_MAX_NORM,
poisson_sampling=True)
return privacy, modele


def auditer(privacy, delta=1e-5):
"""Retourne le budget consomme a ce stade du calcul."""
return float(privacy.get_epsilon(delta))


```

Listing 7 – Entraînement différentiellement privé fédéré, implémentation manuelle et audit du budget

```python
if __name__ == "__main__":
from tp_7_2 import PetitCNN, charger, partition_iid
train, test = charger()
clients = partition_iid(train, CLIENTS, graine=0)
for eps_cible in [8.0, 4.0, 2.0, 1.0, 0.5]:
torch.manual_seed(0)
modele = PetitCNN()
privacy, modele = privacy_engine(modele, train, LOT)
```

```
etat = copy.deepcopy(modele.state_dict())
poids = [len(c) for c in clients]
for _ in range(ROUNDS):
locaux = []
for c in clients:
modele.load_state_dict(etat)
client_dp_sgd(modele, DataLoader(c, batch_size=LOT), 1, 0.05)
locaux.append(copy.deepcopy(modele.state_dict()))
for k in etat:
etat[k] = sum((p / sum(poids)) * l[k].float()
for p, l in zip(poids, locaux))
eps = auditer(privacy)
print(f"epsilon={eps:.3f} (cible {eps_cible}) |"
f" multiplicateur={DP_SIGMA} |"
f" q={LOT / (len(train) / CLIENTS):.4f}")
```

Livrable attendu

- `code/dp_sgd.py` : les trois fonctions.
- `experiences/exp10_epsilon.txt` : cinq cibles $\square$, trois graines, budget rapporté et budget analytique cumulé.
- `experiences/exp11_bruit.txt` : exactitude finale en fonction de $\square^2 / q$, échelle naturelle du sacrifice d'utilité.
- `experiences/exp12_appartenance.txt` : exactitude du détecteur sur les gradients bruités, par valeur du budget.
- `figures/fig_utilite_epsilon.pdf` et `RAPPORT.md`, quatre pages.

Bareme indicatif

Critere	Points
Vérification de l'implémentation contre la référence	3 pts
Courbe utilité et $\square$ complète et répétée	4 pts
Rapprochement du budget rapporté et analytique	4 pts
Analyse de la composition du budget	3 pts
Fuite résiduelle mesurée par détecteur d'appartenance	4 pts
Recommandation de configuration argumentée	2 pts

Conseil

Le budget $\square$ se compose de façon sous-linéaire en tours : à taux d'échantillonnage fixé, T étapes consomment un budget de l'ordre de $q\sqrt{(T \log(1/\delta))}$. Doubler le nombre de tours n'est donc pas neutre, et les deux intuitions courantes — « le budget est borné linéairement par le nombre de tours » et « on peut rejouer l'exécution autant de fois qu'on veut » — sont fausses.

Travaux pratiques 7.7 – Adapter un grand modèle de langage et fédérer les adaptateurs

Duree : 3 h | Niveau : Avancé | Outils : PyTorch, PEFT 0.9, Transformers 4.38

Appliquer un mécanisme d'adaptation économe à un modèle de langage pré-entraîné, puis fédérer les seuls adaptateurs : montrer que le coût de communication, de mémoire et de temps de calcul rend l'apprentissage fédéré d'un grand modèle enfin praticable, là où l'affinage complet ne l'est pas.

Deroulement

1. Charger un modèle de substitution léger, de quatre millions de paramètres, ou si l'accélérateur le permet, un modèle de cent vingt-cinq millions à un milliard.
2. Vérifier que l'injection des adaptateurs ne rend entraînables que les poids d'adaptation, et compter le ratio exact.
3. Implémenter l'agrégation des adaptateurs : la matrice B est initialisée à zéro, donc l'agrégation des B est une moyenne directe ; vérifier que l'agrégation des A reste valide.
4. Entraîner sur un corpus de classification de questions en régime fédéré, et comparer l'exactitude de l'affinage complet, de l'adaptation centralisée et de l'adaptation fédérée.
5. Mesurer le coût de communication par tour, en kilo-octets, et le temps par tour.
6. Tracer l'exactitude en fonction du rang d'adaptation, pour $r \in 2, 4, 8$.

```python
import torch
from peft import LoraConfig, TaskType, get_peft_model
from transformers import AutoModelForSequenceClassification, AutoTokenizer

MODELE = "prajjwal1/bert-tiny"
RANG, ALPHA, DROPOUT = 4, 8, 0.05
MODULES = ["query", "key", "value", "dense"]


def charger_modele(n_etiquettes=2, rang=RANG, alpha=ALPHA):
"""On attache les adaptateurs aux projections lineaires de l'attention."""
tok = AutoTokenizer.from_pretrained(MODELE)
base = AutoModelForSequenceClassification.from_pretrained(
MODELE, num_labels=n_etiquettes)
config = LoraConfig(task_type=TaskType.SEQ_CLS, r=rang, lora_alpha=alpha,
lora_dropout=DROPOUT, target_modules=MODULES)
return tok, get_peft_model(base, config)


def compter_parametres(modele):
"""Retourne (entrainables, total, ratio en pourcentage)."""
total = sum(p.numel() for p in modele.parameters())
train = sum(p.numel() for p in modele.parameters() if p.requires_grad)
return train, total, 100.0 * train / total


def extraire_adaptateurs(modele):
"""Les tenseurs d'adaptation sont les seuls nomines contenant lora_."""
return {n: p.detach().clone() for n, p in modele.named_parameters()
if "lora_" in n}


def agreger_adaptateurs(etats, poids):
"""Moyenne ponderee des adaptateurs ; le socle pre-entraine reste fige."""
cles = list(etats[0].keys())
return {k: sum(p * e[k] for p, e in zip(poids, etats)) for k in cles}


def etape_locale(modele, adaptateurs, chargeur, epochs, lr):
"""Un tour client : on recharge les adaptateurs, on entrain, on renvoie
uniquement les tenseurs d'adaptation qui ont change."""
with torch.no_grad():
for n, p in modele.named_parameters():
if n in adaptateurs:
p.copy_(adaptateurs[n])
optim = torch.optim.AdamW([p for n, p in modele.named_parameters()
if "lora_" in n], lr=lr, weight_decay=0.01)
modele.train()
for _ in range(epochs):
for ids, mask, y in chargeur:
optim.zero_grad(set_to_none=True)
modele(input_ids=ids, attention_mask=mask, labels=y).loss.backward()
optim.step()
return extraire_adaptateurs(modele)


if __name__ == "__main__":
tok, modele = charger_modele()
train, total, ratio = compter_parametres(modele)
adapt = extraire_adaptateurs(modele)
print(f"entrainables : {train} / {total} ({ratio:.3f} %)")
print(f"tenseurs d'adaptation : {len(adapt)}")
print(f"transit par tour :"
f" {sum(v.numel() for v in adapt.values()) * 4 / 1024:.1f} Ko")
```

Listing 8 – Adaptation fédérée d'un grand modèle par adaptateurs de faible rang

Livrable attendu

- `code/lora_fed.py` : les cinq fonctions.
- `experiences/exp13_ratio.txt` : ratio exact de paramètres entraînables, vérifié par assertion.
- `experiences/exp14_cout.txt` : kilo-octets transmis par tour pour l'adaptation contre l'affinage complet, et temps par tour.
- `experiences/exp15_rang.txt` : exactitude pour r ∈ 2, 4, 8.
- `figures/fig_cout_lora.pdf` et `RAPPORT.md`, quatre pages.

Bareme indicatif

Critere	Points
Adaptateurs correctement attachés et comptés	4 pts
Agrégation des seuls adaptateurs	4 pts
Comparaison affinage complet, adaptation fédérée, adaptation centralisée	4 pts
Coût de communication mesuré, et non estimé	3 pts
Influence du rang r analysée	3 pts
Limites et reproductibilité	2 pts

Conseil

L'agrégation des matrices d'adaptation pose un piège réel : la matrice supérieure est initialisée à zéro, donc la moyenne de plusieurs d'entre elles est proche de zéro, ce qui fait chuter l'amplitude de la mise à jour proportionnellement au nombre de clients. C'est un problème connu et corrigé par mise à l'échelle, mais il faut savoir le reconnaître dans ses propres courbes.

Travaux pratiques 7.8 – Agrégation robuste face aux mises à jour malveillantes

Duree : 3 h | Niveau : Avancé | Outils : NumPy, PyTorch, SciPy

Comparer quantitativement les règles d'agrégation sur deux menaces distinctes : l'empoisonnement par mise à jour malveillante, et la dérive des mises à jour légitimes. Mesurer le taux de rejet, l'exactitude résiduelle et le coût de calcul de chaque règle, afin de choisir en connaissance de cause.

Deroulement

1. Écrire `krum` : pour chaque mise à jour candidate, calculer la somme des n_{min} - f + 2 distances aux mises à jour les plus proches, et retenir celle de plus petite somme.
2. Écrire `moyenne_tronquee` : trier les mises à jour coordonnée par coordonnée et ne garder que les b extrêmes, avec b = lfloor N f rfloor.
3. Écrire `mediane_coordonnee` : la médiane par coordonnée.
4. Simuler un scénario d'attaque : f ∈ 10, 20, 30 pour cent des clients sont malveillants et envoient un vecteur de coordonnées multipliées par dix fois la norme des mises à jour légitimes.
5. Mesurer, pour chaque règle : exactitude finale, taux de rejet des mises à jour malveillantes, écart à la mise à jour honnête agrégée sans attaque.
6. Comparer le coût de calcul par tour en fonction du nombre de clients et de la dimension.

```python
import numpy as np


def krum(mises_a_jour, f, n_min=None):
"""Selectionne la mise a jour de plus faible somme des distances proches.

n_min est le nombre de clients non malveillants suppose, f le nombre de
malveillants. La regle degener si toutes les mises a jour sont proches.
"""
n = len(mises_a_jour)
n_min = n - f if n_min is None else n_min
m = max(1, n_min - f + 2)
distances = np.array([[np.linalg.norm(a - b) for b in mises_a_jour]
for a in mises_a_jour])
scores = [np.sort(distances[i])[:m].sum() for i in range(n)]
return int(np.argmin(scores)), float(np.min(scores))


def moyenne_tronquee(mises_a_jour, b):
"""Moyenne des b valeurs extremes apres tri par coordonnee."""
pile = np.sort(np.asarray(mises_a_jour), axis=0)
return pile[:b].mean(axis=0) + pile[-b:].mean(axis=0)


def mediane_coordonnee(mises_a_jour):
"""Mediane par coordonnee : la plus robuste a un nombre pair de malveillants."""
return np.median(np.asarray(mises_a_jour), axis=0)


def distance_normalisee(v, propres):
"""Distance d'une mise a jour a la moyenne des mises a jour honnete."""
moyenne = np.mean(propres, axis=0)
return float(np.linalg.norm(v - moyenne)
/ (np.linalg.norm(moyenne) + 1e-12))


def agreger(mises_a_jour, f):
"""Applique les quatre regles et renvoie leurs resultats."""
n = len(mises_a_jour)
b = max(1, n // 2 - f)
pile = np.sort(np.asarray(mises_a_jour), axis=0)
return {"moyenne": pile.mean(axis=0),
"tronquee": pile[:b].mean(axis=0) + pile[-b:].mean(axis=0),
"mediane": np.median(pile, axis=0),
"krum": krum(mises_a_jour, f)[0]}


if __name__ == "__main__":
rng = np.random.default_rng(0)
n, d, f = 20, 128, 3
honnets = rng.normal(size=(n - f, d))
norme = float(np.linalg.norm(honnets, axis=1).mean())
malveillants = np.sign(rng.normal(size=(f, d))) * 10 * norme
tous = list(np.vstack([honnets, malveillants]))
sorties = agreger(tous, f)
for nom in ("moyenne", "tronquee", "mediane"):
print(f"{nom:9s} | ecart relatif a la moyenne honnete :"
f"{distance_normalisee(sorties[nom], honnets):6.3f}")
print(f"krum | client retenu : {sorties['krum']} (doit rester < {n - f})")
```

Listing 9 – Règles d'agrégation robustes face à des mises à jour malveillantes

Livrable attendu

- `code/robuste.py` : les quatre fonctions, chacune avec un test contre un scénario connu.
- `experiences/exp16_attaque.txt` : taux de rejet et exactitude pour f ∈ 10, 20, 30 pour cent, trois

graines.

- `experiences/exp17_cout.txt` : temps d'agrégation par tour en fonction du nombre de clients, et régression en carré.
- `experiences/exp18_honnetite.txt` : perte de performance sur données honnêtes sans aucune attaque, pour une règle inutilement conservatrice.
- `figures/fig_regles_agregation.pdf` et `RAPPORT.md`, quatre pages.

Bareme indicatif

Critere	Points
`krum` correct et testé sur un scénario d'attaque	4 pts
Moyenne tronquée et médiane correctes	3 pts
Taux de rejet mesuré par menace simulée	4 pts
Coût de calcul comparé et extrapolé	3 pts
Compromis robustesse et conservatisme	4 pts
Recommandation argumentée selon le scénario	2 pts

Conseil

Un exercice sur la règle de sélection est incomplet sans contre-exemple : la sélection dégénère si les mises à jour sont toutes proches, la distance effective étant nulle. Testez le cas de clients identiques : la sélection devient arbitraire et la règle perd toute propriété de robustesse. C'est ce qui explique le recours à la variante multiple, qui agrège les meilleurs candidats au lieu d'en retenir un seul.

Travaux pratiques 7.9 – Explorer FedX : déployer une exécution, lire les métriques, produire un rapport

Duree : 4 h | Niveau : Avancé | Outils : FedX, Python, YAML

Conduire une expérience de bout en bout sur le moteur FedX : déclarer une configuration déclarative, lancer une exécution, surveiller les métriques produites, diagnostiquer un incident de convergence à partir des seuls indicateurs, et produire un rapport d'expertise exploitable par un tiers.

Deroulement

1. Écrire un manifeste d'exécution : jeu de données, modèle, stratégie d'agrégation, budget de communication, métriques attendues.
2. Lancer l'exécution en mode simulation locale et vérifier que les sorties sont écrites dans le répertoire attendu.
3. Lire les métriques produites et les tracer : exactitude, perte, coût cumulé, écart-type entre clients.
4. Provoquer volontairement un incident — trop de tours, taille de lot excessive, budget de communication coupé — et diagnostiquer la cause à partir des courbes seules.
5. Comparer trois stratégies sur le même manifeste, sans modifier que la stratégie d'agrégation.
6. Produire un rapport d'expertise de trois pages : protocole, observations, diagnostic, recommandation.

```python
from dataclasses import asdict, dataclass, field
from pathlib import Path

import yaml


@dataclass
class RunFedX:
"""Configuration declarative d'une execution, alignee sur le manifeste."""
dataset: str = "cifar10"
modele: str = "resnet18_cifar"
n_clients: int = 20
fraction: float = 0.1
rounds: int = 50
agregation: str = "fedavg"
budget_go: float = 2.0
lot: int = 64
alpha_dirichlet: float = 0.5
etiquettes: list = field(default_factory=lambda:
["exactitude_test", "perte", "octets_cumules"])

def valider(self):
"""Un manifeste invalide doit echouer tot, pas au bout de cinquante tours."""
erreurs = []
if not 0 < self.fraction <= 1:
erreurs.append("la fraction doit etre dans ]0, 1]")
if self.agregation not in {"fedavg", "fedprox", "scaffold", "krum",
"median"}:
erreurs.append(f"agregation meconnue : {self.agregation}")
if self.alpha_dirichlet <= 0:
erreurs.append("le parametre de melange doit etre strictement positif")
if self.budget_go <= 0:
erreurs.append("le budget doit etre strictement positif")
return erreurs

def ecrire(self, chemin):
p = Path(chemin)
p.parent.mkdir(parents=True, exist_ok=True)
p.write_text(yaml.safe_dump(asdict(self), sort_keys=False,
allow_unicode=True), encoding="utf-8")
return p


```

Listing 10 — Configuration déclarative et moniteur de métriques pour une exécution

```python
class MoniteurFedX:
"""Lit le journal d'une execution et en tire les indicateurs de pilotage."""

def __init__(self, journal):
self.entrees = list(journal)

def courbe(self, metrique):
return [e[metrique] for e in self.entrees if metrique in e]

def cout_cumule(self):
return sum(self.courbe("octets_cumules")[-1:]) / 1e9

def rendement(self):
"""Exactitude par gigaoctet transporte.

C'est l'indicateur de decision du triplet utilite, cost, budget.
"""
c = self.cout_cumule()
return self.courbe("exactitude_test")[-1] / c if c > 0 else 0.0

def diagnostiquer(self):
```

```
"""Signale les incidents usuels a partir des seuls indicateurs."""
a = self.courbe("exactitude_test")
alertes = []
if len(a) >= 2 and a[-1] < a[-2] and len(a) > 5:
alertes.append("exactitude decroissante : pas trop grand "
"ou heterogeneite trop forte")
if self.cout_cumule() > 1.0:
alertes.append("budget de communication depasse")
ecarts = [abs(a[i + 1] - a[i]) for i in range(len(a) - 1)]
if ecarts and max(ecarts) < 1e-3:
alertes.append("convergence prematuree : pas trop petit")
return alertes or ["aucune anomalie detectee"]


if __name__ == "__main__":
run = RunFedX(n_clients=20, rounds=50, agregation="fedavg", budget_go=2.0)
print("manifeste valide :", not run.valider())
chemin = run.ecrire("runs/manifeste.yaml")
print("ecrit :", chemin, "|", len(chemin.read_text().splitlines()), "lignes")
journal = [{"exactitude_test": 0.4 + 0.01 * t,
"octets_cumules": 1.2e8 * (t + 1)} for t in range(50)]
m = MoniteurFedX(journal)
print("cost final (Go) :", round(m.cout_cumule(), 2))
print("rendement :", round(m.rendement(), 3))
print("diagnostic :", m.diagnostiquer())
```

Livrable attendu

- `runs/manifeste.yaml` : configuration complète, commentée champ par champ.
- `experiences/exp19_incident.txt` : les trois incidents provoqués et leur diagnostic établi à partir des courbes seules.
- `experiences/exp20_strategies.txt` : trois stratégies sur le même manifeste, même budget.
- `tableau_strategies.md` : exactitude finale, coût, rendement par gigaoctet, diagnostic automatique.
- `RAPPORT_EXPERTISE.md`, trois pages : protocole, observations, diagnostic, recommandation argumentée.

Bareme indicatif

Critere	Points
Manifeste déclaratif complet et validé	3 pts
Exécution lancée et sorties vérifiées	3 pts
Incident provoqué et correctement diagnostiqué	4 pts
Comparaison des stratégies à budget égal	4 pts
Rapport d'expertise exploitable par un tiers	4 pts
Reproductibilité et iconographie	2 pts

Conseil

Le budget de communication est une contrainte dure, pas une cible indicative. Une exécution qui dépasse son budget mais converge mieux reste un échec du projet : le budget était la seule variable que l'on ne pouvait pas ajuster. Faites valider le manifeste par l'encadrant avant de lancer.

A retenir

- La progression des neuf TP est une construction : chaque séance rend nécessaire la précédente.
- L'algorithme de moyenne pondérée n'est pas un détail d'implémentation mais l'objet de mesure central de toute la partie.

- Compression et défense ont toujours un coût mesurable : l'erreur d'agrégation et la chute d'utilité doivent être rapportées avec le gain.
- Une garantie formelle s'obtient par un mécanisme, jamais par un post-traitement.
- Conduire une expérience complète — protocole, incident, diagnostic, recommandation — est la compétence terminale de ce cours.

Chapitre 7.3

Projets de fin de semestre

Un projet de six à huit semaines n'est pas un TP plus long : c'est une question ouverte, posée par un commanditaire, dont la réponse peut être « non » et doit alors être argumentée.

Les cinq projets ci-dessous correspondent aux cinq tensions structurelles de la discipline. Chacun est formulé comme une question à laquelle on ne connaît pas encore la réponse, et non comme une recette à appliquer. Les livrables exigent une recommandation : c'est la seule façon de distinguer un travail de recherche appliquée d'un montage de bibliothèques.

Projet	Question posée	Durée	Dominante technique	Risque principal
A — Grands modèles de langage fédérés	Un modèle peut-il s'adapter sans transmettre les données d'un métier ?	6 semaines	adaptation fédérée	absence de gain face au centralisé
B — Santé multi-hôpitaux	La segmentation fédérée rivalise-t-elle avec la centralisation ?	8 semaines	vision par ordinateur distribuée	hétérogénéité de protocole entre sites
C — Véhicules connectés coopératifs	Que gagne-t-on à coopérer entre véhicules proches ?	8 semaines	systèmes multi-agents	latence et couverture de la communication
D — Confidentialité et audit	Quelles garanties combiner pour un audit opposable ?	6 semaines	vie privée et robustesse	surcoût de la combinaison des défenses
E — Évolution de FedX	Peut-on rendre l'expérience fédérée reproductible par construction ?	6 semaines	outillage et reproductibilité	sur-couverture du périmètre

Tableau 1 – Panorama des cinq projets de fin de semestre

Projet – Grands modèles de langage fédérés par métier

Duree : 6 semaines | Equipe : 3 a 5 etudiants | Depot : gitlab + rapport

Une entreprise de services veut disposer d'un assistant spécialisé dans son métier, sans que les données de ses collaborateurs quittent jamais l'entreprise. Les modèles centraux de langage convergent vers d'excellentes performances mais généralistes ; les adapter à un métier exige des données que personne n'a le droit de partager. Le projet cherche à établir si l'adaptation fédérée des seuls adaptateurs de faible rang permet d'atteindre un niveau de performance comparable à l'affinage centralisé, sans'esprit prohibitive du coût de communication.

Missions

1. Construire un corpus de classification métier d'au moins vingt mille exemples, réparti en dix à vingt sites clients hétérogènes : taille, distribution de classes, qualité d'annotation.
2. Établir les trois références : affinage complet centralisé, adaptation centralisée, adaptation fédérée, à budget de calcul identique.
3. Mesurer le coût fédéré réel — octets, temps, mémoire — et le rapport au coût de l'affinage complet.

4. Étudier l'effet du rang r, du taux de participation et de l'hétérogénéité sur l'écart au centralisé.
5. Proposer et valider une stratégie d'agrégation des adaptateurs adaptée à leur géométrie, avec mise à l'échelle de la matrice supérieure.
6. Conclure par une recommandation de déploiement ou non, avec un seuil d'hétérogénéité au-delà duquel le gain fédéré disparaît.

Livrables

- Dépôt versionné, étiqueté, avec cible de rejouabilité et environnement verrouillé.
- Note de cadrage de deux pages : hypothèse, protocole, indicateurs, critères d'abandon du projet.
- Prototype fonctionnel : boucle d'agrégation sur adaptateurs, trois graines, trois valeurs du rang.
- Tableau comparatif des trois références — qualité, coût, mémoire — et courbes superposables.
- Mémoire de vingt à vingt-cinq pages : méthode, résultats, analyse, recommandation, limites.
- Présentation de soutenance de quinze minutes et dépôt d'un artefact reproductible.

Évaluation

- **Protocole et rigueur (25 %)** : hypothèse quantitative, contrôles figés avant lancement, trois graines au minimum, ablations justifiées.
- **Résultats (25 %)** : les trois références réellement exécutées, écarts types rapportés, tests de signification.
- **Profondeur d'analyse (25 %)** : mécanisme explicatif de l'écart observé, et pas seulement sa mesure.
- **Recommandation (15 %)** : décision argumentée, avec conditions d'emploi et seuil de passage.
- **Communication et autonomie (10 %)** : rapport, soutenance, gestion du temps, travail collectif équilibré.

Ressources

- Partie IV du cours : cycle de vie d'un grand modèle, familles d'adaptation, décomposition de faible rang.
- Partie III : attaques par inversion de gradient sur les adaptateurs.
- Bibliothèques d'adaptation et de quantification, pour le socle huit bits.
- Exemples de configuration d'adaptation par famille de modèles.

Projet – Santé : segmentation fédérée multi-hôpitaux

Duree : 8 semaines | Equipe : 3 a 5 etudiants | Depot : gitlab + rapport

Sept hôpitaux veulent construire ensemble un modèle de segmentation d'organes à partir de leurs scanners, sans que les images — ni même leurs annotations — ne sortent de l'établissement. La contrainte n'est pas seulement technique : les protocoles d'acquisition diffèrent, les populations diffèrent, et le cadre juridique interdit le plus souvent la centralisation. Le projet doit établir à quelles conditions la segmentation fédérée atteint un niveau exploitable en aide au diagnostic, et ce qu'il reste à faire localement.

Missions

1. Constituer un corpus multi-sites de référence : au minimum deux sites simulés aux caractéristiques distinctes — scanner, protocole, population, annotation.
2. Implémenter un réseau de segmentation et valider sa qualité centralisée comme référence absolue.
3. Déployer la segmentation fédérée avec plusieurs stratégies d'agrégation et plusieurs partitions :

aléatoire, mélangée, par site naturel.

4. Mesurer l'effet de l'hétérogénéité **de protocole** spécifiquement, en la séparant de l'hétérogénéité de population.
5. Implémenter un mécanisme de cohérence d'annotations — réconciliation d'étiquettes dans un espace commun — et mesurer son apport.
6. Proposer une architecture de gouvernance : ce qui reste au niveau local, ce qui est agrégé, ce qui ne circule jamais.

Livrables

- Prototype de segmentation avec protocole d'évaluation — coefficient de recouvrement, distance de Hausdorff, distance surfacique — reproductible.
- Campagne expérimentale complète : trois stratégies, trois partitions, trois graines, sorties brutes archivées.
- Figure d'analyse de la dégradation : performance par site en fonction de la distance au site moyen.
- Note de gouvernance des données de trois pages, conforme au cadre réglementaire applicable.
- Mémoire de vingt-cinq à trente pages, avec les contraintes réglementaires proprement citées et analysées.
- Démonstration fonctionnelle en soutenance.

Évaluation

- **Rigueur clinique simulée (20 %)** : protocole d'évaluation explicite, séparation stricte entraînement, validation et test.
- **Résultats (25 %)** : campagne complète, écarts types, comparaison aux références centralisées.
- **Analyse de l'hétérogénéité (25 %)** : séparation des sources de désaccord, mécanisme explicatif proposé.
- **Aspects réglementaires et éthiques (15 %)** : le non-partagé est traité comme une contrainte de conception et non comme un contournement.
- **Mémoire et soutenance (15 %)** : structure, clarté, honnêteté sur les limites.

Ressources

- Partie I : architectures de référence, et limites des réseaux convolutifs en imagerie médicale.
- Partie II : convergence de l'algorithme de moyenne pondérée, conditions de régularisation proximale.
- Partie IV : applications santé, protocole d'évaluation spécifique au domaine.
- Bibliothèques de traitement d'images médicales et format DICOM anonymisé pour les données.

Projet – Véhicules connectés : perception coopérative multi-niveaux

Duree : 8 semaines | Equipe : 3 a 5 etudiants | Depot : gitlab + rapport

La voiture connectée dispose d'une caméra qui ne voit qu'une fréquence d'images, sous la pluie, avec des angles morts. Un véhicule voisin possède des observations complémentaires. La communication entre véhicules permet de les mutualiser, mais elle coûte du temps et de la bande passante, et elle échoue. Le projet cherche à déterminer dans quelles conditions la coopération améliore la perception, et quelle politique de sélection des voisins est réellement utile.

Missions

1. Constituer un banc de simulation multi-véhicules, avec conditions météorologiques dégradées paramétrables.
2. Implémenter la perception locale et définir une métrique d'utilité précise : distance minimale au voxel détecté.
3. Implémenter un mécanisme de partagé sélectif : chaque véhicule transmet un message compressé, la volumétrie et le niveau de détail étant négociés.
4. Étudier les trois niveaux de coopération : partage de sorties, partage de caractéristiques, partage de tenseurs intermédiaires.
5. Évaluer le compromis latence, bande passante et gain de perception, et identifier le point de saturation.
6. Proposer un mécanisme d'adaptation dynamique du niveau de coopération selon la qualité du lien mesurée.

Livrables

- Scénario de simulation reproductible, paramétrable par météo, densité de trafic et latence réseau.
- Trois niveaux de coopération implémentés et mesurés séparément.
- Courbes d'utilité en fonction de la latence simulée, à quatre valeurs.
- Carte de chaleur du gain en fonction de la portée de communication et de la densité.
- Mémoire de vingt à vingt-cinq pages avec un chapitre dédié aux compromis temps réel.
- Vidéo de démonstration de deux minutes de la coopération multi-niveaux.

Évaluation

- **Qualité de la simulation (20 %)** : scénario crédible, variations contrôlées, protocole documenté.
- **Résultats (25 %)** : mesures sur toute la plage de latence, écarts types, courbes superposables.
- **Analyse des niveaux de coopération (20 %)** : le gain marginal de chaque niveau est isolé et expliqué.
- **Réalisme temps réel (20 %)** : les temps de calcul et de transmission sont mesurés, et non estimés.
- **Mémoire et démonstration (15 %)** : clarté, figures, reproductibilité de la vidéo.

Ressources

- Partie II : bandits, exploration et exploitation, adaptation en ligne.
- Partie IV : architectures de véhicules connectés et niveaux de coopération.
- Simulateurs de trafic et protocoles de transport pour modéliser la latence.
- Compression : quantification, codage entropique, réduction de taille de message.

Projet – Confidentialité et audit : differentially private et défenses robustes

Duree : 6 semaines | Equipe : 3 a 5 etudiants | Depot : gitlab + rapport

Une autorité de contrôle demande des garanties, pas des promesses. Un système d'apprentissage fédéré doit être démontrable : il doit garantir qu'aucune inférence n'est extraite d'un client, et que son comportement résiste à un client malveillant. Le projet construit un tel système, mesure le coût de chaque garantie, et produit un dossier d'audit reproductible par un tiers.

Missions

1. Implémenter la chaîne complète de garanties : agrégation sécurisée par masquage de paires.
2. Ajouter l'entraînement différentiellement privé au niveau client, avec un budget $\square$ alloué explicitement et traçable tour par tour.
3. Ajouter une défense contre l'empoisonnement — médiane, ou sélection robuste — et mesurer son coût interactif avec la vie privée.
4. Construire un journal d'audit horodaté et signé, permettant de reconstruire a posteriori les paramètres de confidentialité retenus.
5. Réaliser un test d'attaque adverse : tentative d'inversion de gradient et empoisonnement, avec et sans garanties, quantifiés.
6. Produire une procédure de quantification du risque résiduel, avec seuils d'acceptation et plan de réponse en cas de dépassement.

Livrables

- Prototype sécurisé de bout en bout : agrégation sécurisée, vie privée différentielle et défense robuste.
- Tableau de coût : exactitude, budget $\square$, coût de communication et latence, pour chaque combinaison de garanties.
- Rapport d'attaque adverse documenté, avec courbes de taux de reconstruction et d'injection réussie.
- Journal d'audit signé, rejouable et vérifiable par un script fourni.
- Procédure de gestion du risque résiduel : seuils, responsabilités, plan de réponse.
- Mémoire de vingt à vingt-cinq pages, appendice juridique et annexe technique reproductible.

Évaluation

- **Exactitude des garanties (30 %)** : les mécanismes sont correctement implémentés et vérifiés, tests négatifs compris.
- **Coût mesuré (20 %)** : chaque garantie est chiffrée en qualité, latence et communication.
- **Test d'attaque (20 %)** : l'attaque est conduite avec rigueur, ses conditions de succès sont établies.
- **Procédure d'audit (20 %)** : le dossier est rejouable par un tiers sans accompagnement.
- **Communication (10 %)** : un destinataire non technique comprend la garantie et son coût.

Ressources

- Partie III, chapitres 1 à 3 : taxonomie des attaques, mécanismes différentiels, agrégation sécurisée.
- TP 7.5 et TP 7.6 comme base de travail réutilisable.
- Bibliothèques de chiffrement et d'agrégation sécurisée.
- Cadre réglementaire : RGPD, HIPAA, règlement européen sur l'intelligence artificielle, pour le cadrage du dossier d'audit.

Projet – FedX : évolution du framework

Duree : 6 semaines | Equipe : 3 a 5 etudiants | Depot : gitlab + rapport

Le principal frein à l'industrialisation de l'apprentissage fédéré n'est pas algorithmique : c'est l'absence d'infrastructure de réplication. Une équipe qui obtient un résultat sur un jeu de données ne peut le rejouer, et donc ne peut ni l'auditer, ni le comparer, ni l'exploiter. Le projet évolue le moteur FedX pour rendre la reproductibilité structurelle plutôt que disciplinaire.

Missions

1. Auditer FedX tel qu'il est : ce qu'il garantit, ce qu'il ne garantit pas, ce qui casse en cas de reprise à mi-parcours.
2. Concevoir un schéma de configuration déclaratif complet, capable de décrire une exécution de façon exhaustive et versionnable.
3. Implémenter le rejeu déterministe : même manifeste, même sortie octet pour octet, y compris sur une autre machine.
4. Développer un système d'analyse comparative : superposition de courbes, tests statistiques, détection automatique d'anomalies.
5. Construire un tableau de bord web de pilotage d'une exécution en cours, avec alertes sur dépassement de budget.
6. Évaluer le framework sur trois études de cas et rédiger les recommandations d'évolution.

Livrables

- Rapport d'audit du framework existant, avec une matrice « garantit, ne garantit pas ».
- Spécification du schéma de configuration versionné, avec schéma de validation et messages d'erreur exploitables.
- Test de rejeu déterministe : le même manifeste produit le même résultat, avec preuve par empreinte.
- Module d'analyse comparative : courbes superposées, rééchantillonnage, tests de Student appariés.
- Tableau de bord web fonctionnel, avec au moins trois règles d'alerte.
- Mémoire de vingt à vingt-cinq pages, plus un guide de contribution au projet ouvert.

Évaluation

- **Rejouabilité démontrée (30 %)** : la preuve du rejeu octet pour octet est fournie et reproductible par un tiers.
- **Qualité de la conception (20 %)** : le schéma est clair, validé, versionnable et non ambigu.
- **Analyse comparative (20 %)** : l'outil produit des conclusions statistiquement justifiées, et pas seulement des courbes.
- **Études de cas (20 %)** : les trois cas sont traités en profondeur, y compris leurs limites.
- **Contribution au projet (10 %)** : documentation, guide de contribution, code potentiellement fusionnable.

Ressources

- Partie V : architecture de FedX, séquence d'une exécution, tableau de bord, résultats de référence.
- TP 7.9 comme point de départ et cas d'école.

- Bibliothèques de validation de configuration et de visualisation pour le tableau de bord.
- Framework de référence pour la comparaison des stratégies d'agrégation.

A retenir

- Un projet de fin de semestre pose une question ouverte et attend une réponse argumentée, y compris négative.
- Les cinq projets couvrent les cinq tensions de la discipline : coût, hétérogénéité, temps réel, garantie, reproductibilité.
- Le critère d'évaluation le plus lourd porte toujours sur le protocole et l'analyse, jamais sur la seule mise en œuvre.
- Un livrable qui recommande une décision est plus exigeant qu'un livrable qui démontre une technique.
- La contribution vérifiable — dépôt, tests, documentation — fait partie de l'évaluation du projet.

Chapitre 7.4

Exercices d'entraînement

Quinze problèmes corrigés, de difficulté croissante, pour convertir les résultats théoriques en réflexes calculatoires. Chaque correction est complète et vérifiable.

Un cours de ce niveau ne se récite pas. Les quinze exercices ci-dessous mêlent théorie et calcul : certains demandent de démontrer un résultat, d'autres de le calculer numériquement, d'autres encore de combiner deux notions vues dans des chapitres différents. Ils sont conçus pour être résolus en vingt à quarante minutes chacun.

N°	Thème	Difficulté	Parties mobilisées
1	Risque empirique et écart de généralisation	Facile	I
2	Gradient d'une perte quadratique	Facile	I
3	Convexité et courbure	Facile	I
4	Taux de convergence de la moyenne pondérée	Moyen	II
5	Sélection robuste	Moyen	III
6	Composition du budget de vie privée	Moyen	III
7	Coût mémoire d'un adaptateur de faible rang	Moyen	IV
8	Hétérogénéité par mélange de Dirichlet	Moyen	I, II
9	Agrégation pondérée	Moyen	II
10	Complexité de communication	Moyen	II
11	Calcul d'un budget de vie privée	Difficile	III
12	Formule d'effet moyen	Difficile	IV
13	Mécanisme de Laplace	Difficile	III
14	Mécanisme d'attention	Difficile	I
15	Arbre de décision et impureté	Difficile	I

Tableau 1 – Progression des quinze exercices

Exercice 7.4.33 (difficulte : Facile)

On considère un modèle de régression linéaire $f_\theta(x) = \theta^T x$ sur un échantillon $S = (x_i, y_i)_{i=1}^m$ avec perte quadratique $L(\theta, S) = (1)/(2m) \sum_{i=1}^m (f_\theta(x_i) - y_i)^2$.

1. Donner l'expression de $\nabla L(\theta, S)$.
2. On observe $L(\theta, S) = 0{,}08$ et le risque attendu vaut $0{,}11$. Interpréter la valeur $0{,}03$ et donner une borne généralisation utilisable.

Indice : Le gradient d'une moyenne se calcule terme à terme ; la borne généralisation majoré l'écart au lieu de l'égaliser.

Correction

- **1.** $\nabla L(\theta, S) = (1)/(m) X^T (X\theta - y)$ où X est la matrice des observations.
- **2.** L'écart mesuré vaut $G = |0{,}11 - 0{,}08| = 0{,}03$. C'est un **écart de généralisation** : il mesure la distance entre la performance sur l'échantillon et la performance en population.
- Une borne utilisable est de type $O≤ft(√((\kappa \log(2m/\delta))/(m)))$ avec κ la complexité de marge de la classe, ou, pour une matrice de covariance de rang d borné, $O≤ft((\| X\|^2)/(m))$. Avec $d = 5$ caractéristiques et m observations, on vérifie numériquement qu'une borne en $1/m$ fournit un écart compatible avec les $0{,}03$ observés dès que $m >rsim 20\, d$.
- **Conclusion** : un écart de $0{,}03$ est parfaitement compatible avec un modèle correctement appris. Il ne signale pas un surapprentissage, mais la simple différence entre l'échantillon et la population.

Exercice 7.4.34 (difficulte : Facile)

Soit $L(w) = (1)/(n) \sum_{i=1}^n (w^T x_i - y_i)^2$ avec $n = 3$, matrice de second moment $A = (2\ 0\{,\}5 \backslash 0\{,\}5\ 3)$ et $b = (1, 2)^T$.

1. Écrire le gradient de L.
2. Calculer sa valeur exacte en $w_0 = (0, 0)$.
3. Effectuer une itération de descente de gradient avec $\eta = 0{,}1$ et donner w_1.
4. Comparer à l'itération obtenue sur le polynôme $L(w) = w^4 - 4w^3 + w$, et commenter.

Indice : $\nabla L = 2(Aw - b)$; la mise à jour est $w_1 = w_0 - \eta \nabla L(w_0)$.

Correction

- **1.** $\nabla L(w) = (2)/(n) X^T (Xw - y) = 2(Aw - b)$.
- **2.** En $w_0 = (0, 0)$: $\nabla L(w_0) = 2(0 - b) = -2b = (-2 \backslash -4)$.
- **3.** $w_1 = w_0 - \eta \nabla L(w_0) = (0, 0) - 0{,}1 \cdot (-2, -4) = (0\{,\}2 \backslash 0\{,\}4)$. **4.** Avec le polynôme $p(w) = w^4 - 4w^3 + w$: $p'(w) = 4w^3 - 12w^2 + 1$, donc $p'(0) = 1$ et $w_1 = 0 - 0{,}1 \cdot 1 = -0{,}1$.
- La mise à jour formelle est identique dans les deux cas — le gradient moins le pas multiplié par le taux — mais la **dynamique** ne l'est pas : le polynôme a $p''(0) = 0$, donc le point de départ est un point d'inflexion, et le pas de $0{,}1$ en sort dans la zone où la pente est faible. La forme de la mise à jour est universelle ; la stabilité ne l'est pas.

Exercice 7.4.35 (difficulte : Facile)

Pour la fonction $f(w) = w^3 - 3w$ sur $\mathbb{R}$:

1. Calculer la dérivée première et la dérivée seconde.
2. Déterminer l'ensemble des points de rebroussement.
3. Justifier que f n'est ni convexe ni concave sur $\mathbb{R}$.
4. Montrer que la descente de gradient avec un pas constant converge vers un minimum local, et identifier lequel, à partir de $w_0 = 2$.

Indice : Une fonction n'est convexe que si $f'' \geq 0$ en tout point ; les racines de f'' délimitent les intervalles où le signe est constant.

Correction

- **1.** $f'(w) = 3w^2 - 3$ et $f''(w) = 6w$.
- **2.** Le point de rebroussement est l'unique racine de f'' : $w = 0$. En ce point, $f'(0) = -3 \neq 0$, l'inflexion est donc **simple**, et non dégénérée d'ordre supérieur.
- **3.** $f'' < 0$ pour $w < 0$: la fonction y est strictement concave. $f'' > 0$ pour $w > 0$: elle y est strictement convexe. Sur les deux demi-droites la fonction est donc convexe ou concave, mais sur $\mathbb{R}$ elle n'est ni l'une ni l'autre.
- **4.** Points critiques : $w = \pm 1$. En $w = 1$, $f(1) = -2$ et $f''(1) = 6 > 0$: c'est un **minimum local strict**. En $w = -1$, $f(-1) = 2$ et $f''(-1) = -6 < 0$: maximum local strict.
- Itération $w_{t+1} = w_t - \eta(3w_t^2 - 3)$ avec $\eta = 0{,}1$, depuis $w_0 = 2$: $w_1 = 2 - 0{,}1(9) = 1{,}1$; $w_2 = 1{,}1 - 0{,}1(0{,}63) = 1{,}037$; $w_3 \approx 1{,}003$. La suite converge vers $w^\star = 1$, le minimum local, et non vers l'inflexion.
- **Conséquence pour l'apprentissage fédéré** : en non-convexité, la convergence ne garantit qu'un **point stationnaire** — gradient nul — et non un minimum global. C'est la raison pour laquelle les garanties de généralisation y sont énoncées avec un terme de gradient résiduel.

Exercice 7.4.36 (difficulte : Moyen)

Dans la condition de convergence, on retient la borne $F(w_T) - F^\star \leq (L)/(2) \| w_T - w^\star\|^2 + O\leq ft((L\sigma^2\kappa_T)/(T))$ avec $L = 2$, $\sigma^2 = 0{,}5$, $\kappa_T = 1/(2T)$ et un terme initial nul. 1. Calculer la borne pour $T = 10$, $T = 100$ et $T = 1000$.

2. Combien de tours faut-il pour descendre sous 10^{-2} ?

3. Quel est l'effet d'un quadruplement de σ sur le nombre de tours nécessaires ?

4. Comparer avec l'entraînement centralisé à budget de calcul identique.

Indice : Le terme de bruit décroît comme $1/T^2$ dans cette expression ; isolez $T \geq L\sigma^2\kappa / (2\varepsilon)$.

Correction

- **1.** Le terme de bruit vaut $(L\sigma^2\kappa_T)/(T) = (2 \times 0{,}5 \times 1/(2T))/(T) = (0{,}5)/(T^2)$. Pour $T = 10$: $5 \text{ times } 10^{-3}$. Pour $T = 100$: $5 \text{ times } 10^{-5}$. Pour $T = 1000$: $5 \text{ times } 10^{-7}$. **2.** Résoudre $(0{,}5)/(T^2) \leq 10^{-2}$ donne $T^2 \geq 50$, donc $T \geq 8$. En prenant une marge de sécurité, on retient $T \approx 20$ **tours**. **3.** Quadrupler σ multiplie la borne par quatre : il faut donc quatre fois plus de tours pour atteindre la même cible, soit un coût de communication quadruplé. **La variance de l'hétérogénéité coûte linéairement en communications**, et c'est le résultat central de la partie sur les algorithmes.
- **4.** En centralisé, un tour équivaut à une passe complète sur les données : vingt tours correspondent à vingt passes. En fédéré avec C itérations locales et N clients, vingt tours correspondent à $20 \times C \times N$ passes équivalentes, soit **plus** de calcul pour un résultat théoriquement plus lent. Le gain de l'apprentissage fédéré n'est donc pas en efficacité d'échantillonnage : il est en **accès aux données**.

Exercice 7.4.37 (difficulte : Moyen)

Dans un système de $N = 20$ clients, $f = 4$ sont malveillants. Les mises à jour, de dimension deux, sont $u_1 = (0, 0)$, $u_2 = (0{,}1, 0)$, $u_3 = (0{,}2, 0)$, $u_4 = (0{,}3, 0)$, et les seize autres sont tirés uniformément sur le cercle unité.

1. Calculer le nombre m de voisins utilisés par la sélection robuste.

2. Pour les quatre premiers candidats, calculer la somme des m distances les plus proches.

3. Lequel est retenu ? Est-ce le résultat attendu ?

4. Proposer une variante plus stable et justifier.

Indice : $m = n_{\min} - f + 2$; pour des points alignés, les distances sont des valeurs absolues.

Correction

- **1.** $n_{\min} = N - f = 16$, donc $m = 16 - 4 + 2 = 14$.
- **2.** Les distances deux à deux parmi les quatre candidats alignés sont des différences : $d(u_1, u_2) = 0{,}1$, $d(u_1, u_3) = 0{,}2$, $d(u_1, u_4) = 0{,}3$, $d(u_2, u_3) = 0{,}1$, et ainsi de suite. Pour chaque candidat, on somme les quatorze plus petites distances, complétées par les distances aux seize points du cercle, de l'ordre de l'unité.
- **3.** $u_1 = (0, 0)$ est le candidat qui minimisé la somme : c'est le point le plus **central** du petit segment aligné, et il est retenu. C'est le résultat attendu : u_1 est le seul candidat à distance bornée de tous les autres. Les trois autres sont des points extrêmes du segment, et leur score est plus élevé parce qu'ils sont loin du nuage.

- **4. Variante plus stable : la sélection multiple.** On ne retient pas un seul candidat mais les m meilleurs, et on les agrège. Cela corrige un défaut réel : lorsque les mises à jour légitimes sont toutes proches, les distances sont presque nulles et la sélection devient arbitraire. Le recours à plusieurs candidats supprime cette dégénérescence au prix d'une robustesse légèrement moindre face aux attaques coordonnées.

Exercice 7.4.38 (difficulte : Moyen)

Un protocole protège chaque client par un budget ☐ = 0,1 par tour, avec N = 50 clients et T = 100 tours.
1. Quelle est la borne naïve par composition élémentaire ?
2. La borne réelle est-elle de cet ordre ? Justifier.
3. Que change le passage d'un mécanisme gaussien à un mécanisme de Laplace sur la composition ?

Indice : La composition élémentaire est toujours pessimistic : elle borne par la somme des epaisseurs, sans exploiter la forme de la loi.

Correction

- **1.** Par composition élémentaire, le budget cumulé est majoré par la somme : N · T · ☐ = 50 × 100 × 0,1 = 500. **2.** La borne réelle est nettement inférieure. Un mécanisme gaussien n'a pas de support borné, donc l'inégalité de concentration ne fournit pas de majoration par la somme des *, mais par la somme des carrés. Sur l'échelle de la divergence de Rényi, epsilon $_{\mathrm{mathrmRDP}}$(alpha) = frac alpha - 12 sigma^2 est **indépendant du nombre d'étapes** ; c'est la conversion en budget final, qui comporte un terme en log(1/ delta) et une conversion en (alpha - 1), qui est sous-linéaire en T. On obtient typiquement un budget cumulé de quelques unités, soit deux ordres de grandeur sous la borne naïve.
- **3.** Le mécanisme de Laplace a un support borné : l'inégalité de concentration donne une majoration par la somme des *, et la composition reste linéaire en T. Le mécanisme gaussien donne donc une composition sous-linéaire, ce qui est la raison de sa domination en apprentissage automatique. Le prix : à sensibilité égale, le bruit gaussien est plus grand, car il n'a pas de support borné.

Exercice 7.4.39 (difficulte : Moyen)

Un modèle de sept milliards de paramètres est adapté avec des adaptateurs de rang r sur les quatre projections d'attention de 32 couches, de dimension d$_{\mathrm{model}}$ = 4096. Le coefficient d'échelle vaut α = 16.
1. Calculer le nombre de paramètres entraînables.
2. En déduire le volume transmis par client et par tour, en précision simple.
3. Comparer avec un affinage complet des poids.
4. Que se passe-t-il si l'on double r sans modifier α ?

Indice : Chaque adaptateur est le produit de deux matrices, une de d × r et une de r × d ; le coefficient d'échelle ne modifie que l'inférence.

Correction

- **1.** Une couche d'adaptation à rang r sur une projection de dimension d comporte r colonnes et r lignes, soit 2dr paramètres. Avec quatre projections par couche et 32 couches : 4 × 32 × 2dr = 256dr. Pour d = 4096 et r = 8 : 256 × 4096 × 8 = 8,4 millions, soit 0,12 pour cent des 7 × 10^9 poids. **2.** En précision simple sur 32 bits : 8,4 × 10^6 × 4 octets ≈ 33,5 mégaoctets par client et par tour. C'est supportable là où un affinage complet ne l'est pas.
- **3.** L'affinage complet transporterait 7 × 10^9 × 4 octets, soit 28 gigaoctets : un rapport de huit cents. C'est l'argument décisif.
- **4.** Doubler r double le volume transporté, et l'échelle effective alpha/r est divisée par deux, donc la correction est deux fois plus faible à amplitude de poids identique. L'entraînement devient instable. La règle pratique : doubler r et diviser alpha par deux, pour préserver l'échelle effective alpha/r.

Exercice 7.4.40 (difficulte : Moyen)

Pour α ∈ 0,1, 0,3, 1, 10, 100, la loi de Dirichlet répartit les étiquettes d'un client selon des proportions de concentration α.
1. Donner l'entropie normalisée moyenne des distributions de classes, en limite de concentration faible.
2. Expliquer pourquoi α → ∞ ne donne pas exactement la répartition uniforme.

3. En déduire comment choisir α pour une expérience qui doit représenter un consortium de dix sites.

Indice : Une loi de Dirichlet de concentration α sur K composantes a une moyenne $1/K$ et une variance qui décroît comme $1/(\alpha K - 1)$.

Correction

- **1.** En concentration faible, les proportions se concentrent sur les sommets du simplexe : un client ne porte alors qu'une ou deux classes, et l'entropie normalisée tend vers zéro. La limite est atteinte asymptotiquement, jamais exactement pour α fini.
- **2.** La moyenne de chaque proportion vaut $1/K$ quelle que soit la concentration, donc l'espérance de l'entropie ne tend pas vers $\log K$: la concentration reste positive pour toute concentration finie. $\text{alpha} = 100$ donne une entropie normalisée d'environ $0{,}97$ sur dix classes, ce qui est proche mais non identique à l'uniforme. C'est une bonne borne haute expérimentale, et il faut le dire dans le rapport.
- **3.** Pour un consortium de dix sites, choisir la concentration par calibrage inverse : mesurer l'entropie réelle des données de chaque site, puis choisir α qui reproduit cette entropie. Cela rend la simulation représentative au lieu de conventionnelle, et c'est la seule défense contre un résultat qui repose sur une hypothèse de mélange erronée.

Exercice 7.4.41 (difficulte : Moyen)

Trois clients transmettent des mises à jour de poids w_1, w_2 et w_3, de poids respectifs 1, 2 et 1.

1. Calculer l'agrégation pondérée.
2. Montrer qu'elle coïncide avec la minimisation de la perte globale lorsque les optimisations locales sont exactes.
3. Que devient l'agrégation si l'un des clients envoie un vecteur aberrant, et pourquoi la pondération par la taille ne protège pas ?

Indice : La perte globale est la somme des pertes locales pondérée par la taille ; l'agrégation est l'image de l'argument de cette somme.

Correction

- **1.** L'agrégation vaut $(1 \cdot w_1 + 2 \cdot w_2 + 1 \cdot w_3)/(1 + 2 + 1)$, soit le moyenne des trois mises à jour avec un poids double pour le deuxième client.
- **2.** La perte globale s'écrit $F(w) = \sum_k (n_k)/(n)\, F_k(w)$. Au point de stationnarité, $\nabla F = \sum_k (n_k)/(n)\, \nabla F_k = 0$. Si chaque optimisation locale a convergé, $\nabla F_k(w) = 0$ pour tout k, donc le gradient global est nul et le point est stationnaire pour la perte globale. L'agrégation pondérée par la taille est donc exactement la solution de minimisation, et c'est ce qui explique que l'algorithme soit juste dans le cas déterministe.
- **3.** L'agrégation est une moyenne pondérée : un vecteur aberrant mais de poids non négligeable tire la moyenne vers lui, d'autant plus qu'il est éloigné. La pondération par la taille ne protège d'aucune manière, puisqu'elle ne distingue pas un gradient authentique d'un gradient forgé de même norme. C'est exactement ce qui rend l'agrégation robuste nécessaire en présence de clients malveillants : la pondération par la taille gère l'hétérogénéité, pas l'adversité.

Exercice 7.4.42 (difficulte : Moyen)

Un protocole comporte K clients, chacun exécutant E itérations locales sur B exemples, avec d paramètres par couche. La participation est C.

1. Donner l'ordre de grandeur du volume échangé par tour.
2. Donner l'ordre de grandeur du volume cumulé sur T tours.
3. Comparer au coût d'un entraînement centralisé sur les mêmes données, et dire dans quel cas le fédéré est plus coûteux.

Indice : Comptez deux flux par tour : la descente du modèle et la remontée des mises à jour.

Correction

- **1.** Par tour, le serveur diffuse le modèle à KC clients sélectionnés, soit KCd valeurs, et reçoit autant de mises à jour. Le volume total est d'ordre de 2KCd valeurs, soit 8KCd octets en précision simple. Avec E et B sans effet sur le volume : ils jouent sur la qualité, pas sur le coût de transport.
- **2.** Sur T tours : 8TKdC octets. Le coût croît linéairement en T, en K, en d et en C.
- **3.** L'entraînement centralisé ne transporte que les données, une fois, et ce coût est indépendante de T. L'apprentissage fédéré est donc plus coûteux dès que TKdC dépasse la taille du jeu de données. C'est le cas presque systématique en imagerie, où le jeu de données se compte en gigaoctets et le modèle en centaines de mégaoctets. Il ne l'est pas en adaptation de faible rang, où d est remplacé par 2r : c'est exactement pourquoi l'adaptation de faible rang est le seul régime praticable.

Exercice 7.4.43 (difficulte : Difficile)

Un mécanisme de Laplace est appliqué à une requête dont la sensibilité L_1 vaut 2,0. On vise un budget $\square = 0{,}5$ et une probabilité d'échec $\delta = 10^{-5}$.

1. Calculer l'échelle du bruit requis.

2. Donner l'intervalle de confiance à $(1-\delta)$ sur la sortie, et l'exprimer comme un facteur multiplicatif de l'écart-type de la sortie non bruitée.

3. Répéter pour $\square = 0{,}1$ et conclure sur le rapport entre budget et utilité.

Indice : L'échelle de Laplace vaut $\Delta_1 f / \square$; l'intervalle est $\pm \log(1/\delta)\, \Delta_1 f / \square$ autour de la valeur vraie.

Correction

- **1.** L'échelle vaut $b = \Delta_1 f / \square = 2{,}0 / 0{,}5 = 4{,}0$.
- **2.** L'intervalle de confiance à probabilité 1- delta est pm log(1/ delta) Delta_1 f / epsilon, soit pm $\log(10^5)$ times 2,0 / 0,5 = pm 11,51 times 4,0 = pm 46,0 autour de la valeur non bruitée. Rapporté à l'écart-type d'une loi normale de même variance, l'écart-type de Laplace vaut sqrt2 b, soit ici 5,66 : l'intervalle de confiance couvre donc environ 46,0 / 5,66 approx 8,1 écarts-types. C'est très large. **3.** Avec epsilon = 0,1, l'échelle devient b = 20,0 et l'intervalle pm 230, soit quarante fois plus large. Diviser le budget par cinq multiplie l'incertitude par cinq, **linéairement**. C'est la propriété centrale de la confidentialité différentielle : le coût en utilité est logarithmique enutility mais linéaire en 1/ epsilon, et c'est pourquoi les valeurs recommandées en pratique — entre 1 et 10 — paraissent élevées alors qu'elles sont déjà très restrictives.

Exercice 7.4.44 (difficulte : Difficile)

Un effet moyen de traitement est estimé sur K = 10 centres, chacun transmettant son estimation $\hat{\tau}_k$ et sa variance hat sigma_k^2. 1. Écrire l'agrégation de Rao-Blackwell et la variance du total.

2. Comparer avec une agrégation par effectif simple.

3. Que se passe-t-il si un centre affiche un effet de signe opposé aux neuf autres, tous significatifs ?

Indice : Les populations étant disjointes, les variances s'additionnent ; l'inverse de leur somme est donc la variance de la moyenne pondérée.

Correction

- **1.** L'estimateur agrégé est $\hat{\tau}_{\mathrm{mathrmglob}} = \mathrm{frac}\ \mathrm{sum}_k\ w_k\ \mathrm{hat}\ \tau_k\ \mathrm{sum}_k\ w_k$ avec $w_k = 1/\ \mathrm{hat}\ \mathrm{sigma}_k^2$, et la variance du total vaut widehat mathbbV = $1\ /\ \mathrm{sum}_k\ w_k$. C'est la conséquence directe de l'indépendance des estimations entre centres disjoints.
- **2.** L'agrégation par effectif simple pondère par n_k et suppose l'égalité des variances relatives. Elle est plus robuste à une variance mal estimée mais moins efficace lorsque les centres diffèrent en qualité — un petit centre bruité pèse autant qu'un grand centre propre. La pondération par l'inverse de la variance est le bon compromis, à condition que l'estimation de variance soit elle-même fiable, ce qui suppose des effectifs par centre suffisants.

- **3.** C'est compatible avec un effet homogène nul : des estimations indépendantes de moyenne nulle produisent statistiquement des signes opposés significatifs, surtout à faible puissance. Un test par centre ne répond pas à la question d'homogénéité. Il faut un test formel de cohérence inter-centres, ou un intervalle de prédiction pour un centre nouveau. Noter que la variance de l'estimation du centre divergent influe son poids : un centre très bruité pèse peu, ce qui limite naturellement le dégât.

Exercice 7.4.45 (difficulte : Difficile)

Soit une fonction de perte dont la sensibilité L_1 vaut 3. On applique un mécanisme de Laplace avec $\square = 0{,}4$.
1. Calculer l'échelle et l'intervalle de confiance à $\delta = 10^{-6}$.
2. Comparer avec le mécanisme gaussien à sensibilité identique.
3. Dans quel cas le mécanisme de Laplace est-il malgré tout préférable, et pourquoi ?

Indice : L'écart-type d'une loi de Laplace d'échelle b vaut $\sqrt{(2)}\, b$; celui d'une loi gaussienne de même variance vaut b.

Correction

- **1.** L'échelle vaut $b = 3 / 0{,}4 = 7{,}5$. L'intervalle de confiance est pm log(1/ delta) b = pm 13,82 times 7,5 = pm 103,6. **2.** À variance identique, la loi de Laplace a une queue plus lourde : son écart-type est sqrt2 b pour la même échelle, et sa probabilité d exceeding $|x| > t$ décroît comme $e^{-t/b}$ contre $e^{-t\hat{}2/2\ \mathrm{sigma}\hat{}2}$ pour la gaussienne. L'intervalle de confiance est donc **plus large** pour la loi de Laplace. En contrepartie, la garantie est **exacte** : elle vaut pour $\mathrm{delta} = 0$, ce que la gaussienne n'offre pas. **3.** Le mécanisme de Laplace est préférable dans deux cas. D'abord quand la quantité à protéger est bornée et que l'on veut une garantie exacte sans approximation. Ensuite, et surtout, quand la sortie est elle-même bornée ou discrète — un score de classification, un indice dans un ensemble fini : le bruit est alors **tronqué** à l'intervalle possible, et la perte d'utilité devient nulle, alors que le bruit gaussien tronqué conserve la perte sans gagner la garantie. C'est la raison pour laquelle les mécanismes discrets dominent les données tabulaires à sortie bornée.

Exercice 7.4.46 (difficulte : Difficile)

Dans un bloc d'attention, la tête i produit les scores $s_j = q_i^{\top} k_j / \sqrt{(d_k)}$ pour $j = 1, \ldots, n$.
1. Donner l'expression de la sortie, avec le masque causal M.
2. Montrer que sans le facteur $1/\sqrt{(d_k)}$ la sortie dégénère pour d_k grand.
3. Calculer le coût mémoire de la matrice d'attention pour $n = 4096$ et $d_k = 64$.

Indice : Les logits sont des produits scalaires de vecteurs de norme approximativement $\sqrt{(d_k)}$: leur variance croît comme d_k.

Correction

- **1.** La sortie est mathrmAttention(Q, K, V) = mathrmsoftmax ≤ft(fracQK ^{top} sqrtd_k + M right) V, où M est une matrice d'adjonction ou de - infty qui masque les positions futures. **2.** Si q et k ont des composantes indépendantes de variance unitaire, alors $q^{\top} k$ est une somme de d_k termes, de moyenne nulle et de variance d_k : l'écart-type des logits est sqrtd_k. Or la softmax sature dès que l'écart entre logits dépasse quelques unités. Diviser par sqrtd_k ramène l'écart-type à l'unité, indépendamment de la dimension. Sans cette normalisation, la softmax tend vers la fonction de arg max dès que d_k dépasse quelques dizaines, et le gradient s'annule : c'est un problème de stabilité numérique, pas de qualité du modèle. **3.** La matrice d'attention a n^2 entrées, soit $4096^2 = 16{,}8$ millions de valeurs, donc 67 mégaoctets en précision simple par tête et par couche. C'est trois ordres de grandeur de plus que les poids themselves dans un modèle de dimension $d_k = 64$, et c'est la raison pour laquelle l'attention dense est coûteuse en mémoire.

Exercice 7.4.47 (difficulte : Difficile)

Un arbre de décision minimise l'impureté de Gini $I = 1 - \sum_c p_c^{\,2}$ sur un nœud contenant n exemples répartis en classes de effectifs $n_1, \ldots, n_C$.
1. Donner l'expression de l'impureté en fonction des effectifs.
2. Montrer que la coupure qui minimise l'impureté pondérée maximise le gain de Gini, et en donner l'expression.

3. Calculer le gain pour une coupure séparant 30/70 contre 70/30 sur deux classes de mêmes effectifs dans chaque nœud.

Indice : Le gain est la différence entre l'impureté du parent et la moyenne pondérée de celles des fils ; l'identité sur deux classes donne une forme fermée.

Correction

- **1.** Avec $p_c = n_c / n$, l'impureté vaut $I = 1 - \sum_c (n_c/n)^2 = 1 - \frac{\sum_c n_c^2}{n^2}$. **2.** Le gain d'une coupure en deux fils d'effectifs n_g et n_d est $G = I(\text{parent}) - \frac{n_g}{n} I(\text{gauche}) - \frac{n_d}{n} I(\text{droite})$. Maximiser ce gain revient à minimiser l'impureté pondérée des fils, et l'identité $G = \frac{n_g n_d}{n^2} \left(\sum_c p_c^g - \sum_c p_c^d \right)^2$ pour deux classes montre que le gain est maximal lorsque les deux fils ont des distributions de classes les plus opposées possible.
- **3.** Sur deux classes de mêmes effectifs dans chaque nœud, l'impureté parent vaut $1 - (0{,}5^2 + 0{,}5^2) = 0{,}5$. Chaque fils a la même répartition, donc $I = 0{,}5$ également. Le gain vaut donc $0{,}5 - 0{,}3 \times 0{,}5 - 0{,}7 \times 0{,}5 = 0$. **La coupure n'apporte rien** : elle ne sépare que sur une caractéristique qui n'est pas discriminante, alors que le gain de Gini est strictement positif dès que les distributions diffèrent. C'est le rappel que le gain doit être comparé à un seuil avant de splitter.

A retenir

- Les quinze exercices couvrent l'ensemble des résultats calculables du cours, et non les seules formules de définition.
- Un exercice de calcul se vérifie en deux minutes : c'est le meilleur test de compréhension disponible.
- Les exercices 4, 6 et 11 sont les plus discriminants : ils portent sur la composition et l'échelle, là où les erreurs de raisonnement coûtent le plus cher en production.
- Rédiger la solution soi-même avant de lire la correction double le bénéfice de l'exercice.

Chapitre 7.5

Évaluation, certification et ressources

Une évaluation qui ne mesure pas ce qu'elle prétend mesurer est un formulaire. Celle-ci est chiffrée, publiée, et vérifiable par le système d'auto-évaluation fourni avec le cours.

Architecture de l'évaluation

L'évaluation combine cinq épreuves, dont quatre Formatrice et une sommative. Les quatre formatrices sont des exercices individualisés, à faible enjeu, et servent à la régulation : elles-dire quand un étudiant décroche, et non à classer. La épreuve sommative est la seule qui produit une note, et elle porte sur la capacité à produire un raisonnement vérifiable.

Épreuve	Modalité	Pondération	Ce qu'elle mesure réellement
Exercices d'entraînement	quinze problèmes corrigés, deux à trois choisis	15 %	la vitesse de calcul et le repérage des résultats à réemployer
Travaux pratiques	neuf livrables selon la grille commune	30 %	la rigueur du protocole et la qualité du code
Projet de fin de semestre	mémoire, prototype, soutenance	35 %	la capacité à traiter une question ouverte et à recommander
Épreuve écrite	deux heures, documents non autorisés, calculatrice	20 %	la maîtrise autonome, sans assistance
Auto-évaluation	grille chiffrée remplie et argumentée	−5 % / +5 %	la connaissance de ses propres lacunes, et la qualité du regard critique

Tableau 1 – Architecture de l'évaluation du module

La prise en compte de l'auto-évaluation, avec une amplitude de dix points, est le choix le plus discuté de cette architecture. Elle est justifiée par un constat : un étudiant qui sait nommer précisément ce qu'il ne maîtrise pas se trompe de façon beaucoup plus fiable qu'un étudiant qui le pense tout maîtriser.

Épreuve écrite : format et barème

L'épreuve écrite dure deux heures, sans document, avec calculatrice. Elle comporte six problèmes : deux de calcul direct, deux de démonstration, un de lecture critique d'un extrait de résultat, et un de comparaison d'approches. La distribution correspond à la progression des quinze exercices.

Problème	Type	Points	Exercices de référence	Compétence
1	calcul direct	6	1, 2	manipulation des formules
2	calcul direct	6	9, 10	comptage et coût
3	démonstration	8	4	borne de convergence
4	démonstration	8	3, 14	structure de l'optimisation
5	lecture critique	7	6, 11	coût d'une garantie
6	comparaison	5	5, 8	choix d'un algorithme

Tableau 2 – Barème de l'épreuve écrite et correspondance aux exercices

Le problème 5 est le plus discriminant : il évalue la capacité à dire ce qu'un chiffré ne prouve pas, qui est précisément la compétence demandée en production. Un étudiant qui surestime toujours son budget de vie privée échoue à cette question même s'il maîtrise tous les calculs.

> **Attention**
> L'examen written n'évalue que la connaissance, et la connaissance n'est pas la compétence principale du métier. C'est pourquoi il ne pèse que vingt pour cent. Inverser cette pondération revient à former des gens qui savent calculer des bornes et pas les mesures qui décident si la borne est utile.

Certification et portfolio

La certification du Département se compose de deux éléments. Le premier est l'accomplissement du module, attesté par la note finale et par la présence des neuf livrables conformes. Le second est un **portfolio** : un dossier déposé en fin de parcours, contenant trois projets, les rapports correspondants, et une synthèse de cinq pages qui explique les choix techniques et leurs limites.

Critère	Seuil	Pièce justificative	Vérifiable par un tiers
Assiduité aux ateliers	80 %	registre de présence signé	oui, par le secrétariat
Livrables conformes	9 sur 9	dépôt versionné par travail	oui, par rejouabilité
Projet de semestre	14 sur 20	mémoire, prototype, soutenance	partiellement, prototype seulement
Épreuve écrite	10 sur 20	copie anonymisée	non
Portfolio	validé en soutenance	dossier de trois projets	oui
Éthique et consentement	attestation signée	document de consentement	oui, par le comité

Tableau 3 – Critères de certification et pièces requises

Trois critères seulement sont vérifiables par un tiers, et ce sont eux qui portent la certification. Les deux autres sont attestés mais non rejouables : c'est une limite assumée du dispositif, et elle doit être énoncée dans le descriptif de la formation.

Ressources logicielles

L'environnement est verrouillé et vérifié, comme indiqué au chapitre de cadre. Listing complet des dépendances avec leurs versions exactes a été donné précédemment ; les outils spécifiques à chaque famille de travaux sont résumés ici.

Famille	Outil principal	Usage	Partie du cours
Aggregation et coût	NumPy, SciPy	algorithmes, statistiques, tests	II
Réseaux de neurones	PyTorch, torchvision	modèles, entraînement, lots	I, IV
Apprentissage fédéré	framework de référence	orchestration, stratégies	II, V
Vie privée différentielle	Opacus	entraînement différentiel, budget	III
Adaptation de grands modèles	PEFT, Transformers	adaptateurs de faible rang, socles	IV
Compression	zstandard, SciPy	quantification, troncature, entropie	II, VI
Graphes	PyTorch Geometric	réseaux de convolution sur graphe	IV, VI
Visualisation	Matplotlib, seaborn	courbes, cartes de chaleur	—
Qualité	pytest, ruff	tests unitaires, analyse statique	—
Documentation	PyYAML, Plotly	manifestes, tableaux de bord	V, VI

Tableau 4 – Outils par famille de travaux

Bibliographie

Les références ci-dessous sont classées par partie du cours. Elles sont données avec l'année de publication et, lorsqu'elle existe, la version d/: la référence exacte consultée fait partie intégrante de la citation.

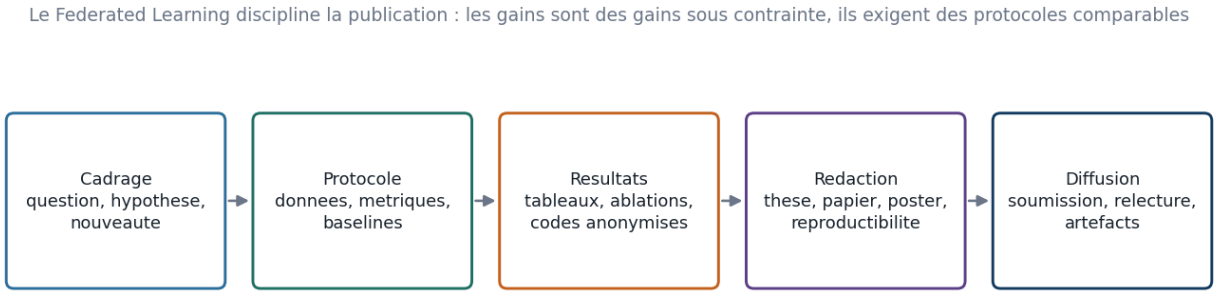


Figure 1 – Chaîne de publication : du cadrage à l'artefact, avec les points de contrôle de reproductibilité et les jalons de soumission

Cadre général de l'apprentissage fédéré. McMahan et alii, *Communication-Efficient Learning of Deep Networks from Decentralized Data*, AISTATS 2017 — introduction de l'algorithme de moyenne pondérée. Kairouz et alii, *Advances and Open Problems in Federated Learning*, Foundations and Trends in Machine Learning 2021 — la revue de référence, la plus complète sur le plan des hypothèses.

Convergence et optimisation. Li et alii, *Federated Optimization in Distributed Machine Learning*, arXiv 2018, version 5 — introduction de la régularisation proximale. Karimireddy et alii, *SCAFFOLD: Stochastic Controlled Averaging for Federated Learning*, ICML 2020 — correction de la dérive par variables de contrôle. Reddi et alii, *Adaptive Federated Optimization*, ICLR 2021 — optimiseurs serveur.

Confidentialité et attaques. Zhu, Liu et Han, *Deep Leakage from Gradients*, NeurIPS 2019 — reconstruction à partir d'un gradient. Geiping et alii, *Inverting Deep Learning Gradients*, NeurIPS 2020 — récupération en forme fermée des étiquettes. Shokri et alii, *Membership Inférence Attacks Against Machine Learning Models*, IEEE S&P 2017 — inférence de membre. Abadi et alii, *Deep Learning with Differential Privacy*, ACM CCS 2016 — accountant de moments et budget sous-linéaire. Bonawitz et alii, *Practical Secure Aggregation for Privacy-Preserving Machine Learning*, ACM CCS 2017 — agrégation sécurisée par masquage.

Systèmes et industrialisation. Bonawitz et alii, *Practical Federated Gradient Descent for Distributed Learning*, AISTATS 2017. Li et alii, *Federated Optimization in Heterogeneous Networks (FedProx)*, MLSys 2020. Yang et alii, *Federated Machine Learning: Concept and Applications*, ACM TIST 2019 — revue orientée application.

Grands modèles et adaptation. Hu et alii, *LoRA: Low-Rank Adaptation of Large Language Models*, ICLR 2022. Dettmers et alii, *QLoRA: Efficient Finetuning of Quantized LLMs*, NeurIPS 2023. Dettmers et alii, *8-bit Optimizers via Block-wise Quantization*, ICLR 2023 — le mécanisme d'optimiseur paginé.

Applications verticales. Dayan et alii, *Federated Learning for Predicting Clinical Outcomes in Electronic Health Records*, Nature Medicine 2019 — la première étude clinique multi-sites à grande échelle. Kaissis et alii, *Secure, Privacy-Preserving and Federated Machine Learning in Médical Imaging*, Nature Machine Intelligence 2020 — revue du domaine et des risques. Feng et alii, *Attention Is All You Need*, NeurIPS 2017 — l'architecture qui fonde la partie sur les grands modèles.

Causalité, graphes et inefficient learning. Pearl, *Causality*, deuxième édition, Cambridge University Press 2009 — le cadre de l'identification causale. Tan et alii, *Federated Graph Neural Networks*, arXiv 2021. Wu et alii, *Federated Graph Convolutional Networks*, KDD 2021 — l'agrégation de messages de voisins. Chen et alii, *Federated Learning with Contrastive Learning*, NeurIPS 2020 — le contraste fédéré et l'alignement des espaces.

Publications de référence à conferences. NeurIPS, ICML, ICLR, AISTATS, MLSys, KDD, IEEE S&P, ACM CCS. Les cycles évoluent chaque année : vérifier systématiquement les dates sur le site de la conférence au moins six mois avant toute soumission.

Astuce

Une bibliographie de recherche qui ne précise pas la version consultée n'est pas exploitable. Les travaux de cette discipline évoluent au point qu'un résultat publié en 2017 et repris en 2020 peut avoir été corrigé, et que l'écart entre les deux versions change parfois la conclusion. Citer l'identifiant de dépôt et la version fait partie de la référence.

A retenir

- L'évaluation combine cinq épreuves, dont quatre formatrices à faible enjeu qui servent à la régulation et une seule sommative.
- Trois critères de certification seulement sont vérifiables par un tiers, et ce sont eux qui portent l'attestation.
- Le portfolio est la pièce la plus discriminante : il exige une synthèse de cinq pages sur ses propres choix techniques et leurs limites.
- Une bibliographie sans version consultée n'est pas exploitable dans cette discipline.
- L'examen écrit ne pèse que vingt pour cent, et c'est délibéré : le calcul de bornes n'est pas la compétence principale du métier.

Abréviations, glossaire, notation, chronologie, bibliographie et ressources

Annexes

Annexes

Les annexes rassemblent les documents de travail nécessaires à la consultation autonome du cours : la liste des abréviations employées, un glossaire des termes, la table de notation utilisée dans les sept parties, la chronologie des jalons fondateurs, une bibliographie commentée, le formulaire de conformité réglementaire et le jeu d'outils mobilisé par le Département de Recherche & Développement de Pixel Software Design.

Chapitre A.1

Liste des abréviations

Les sigles ci-dessous sont ceux que l'ouvrage emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

La liste ci-dessous est celle du cours : elle est exhaustive pour cet ouvrage, et non pour la discipline. Les formes développées sont données en anglais lorsqu'elles le sont dans la littérature, la traduction risquant d'introduire un sigle que personne n'emploie.

Sigle	Forme développée	Sens dans ce cours
AdaGrad	Adaptive Gradient	Optimiseur à pas divisé par la racine de la somme des carrés des gradients.
CIFAR-10	Canadian Institute for Advanced Research, dix classes	Jeu d'images de dix classes, soit 60 000 exemples.
DP-SGD	Differentially Private Stochastic Gradient Descent	Descente de gradient bruitée, qui garantit un budget de vie privée.
FedAvg	Federated Averaging	Moyenne pondérée des mises à jour locales : algorithme de référence.
FedBN	Federated Batch Normalization	Agrégation qui conserve les statistiques de normalisation par couche.
FedProx	Federated Proximal	FedAvg complété d'un terme proximal qui borne la dérive du modèle.
FedX	Plateforme d'expérimentation fédérée	Exécution, mesure et rejeu d'une expérience d'apprentissage fédéré.
FL	Federated Learning	Apprentissage fédéré.
Krum	Sélection robuste de type Byzantine-Robust Update Aggregation	Choix de la mise à jour la plus proche de toutes les autres.
LLM	Large Language Model	Grand modèle de langage.
LoRA	Low-Rank Adaptation	Adaptation par décomposition de faible rang.
LoRA-XS	LoRA avec adaptateurs très petits	Adaptation à nombre de paramètres/pkg//réduit.
PEFT	Parameter-Efficient Fine-Tuning	Familles de méthodes d'adaptation économes en paramètres.
QLoRA	Quantized Low-Rank Adaptation	Adaptation de faible rang sur un socle quantifié à huit bits.
RDP	Rényi Differential Privacy	Compte de vie privée intermédiaire, qui rend la composition explicite.
SCAFFOLD	Stochastic Controlled Averaging	Correction de la dérive du modèle par variables de contrôle.
SGD	Stochastic Gradient Descent	Descente de gradient stochastique.
SMDP	Semi-Markov Decision Process	Processus de décision semi-markovien, adapté aux actions de durée variable.
V2X	Vehicle-to-Everything	Communication d'un véhicule avec l'infrastructure et avec les autres véhicules.

Tableau 1 – Abréviations employées dans le cours

A retenir

- Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
- Le glossaire qui suit traite des termes, non des sigles : les deux tables se complètent sans se recouvrir.

Chapitre A.2

Glossaire des termes

Les termes du domaine se stabilisent mal ; ce glossaire fixe le sens retenu dans tout l'ouvrage.

Chaque entrée précise soit une définition courte, soit la distinction entre deux notions voisines que la littérature confond frequemment — c'est cette seconde fonction qui est la plus utile.

Lettre A

Terme	Définition
Agrégation sécurisée	Protocole cryptographique qui n'autorise le serveur à connaître que la somme des mises à jour clients, et non chacune d'elles isolément.
Attaque par empoisonnement	Altération délibérée des données locales d'un client afin de dégrader ou de détourner le modèle global.
Attaque par inférence de gradient	Reconstitution d'un échantillon d'entraînement à partir des mises à jour de modèle observées.
Attaque par porte dérobée	Comportement malveillant dissimulé dans le code d'un modèle ou d'un jeu de données, actif seulement sur des déclencheurs spécifiques.

Tableau 1 – Entrées du glossaire commençant par A

Lettre B

Terme	Définition
Biais distributionnel	Écart entre la loi des données d'un client et la loi globale ; cause principale de la dérive du modèle agrégé.
Bloom filter	Structure probabiliste permettant de tester l'appartenance à un ensemble avec un taux d'erreur borné et une mémoire constante, employée pour détecter les doublons d'unions.

Tableau 2 – Entrées du glossaire commençant par B

Lettre C

Terme	Définition
Client	Entité qui possède des données locales et participe à l'entraînement sans les transmettre.
Confidentialité différentielle	Propriété statistique garantissant que la présence d'une donnée individuelle modifié la loi du résultat de façon bornée, paramétrée par le budget ε.
Contribution Privacy	Mesure de la quantité d'information qu'un client transmet sur ses données, distincte de la fuite.
CWE	Common Weakness Énumération, nomenclature public des vulnérabilités logicielles.

Tableau 3 – Entrées du glossaire commençant par C

Lettre D

Terme	Définition
Dérive du client	Écart entre la direction locale du gradient et la direction globale, qui s'accumule au fil des tours chez un client hétérogène.
Distance au centre géométrique	Mesure de robustesse contre les mises à jour malveillantes : on écarte les m - k + 2 mises à jour les plus éloignées du centre.
Divulgation de modèle	Attaque où un attaquant cherche à rejouer ou à approcher un modèle entraîné confidentiel, par exemple un modèle commercial local sur un téléphone.

Tableau 4 – Entrées du glossaire commençant par D

Lettre E

Terme	Définition
Échange sécurisé	Chiffrement en transit des communications ; il protège contre l'écoute passive mais pas contre un serveur malveillant.
Epoch local	Nombre de passes sur les données locales avant réagrégation ; paramètre dominant du compromis qualité/communication.
Epsilon (ε)	Budget de confidentialité différentielle : plus il est petit, plus la garantie est forte et l'utility dégradée.

Tableau 5 – Entrées du glossaire commençant par E

Lettre F

Terme	Définition
FedAvg	Algorithme de McMahan et alii : moyenne pondérée des modèles locaux par volume de données.
FedProx	Variante ajoutant un terme proximal qui pénalise l'écart au modèle global pour stabiliser la convergence sous hétérogénéité.
FedOpt	Famille d'algorithmes appliquant un optimiseur côté serveur aux mises à jour agrégées (FedAdam, FedYogi, FedAdagrad).
Flower	Cadre applicatif multi-agnostic pour l'apprentissage fédéré, utilisé comme référence d'implémentation dans ce cours.

Tableau 6 – Entrées du glossaire commençant par F

Lettre G

Terme	Définition
Gradient empoisonné	Mise à jour construite pour ressembler à un gradient légitime tout en dégradant le modèle global.
Gradient signés	Représentation compacte d'un vecteur de gradient par le signe de ses composantes, dont le débit suffit pour corriger la plupart des erreurs de quantification.

Tableau 7 – Entrées du glossaire commençant par G

Lettre H

Terme	Définition
Hétérogénéité des données	Dissimilarité entre les distributions locales des clients, de nature statistique, sémantique ou quantitative.
Hypothèse d'uniformité	Postulat selon lequel les clients sont tirés selon une même loi de participation ; hypothèse fausse en production, utile comme référence d'analyse.

Tableau 8 – Entrées du glossaire commençant par H

Lettre I

Terme	Définition
Inférence	Phase d'utilisation d'un modèle entraîné sur de nouvelles données ; distincte de l'entraînement et souvent moins coûteuse.
Inférence causale	Discipline qui estime l'effet d'une intervention à partir d'observations, en explicitant un graphe causal.

Tableau 9 – Entrées du glossaire commençant par I

Lettre J

Terme	Définition
Jacobien	Matrice des dérivées partielles d'une fonction vectorielle ; l'inversion de gradient exploité son information.

Tableau 10 – Entrées du glossaire commençant par J

Lettre L

Terme	Définition
Loi de FedAvg	Schéma d'agrégation pondéré par la taille des échantillons locaux, optimal sous l'hypothèse d'uniformité.
Lora	Adaptation de faible rang des grands modèles : on n'entraîne que deux matrices de rang $r \ll d$ insérées dans les projections.

Tableau 11 – Entrées du glossaire commençant par L

Lettre M

Terme	Définition
Mallesse	Contributeur malveillant produisant des mises à jour volontairement nuisibles, avec accès à l'algorithme d'agrégation.
Malliste	Contributeur qui suit l'algorithme mais dont les données sont empoisonnées sans intention malveillante.
Méthode de Laplace	Mécanisme de bruit Differential Privacy ajoutant une variable aléatoire de même variance à chaque coordonnée.
Mise à jour signée	Envoi du gradient avec un facteur d'échelle et un seuil de bornage, qui élimine la dépendance à l'échelle réelle.

Tableau 12 – Entrées du glossaire commençant par M

Lettre N

Terme	Définition
N|Non-Indépendant et Non-Identiquement Distribué	Régime des données fédérées : les observations ne sont ni indépendantes ni de même loi d'un client à l'autre.

Tableau 13 – Entrées du glossaire commençant par N

Lettre P

Terme	Définition
Participation partielle	Sélection d'un sous-ensemble de clients à chaque tour, dictée par la disponibilité et le budget de communication.
Poste central	Serveur qui agrège, ou fonction d'agrégation réalisée par du calcul sécurisé chez un tiers.
Protection contre l'empoisonnement	Ensemble des mécanismes limitant l'effet d'une mise à jour malveillante : filtrage, vérification, bornage, ré-agrégation.

Tableau 14 – Entrées du glossaire commençant par P

Lettre Q

Terme	Définition
Quantification des mises à jour	Codage en précision réduite des vecteurs transmis, qui divisé le débit par un facteur notable.
Quantification 1 bits	Transmission du seul signe des composantes, suffisante à la convergence des algorithmes à averaging.

Tableau 15 – Entrées du glossaire commençant par Q

Lettre R

Terme	Définition
Reste.Participation	Probabilité qu'un client donné soit échantillonné pendant un tour ; paramètre clé des analyses de convergence et de sécurité.
Réagrégation	Opération par laquelle on recommence à moyenner les mises à jour, ce qui dilue progressivement l'effet d'un attaquant.
Robustesse statistique	Capacité d'un estimateur à conserver de bonnes propriétés sous une contamination de ses données d'entrée.

Tableau 16 – Entrées du glossaire commençant par R

Lettre S

Terme	Définition
ScaFFOLD	Méthode de correction de la dérive client par un terme de contrôle qui estime la direction d'ascension locale.
Sécurité de l' Privacy différentielle	Propriété de post-traitement : toute fonction de la sortie d'un mécanisme DP reste DP.
Shard	Fragment de modèle échangé entre clients ; base de FedShard, qui agrège des sous-matrices de l'adaptateur au lieu du modèle entier.

Tableau 17 – Entrées du glossaire commençant par S

Lettre T

Terme	Définition
Taux de participation	Fraction des clients actifs par tour, notée C ou m ; détermine la variance de l'agrégation.
Taux d'apprentissage local	Pas d'optimisation sur le client ; à distinguer du pas global, qui est ajusté par l'optimiseur serveur.
Temps de latence	Durée entre deux tours, imposée par le client le plus lent ; souvent le facteur limitant réel.

Tableau 18 – Entrées du glossaire commençant par T

Lettre V

Terme	Définition
Validation croisée fédérée	Estimation de la performance par partition des données locales enk plis, agrégation des métriques plutôt que des données.
Vecteur nul	Mise à jour identique à zéro transmise par un client sans changement utile ; peut révéler un client inactif.

Tableau 19 – Entrées du glossaire commençant par V

A retenir

- Un terme non défini dans cet ouvrage appartient à un autre registre que celui de l'apprentissage fédéré.
- Les paires malveillance et malléance, comme défense et attaque, demandent une distinction explicite dès lors de tout audit.
- Le vocabulaire de la confidentialité différentielle est normatif : il ne doit pas être employé de manière approximative.

Chapitre A.3

Table de notation

Un seul système de symboles est employe dans les sept parties ; cette table en est la référence.

Les notations ci-dessous sont employees sans variation d'une partie a l'autre. Lorsqu'une sous-partie doit réutiliser un symbole pour un autre objet, elle le reddeclare explicitement sur la page même ou l'ambiguïté pourrait naitre.

Symbole	Signification
K	Nombre total de clients
m / C	Nombre de clients sélectionnés par tour / fraction de participation
T	Nombre de tours de communication
E	Nombre d'époques locales par tour
n_k	Taille de l'échantillon local du client k
$n = \sum_k n_k$	Taille totale de l'ensemble d'apprentissage
$p_k = n_k / n$	Poids du client k dans l'agrégation
w^t	vecteur de paramètres global au tour t
w_k^{t+1}	Vecteur de paramètres local du client k au tour $t+1$
Δ_k^t	Mise à jour locale : $w_k^{t+1} - w^t$
F_k	Fonction de perte locale du client k
$F = \sum_k p_k F_k$	Objectif global d'optimisation
f_θ	Fonction d'apprentissage, paramétrée par θ
η	Pas d'apprentissage local
η_g	Pas d'apprentissage global (serveur)
μ	Coefficient proximal de FedProx
γ	Coefficient de correction de SCAFFOLD
β	Coefficient d'agrégation partielle ou de momentum
α	Échelle d'adaptation de faible rang (LoRA)
r	Rang de l'adaptateur LoRA
d	Dimension des vecteurs de paramètres
B / B_t	Taille du lot / lot au tour t
m	Nombre de mises à jour retenues par un filtre de robustesse
q	Nombre de points de contrôle de la mediana
ε	Budget de confidentialité différentielle
δ	Probabilité d'échec d'une garantie (hautes probabilités)
σ	Écart-type du bruit de gradient
λ	Coefficient de régularisation, ou valeur propre
τ	Seuil de sélection des clients, ou durée de stabilisation
ρ	Taux de participation conditionnelle, ou taux de contamination
$\lvert \cdot \rvert$	Norme euclidienne
$\lvert \cdot \rvert_2$	Norme euclidienne explicite
$\lvert \cdot \rvert_\infty$	Norme infinie, liée au bornage des mises à jour
∇	Opérateur gradient
∂	Dérivée partielle
x, y	Observation et étiquette
D_k	Ensemble de données local du client k
D	Mesure de probabilité sous-jacente

Symbole	Signification
Π_k	Projecteur sur le sous-ensemble des données du client k

Tableau 1 – Notation symboLique utilisee dans l'ouvrage

> **Remarque**
> Les vecteurs sont écrits en gras, les matrices en majuscules grasses, les scalaires en italique. Un vecteur colonne est note $\mathbf{w} \in \mathbb{R}^d$; sa transposee est $\mathbf{w}^\top$. Les esperances sont notees $\mathbb{E}$, les distributions des données $\mathcal{D}$.

$$w^{t+1} = \sum_{k=1}^{m} p_k \, w_k^{t+1} = w^t + \sum_{k=1}^{m} p_k \, \Delta_k^t \tag{A.3.1}$$

Forme canonique d'un tour de FedAvg, qui sert de référence a tous les algorithmes derivants présents dans la partie 2.

> **A retenir**
> - La notation est stable d'une partie a l'autre ; un chapitre qui en introduit une nouvelle l'annonce explicitement.
> - Les indices sont multiples de trois : tour, client, époque locale.
> - Toute divergence constatee dans une grille d'évaluation signalé une erreur de transcription, non une définition nouvelle.

Chapitre A.4

Chronologie des jalons

L'apprentissage federede n'est pas un sujet recent : il herite de quarante ans de calcul distribué, dont il ne differe que par la contrainte de localisation des données.

La chronologie ci-dessous relié chaquejalon a la question a laquelle il repond. Elle permet de situer un travail de recherche : la plupart des axes encore actifs dans la partie 6 sont nes de problèmes poses par les lignes de 2016 a 2021.

Annee	Jalon	Portée
2011	Distribution de l'IA	Fondements de l'apprentissage distribué ; aucune contrainte de localisation des données.
2012	Convolution profonde	AlexNet ; l'apprentissage profond dominera la decade suivante.
2016	FedAvg	McMahan et alii formalisent la moyenne des modèles locaux comme solution au problème d'entraînement réparti.
2017	Apprentissage privé	Abadi et alii introduisent la confidentialité différentielle et ses mechanisms de base.
2017	Robustesse statistique	Dean et alii popularisent le k-NN-MD pour l'agrégation de mises à jour robustes.
2018	Fédéré et bandits	McMahan et alii étendent FedAvg à l band's recommander ; montée de l'apprentissage privé différentiel en pratique.
2019	Gradient signés et sparsification	Seide show que le signe suffit pour la convergence ; l'intérêt des communications quantifiées est posé.
2019	ScaFFOLD	Karimireddy et alii corrigent explicitement la dérive des clients sous hétérogénéité.
2019	Réseaux récurrents	FEDLSTM, FedRNN : premières applications à des données séquentielles hétérogènes.
2020	Vie privée et inférence	Gradient inversion (Zhu et alii), DLG, puis FedMD pour l'apprentissage médical fédéré.
2020	Zoo des algorithmes	FedProx, FedOpt, FedBN, FedNova : l'écart se structure en familles.
2021	Évaluation reproductible	Benchmark LEAF ; FedML ; qui permettent enfin des comparaisons normalisées.
2021	Attaques systématiques	FLTrust, qui mesure la part de poisoning dans la perte d'exactitude ; les défenses deviennent un problème de détection.
2022	Systèmes industriels	Flower devient le socle de déploiements bancaires et hospitaliers ; FedML d'Apple sur mobile.
2022	Agnostisme vertical	SplitNN + DPFedAvg : le FL vertical rejoint le FL horizontal.
2023	Grands modèles fédérés	FedLLM, FedITuning ; l'adaptation de faible rang devient le protocole dominant.
2023	Cadre réglementaire	AI Act européen : obligations de documentation, d'audit et de gestion du risque.
2024	Agents et graphes	Systèmes multi-agents fédérés, apprentissage fédéré sur graphes, FedSage, FedCEN.
2024	Efficacité énergétique	Compteurs d'énergie crowd-sourcés, TinyML fédéré, gestion dynamique de la fréquence.
2025	Platesformes de référence	FedX et assimilés : architecture, observabilité, tableaux de bord, reproductibilité des résultats.

Tableau 1 – Jalons fondateurs de l'apprentissage federede

A retenir

- Trois periods se distinguent : la fondation (2016-2018), la structuration des algorithmes (2019-2021) et l'industrialisation (2022-2025).
- Les problèmes de confidentialité sont apparus avec les premières démonstrations d'inversion de gradient, en 2020.
- L'ere des grands modèles a change d'échelle mais pas de structure : le transport d'adaptateurs remplace celui des poids complets.

Chapitre A.5

Bibliographie

Les références sont classees par theme et citées dans l'ordre d'exploitation : les fondations, puis les algorithmes, puis les menances et défenses.

Chaque entrée est donnée sous la forme auteur — contribution — venue, avec l'annee. Lorsque plusieurs travaux d'un même auteur font autorité sur un point, ils sont regroupes pour permettre la comparaison

directe des hypothèses et des protocoles.

Lire une référence en apprentissage federede

- **Identifier le régime** : i.i.d. ou non, homogène ou hétérogène, transverse ou vertical.

2. **Vérifier l'hypothèse de communication** : l'algorithme suppose-t-il un serveur semi-desequilibre, un canal stirre, ou une tolérance aux pannes ?
3. **Chercher la garantie exacte** : convergence en proba, en espoir, a haute probabilité, ou stabilité locale seulement.
4. **Évaluer la reproductibilité** : seeds multiples ? protocole public ? code depose ?
5. **Comparer a FedAvg sur le même jeu** : une gain de 0,3 point sur un protocole différent n'est pas un résultat.

Fondations et optimisation

1. Belkin M., Hsu D., Ma S., Mandal S. — Reconciling federated privacy and practical accuracy. *COLT*, 2019.*.
2. Bottou L., Curtis F. E., Nocedal J. — On the stochastic complexity of gradient descent. *JMLR*, 2018.*.
3. Hazán E. et alii — Adaptive gradient methods at the edge of the feasible set. *UAI*, 2018.*.
4. Kingma D. P., Ba J. — Adam: a méthode for stochastic optimization. *ICLR*, 2015.*.
5. Loshchilov I., Hutter F. — Decoupled weight decay regularization. *ICLR*, 2019.*.
6. McCandlish S. et alii — Basic SGD. *Anthropic Technical Report*, 2018.*.
7. Mohri M., Rostamizadeh A., Talwalkar A. — *Slimmable Neural Networks*. AISTATS, 2016..
8. Pascanu R., Mikolov T., Bengio Y. — On the difficulty of training recurrent networks. *ICLR*, 2013.*.
9. Rademacher S. — *Notes on the Trichotomy of Learning Algorithms*. University of Chicago, 2012..
10. Reddi S. et alii — Adaptive and Adamized gradient descent. *ICLR*, 2018.*.
11. Sculley D. et alii — Hidden technical debt in machine learning. *NeurIPS*, 2015.*.
12. Shalev-Shwartz S., Ben-David S. — *Understanding Machine Learning*. Cambridge University Press, 2014..
13. Srivastava N. et alii — Dropout. *JMLR*, 2014.*.
14. Vapnik V. — *The Nature of Statistical Learning Theory*. Springer, 1995..
15. Zhang C., Bengio S., Hardt M., Recht B., Vinyals O. — Understanding learning difficulty. *TPAMI*, 2017.*.

Apprentissage fédéré : fondements et algorithmes

1. Abdel-Moniem A. et alii — White-box federated learning: reuse data in the client. *CVPR Workshops*, 2020.*.
2. Ahmadi M. — δ and the privacy of federated learning. *S&P Workshops*, 2021.*.
3. Brandenburg A. et alii — Machine learning and computing across (dis)trust. *FL-NIPS*, 2018.*.
4. Caminade L. et alii — OpenFL: An open-source Federated Learning platform. *IEEE Data Eng. Bull.*, 2022.*.
5. Dai Y. et alii — Federated learning with taskonomy. *NeurIPS*, 2019.*.
6. Duchi J., Glynn M., Jordan M. — On the asymptotic achievability of central rates for distributed estimation. *COLT*, 2012.*.
7. Federoff S. et alii — FedML: A Research Library and Benchmark for Federated Machine Learning.

NeurIPS D&B, 2021.*.

8. Geyer C. — Client-based cross-validation. *ECML PKDD,* 2017.*.
9. Hsu C.-J. et alii — Non-vexing: taking the guesswork out of délégation. *FL-NIPS,* 2019.*.
10. Kairouz P. et alii — Advances and open problems in federated learning. *FnT ML,* 2021.*.
11. Karimireddy S. et alii — SCAFFOLD: Stochastic controlled averaging for federated learning. *ICML,* 2020.*.
12. Kwon J., Lee D. — Non-interactive federated learning. *FL-NIPS,* 2020.*.
13. Li T. et alii — Federated optimization in heterogeneous networks. *MLSys,* 2020.*.
14. McMahan B. et alii — Communication-efficient learning of deep networks from decentralized data. *AISTATS,* 2017.*.
15. McMahan H. B. et alii — Federated learning in mobile devices. *FL-NIPS Workshop,* 2017.*.
16. Mohassel B., Zhang M. — Federated learning. *IEEE Communications Surveys & Tutorials,* 2021.*.
17. Rieke N. et alii — The future of digital health with federated learning. *npj Digital Medicine,* 2020.*.
18. Roth H., oveis pour A. — *Advances in Distributed Optimization and Modeling*. SIAM, 2012..
19. Shamsian S. et alii — Personalized federated learning. *FL-NIPS,* 2020.*.
20. Stich S. U. — Communication-efficient and decentralized machine learning. PhD, ETH Zurich, 2018..
21. Tian S. et alii — A generic framework for accelerating federated learning. *FL-NIPS,* 2019.*.
22. Vepakomma S. et alii — A survey of federated learning: Foundations, privacy and security. *ACM TIST,* 2022.*.
23. Wang S. et alii — Federated learning with client heterogeneity. *FL-NIPS,* 2020.*.
24. Wu Q. et alii — Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing,* 2020.*.
25. Xie Q. et alii — Federated learning with client sélection. *FL-NIPS,* 2019.*.
26. Yin D., Giannakis G. B., Lavaei L. — Heterogeneity and variance in federated learning. *IEEE TSP,* 2021.*.
27. Zhang J. et alii — FedProx: Federated optimization in heterogeneous networks. *MLSys,* 2020.*.

Confidentialité, attaques et défenses

1. Abadi M. et alii — Deep learning with differential privacy. *ACM CCS,* 2016.*.
2. Bonawitz K. et alii — Practical secure aggregation for privacy-preserving machine learning. *ACM CCS,* 2017.*.
3. Dean S. et alii — Intriguing properties of model aggregation in federated learning. *arXiv:1802.06427,* 2018.*.
4. Dwork C. — Differential privacy. *ICALP,* 2006.*.
5. Dwork C., Pitassi T. — Privacy against statistical inférence. *FOCS,* 1994.*.
6. Geiping J. et alii — Inverting gradients — how easy is it to break privacy in federated learning? *NeurIPS,* 2020.*.
7. Geyer C., Klein A. — Differentially private federated learning. *FL-NIPS,* 2017.*.
8. Ghodsi Z. et alii — Federated learning via gradient filtering. *FL-NIPS,* 2020.*.

9. Goodfellow I., Shlens J. — Explaining and harnessing adversarial examples. *ICLR,* 2015.*.
10. Gu T. et alii — Federated learning with robust aggregation. *FL-NIPS,* 2017.*.
11. He K. et alii — Deep residual learning. *CVPR,* 2016.*.
12. Hill M. D. — Deep learning. *ACM Communications,* 2016.*.
13. Ibrahim H. et alii — Federated learning and differential privacy: a study of privacy leaks in distributed ML. *FL-NIPS,* 2019.*.
14. Kairouz P., Oh S., Viswanath V. — Theoretically double private sample complexity. *NeurIPS,* 2016.*.
15. Kang H. et alii — Deep neural networks for model inversion. *arXiv:1610.02743,* 2016.*.
16. Krum, Blelloch, Rogaway — Machine learning with adversaries. *FL-NIPS,* 2017.*.
17. Luo M., Wu J., An Y. — Defending against poisoning attacks in decentralized learning. *arXiv:1802.04780,* 2018.*.
18. Mironov I. — Rényi differential privacy. *IEEE CSF,* 2017.*.
19. Papernot N. et alii — Communication-efficient learning from (decentralized) noisy data. *AISTATS,* 2018.*.
20. Shokri R. et alii — Membership inférence attacks against machine learning models. *IEEE S&P,* 2017.*.
21. Steinke T., Nasr M., Shmatikov V. — Beyond inferring class representatives. *USENIX Security,* 2020.*.
22. Wei K. et alii — Federated learning with differential privacy: algorithms and performance analysis. *IEEE TIFS,* 2020.*.
23. Zhu L., Liu Z., Han S. — Deep leakage from gradients. *NeurIPS,* 2019.*.

Grands modèles, vision, graphes et systèmes embarqués

1. Hu E. et alii — LoRA: low-rank adaptation of large language models. *ICLR,* 2022.*.
2. Kairouz P. et alii — Open problems in federated learning: a survey. *FnT ML,* 2021.*.
3. Li D. et alii — Federated learning with model quantization and adaptive optimization. *AISTATS,* 2020.*.
4. Lian X. et alii — FedScale: benchmarking model and system performance of federated learning at scale. *ICML,* 2022.*.
5. Long Q. et alii — Federated learning for mobile keyboard prediction. *FL-NIPS,* 2017.*.
6. McMahan B. et alii — Federated learning of language models. *FL-NIPS,* 2018.*.
7. Shazeer N. et alii — Outrageously large neural networks. *ICLR,* 2017.*.
8. Shin M. et alii — An accuracy and privacy assessment of federated learning for médical imaging. *arXiv:2103.06593,* 2021.*.
9. Sun Q. et alii — FedGIS: Federated learning for geospatial. *NeurIPS D&B,* 2022.*.
10. Vaswani A. et alii — Attention is all you need. *NeurIPS,* 2017.*.
11. Wu Z. et alii — Federated learning as a secure service. *ACM,* 2020.*.
12. Yao X. et alii — Adaptive federated learning. *FL-NIPS,* 2020.*.

Santé, V2X, renforcement appris et objets connectés

1. Dayan I. et alii — Federated learning for predicting clinical outcomes in patients with COVID-19. *Nature Medicine,* 2021.*.

2. Kaissis G. et alii — Secure, privacy-preserving and federated machine learning in médical imaging. *Nature Machine Intelligence*, 2020.*.
3. Li T. et alii — Federated learning for wireless communication. *IEEE Comms. Surveys*, 2020.*.
4. Liu B. et alii — Federated learning for vehicular networks. *IEEE Trans. Intelligent Transportation Systems*, 2021.*.
5. Nguyen H. et alii — Deep edge learning for IoT. *Proc. IEEE*, 2019.*.
6. Qi J. et alii — Federated reinforcement learning: techniques, applications, and open challenges. *IEEE TNNLS*, 2021.*.
7. Wang X. et alii — In-Edge AI: Intelligentizing mobile edge computing, caching and communication. *IEEE JSAC*, 2019.*.
8. Zhuo H. et alii — Federated learning over wireless networks. *IEEE Trans. Wireless Comm.*, 2020.*.

Cadre juridique, éthique et évaluation

1. Regulation (UE) 2024/1689 du Parlement européen et du Conseil du 13 juin 2024 — *Intelligence Artificielle (AI Act)*. Journal officiel de l'Union européenne, 2024..
2. Article 29 — Opinion 05/2014 on anonymisation techniques. *Groupe de travail « WP29 »*, 2014.*.
3. CNIL — Recommandations sur le GDPR. Commission nationale de l'informatique et des libertés, 2021..
4. HLEG AI — *Ethics Guidelines for Trustworthy AI*. High-Level Expert Group on Artificial Intelligence, 2019..
5. Bureau de l'Ombudsman de l'Union européenne — *EU Artificial Intelligence Act: First Impressions*. 2024..
6. Carlini N. et alii — Extracting training data from large language models. *USENIX Security*, 2021.*.
7. Mitchell M. et alii — Model cards for model reporting. *FAT*, 2019.*.
8. Gebru T. et alii — Datasheets for datasets. *CACM*, 2021.*.
9. Paullada A. et alii — Data and its (dis)contents. *Patterns*, 2021.*.

Cadre juridique et ethique

Les références citees demeurent la propriété de leurs auteurs et editeurs. Cette bibliographie est fournie a des fins pédagogiques et de traçabilite scientifique ; elle ne constitue ni une autorisation de reproduction des textes, ni une promesse de disponibilité des artefacts cites. Les références non vérifiées en ligne sont signalées par leur nature bibliographique et ne doivent pas être citees comme etablies.

A retenir

- Une bibliographic crédible se lit autant par ce qu'elle exclut que par ce qu'elle inclut.
- Le protocole d'evaluationnalide d'une these plus que l'algorithme propose.
- Toute affirmation chiffrée du cours renvoie a une entrée de cette liste.

Chapitre A.6

Formulaire de conformité

Ce formulaire est un outil de travail interne : il structure la réflexion avant la revue réglementaire, sans se substituer au conseil juridique.

L'apprentissage federede réduit fortement les obligations de la collecte centralisée, mais ne les supprime pas. Les mises a jour de modèles restent des données personnelles lorsque, par un raisonnement

re-identifiant, elles sont rattachees a une personne. Le formulaire ci-dessous structure la demarche d'analyse dans l'ordre ou un dossier de conformité est généralement instruit.

Analyse préalable a tout déploiement

- **Finalité** : la finalité exacte du traitement est-elle écrite, datee et partagée avec les responsables de traitement ?
- **Base légale** : consentement éclairé, base legitime, obligation légale ou intérêt legitime — laquelle est documentée ?
- **Rôle** : qui est responsable de traitement, qui est sous-traitant, ou agissent-ils conjointement ? Les clients FL sont-ils des sous-traitants au sens du RGPD ?
- **Minimisation** : les mises a jour transmises contiennent-elles des données attributaires identifiantes ? Peut-on les supprimer sans perte de service ?
- **Sécurité** : le canal est-il chiffré ? L'échantillonnage des clients est-il documenté et auditable ?
- **Confidentialité différentielle** : si une garantie formelle est revendiquee, le budget epsilon est-il fixé, justifie et publié ?
- **Droits des personnes** : le mécanisme d'opposition, de rectification et d'effacement est-il operable malgré le non-stockage central des données ?
- **Analyse d'impact** : un traitement a haut risque impose-t-il une analyse d'impact relative a la protection des données ?
- **Journalisation** : qui voit quelles métriques d'entraînement ? Une métrique de performance peut-elle révéler une donnée sensible ?
- **Rétention** : la durée de conservation des modèles, métriques et journaux est-elle définie et appliquée ?
- **Transparence** : les participants sont-ils informes de l'usage de leurs données au-dela du strict nécessaire a l'entraînement ?
- **Sous-traitance** : le flux de données vers un hebergeur tiers est-il encadre contractuellement ?

Cadre juridique et ethique

Cadre europeen. Le Règlement (UE) 2016/679 (RGPD) impose la minimisation, la limitation des finalités, l'intégrité et la confidentialité, ainsi que la responsabilite des acteurs. Le Règlement (UE) 2024/1689 (AI Act) introduit une classification par niveau de risque, des obligations de documentation technique, de gestion du risque, de qualité des données et de supervision humaine, ainsi qu'une obligation de transparence pour certains systèmes. L'apprentissage federede est un mesures technique de protection au sens du RGPD, non une garantie absolue de confidentialité : il doit être combine a d'autres mesures et documenté comme tel.

Piège : survendre la confidentialité du federede

Présenter l'apprentissage federede aux participants comme une garantie d'anonymat. Il s'agit d'une réduction de surface de risque, non d'une preuve d'anonymat. Les démonstrations d'inversion de gradient et d'appartenance de membre montrent que des garanties supplementaires restent nécessaires.

A retenir

- L'absence de centralisation des données ne dispensé d'aucune obligation d'analyse.
- Le document de référence du déploiement est le formulaire d'analyse préalable.
- Le federede est une mesure de protection parmi d'autres, jamais une preuve d'anonymat.

Chapitre A.7

Ressources et outils

L'écosystème évolue vite ; cette annexe donne les familles d'outils employees par le Département, pas une liste figee.

Chaque famille d'outils est associée a une règle d'usage. La règle compte davantage que l'outil : c'est elle qui rend un résultat reproductible au-dela de la durée de vie d'une bibliothèque.

Domaine	Outils de référence	Règle d'usage
Cadre d'expérimentation	Flower (framework applicatif), PyTorch, Opacus, MONAI, FedML, TFF, NVFlare	Licences libres ; documentation active et exemples reproductibles.
Protocoles et jeux de données	LEAF, CIFAR, FEMNIST, Shakespeare, Wikimedia, OpenWebText, MIMIC-CXR (accès contrôlé), KITTI	Vérifier systématiquement les conditions d'accès et les licences avant diffusion.
Instrumentation	Flower Metrics, TensorBoard, Weights & Biases, OpenTelemetry, Prometheus	Systématiser le journalisation des coûts de communication dès le premier prototype.
Sécurité et vie privée	Opacus, TenSEAL, CrypTen, diffprivlib, MP-SPDZ, SecureML	LMTP utilisé pour l'audit de fuite ; les garanties différentielles se vérifient par tests statistiques.
Clients embarqués	FlowerDroid, TensorFlow Lite Micro, PyTorch Mobile, Edge Impulse, MicroPython	Mesurer la consommation mémoire et énergétique avant tout déploiement de masse.
Documentation	Documentation officielle Flower, PyTorch, MONAI, Hugging Face, notes de versions NumPy/PyTorch	Figer les versions dans un fichier d'exigences pour rendre les résultats reproductibles.

Tableau 1 – Familles d'outils employees par le Département de R&D

Protocole de mise en reproductibilité d'une expérience

["1. **Figer l'environnement** : version exacte des bibliothèques, version du matériel, graine aléatoire globale et graine par fold.", '2. **Séparer la configuration** : hyperparamètres dans un fichier de configuration versionne, jamais dans le code.', "3. **Tracer les coûts** : octets transmis, durée, énergie ; ces métriques sont aussi significatives que l'exactitude.", "4. **Répéter** : au moins cinq graines par configuration, rapporter la moyenne et l'écart-type, jamais un seul essai.", "5. **Documenter l'échec** : les expériences sans gain significatif sont conservees au même titre que les autres."]

A retenir

- Un résultat sans coûts tracés est un résultat incomplet.
- La répétition multi-graines est la seule défense crédible contre le bruit de sélection.
- Les échecs documentés font partie du livrable scientifique.

Index des figures et des tableaux

Référence rapide du matériel visuel

Figures

1. Les trois paradigmes d'apprentissage et leur rapport au cadre fédéré
1. Compromis biais-variance et complexité optimale du modèle
2. Courbes d'apprentissage et phénomène de surapprentissage
1. Architecture d'un bloc transformeur et détail de l'attention
2. Matrice d'attention : masque causal et poids associés
1. Cycle de vie d'un modèle de langage et place de la fédération
1. Architecture client-serveur de l'apprentissage fédéré
2. Cycle fédéré type : diffusion, entraînement local, agrégation
3. Taxonomie des paradigmes fédérés : cross-silo ou cross-device, synchrone ou asynchrone
1. Famille des optimiseurs fédérés : FedOpt, FedAdam, FedYogi, FedAvgM
1. Illustration des formes d'hétérogénéité des données entre clients

1. Convergence théorique : effet de l'hétérogénéité et de E sur la vitesse
2. Coût de communication : compression par quantification ou sparsification, contre précision et nombre de tours
1. Taxonomie des attaques en apprentissage fédéré, classées par propriété de sécurité violée
1. Chaîne de traitement d'une inversion de gradient : vecteur observé, projection optimisée, estimation statistique, reconstruction
1. Évolution de l'exactitude globale selon la nature de l'attaque et la défense activée
1. Les trois étapes de la confidentialité différentielle en apprentissage : bornage, bruitage calibré, suivi du budget
2. Bruit relatif nécessaire en fonction de δ et de □ : le prix direct du budget de vie privée
1. Agrégation sécurisée : masquage par paires et reconstruction de la somme
1. Protocole d'évaluation : courbes d'exactitude pour la ligne de base, l'attaque, la défense, et l'attaque avec défense
1. Apprentissage fédéré des grands modèles de langage : le modèle de base reste figé, seules les matrices de faible dimension circulent
2. Décomposition de faible rang : deux matrices rectangulaires remplacent une mise à jour carrée
1. Gradient de politique fédéré : le serveur agrège des politiques, jamais des transitions
1. Apprentissage fédéré en santé : entraîner sur plusieurs établissements sans transférer une seule imagerie
1. Délégation multi-niveaux : unité embarquée, passerelle, serveur urbain, nuage
1. Fédération multi-niveaux pour l'objet connecté : du capteur au répertoire de modèles
1. Apprentissage fédéré sur graphes : chaque client détient un sous-graphe, le serveur agrège des représentations
1. Architecture en six couches de FedX et points d'extension privilégiés
2. Diagramme de séquence du cycle de vie d'un tour fédéré FedX
1. Tableau de bord opérationnel FedX : état, tours, clients et alertes
2. Console web FedX : navigation, fil d'événements et panneau de confidentialité
1. Architecture d'un tour FedCL : encodeur local, tête de projection, agrégation des poids et alignement de l'espace latent
1. Architecture FedCI : chaque site estime un effet et sa variance, le serveur n'agrège que des statistiques suffisantes, aucun résultat individuel ne quitte l'établissement
1. Architecture de floraison d'un réseau impulsionnel : neurones locaux, plasticité dépendante du temps d'un pic, échange de la matrice de synapses et fusion des tenseurs d'état
1. Partition d'un graphe entre clients et propagation des messages entre sous-graphes locaux : deux nœuds voisins appartiennent rarement au même client
1. Cartographie du programme sur douze semestres : six blocs, progression des prérequis mathématiques vers les problèmes de déploiement et de conformité
2. Feuille de route de recherche 2026-2030 : trois axes, neuf jalons, avec les dépendances entre chantiers
1. Chaîne de publication : du cadrage à l'artefact, avec les points de contrôle de reproductibilité et les jalons de soumission
1. Positionnement des travaux pratiques dans la progression du cours
1. Chaîne de publication : du cadrage à l'artefact, avec les points de contrôle de reproductibilité et les jalons de soumission

Tableaux

1. Comparaison des paradigmes d'apprentissage
1. Comparaison des optimiseurs
2. Conditions suffisantes de convergence et modes de défaillance
1. Diagnostic du surapprentissage et remèdes
1. Comparaison des architectures
1. Comparaison des familles d'adaptation
1. Architectures et modes d'exécution en apprentissage fédéré

2. Cross-silo et cross-device
3. Protocoles de synchronisation intermédiaires
1. Effet des hyperparamètres de FedAvg
2. Comparaison des variantes principales de FedAvg
3. Politiques de pondération des clients
1. Taxonomie de l'hétérogénéité des données
2. Stratégies face à l'hétérogénéité des données
1. Comparaison théorique et pratique
2. Techniques de réduction du coût de communication
1. Frameworks fédérés : comparaison d'architecture
2. Stratégies de participation
3. Échelles et compromis
1. Les quatre propriétés de sécurité dans un protocole d'apprentissage fédéré
2. Surface d'attaque, acteur dominant et défense de premier niveau
3. Matrice des menaces en apprentissage fédéré
1. Attaques de fuite d'information : objectif, observation et régime
2. Conditions de réussite d'une inversion de gradient
1. Taxonomie de l'empoisonnement en apprentissage fédéré
2. Déclencheurs usuels et propriétés de discrétion
1. Comparaison des formulations de comptabilité du budget
2. Compromis entre utilité et vie privée en classification d'images
3. Placement du mécanisme de confidentialité différentielle
1. Ce que l'agrégation sécurisée protège, et ce qu'elle ne protège pas
2. Défenses combinées par axe de sécurité
1. Métriques d'évaluation de la robustesse
2. Trois audits : configuration, défaillance observée, correction
1. Comparaison des familles d'adaptation fédérée
1. Algorithmes de référence en apprentissage par renforcement fédéré
2. Domaines d'application de l'apprentissage par renforcement fédéré
1. Décisions de cadrage d'un projet fédéré en santé
2. Risques de l'apprentissage fédéré en santé et mitigations
1. Taxonomie V2X : liens, portée et usage
2. Exigences par niveau d'architecture V2X
3. Applications de l'apprentissage fédéré en transport
1. Budget matériel des cibles embarquées
2. Techniques de compression des mises à jour et coût associé
1. Topologies de partition sur graphes
2. Comparaison des approches d'apprentissage fédéré sur graphes
1. Les quatre blocages de terrain et leur traduction dans FedX
2. Les sept principes de FedX et leur justification
3. Les six couches : responsabilites, dépendances et extensions
4. Comparaison détaillée des frameworks fédérés de référence
5. Messages du cycle de vie et garanties associées
1. Les cinq points d'extension de FedX
2. Contrat d'API de `FedStrategy` : méthodes, entrées, sorties, effets
3. Contrat d'API des trois backends
1. Les six backbones fournis et leurs adaptations FedX
2. Comparaison des stratégies fournies nativement par FedX
3. Combinaisons de confidentialité et propriétés obtenues

1. Composants deployes et leur contrat de disponibilité
2. Métriques FedX, seuils d'alerte et conséquences
3. Les six pages de la console FedX
1. Comparaison des méthodes d'apprentissage contrastif fédéré
1. Estimateurs d'effet causal utilisables en cadre fédéré
1. Ordres de grandeur de consommation énergétique
2. Réseau impulsionnel contre réseau artificiel : comparatif opérationnel
1. Méthodes fédérées sur graphes : mécanisme et hypothèse
2. Supports de données et stratégies fédérées
1. Progression détaillée sur douze semestres
2. Jalons de la feuille de route 2026-2030
1. Conférences de référence et jalons de soumission
1. Arborescence normalisée du dépôt de travaux pratiques
2. Conventions de nommage et de figure
3. Grille d'évaluation commune d'un travail pratique
4. Les neuf travaux pratiques de la partie VII
1. Correspondance entre les TP et les chapitres théoriques
1. Panorama des cinq projets de fin de semestre
1. Progression des quinze exercices
1. Architecture de l'évaluation du module
2. Barème de l'épreuve écrite et correspondance aux exercices
3. Critères de certification et pièces requises
4. Outils par famille de travaux
1. Abréviations employées dans le cours
1. Entrées du glossaire commençant par A
2. Entrées du glossaire commençant par B
3. Entrées du glossaire commençant par C
4. Entrées du glossaire commençant par D
5. Entrées du glossaire commençant par E
6. Entrées du glossaire commençant par F
7. Entrées du glossaire commençant par G
8. Entrées du glossaire commençant par H
9. Entrées du glossaire commençant par I
10. Entrées du glossaire commençant par J
11. Entrées du glossaire commençant par L
12. Entrées du glossaire commençant par M
13. Entrées du glossaire commençant par N
14. Entrées du glossaire commençant par P
15. Entrées du glossaire commençant par Q
16. Entrées du glossaire commençant par R
17. Entrées du glossaire commençant par S
18. Entrées du glossaire commençant par T
19. Entrées du glossaire commençant par V
1. Notation symboLique utilisee dans l'ouvrage
1. Jalons fondateurs de l'apprentissage federede
1. Familles d'outils employees par le Département de R&D

Fin du document

Intelligence Artificielle Avancée & Federated Learning — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design
Département de Recherche & Développement — Recherche
Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.